



חיכון היובל - ספר פרויקט

"The Simian Legend"

אורי רוגל

326551058

מנחה - אילנה ויסמן

תכנון ותכנות מערכות טלפונים חכמים



"15/4/2026"

"כ"ח בניסן התשפ"ו"



תוכן עניינים

5.....	מבוא
5	הרקע לפרויקט
5	תהליך המחקר
5	למידה עצמאית
5	בדיקת אפליקציות קיימות
5	אתגרים מרכזיים
7.....	תיאור תחום הידע - פרק מילולי
10.....	מבנה/ארכיטקטורה של הפרויקט
10.....	הסבר על המסכים
10.....	מסך הפתיחה - SplashActivity
11.....	מסך ההתחברות - LoginActivity
12.....	מסך ההרשמה - RegisterActivity
13.....	מסך ה"משגר" - LauncherActivity
13.....	פרגמנט הגדרות - SettingsFragment
14.....	פרגמנט שפה - LanguageFragment
14.....	פרגמנט מסגרות - FramesFragment
15.....	מסך המשחק - GameActivity
17.....	תרשים זרימת מסכים
18.....	תרשים UML
19.....	מימוש הפרויקט
19.....	ניהול מידע בענן (Cloud & Data Management)
19.....	מחלקת UserDataManager
22.....	מחלקת BaseDocument
25.....	מחלקת ProfileDoc
27.....	מחלקת StatsDoc
29.....	מחלקת ProgressDoc
32.....	מחלקת CosmeticDoc
34.....	מחלקת LoginActivity
37.....	מחלקת RegisterActivity
40.....	תשתית וליבת המנוע (Core Engine & Lifecycle)
40.....	מחלקת GameLoop
43.....	מחלקת GamePanel
45.....	מחלקת GameActivity
48.....	מחלקת GameStateInterface (ממשק)
48.....	מחלקת SplashActivity
51.....	מחלקת BaseState
52.....	מחלקת MenuManager
54.....	מחלקת BaseMenu
56.....	מחלקת BitmapManager
59.....	מערכת הישויות והרכיבים (Entity System & Components)
59.....	מחלקת Entity
61.....	מחלקת GameCharacters (Enum)
63.....	מחלקת AnimatedObject
63.....	מחלקת AnimatedType (Enum)
64.....	מחלקת BreakableEntity
66.....	מחלקת BrokenParticle
68.....	מחלקת StaticObject
70.....	מחלקת Coin



72.....	HealthComponent	מחלקת
74.....	StaminaComponent	מחלקת
76.....	JumpComponent	מחלקת
78.....	MovementComponent	מחלקת
80.....	CombatComponent	מחלקת
83.....	EmoteComponent	מחלקת
85.....	AnimationComponent	מחלקת
87.....	User Interface & Graphics))	ממשק משתמש וגרפיקה
87.....	BitmapMethods (ממשק)	מחלקת
89.....	CustomDialog	מחלקת
91.....	TextRenderer	מחלקת
93.....	UpgradeState	מחלקת
95.....	MainMenu	מחלקת
97.....	OptionsMenu	מחלקת
98.....	VideoSettingsMenu	מחלקת
100.....	MusicSoundsMenu	מחלקת
101.....	NpcHealthBar	מחלקת
103.....	DialoguePanel	מחלקת
105.....	StaminaDisplay	מחלקת
108.....	XpDisplay	מחלקת
110.....	CoinDisplay	מחלקת
112.....	TapEffect	מחלקת
114.....	CharacterRenderer	מחלקת
118.....	BaseButton	מחלקת
121.....	CircleButton	מחלקת
122.....	RectButton	מחלקת
123.....	CustomSeekBar	מחלקת
126.....	CustomRadioGroup	מחלקת
129.....	CustomSwitch	מחלקת
132.....	CustomUpgrade	מחלקת
134.....	Joystick	מחלקת
137.....	View & World Management))	ניהול עולם ותצוגה
137.....	CameraManager	מחלקת
140.....	SceneManager	מחלקת
142.....	BaseScene	מחלקת
145.....	DeathScreen	מחלקת
148.....	DialogState	מחלקת
151.....	WeatherEffect	מחלקת
153.....	Leaf	מחלקת
155.....	Cloud	מחלקת
157.....	Content & Customization))	ניהול תוכן וקוסמטיקה
157.....	FrameUnlockCondition	מחלקת
158.....	CircleFrameSeries	מחלקת
159.....	FrameData	מחלקת
160.....	FramesAdapter	מחלקת
162.....	FrameSeriesAdapter	מחלקת
164.....	Logic Utilities & Algorithms))	כלי עזר ואלגוריתמים
164.....	TmxLoaderUtils	מחלקת
168.....	ObjectPoolManager	מחלקת
170.....	DialogueManager	מחלקת



174.....	מחלקת Quest
176.....	מחלקת CollisionUtils
178.....	מערכת השמע (Audio System)
178.....	מחלקת MusicManager
181.....	מחלקת SoundManager
184.....	מדריך למשתמש
184.....	דרישות התקנה וסביבת עבודה
184.....	אופן פעולת האפליקציה
184.....	אופן המשחק
185.....	הודעות למשתמש ומשוב (Alerts & Feedback)
185.....	מגבלות ואילוצים
186.....	רפלקציה
187.....	ביבליוגרפיה
188.....	נספחים
188.....	נספח א': קוד המקור המלא של הפרויקט
189.....	נספח ב': תהליך עיצוב השלבים - המחשה ויזואלית בעזרת Tiled
191.....	נספח ג': ממשק המשתמש, התראות ומשוב חזותי
19190.....	נספח ד': ממשק המשתמש (UI) ותפריטי המשחק



מבוא

הרקע לפרויקט

- **שם האפליקציה:** "The Simian Legend"
- **תיאור הפרויקט:** הפרויקט הינו אפליקציית משחק מסוג RPG וסינגלפלייר עלילתי. במשחק השחקן ייתקל במגוון דמויות ואויבים, עם חלקם יהיה ניתן לנהל אינטראקציה ובכך יגלה ויבין את סיפורו של המשחק, ועם חלקם להילחם ובכך לעלות רמות. ככל שיעלה ברמות ויתקדם בסיפור - כך גם תעלה דרגת הקושי. במהלך המשחק יהיה ניתן לשפר את מדדי השחקן ולחזקו ככל הניתן.
- **קהל היעד:** קהל היעד הוא כל חובבי האקשן וההרפתקאות ומתאים לילדים ומבוגרים כאחד (גילי 5+).
- **מדוע בחרתי בפרויקט זה:** בילדותי נהגתי לשחק משחקים רבים - שולחן ומובייל - תמיד תהיתי איך יוצרים את המשחקים האלה ואם גם אני מסוגל לעשות דברים שכאלו. אני מחשיב את עצמי אדם יצירתי, כך שכאשר נפתחה לי האופציה להתנסות בכך בבית הספר, לא היססתי והתחלתי מיד לעבוד על משחק לטעמי.

תהליך המחקר

למידה עצמאית

במהלך עשיית הפרויקט התמקדתי בעיקר על שימוש ומימוש ב-canvas של אנדרואיד סטודיו באמצעות הדפסה על SurfaceView: תוך למידה על מפות ביטים (bitmaps), שינוי קנה מידתן (rescale) וקנה ה-canvas, אירועי מגע (touch events), ויצירת והמרת אלמנטים (views) מוכרים (כמו כפתור, מתג, "סרגל הזזה" - seekbar, ועוד) לכאלו שאני מדפיס בעצמי באמצעות ה-canvas ומתמודד עם אירועי המגע. כל זאת למדתי וחקרתי בעצמי, שכן נושא ה-SurfaceView הוא נושא מתקדם שאינו נלמד בחומר. יתר על כן, למדתי וחקרתי באופן עצמאי על שימוש בבסיס הנתונים Firebase והתמקדתי בעיקר בשימוש של קריאה וכתיבה ב-Firestore. כמו כן, למדתי רבות על שימוש ב-Tiled, שהוא תוכנת יצירת מפות דו-מימדיות בצורה ויזואלית וליצואן בקבצי CSV, ובכך ניתן ליצור ולעצב את המפות כולל הצבת אובייקטים בצורה הרבה יותר קלה ויעילה.¹

בדיקת אפליקציות קיימות

מבחינה ויזואלית, לקחתי הרבה השראה ממשחקי Supercell, כמו: "Clash Royal" ו-"Brawl Stars", ו-"Squad Busters". משחקי RPG שהייתה להם השפעה רבה על החשיבה וההבנה שלי כיצד משחקים מהסוג הזה אמורים להראות ולהתבצע הם "Slayer Legend" (זהו גם משחק פיקסל-ארט כמו שלי) ו-"Survivor.io". ניסיתי לשוות לאפליקציה שלי מראה אסתטי ומקצועי ואני מרגיש שהמשחק שיצרתי הוא שילוב של כל המשחקים והידע שהיה לי בתוך הראש שרק חיפש דרך להתבטא, ולמרות שלא אני ציירתי ויצרתי את רוב הערכה בה אני משתמש במשחק (למרות שכן יש הרבה דברים שיצרתי בעצמי ואני מאוד מתרשם מכך), אני מאמין שהצלחתי להשתמש בה בצורה ייחודית לי.

אתגרים מרכזיים

כאשר התחלתי לעבוד על הפרויקט ובכל שלביו, עמדו בפני מספר אתגרים מרכזיים:

- כיצד להזיז את הדמויות?
- כיצד לקיים את האנימציה?
- כיצד לנהל את מערכת ההתקפה של הדמויות ואת המפגשים ביניהן?
- כיצד לעצב את המפה?
- כיצד לטעון נתונים מתוכנת Tiled בה אני מעצב את המפה לכדי מפה ואובייקטים?

¹ לפירוט טכני על אופן טעינת המפות למשחק ודוגמאות מהמערכת, ניתן לעיין בנספח ב': [תהליך עיצוב השלבים בעזרת Tiled](#).



- כיצד תראה מערכת שדרוג המדדים במשחק וכיצד תפעל?
- כיצד ניתן להפעיל את האפליקציה גם במצב אופליין?
- איך ניתן לשמור על רזולוציה זהה בין מסכים בגדלים שונים?
- איך להשמיע מוזיקה ואפקטים קוליים?
- איך להגדיר את עוצמת האפקטים הקוליים בהתאם למיקומם ביחס לשחקן?
- איך לנהל את המשימות כך שתיווצר עלילה?
- איך לתת משימות לקידום העלילה ללא מתן הרגשה לשחקן של "להכריח" אותו?
- איך ליצור מערכת שמתחשבת ומציירת "אורות" שיבלטו בסביבה?
- איך לייעל את מערכת התאורה ולהשאיר את ביצועי ההרצה גבוהים?
- מה הדרך הכי נכונה עבורי לעצב את ממשק המשתמש שיהיה הכי נוח?
- מה פורמט התמונה הכי מומלץ ויעיל להשתמש בו בפרויקט?
- התקלויות בבאגים.



תיאור תחום הידע - פרק מילולי

פרק זה מתאר את האובייקטים, סוגי הנתונים, ייצוג המידע והפעולות המרכזיות שנעשות עליו בפרויקט.

כששחקן פותח את המשחק בפעם הראשונה, הוא אינו נחשף למסמך טכני - אלא נזרק ישירות אל סצנת פתיחה (CutScene) המציגה את עולם המשחק ומציבה את המשימה הראשונה. לאחר שהסצנה מסתיימת, הדמות משוגרת לנקודת ההתחלה בכפר. על המסך מופיעה משימת פתיחה המבקשת ממנו "לדבר עם אביך". השחקן משוטט ברחובות, נכנס לבתים, מתבונן בדמויות השכנות - הנפח, חברו הטוב והשומרים - ומקבל במהרה תחושה שהוא חלק ממערכת חיה ודינמית: דמויות מדברות עמו, פריטים נעלמים כאשר הוא אוסף אותם, ודלתות נפתחות כאשר מתאפשר לו להיכנס למרתף ישן מתחת לאחד הבתים.

מאחורי האשליה הזו עומדת מערכת ברורה ומוגדרת: כל ישות בעולם - דמות, חפץ, שער או מטבע - מיוצגת כאובייקט בעל מיקום וגבולות פיזיים, שחלק מהם ניתן להרוס כדי לפנות מקום לשחקן. המיקום מיוצג באמצעות מלבן (hitBox) בעל קואורדינטות מספריות, המאפשר למנוע המשחק לזהות התקרבות, התנגשויות ואינטראקציות עם הסביבה. כאשר השחקן קופץ או תוקף, המנוע בודק חיתוך בין מלבנים - מנגנון פשוט ויעיל שמייצר תחושת פיזיקה חלקה בעולם דרמטי.

המפה עצמה בנויה כרשת דרמטית של משבצות (tiles) המרכיבות את הכפר, הבתים והמרתפים. ייצוג זה מאפשר לזהות בקלות מה נמצא בכל אזור, להגדיר מעברים נסתרים ולנווט נתיבי תנועה של דמויות לא-נשלטות (NPC). בזמן ריצה, כל הישויות הפעילות נשמרות במבני נתונים דינמיים (רשימות) - רשימת NPC, רשימת אויבים, רשימת חפצים וכדומה. בכל פריים מתבצע מעבר על רשימות אלו לצורך עדכון וציור. מבנה זה מאפשר הוספה והסרה של ישויות בזמן אמת - למשל יצירת אויב חדש במהלך קרב או הסרת מטבע שנאסף - מבלי לפגוע בזרימת המשחק.

כאשר השחקן מתקרב ל-NPC ומתחיל דיאלוג, אין מדובר בטקסט בלבד - המשימה מתעדכנת בהתאם. שיחה יכולה לפתוח משימה חדשה, להעניק נשק ראשון או להוסיף ניסיון (XP) לשחקן. הניסיון והמטבעות הם משאבים מרכזיים במערכת: XP מאפשר עליית רמות, וכל עליית רמה מעניקה נקודות שדרוג שבעזרתן השחקן בוחר לשפר חיים, כוח או מהירות תקיפה. המטבעות משמשים לרכישת פריטים קוסמיים ומסגרות, כאשר חלקם נפתחים רק לאחר השלמת משימות מסוימות - ובכך נוצר חיבור ישיר בין התקדמות עלילתית לבין מערכת הכלכלה של המשחק.

אחד המרכיבים המרכזיים המשפיעים על אופי המשחק הוא מד האנרגיה. מד זה מייצג משאב שמתחדש בהדרגה אך נצרך בפעולות כמו קפיצה והתקפה. מבחינה עלילתית, הוא מאלץ את השחקן לחשוב ולפעול באופן מחושב ולא לבצע פעולות ללא הגבלה. מבחינה טכנית, מדובר בערך מספרי המתעדכן בכל פריים ונבדק על ידי לוגיקת המשחק כדי לקבוע האם פעולה מסוימת אפשרית.

ככל שהעלילה מתקדמת, נחשף השחקן לאויבים וישויות חדשות, והקומבט הופך למרכזי יותר בחוויה. הלחימה מרגישה טבעית ואינטואיטיבית: השחקן לוחץ להתקפה, מתבצעת אנימצית תקיפה, ואם הפגיעה מצליחה - מופעל מנגנון הדיפה (knockback). הישות הנפגעת נדחפת לאחור ונכנסת למצב זמני שבו אינה יכולה לתקוף. תחושת ההדיפה מעניקה לקרב משקל ומשמעות לכל פעולה.

האויבים אינם ישויות פשוטות או "בוטים" בסיסיים; גם להם קיימים רכיבים דומים לאלו של השחקן - אנרגיה, יכולת קפיצה, מהירות תנועה ומד חיים - אך בערכים שונים בהתאם לסוג הדמות. יש אויבים זריזים ומהירים, אחרים כבדים וחזקים יותר. מכיוון שגם לשחקן וגם לאויבים יש יכולת קפיצה, הם יכולים לקפוץ זה מעל זה. בעת קפיצה מתבצע שינוי זמני במיקום האנכי של ה-virtual z-offset (hitBox), מה שמאפשר מעבר מעל ישות אחרת במקום התנגשות ישירה. יכולת זו מוסיפה רובד טקטי משמעותי ללחימה.

בנוסף ליכולות הפיזיות, קיימת מערכת מצבים המדמה "רגשות". כאשר אנרגיית ישות (אויב או NPC) מתרוקנת, היא נכנסת למצב פחד: מעליה מופיע סימון ויזואלי מתאים והיא פונה להימלט מהשחקן. כיוון



התנועה שלה מחושב כך שתרחק ממנו עד שהאנרגיה תתחדש לרמה מספקת. כאשר האנרגיה מתמלאת, הישות מציגה סימון יהיר וחוזרת למצב תקיפה. עם זאת, אם מד החיים שלה יורד מתחת לסף קריטי, היא נכנסת למצב לחימה עד הסוף ומתעלמת ממגבלת האנרגיה - מה שיוצר קרב אינטנסיבי ומאתגר יותר.

כל ההתנהגויות הללו מנוהלות בתוך לולאת העדכון של מנוע המשחק. בכל פריים מתבצעים עדכוני מדדים (התחדשות אנרגיה, תנועה), בדיקות תנאים (קרבה לשחקן, רמת אנרגיה נמוכה) ומעבר בין מצבים כגון Dead, Idle, Pursue, Flee, Attack. מצבי ה"רגש" מתורגמים לאנימציות ולאייקונים ויזואליים המופיעים מעל הדמות, וכן משפיעים על פרמטרי ה-AI כמו מהירות תנועה או דפוס תקיפה.

מנגנון ה-knockback פועל באמצעות חישוב וקטור דחיפה על ה-hitBox של הנפגע והגדרת מצב זמני של Stun. לכל נשק מוגדרים ערכי הדיפה שונים - הן בעוצמה והן במשך ההמומה - כך שנשקים קלים ומהירים דוחפים פחות, בעוד נשקים כבדים גורמים להדיפה משמעותית יותר אך בקצב איטי.

כאשר אויב מובס, מופעל מנגנון תגמול המחלק XP ומטבעות. לכל סוג אויב מוגדרים ערכי מינימום ומקסימום עבור תגמולים אלה, וכן משקל הסתברותי (weight). בעת חיסול האויב האלגוריתם בוחר ערך אקראי בטווח שהוגדר, תוך התחשבות במשקל ההסתברותי של אותו סוג דמות. כך מתקבלת מערכת מאוזנת: התגמול מרגיש אקראי ומגוון, אך נשלט ומתוכנן לאורך זמן.

המידע שהשחקן צובר אינו נשמר רק במהלך המשחק הפעיל. פרטי הפרופיל, מדדי הסטטוס, התקדמות במשימות ופריטים קוסמיים נשמרים בענן באמצעות Firebase Firestore, כך שהשבון נגיש ממכשירים שונים. מבנה ה-NoSQL מאפשר גמישות בהוספת שדות חדשים ללא צורך בשינוי מבנה טבלאי מורכב.

כדי למנוע עיכובים בזמן טעינה, נעשה שימוש במנגנון Cache מקומי הטוען את מסמכי המשתמש בתחילת ההפעלה ושומר עותק בזיכרון. במהלך המשחק נעשות קריאות מקומיות בלבד, ועדכוני ענן מתבצעים רק כאשר יש שינוי מהותי (כגון עליית רמה או רכישת פריט). הכתיבה לענן נעשית באופן מבוקר, תוך מיזוג שדות, כדי למנוע דריסה של מידע קיים.

בנוסף, מערכת השמירה של המשחק אינה מוגבלת להתקדמות אחת בלבד, אלא מאפשרת ניהול של מספר סלוטים (Slots), כאשר כל סלוט מייצג התקדמות נפרדת בעולם המשחק. כך השחקן יכול להתחיל משחק חדש מבלי לאבד התקדמות קודמת, או לנהל מספר מסלולי משחק במקביל. כל סלוט נשמר כיחידת מידע עצמאית בענן, הכוללת את נתוני ההתקדמות, הסטטוס והמשימות של אותו משחק, וכאשר השחקן בוחר סלוט מסוים - הנתונים הרלוונטיים נטענים לזיכרון ומנהלים את חווית המשחק הנוכחית. בנוסף, קיימת אפשרות לאפס או למחוק סלוט קיים, פעולה אשר מוחקת את נתוני ההתקדמות של אותו סלוט ומאפשרת התחלה חדשה. מבחינה מערכתית מדובר בפעולת מחיקה משמעותית, ולכן היא מתבצעת באופן מבוקר כדי למנוע מחיקה לא מכוונת של נתונים.

סדר הציור של הישויות נקבע לפי ציר ה-Y: ישויות הנמצאות נמוך יותר על המסך מצוירות אחרונות, וכך נוצר אפקט עומק בעולם דו-ממדי ללא שימוש במנוע תלת-ממדי.

בנוסף לחלק המשחקי, המשתמש יכול להתאים את חווית המשחק דרך מסך הגדרות: שינוי שפה, שליטה ברמת הזום, הצגת FPS, הצגת או הסתרת hitBoxes והפעלת רטט (haptics). העדפות אלו נשמרות הן מקומית והן בענן, ולכן משפיעות מיידית על החוויה ונשמרות גם בין מכשירים.

מרכיב נוסף שמעמיק את תחושת העולם הוא מערכת התאורה הדינמית של המשחק. אזורי מסוימים במפה - כמו מרתפים, מבנים נטושים או לילה באזורים חיצוניים - מוצגים כשהם מוחשכים חלקית. במקום לשנות את המפה עצמה, מנוע המשחק מצייר שכבת חושך על גבי הקנבס, ומעליה נוצרים מקורות אור (Light Sources) המוחקים חלק מהחושך ברדיוס מסוים. כל מקור אור מוגדר עם מיקום, רדיוס וצבע, ויכול להיות סטטי או דינמי - למשל לפיד מהבהב או אור פועם. במהלך ציור הפריים נוצרת מסכת תאורה (Light Mask) שמחשיכה את הסצנה, ואז מנוקים ממנה אזורי האור סביב לפידים, דמויות או אפקטים. כך מתקבל אפקט של עולם חשוך שבו האור חושף רק את הסביבה הקרובה, מה שמעצים את האווירה ומוסיף עומק ויזואלי למשחק.



לצד המערכת הוויזואלית פועלת גם מערכת סאונד המשלימה את החוויה. מוזיקת רקע משתנה בהתאם לאזור או למצב המשחק - מוזיקה רגועה בכפר, מוזיקה מתוחה יותר בעת קרב או באזורים מסוכנים. בנוסף למוזיקה, פעולות רבות במשחק מלוות באפקטים קוליים (SFX): פגיעה באויב, קפיצה, איסוף מטבע או פתיחת דלת. מערכת ניהול הצלילים מרכזת את הפעלת האפקטים ומאפשרת שליטה בעוצמת השמע דרך הגדרות המשתמש. כל אפקט מושמע בזמן הפעולה המתאימה וכך מחזק את תחושת המשוב של המשחק - השחקן לא רק רואה את הפעולה אלא גם שומע אותה. שילוב המוזיקה והאפקטים הקוליים תורם ליצירת חוויה עשירה וסוחפת יותר.

לבסוף, למשחק קיימים שני מצבי סיום - זמני וסופי. במהלך הקרבות השחקן עשוי לאבד את כל נקודות החיים שלו. במקרה כזה מופעל מנגנון מוות: הדמות מסומנת כמתה והמשחק מחזיר את השחקן לנקודת הביקורת (Checkpoint) האחרונה שנשמרה. החזרה מתבצעת עם המדדים שהיו תקפים באותה נקודה, ובכך נשמר איזון בין ענישה לבין המשכיות.

בנוסף, ניתן להגיע לסיום עלילתי מלא. כאשר השחקן משלים את כל המשימות המרכזיות הקיימות בקו העלילה הנוכחי, המערכת מזהה את השלמת הסיפור ומפעילה רצף סיום מתאים. כך המשחק אינו מסתיים רק בכישלון אפשרי, אלא גם בהצלחה נרטיבית - השלמת המסע, פיתוח הדמות וסגירת מעגל העלילה.



מבנה/ארכיטקטורה של הפרויקט

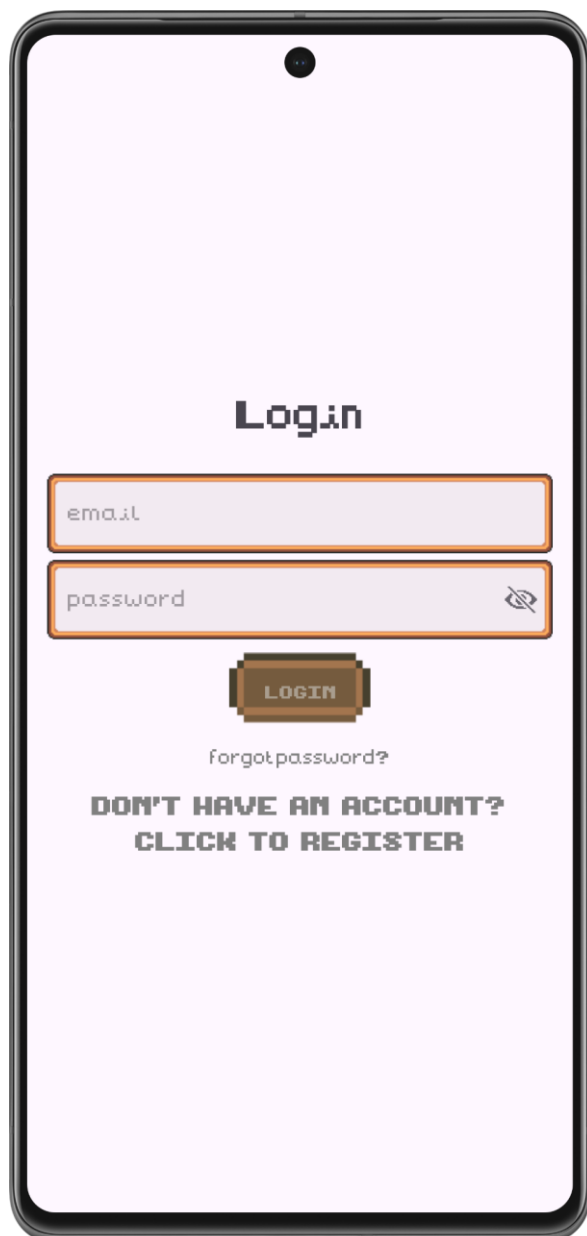
הסבר על המסכים

מסך הפתיחה - SplashActivity



מסך זה הינו המסך הראשון שמופיע כאשר נכנסים לאפליקציה, מטרתו היא להוות טעינה קצרה של נתוני הענן לפני שעוברים למסך המרכזי, כדי למנוע מצב של אי נעימות בו מחלקה מנסה לגשת לנתונים מהענן לפני שהם סיימו להיטען. המסך מציג את שם האפליקציה עם לוגו שרץ עם אנימציה בצורה אינסופית שמראה כי קורה משהו.

ניווט: אין למשתמש שליטה לאן המסך יעביר אותו, משתמש שהיה מחובר לחשבון לפני שנכנס לאפליקציה יעבור ל-`LauncherActivity`, ובמידה שלא היה מחובר, או שזו פעם ראשונה שלו באפליקציה - יועבר ל-`LoginActivity`. אין למשתמש איך לחזור למסך זה.



מסך ההתחברות - LoginActivity

מסך זה מטרתו היא התחברות לחשבון קיים.

ניווט²: ניתן לגשת אליו דרך ה-Fragment Settings וה-RegisterActivity ולעבור ממנו ל-LauncherActivity.

מה יש במסך?

- במסך יש שתי תיבות טקסט - בהן אפשר לרשום ולהזין את פרטי ההזדהות שלך (אימייל וסיסמה).
- כפתור אחד - לאחר שהזנת את פרטי ההזדהות האישיים, במידה שהפרטים נכונים ושמורים במערכת (והאימייל אומת - במידה שלא, ישלח אימייל לאימות), לחיצה על הכפתור תוביל אותך למסך המשגר (launcher).
- שני אלמנטים לחיצים של טקסט, לחיצה על "forgot password?" (במידה ומילאת את האימייל) תשלח אימייל לאיפוס הסיסמה. לחיצה על "don't have an account? click to register" תעביר אותך למסך ההרשמה.

² בכניסה הראשונית לאפליקציה, במידה שהמשתמש לא מחובר ה-SplashActivity יעביר אליו ישירות



Register

nickname

email

password

REGISTER

ALREADY HAVE AN ACCOUNT?
CLICK TO LOGIN

מסך ההרשמה - RegisterActivity

מסך זה הינו המסך המשלים למסך ההתחברות שהוצג קודם, במידה ואין למשתמש חשבון הוא יעבור למסך הזה ויבצע הרשמה לאפליקציה וייצור חשבון קיים. **ניווט** : ניתן לגשת אליו דרך ה-LoginActivity ולצאת אליו.

מה יש במסך?

- במסך יש שלוש תיבות טקסט - בהן אפשר לרשום ולהזין את פרטי ההזדהות שלך (אימייל וסיסמה) וליצור כינוי רשמי ויחיד למשתמש בחשבון (האפליקציה בודקת ולא מאפשרת רישום עם כינוי שכבר קיים).
- כפתור אחד - לאחר שהזנת את פרטי ההזדהות האישיים, במידה שאין כבר משתמש עם אותו האימייל והכינוי, לחיצה על הכפתור תוביל אותך חזרה למסך ההתחברות, כדי להתחבר עם החשבון החדש.
- אלמנט טקסט לחץ אחד - לחיצה על " already have an account? click to login" תעביר אותך למסך ההתחברות שם תוכל להזין פרטי חשבון קיים ולהתחבר לאפליקציה.



מסך ה"משגר" - LauncherActivity

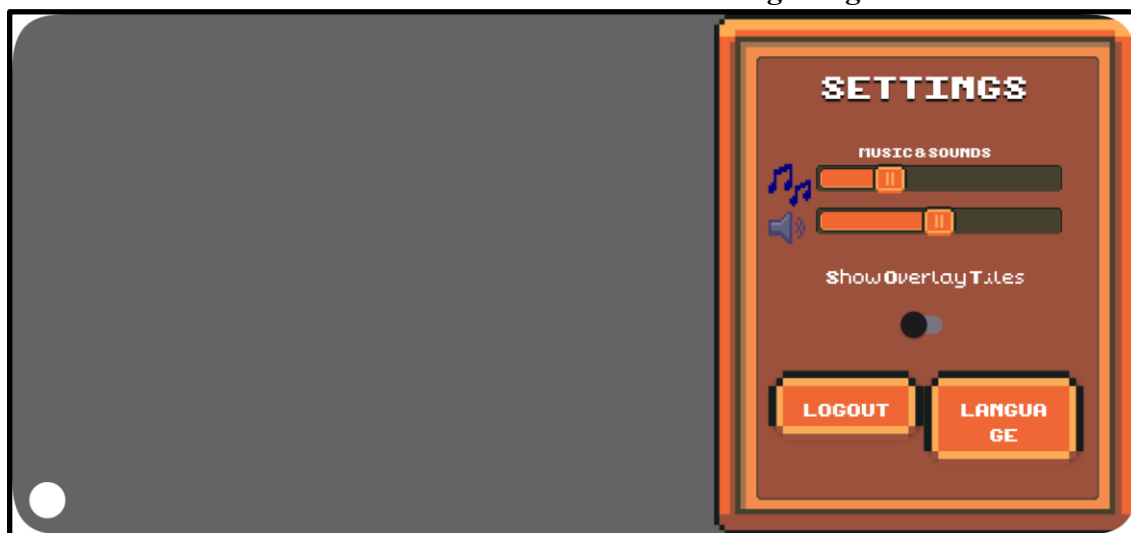


מסך זה הינו מסך ה"שיגור" למשחק עצמו ומהווה את המסך ה"ראשי" של האפליקציה (תמיד מתחילים במסך ההתחברות, אך במידה שאתה מחובר יעביר אותך ישירות למסך הנ"ל).
ניווט : ממנו ניתן להגיע ל-ProfileFragment, SettingsFragment ול-GameActivity, ואליו ניתן להגיע דרך LoginActivity, GameActivity, SettingsFragment, LanguageFragment.

מה יש במסך?

- במסך יש שני כפתורים - כפתור הממוקם למעלה מימין ופותח את פרגמנט ההגדרות, וכפתור "play" אשר מעביר אותך למסך המשחק.
- שני אלמנטי תמונה - אחת המציגה תמונה של פרופיל המשתמש, והשניה מציגה את הפריים הנבחר על ידי, על ידי לחיצה על אותו הפריים נפתח ומוצג ProfileFragment.

פרגמנט הגדרות - SettingsFragment



פרגמנט זה הינו הדרך של המשתמש לשנות את הגדרות האפליקציה, כמו ווליום, שפה, ועוד.
ניווט : ממנו ניתן להגיע ל-LoginActivity, LanguageFragment, LauncherActivity (באמצעות התנתקות מהחשבון). אליו ניתן להגיע דרך LauncherActivity.

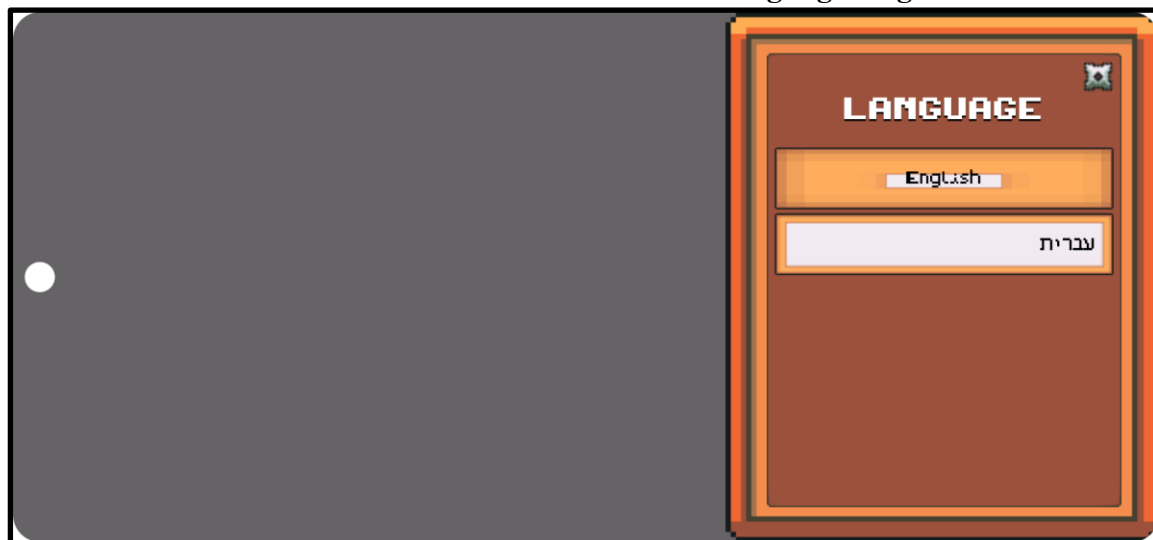
מה יש בפרגמנט?

- בפרגמנט יש שני "סליידרים" (seekbar), אחד שולט על עוצמת המוזיקה של האפליקציה, והשני שולט על עוצמת הצלילים והאפקטים הקוליים של האפליקציה.



- שני כפתורים - כפתור התנתקות שיפתח דיאלוג ששואל אם המשתמש באמת מעוניין להתנתק מהחשבון, וכפתור "שפה" שפותח פרגמנט חדש בו אפשר לשנות את שפת האפליקציה.

פרגמנט שפה - LanguageFragment



פרגמנט זה הינו הדרך של המשתמש לשנות את שפת האפליקציה בהתאם לבחירתו.
ניוט: ממנו ניתן להגיע ל-SettingsFragment ול-GameActivity, ואליו ניתן להגיע דרך SettingsFragment.

מה יש בפרגמנט?

- בפרגמנט ישנם כפתורים לשינוי שפה (כרגע שניים), ובהתאם לכפתור עליו המשתמש לוחץ המערכת משנה את שפתה לשפה הרצויה.

פרגמנט מסגרות - FramesFragment



פרגמנט זה משמש כגלריה מרכזית עבור פריטי העיצוב (מסגרות לתמונת הפרופיל) של השחקן. תפקידו העיקרי הוא לאפשר לשחקן לצפות בכל סדרות המסגרות הקיימות במשחק, לראות אילו מהן כבר נאספו, ולעקוב אחר התקדמות האיסוף הכללית שלו. הוא מציג רשימה מאורגנת של "סדרות" (קולקציות) של מסגרות מעגליות. בראש המסך מופיעה כותרת המלווה במונה התקדמות המציג כמה מסגרות השחקן



צבר מתוך הסך הכל הכללי הקיים במשחק. המסך מעוצב כחלון המופיע מעל ממשק הפרופיל, ומאפשר דפדוף מהיר ונוח בין קטגוריות העיצוב השונות.

מה יש בפרגמנט?

- כפתור סגירה (X): כפתור גרפי הממוקם בפינת המסך. לחיצה עליו סוגרת את הגלריה ומחזירה את המשתמש למסך הפרופיל הראשי.
- מונה איסוף (Collection Counter): שדה טקסט בולט המציג את הסטטוס הנוכחי של השחקן (לדוגמה: "3/67"). המונה מתעדכן בזמן אמת במידה והשחקן פותח מסגרות חדשות.
- רשימת סדרות מרכזית (Main List): אזור התצוגה המרכזי של המסך המאפשר גלילה אנכית. בתוך רשימה זו מוצגות כל סדרות המסגרות (למשל: סדרת מסגרות "זהב", סדרת "טבע" וכו').
- כרטיסיית סדרה (Item Row): כל שורה ברשימה מייצגת סדרה שלמה, הכוללת את שם הסדרה ואת הפריטים השייכים לה.

תרשים זרימה מילולי (מהלך האירועים במסך):

1. **כניסה לפרגמנט:** המשתמש לוחץ על כפתור העריכה או בחירת המסגרת בפרגמנט הפרופיל.
2. טעינת נתונים: המערכת סורקת את מאגר המידע של השחקן בענן, בודקת אילו מסגרות שייכות לו, ומחשבת את היחס בין הפריטים שנאספו לפריטים הקיימים.
3. תצוגה ראשונית: המסך נפתח, המונה העליון מתעדכן במספר המדויק, והרשימה המרכזית נפרסת בפני המשתמש.
4. אינטראקציה: המשתמש גולל ברשימת הסדרות. במידה ומתבצעת פעולה שמשנה את מצב האיסוף (כמו רכישה בתוך תתי-הרשימות), המערכת מזהה את השינוי ומעדכנת באופן מיידי את המונה בראש המסך ללא צורך בטעינה מחדש של כל הגלריה.
5. **יציאה:** המשתמש לוחץ על כפתור הסגירה; המסך הנוכחי מוסר והמשתמש חוזר בדיוק לנקודה בה הפסיק בפרגמנט הפרופיל. או שיכול ללחוץ מחוץ לשטח הפרגמנט וכך יחזור ישירות למסך "המשגר".

מסך המשחק - GameActivity



מסך זה הינו המסך בו קורה כל הקסם - המשתמש נכנס והופך לשחקן המיוצג על ידי קוף חמוד (שעוצב על ידי!) ונכנס לעולם קסום בו יש אויבים ובתים (!) **ניווט:** ממנו ניתן להגיע ל-LauncherActivity ואליה ניתן להגיע דרך האחרון.

מה יש במסך?

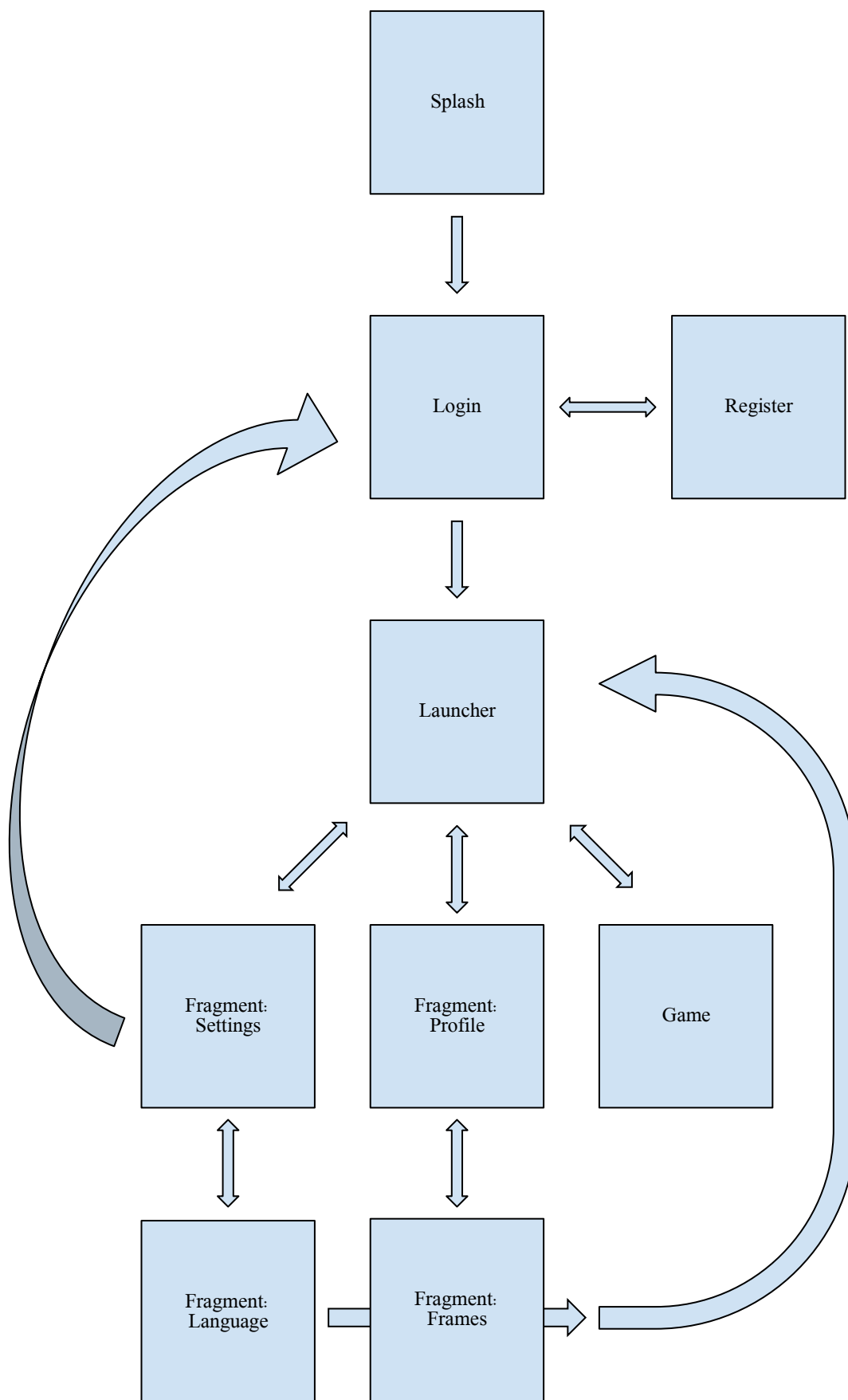
- *יש לציין שאין במסך רכיבי xml, אלא כל מה שבו נוצר ומוצג על ידי canvas
- במסך יש 5 כפתורים: כפתור התקפה שמיוצג על ידי אגרוף ומפעיל התקפה מצד השחקן, כפתור קפיצה שמיוצג ליד חץ כלפי מעלה, גיוסטק שמזיז את השחקן בהתאם למיקומו שנשלט על ידי



- השחקן, כפתור תפריט המוצג על ידי הכפתור העליון הימני שמצויר בו בית - הכפתור פותח את תפריט-תוך המשחק. הכפתור האחרון מיוצג בתמונת הקוף העליונה השמאלית - ולחיצה עליו תוביל למצב שיפור הדמות.
- בנוסף, ישנו כפתור נוסף שמופיע במקרה מסוים - זהו כפתור שיחה ואינטראקציה. הכפתור מופיע כאשר השחקן מתקרב לדמות שיש בה את האופציה לשיחה, וכאשר השחקן ילחץ על הכפתור הוא יכנס למצב דיאלוג עם אותה הדמות.

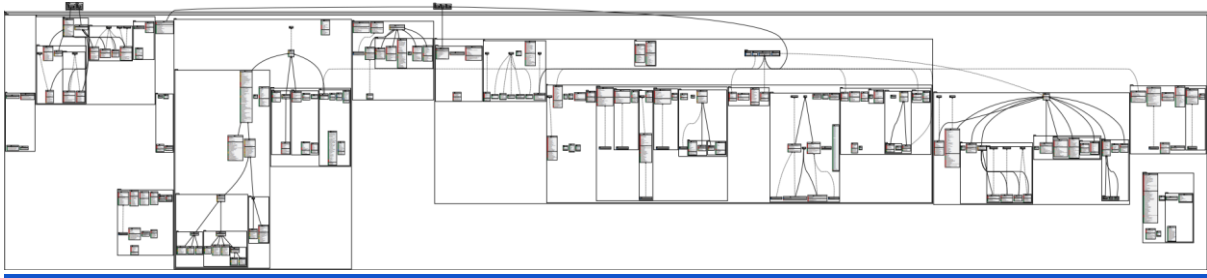


תרשים זרימת מסכים





תרשים UML



כדי לראות את התרשים ברזולוציה גבוהה יש להיכנס לקישור ולהוריד את קובץ ה-PDF.



מימוש הפרויקט

חלק זה מרחיב על שלבי התכנון ומתאר את המימוש הטכני של האפליקציה. הוא מחולק לשלושה חלקים עיקריים: לוגיקה ומנוע המשחק, גרפיקה וממשק משתמש, וניהול מידע בענן.

ניהול מידע בענן (Cloud & Data Management) מחלקה זו מהווה את השכבה המקשרת בין האפליקציה לשרתי *Firebase*.

מחלקת `UserDataManager`

מיקום: `com.example.tutorialgame.cloud.UserDataManager`

תפקיד המחלקה: מחלקה זו היא רכיב הליבה וה"מוח" של כל מערך ניהול המידע של המשתמש באפליקציה. היא משמשת כמנהל-על (Orchestrator) ונקודת גישה מרכזית (Facade) לכל הנתונים השמורים בענן.

היא אחראית על:

1. ריכוז גישה: להחזיק מופעים של כל מחלקות המסמכים (`ProfileDoc`, `StatsDoc`, וכו') ולאפשר גישה אליהם משאר חלקי האפליקציה.
 2. ניהול תהליך טעינה מורכב: לתזמר את הטעינה האסינכרונית של כל חלקי המידע של המשתמש, שהם למעשה מפות מקוננות בתוך מסמך אחד.
 3. דיווח על סטטוס: להודיע למערכת (בדרך כלל ל-`SplashActivity`) מתי כל הנתונים נטענו בהצלחה והאפליקציה יכולה להמשיך, או אם אירע כשל קריטי בתהליך.
 4. יצירת משתמש חדש: להפעיל את תהליך היצירה הראשוני של כל מסמכי הנתונים בענן עבור משתמש חדש.
- בארכיטקטורה זו, אף חלק אחר באפליקציה לא ניגש ישירות למסמכי הנתונים. כולם פונים דרך `UserDataManager`, מה שמבטיח שהגישה לנתונים היא תמיד לנתונים שעברו טעינה ואימות.

הסבר על תכונות המחלקה:

- `private ProfileDoc profileDoc, StatsDoc statsDoc, ProgressDoc progressDoc, CosmeticDoc cosmeticDoc, WorldStateDoc worldStateDoc`
 - תחום הכרה: `private` - ניתן לגשת אליהם רק מתוך המחלקה. הגישה החיצונית אליהם מתבצעת דרך מתודות `get` ציבוריות.
 - תפקיד ושימוש: אלו הם המופעים של המחלקות המנהלות כל פיסת מידע. לאחר שהנתונים נטענים מהענן, הם נשמרים בתוך המופעים האלה ומשמשים כ-`Cache` מקומי ומהיר.
- `private int successfulLoads`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: זהו מונה (counter) פנימי. מכיוון שתהליך הטעינה של כל מסמך (`profile`, `stats`, ...) הוא אסינכרוני ומתרחש במקביל, המונה סופר כמה מהם הסתיימו. הוא קריטי כדי לדעת מתי כל ארבעת התהליכים הסתיימו.
- `private boolean hasLoadingFailed`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: זהו דגל (flag) בטיחות למניעת "מצב מרוץ" (Race Condition). אם טעינת אחד המסמכים נכשלת, הדגל מורם. אם לאחר מכן טעינה של מסמך אחר מסתיימת בהצלחה, הדגל ימנע מהמערכת לדווח בטעות על הצלחה כוללת.



- `private final DocumentReference userDoc`
 - תחום הכרה: `private final`. משמעותו של `final` היא שלאחר האתחול הראשוני בבנאי, לא ניתן לשנות את הרפרנס הזה.
 - תפקיד ושימוש: רפרנס (הפניה) למסמך הראשי והיחיד של המשתמש באוסף `users` ב-Firestore. כל מחלקות ה-`Doc` מקבלות את הרפרנס הזה בבנאי שלהן, כך שכולן פועלות על אותו מסמך מרכזי.
- `public interface OnDataLoadedListener`
 - תחום הכרה: `public` - הממשק חשוף וניתן למימוש על ידי מחלקות חיצוניות (כמו `SplashActivity`).
 - תפקיד ושימוש: זהו חוזה (contract) של `Callback`. הוא מגדיר שתי מתודות, `onDataLoadSuccess` ו-`onDataLoadFailed`, ומאפשר ל-`UserDataManager` לדווח בחזרה לקוד שקרא לו על תוצאת תהליך הטעינה המורכב.

הסבר פעולות מרכזיות:

- `initializeNewUserData(String email, String nickname)`
 - פעולה זו נקראת רק פעם אחת, לאחר הרשמה של משתמש חדש. היא מאתחלת את כל מנהלי המסמכים (`ProfileDoc` וכו') וקוראת למתודות ה-`create` של כל אחד מהם, כדי ליצור את המפות המקוננות עם ערכי ברירת המחדל במסמך המשתמש ב-Firestore.
- קוד:

```
public void initializeNewUserData(String email, String nickname) {
    if (profileDoc == null || progressDoc == null || statsDoc == null || cosmeticDoc == null) {
        profileDoc = new ProfileDoc(userDoc, null); // כאן callback ל-צורך אי'
        progressDoc = new ProgressDoc(userDoc, null);
        statsDoc = new StatsDoc(userDoc, null);
        cosmeticDoc = new CosmeticDoc(userDoc, null);
        worldStateDoc = new WorldStateDoc(userDoc, null);
    }
    profileDoc.createProfile(nickname, email);
    progressDoc.createProfile();
    statsDoc.createDefaults();
    cosmeticDoc.createDoc();
    worldStateDoc.createDefaults();
}
```

- `loadAndCache(OnDataLoadedListener listener)`
 - זוהי הפעולה המורכבת והחשובה ביותר, המנהלת את סדר הטעינה האסינכרוני.
 - תרשים זרימה (סדר פעולות):
 - i. איפוס מצב: מאפס את המונה `successfulLoads` ל-0 ואת הדגל `hasLoadingFailed` ל-`false`.
 - ii. יצירת `Listener` בטוח: יוצר `OnDataLoadedListener` פנימי (`safeListener`) שכל תפקידו הוא לוודא שהמתודה `onDataLoadFailed` של ה-`Listener` המקורי תקרא רק פעם אחת.
 - iii. אתחול מנהלי המסמכים: מאתחל את `profileDoc`, `statsDoc`, `progressDoc` ו-`cosmeticDoc`. לכל אחד מהם הוא מעביר בבנאי שני דברים:
 - `userDoc`: הרפרנס למסמך הראשי בענן.
 - `checkAllDataLoaded(listener)` -> `()`: פעולת `Lambda` (ביטוי פונקציונלי) המשמשת כ-`Callback`. כל מסמך, כשיסיים את הטעינה שלו (בהצלחה או בכישלון), יפעיל את המתודה `checkAllDataLoaded`.
 - iv. בקשה ראשונית מהענן: קורא ל-`userDoc.get()` כדי לבדוק אם המסמך הראשי של המשתמש בכלל קיים.
 - v. עם קבלת תשובה:



- אם המסמך קיים (task.isSuccessful): מפעיל את מתודת loadAndCache על כל אחד מארבעת מנהלי המסמכים. מכאן, כל אחד מהם מתחיל את תהליך הטעינה שלו במקביל ובאופן עצמאי.
- אם המסמך לא קיים (שגיאת רשת קריטית): קורא מיד ל-safeListener.onDataLoadFailed().

○ קוד:

```
public void loadAndCache(OnDataLoadedListener listener) {
    this.successfulLoads = 0;
    this.hasLoadingFailed = false;

    // כישלון היה שכבר אחרי onSuccess - נקרא שלא שמוודא פנימי 'OnDataLoadedListener' ו'וצר'
    OnDataLoadedListener safeListener = new OnDataLoadedListener() {
        @Override
        public void onDataLoadSuccess() {} // לא רלוונטי
        @Override
        public void onDataLoadFailed() {
            if (!hasLoadingFailed) {
                hasLoadingFailed = true;
                listener.onDataLoadFailed();
            }
        }
    };

    // מודולים אתחול
    profileDoc = new ProfileDoc(userDoc, () -> checkAllDataLoaded(listener));
    progressDoc = new ProgressDoc(userDoc, () -> checkAllDataLoaded(listener));
    statsDoc = new StatsDoc(userDoc, () -> checkAllDataLoaded(listener));
    cosmeticDoc = new CosmeticDoc(userDoc, () -> checkAllDataLoaded(listener));
    worldStateDoc = new WorldStateDoc(userDoc, () -> checkAllDataLoaded(listener));

    userDoc.get().addOnCompleteListener(task -> {
        if (task.isSuccessful()) {
            profileDoc.loadAndCache(safeListener);
            progressDoc.loadAndCache(safeListener);
            statsDoc.loadAndCache(safeListener);
            cosmeticDoc.loadAndCache(safeListener);
            worldStateDoc.loadAndCache(safeListener);
        } else safeListener.onDataLoadFailed();
    });
}
```

- checkAllDataLoaded(OnDataLoadedListener listener)
 - מתודה private ו-synchronized. היא נקראת 5 פעמים, פעם אחת בסוף הטעינה של כל מסמך. ההכרזה synchronized מבטיחה שאם שני תהליכים יסתיימו בדיוק באותו הזמן, לא ייווצר "מצב מרוץ" והמונה ייספר נכון.
 - תרשים זרימה (סדר פעולות):
 - i. בודק אם הדגל hasLoadingFailed כבר הורס. אם כן, יוצא מיד כדי לא לדווח על הצלחה בטעות.
 - ii. מעלה את המונה successfulLoads ב-1.
 - iii. בודק אם המונה הגיע ל-4. רק אם כן, זה אומר שכל התהליכים הסתיימו ואף אחד לא נכשל. במצב זה, קורא ל-listener.onDataLoadSuccess() כדי להודיע שהכל מוכן.

○ קוד:

```
private synchronized void checkAllDataLoaded(OnDataLoadedListener listener) {
    if (hasLoadingFailed) return;
    successfulLoads++;
    if (successfulLoads == 5) listener.onDataLoadSuccess();
}
```

**מחלקת BaseDocument**

מיקום: com.example.tutorialgame.cloud.BaseDocument

תפקיד המחלקה: זוהי מחלקה אבסטרקטית המשמשת כשלד ותבנית בסיס לכל המחלקות אשר מנהלות מסמכים ספציפיים (כמו ProfileDoc, StatsDoc) בבסיס הנתונים Firestore. מטרתה המרכזית היא לרכז את הלוגיקה המשותפת, המסובכת והמועדת לטעויות של טעינת נתונים אסינכרונית מהענן, ובכך למנוע שכפול קוד ולהבטיח עקביות.

היא אחראית על:

1. ניהול תהליך הטעינה האסינכרוני מול Firestore.
2. תמיכה מובנית במצב אופליין (Offline Mode) - אם אין חיבור לאינטרנט, היא טוענת אוטומטית את הנתונים מזיכרון המטמון (cache) של המכשיר.
3. טיפול מרכזי בשגיאות תקשורת או שגיאות בנתונים.
4. הבטחה שהמחלקה היורשת צריכה לממש רק את הלוגיקה העסקית הספציפית שלה: איך לפרש את הנתונים ומה שם המפתח שלה בענן.

הסבר על תכונות המחלקה:

- `protected final Runnable onFinishedLoading`
 - תחום הכרה: `protected final` - זמין למחלקה זו ולכל המחלקות היורשות ממנה.
 - תפקיד ושימוש: זוהי פעולת Callback (התקשרות-חזרה). המחלקה מקבלת אותה מהאובייקט המנהל אותה (`UserDataManager`). תפקידה הוא לאותת למנהל שתהליך הטעינה של מסמך ספציפי זה הסתיים - בין אם בהצלחה ובין אם בכישלון.
- `protected final DocumentReference docRef`
 - תחום הכרה: `protected final`.
 - תפקיד ושימוש: זהו ה"מצביע" או הרפרנס למסמך הראשי של המשתמש המחובר בתוך אוסף (`collection`) ה-`users` ב-Firebase. כל פעולות הקריאה והכתיבה לענן מתבצעות דרך אובייקט זה. הוא משותף לכל מחלקות ה-`Doc` השונות, מכיוון שכולן כותבות וקוראות מאותו מסמך משתמש מרכזי.

הסבר פעולות מרכזיות:

- `protected abstract void parseData(@NonNull Map<String, Object> data)`
 - זוהי מתודה אבסטרקטית, כלומר אין לה גוף מימוש במחלקת `BaseDocument`. היא מחייבת כל מחלקה שיורשת ממנה לספק לה מימוש משלה. תפקידה הוא לקבל מפת נתונים גולמית (שהגיעה מה-JSON בענן) ולפרש אותה, כלומר, להכניס את הערכים לתוך המשתנים המקומיים (ה-`Cache`) של המחלקה היורשת. לדוגמה, `ProfileDoc` תממש אותה כדי לחלץ את ה-"nickname" וה-"email".
- `protected abstract String getDocName()`
 - מתודה אבסטרקטית נוספת. מחייבת כל מחלקה יורשת להחזיר את שם המפתח (ה-`key`) שלה בתוך מסמך המשתמש הראשי. לדוגמה, `ProfileDoc` תחזיר "profile", ו-`StatsDoc` תחזיר "stats". מתודת `loadAndCache` משתמשת בשם זה כדי לשלוף את החלק הרלוונטי מה-JSON.
- `public void loadAndCache(UserDataManager.OnDataLoadedListener listener)`
 - זוהי הפעולה המרכזית והחשובה ביותר במחלקה. היא מנהלת את כל תהליך הטעינה וכוללת לוגיקה מורכבת של טיפול בשגיאות וזרימת נתונים אסינכרונית.
 - **תרשים זרימה (סדר פעולות):**
 - i. התחלה: קריאה ל-`docRef.get()` כדי לבקש את מסמך המשתמש מהענן. הפעולה אינה מציינת מקור (`Source`), ולכן Firestore יפעל כך:



- ינסה להביא את הנתונים המעודכנים ביותר מהשרת.
- אם השרת אינו זמין (אין אינטרנט), יחזיר אוטומטית את הגרסה האחרונה השמורה בזיכרון המטמון של המכשיר.
- ii. המתנה לסיום : הוספת `addOnCompleteListener` שממתין לתשובה מהשרת או מהמטמון.
- iii. עם קבלת תשובה :
- בדיקת הצלחה (`task.isSuccessful()`) :
 - a. אם כן (`true`) :
 - i. קבלת ה-`DocumentSnapshot` (תמונת המצב של הנתונים).
 - ii. בדיקה שה-`snapshot` קיים ולא ריק (`snapshot.exists()`).
 - iii. שליפת המידע הספציפי : שימוש ב-`snapshot.get(getDocName)` כדי לחלץ את המפה המקוננת הרלוונטית (למשל, מפת ה-`"profile"`).
 - iv. בדיקה שהמפה הספציפית אינה ריקה (`specificData != null`).
 - v. הפעלת הלוגיקה הספציפית : קריאה למתודה `parseData(specificData)`, כדי שהמחלקה היורשת תעבד את הנתונים שלה.
 - b. אם לא (`false`) או אם אחת הבדיקות נכשלה :
 - i. התהליך נכנס לבלוק ה-`catch (Exception e)`.
 - ii. שגיאה נרשמת ב-`Logcat` של אנדרואיד.
 - iii. קריאה ל-`listener.onDataLoadFailed()` כדי להודיע למנהל הראשי (`UserDataManager`) שהייתה שגיאה קריטית בתהליך הטעינה.
 - iv. בלוק `finally` (מתבצע תמיד) :
- לא משנה אם התהליך הצליח או נכשל, מתבצעת קריאה ל-`onFinishedLoading.run()`. זהו האות ל-`UserDataManager` שהטיפול במסמך זה הסתיים, והוא יכול להמשיך לספור את המסמכים הבאים.

○ קוד:

```
@SuppressWarnings("unchecked")
public void loadAndCache(UserDataManager.OnDataLoadedListener listener) {
    docRef.get().addOnCompleteListener(task -> {
        try {
            if (task.isSuccessful()) {
                DocumentSnapshot snapshot = task.getResult();
                if (snapshot != null && snapshot.exists()) {
                    Map<String, Object> specificData = (Map<String, Object>)
                    snapshot.get(getDocName());

                    if (specificData != null) {
                        parseData(specificData);
                        if (snapshot.getMetadata().isFromCache()) {
                            Log.d("BaseDocument", getDocName() + " data loaded from cache.");
                        } else {
                            Log.d("BaseDocument", getDocName() + " data loaded from server.");
                        }
                    } else {
                        throw new Exception("Document part '" + getDocName() + "' not found.");
                    }
                } else {
                    throw new Exception("User document does not exist. This may be a new user.");
                }
            } else {
                // ...
            }
        }
    });
}
```



```
        throw task.getException() != null ? task.getException() : new Exception("Failed  
to load user document.");  
    }  
    } catch (Exception e) {  
        Log.e("BaseDocument", "Error loading document part: " + getDocName(), e);  
        if (listener != null) {  
            listener.onDataLoadFailed();  
        }  
    } finally {  
        if (onFinishedLoading != null) {  
            onFinishedLoading.run();  
        }  
    }  
    });  
}
```


**מחלקת ProfileDoc**

מיקום: com.example.tutorialgame.cloud.document.ProfileDoc

תפקיד המחלקה: מחלקה זו משמשת כיחידת ניהול הנתונים עבור פרטי הזהות האישיים והיסטוריית הגישה של המשתמש. תפקידה המרכזי הוא לסנכרן בין פרטי הפרופיל (כגון כינוי ואימייל) לבין מסד הנתונים בענן (Firebase Firestore), תוך ניהול לוגיקת ה-"Daily Login" המעניקה לשחקן תגמולים על התמדה. המחלקה מיישמת פענוח נתונים הגנתי כדי להבטיח יציבות גם כאשר מבנה הנתונים בענן משתנה או כאשר מידע מסוים חסר.

היא אחראית על:

1. ניהול זהות משתמש: טעינה, שמירה ועדכון של הכינוי והאימייל המקושרים לחשבון.
2. מעקב אחר התחברות יומית: זיהוי כניסות בנפרד בכל יום קלנדרי ועדכון מוני ההתמדה ב-ProgressDoc.
3. ניהול נתוני זמן: שימוש בפורמט תאריך תקני (ISO) לתיעוד מועדי יצירת החשבון והכניסה האחרונה.
4. אבטחת סוגי נתונים: הבטחת תקינות המחרוזות הנשלפות מהענן למניעת שגיאות הרצה (Crashes).

הסבר על תכונות המחלקה:

- `private static final String KEY_ROOT, KEY_NICKNAME, KEY_EMAIL, KEY_LAST_LOGIN, KEY_CREATED`
 תחום הכרה: `private static final` (פרטי, סטטי וקבוע).
 תפקיד ושימוש: מפתחות מחרוזות המייצגים את נתיבי השדות ב-Firebase. שימוש זה מבטיח אחידות ומונע שגיאות כתיב (Typo) בכל רחבי הקוד המבצע עדכונים לענן.
- `private String nickname, email, lastLoginDate`
 תחום הכרה: `private`.
 תפקיד ושימוש: משתני הזיכרון המקומיים השומרים את נתוני הפרופיל לאחר שנשלפו מהענן. הם מאותחלים כמחרוזות ריקות כדי להבטיח שהאובייקט תמיד במצב תקין.
- `private final SimpleDateFormat dateFormat`
 תחום הכרה: `private final`.
 תפקיד ושימוש: כלי עזר לעיבוד תאריכים בפורמט "yyyy-MM-dd". משמש להשוואת ימי התחברות ללא תלות בשעות ודקות.

הסבר פעולות מרכזיות:

- `protected void parseData(@NonNull Map<String, Object> data)`
 פעולה המפענחת את המידע הגולמי המגיע מ-Firebase. הפעולה משתמשת בגישה הגנתית (Safe Casting). היא בודקת עבור כל שדה האם הוא אכן מטיפוס `String`. אם כן, המידע מעודכן. אם לא (למשל, שדה שטרם נוצר עבור משתמש ותיק), המשתנה המקומי שומר על ערכו הקודם או על ערך הדיפולט, ובכך נמנעת קריסה של היישום.
- `public void updateLogin()`
 פעולה לניהול התמדת השחקן. בכל כניסה למשחק, הפעולה משווה את התאריך הנוכחי לתאריך ה-`lastLoginDate` השמור בענן. אם התאריכים שונים, המשמעות היא שזהו יום חדש. במקרה כזה, הפעולה מעדכנת את התאריך בענן וקוראת למסמך ההתקדמות (ProgressDoc) כדי להעלות את מונה ימי ההתחברות.
- **תרשים זרימה (סדר פעולות):**
 - i. קבלת תאריך המערכת הנוכחי.
 - ii. דיקה: האם התאריך הנוכחי שונה מהתאריך השמור?
 כן:
 - a. עדכון המשתנה המקומי `lastLoginDate`.



- b. קריאה ל- MyApp.getProgress().increaseDaysLoggedIn().
- c. שליחת עדכון (Update) לשדה ב-Firebase.
- לא: סיום ללא שינוי (המשתמש כבר נכנס היום).

○ קוד:

```
public void updateLogin() {
    String todayDate = dateFormat.format(Calendar.getInstance().getTime());
    if (!Objects.equals(lastLoginDate, todayDate)) {
        lastLoginDate = todayDate;
        MyApp.getProgress().increaseDaysLoggedIn();

        docRef.update(KEY_LAST_LOGIN, lastLoginDate);
    }
}
```

בסיס נתונים (Firestore) (Firebase)

המחלקה מנהלת את מסמך ה-profile בתוך אוסף המשתמש.

- סקירת המידע: המידע נשמר כמבנה מקונן (Map):

○ nickname(String)

○ email(String)

○ lastLoginDate(String)

- פעולות:

○ טעינה: המידע נטען בשיטת Lazy Parsing בעת קבלת עדכון Snapshot מהשרת.

○ עדכון: שימוש במתודת update עבור שדות ספציפיים (כמו nickname) מבטיח מינימום תעבורת רשת ושמירה על שלמות שאר הנתונים במסמך.

- הצגה בפורמט מסמך (JSON-like) (NoSQL):

▼ profile

created: "2026-03-14"

email: "orirogel007@gmail.com"

lastLoginDate: "2026-03-15"

nickname: "macaco"



מחלקת StatsDoc

מיקום: StatsDoc.cloud.document.tutorialgame.example.com

תפקיד המחלקה: מחלקה זו משמשת כיחידת ניהול הנתונים עבור הסטטיסטיקות הקבועות של השחקן ("הפוטנציאל של הדמות"). תפקידה המרכזי הוא לסנכרן בין ערכי המקסימום של השחקן (כגון חיים מירביים, עוצמת התקפה ומהירות) לבין מסד הנתונים בענן (Firestore). המחלקה מאפשרת ניהול שדרוגים קבועים, הבטחת שלמות נתונים בזמן עדכונים אטומיים, ומתן ערכי ברירת מחדל בטוחים למשתמשים חדשים או במקרים של נתונים חסרים.

היא אחראית על:

1. סנכרון נתוני יכולות: טעינה ושמירה של ערכי המקסימום המגדירים את עוצמת הדמות.
2. ניהול שדרוגים אטומיים: ביצוע עדכונים ישירים לשדות ב-Firebase המונעים דריסת נתונים וסנכרון מול נקודות השדרוג של השחקן.
3. פענוח נתונים הגנתי: הבטחת יציבות האפליקציה על ידי אימות טיפוס הנתונים המגיעים מהענן ושימוש בערכי ברירת מחדל בעת הצורך.
4. תשתית לשדרוג דמות: מתן ממשק נוח למסד השדרוגים (UpgradeState) לעדכון ערכים לוגיים וכתיבתם לענן בבת אחת.

הסבר על תכונות המחלקה:

- `private static final String KEY_ROOT, KEY_MAX_HEALTH, KEY_STRENGTH, KEY_ATTACK_SPEED, KEY_MAX_STAMINA`
 - תחום הכרה: `private static final` (פרטי, סטטי וקבוע).
 - תפקיד ושימוש: קבועי מחרוזת המגדירים את שמות המפתחות במסד הנתונים. שימוש זה מונע שגיאות כתיב (Typo) ומאפשר תחזוקה קלה של מבנה הנתונים ב-Firebase במקום מרכזי אחד.
- `public static final int INIT_HEALTH, INIT_STRENGTH, INIT_ATTACK_SPEED, INIT_STAMINA`
 - תחום הכרה: `public static final` (ציבורי, סטטי וקבוע).
 - תפקיד ושימוש: ערכי ברירת המחדל ההתחלתיים עבור שחקן חדש.
- `private int maxHealth, strength, attackSpeed, maxStamina`
 - תחום הכרה: `private` (פרטי).
 - תפקיד ושימוש: המשתנים הלוגיים המקומיים השומרים את הסטטיסטיקות העדכניות של השחקן בזיכרון ה-RAM לאחר הטעינה.

הסבר פעולות מרכזיות:

- `protected void parseData(@NonNull Map<String, Object> data)`
 - פעולה המפענחת את המידע הגולמי המגיע מ-Firebase והמרתו למשתני המחלקה. הפעולה משתמשת בגישה הגנתית (Defensive Parsing). עבור כל שדה, היא בודקת אם הערך קיים ואם הוא אכן מטיפוס `Number` (הטיפוס הכללי ב-Firebase למספרים). במידה והבדיקה מצליחה, הערך מעודכן. במידה ולא (למשל עבור משתמש ישן ששדה מסוים חסר לו), המשתנה שומר על ערך ברירת המחדל שלו (INIT), ובכך מונעת את קריסת האפליקציה.
- `public void updateStat(String field, int increment, int price)`
 - הפעולה המרכזית לביצוע שדרוג לדמות. הפעולה מקבלת את שם השדה לשדרוג, את ערך ההוספה ואת המחיר בנקודות שדרוג. היא מעדכנת את המשתנה המקומי ושולחת פקודת `FieldValue.increment` ל-Firebase. שימוש ב-`increment` ב-Firebase הוא קריטי כיוון שהוא אטומי - השרת מחשב את הערך החדש בעצמו, מה שמונע תקלות במקרה של מספר עדכונים בו-זמנית. בנוסף, היא פוקדת על מסמך ההתקדמות (ProgressDoc) להפחית את הנקודות שנוצלו.



○ קוד:

```
public void updateStat(String field, int increment, int price) {
    switch (field) {
        case "maxHealth":
            maxHealth += increment;
            docRef.update(KEY_MAX_HEALTH, FieldValue.increment(increment));
            break;
        //case ...
        default:
            throw new IllegalArgumentException("Unknown stat field: " + field);
    }
    incrementUpgradeCount(price);
}
```

בסיס נתונים (Firestore)

המחלקה מנהלת את מסמך ה-stats בתוך אוסף המשתמש.

- סקירת המידע: המידע נשמר כמבנה מקונן (Map):

○ maxHealth(Number)

○ strength(Number)

○ attackSpeed(Number)

○ maxStamina(Number)

- פעולות:

○ טעינה: המידע נטען אוטומטית בעת עליית האפליקציה ב-SplashActivity דרך ה-

UserDataManager.

○ יצירה: בעת רישום משתמש חדש, נקראת הפעולה createDefaults() המייצרת את

המבנה ההתחלתי בעזרת SetOptions.merge().

○ עדכון: כתיבה מתבצעת בשיטת Partial Update (עדכון שדה ספציפי) כדי לחסוך ברוחב

פס ולמנוע דריסת נתונים אחרים במסמך.

- הצגה בפורמט מסמך (JSON-like) (NoSQL):

▼ stats

attackSpeed: 300

maxHealth: 100

maxStamina: 50

strength: 0



מחלקת ProgressDoc

מיקום: com.example.tutorialgame.cloud.document.ProgressDoc

תפקיד המחלקה: מחלקה זו מנהלת את כל מדדי ההתקדמות הכמותיים וההישגים של השחקן במשחק. תפקידה המרכזי הוא לסנכן בזמן אמת בין התקדמות השחקן בעולם (צבירת ניסיון, עליית רמות, הבסת אויבים) לבין מסד הנתונים בענן (Firebase Firestore). המחלקה מבטיחה שלמות נתונים (Data Integrity) על ידי שימוש בעדכונים אטומיים, ומיישמת פיענוח נתונים הגנתי כדי להבטיח יציבות מול שינויים במבנה הנתונים בענן.

היא אחראית על:

1. ניהול מערכת הניסיון (XP): עדכון נקודות הניסיון וביצוע בדיקות אוטומטיות לעליית רמה.
2. בקרה על רמות (Leveling): ניהול המעבר בין דרגות והענקת נקודות שדרוג לשחקן.
3. מעקב הישגים: תחזוקת מונים עבור אויבים שהובסו, משימות שהושלמו וימי התחברות.
4. ניהול אינטראקציות: רישום דמויות ייחודיות (NPCs) שהשחקן פגש והענקת תגמולי גילוי.
5. סנכרון מטא-דאטה: עדכון פרטי הרמה בסלולר השמירה לצורך תצוגה בתפריט בחירת הסלטים.

הסבר על תכונות המחלקה:

- `private static final String KEY_XP, KEY_LEVEL` (וכו')
 - תחום הכרה: `private static final`.
 - פקיד ושימוש: קבועי מחרוזת המגדירים את שמות השדות במסד הנתונים. שימוש זה מונע שגיאות כתיב ומאפשר תחזוקה קלה של מבנה הנתונים ממקום מרכזי אחד.
- `private int xp, level, upgradePoints`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתני הזיכרון המקומיים השומרים את נתוני ההתקדמות הנוכחיים ב-RAM. הם מאותחלים בערכי ברירת מחדל (0 ו-1) כדי להבטיח מצב תקין תמיד.
- `private List<String> metCharacters`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: רשימה של מזהי דמויות שהשחקן כבר שוחח איתן. משמשת למניעת כפל נקודות ניסיון על אותה דמות.

הסבר פעולות מרכזיות:

- `public void updateXp(int value)`
 - מעדכנת את נקודות הניסיון של השחקן. הפעולה מעלה את ערך ה-XP המקומי ושולחת עדכון אטומי ל-Firebase באמצעות `FieldValue.increment`. שימוש זה קריטי כדי להבטיח שגם אם השחקן הורג מספר אויבים בו-זמנית, השרת יספור את כל הנקודות נכון. לאחר העדכון, הפעולה מפעילה את `checkLevelUp`.
 - קוד:

```
public void updateXp(int value) {
    if (value == 0) return;
    this.xp += value;
    docRef.update(KEY_XP, FieldValue.increment(value));
    checkLevelUp();
}
```

- `private void checkLevelUp()`

- מנהלת את לוגיקת עליית הדרגה. הפעולה רצה כלולאת `while` הבודקת אם כמות ה-XP הנוכחית עוברת את הרף הנדרש לרמה הבאה (המחושב לפי נוסחה ריבועית). אם כן, היא מפחיתה את הניסיון "המשומש", מעלה את הרמה, ומעניקה 3 נקודות שדרוג. התהליך חוזר על עצמו עד שהניסיון הנותר קטן מהרף הבא (תמיכה בעליית מספר רמות בבת אחת). בסיום, היא מעדכנת את המטא-דאטה של הסלולר.



○ תרשים זרימה (סדר פעולות):

- i. כניסה ללולאה: האם $XP \leq$ ניסיון נדרש?
1. כן:

- a. הפחתת הניסיון הנדרש מהיתרה המקומית ובענן.
- b. קריאה ל-`increaseLevel()` (עדכון רמה בזיכרון ובענן).
- c. קריאה ל-`increaseUpgradePoints(3)` (הענקת נקודות).
- d. חזרה לתחילת הלולאה.
2. לא: יציאה מהלולאה.

- ii. עדכון רמת הסלוט ב-`SlotsMetadata` לסנכרון התצוגה.

○ קוד:

```
private void checkLevelUp() {
    boolean leveled = false;
    while (xp >= neededXpForLevelUp()) {
        int needed = neededXpForLevelUp();
        this.xp -= needed;
        docRef.update(KEY_XP, FieldValue.increment(-needed));

        increaseLevel();
        increaseUpgradePoints(3); // Reward 3 points per level
        leveled = true;
    }

    if (leveled) {
        // Sync new level to slot metadata for display in slot selection
        MyApp.getCloudManager().getSlotsMetadata().updateLevel(MyApp.getCloudManager().getActiveSlotId(), level);
    }
}
```

● public void registerEncounter(String charId)

- מתעדת מפגש עם דמות חדשה בעולם. הפעולה בודקת אם מזהה הדמות כבר קיים ברשימה. במידה ולא, היא מוסיפה אותו לרשימה המקומית ומשתמשת ב-`FieldValue.arrayUnion` כדי להוסיף אותו לענן בצורה בטוחה. כשכר על הגילוי, היא מעניקה לשחקן 5 נקודות ניסיון ומציגה הודעת `Toast` מעוצבת.

○ קוד:

```
public void registerEncounter(String charId) {
    if (metCharacters != null && !metCharacters.contains(charId)) {
        metCharacters.add(charId);
        docRef.update(KEY_MET_CHARS, FieldValue.arrayUnion(charId));
        updateXp(5);

        BaseActivity activity = (BaseActivity) BaseActivity.getContext();
        if (activity != null) {
            activity.showToast(MessageFormat.format("{0} {1} {2} 5 xp",
                activity.getString(R.string.new_char_met), charId,
                activity.getString(R.string.you_gained)),
                Toast.LENGTH_LONG);
        }
    }
}
```

בסיס נתונים (Firestore)

המחלקה מנהלת את האובייקט המקוון `progress` בתוך מסמך הסלוט של המשתמש.

- סקירת המידע: המידע נשמר כמבנה מקוון (`Map`):
- פעולות:



- טעינה: המידע נטען בשיטת Lazy Parsing בעת קבלת עדכון Snapshot. שימוש ב- instanceof מבטיח טעינה בטוחה גם אם שדה חסר.
- עדכון אטומי: שימוש נרחב ב- FieldValue.increment מבטיח סנכרון מושלם של מונים גם בביצועים מהירים.
- הצגה בפורמט מסמך (NoSQL JSON-like):

```
▼ progress
  daysLoggedIn: 0
  enemiesDefeated: 6
  level: 3
  ▼ metCharacters
    0 "King"
  questsCompleted: 5
  upgradePoints: 7
  upgradesDone: 1
  xp: 23
```



מחלקת CosmeticDoc

מיקום: com.example.tutorialgame.cloud.document.CosmeticDoc

תפקיד המחלקה: מחלקה זו משמשת כיחידת ניהול הנתונים עבור כלל המשאבים ה"קוסמיים" והכלכליים של המשתמש. היא אחראית על סנכרון יתרת המטבעות, ניהול רשימת המסגרות (Frames) שנרכשו, ומעקב אחר המעטפת היוזואלית הפעילה של השחקן. המחלקה מבטיחה שלמות נתונים (Data Integrity) מול שרתי Firebase Firestore על ידי שימוש בעדכונים אטומיים, ומונעת מצבי קצה של "חסור סנכרון" בעת רכישות או איסוף מטבעות בזמן אמת.

היא אחראית על:

1. ניהול כלכלת המשחק: עדכון ושמירה של מאזן המטבעות תוך שימוש בשיטות עדכון בטוחות.
2. בקרת רכישות: אימות יכולת תשלום וביצוע הורדת יתרה אטומית.
3. ניהול נכסים (Assets Inventory): תחזוקת רשימת המסגרות הזמינות למשתמש.
4. שיקוף מצב ויזואלי: שמירת המידע על המסגרת שנבחרה כרגע להצגה ב-UI.

הסבר על תכונות המחלקה:

- `private static final String KEY_COINS, KEY_AVAILABLE_FRAMES`
 תחום הכרה: `private static final` (פרטי, סטטי וקבוע).
 תפקיד ושימוש: קבועי מחרוזות המגדירים את שמות השדות במסד הנתונים של Firestore. השימוש בקבועים מונע שגיאות כתיב (Typo) ומאפשר תחזוקה קלה של מבנה הנתונים ממקום מרכזי אחד.
- `private int coinsLeft`
 תחום הכרה: `private`.
 תפקיד ושימוש: המשתנה הלוגי המקומי המייצג את כמות המטבעות הזמינה לשחקן.
- `private List<String> availableFrames`
 תחום הכרה: `private`.
 תפקיד ושימוש: רשימה של מזהי מסגרות (Strings) שהמשתמש פתח או רכש.

הסבר פעולות מרכזיות:

- `public boolean purchase(int price)`
 פעולה המבצעת רכישה לוגית ופיזית של פריט. הפעולה מהווה שכבת הגנה (Guard) לפני הוצאת כסף. היא בודקת תחילה אם לשחקן יש יתרה מספקת. אם כן, היא מעדכנת את הזיכרון המקומי ושולחת פקודת `FieldValue.increment` שלילית ל-Firebase. שימוש בערך שלילי ב-`increment` מבטיח שהפחתת המטבעות תהיה אטומית - כלומר, השרת יחשב את היתרה החדשה ולא יכתוב ערך שעלול להיות לא מעודכן עקב השהיית רשת.
- קוד:

```
public boolean purchase(int price) {
    if (coinsLeft < price) return false;

    coinsLeft -= price;
    docRef.update(KEY_COINS, FieldValue.increment(-price));
    return true;
}
```

- `protected void parseData(@NonNull Map<String, Object> data)`
 פעולה המפענחת את מסמך ה-JSON המגיע מהענן לאובייקט Java. הפעולה משתמשת בגישה הגנתית (Safe Parsing). היא עוטפת את התהליך בבלוק `try-catch` למקרה של נתונים פגומים, ומשתמשת במחלקה `Number` כדי לבצע המרה בטוחה של מספרים (מכיוון ש-Firebase נוטה להחזיר מספרים כ-Long, וקוד המשחק משתמש ב-int).
- `public void addAvailableFrame(String frameName)`



- פעולה המוסיפה לרשימת המסגרות של השחקן מסגרת חדשה אם היא אינה קיימת כבר ברשימה.

○ קוד:

```
public void addAvailableFrame(String frameName) {  
    if (availableFrames != null && !availableFrames.contains(frameName)) {  
        availableFrames.add(frameName);  
        docRef.update(KEY_AVAILABLE_FRAMES, FieldValue.arrayUnion(frameName));  
    }  
}
```

בסיס נתונים (Firestore) (Firebase)

המחלקה מנהלת את מסמך ה-cosmetic בתוך אוסף המשתמש.

- סקירת המידע: המידע נשמר כמבנה מקוון (Map):

○ coinsLeft (Number)

○ currentFrame (String)

○ availableFrames (Array of Strings)

- פעולות:

○ טעינה: המידע נטען בשיטת "Lazy Parsing" ברגע שמתקבל Snapshot מהשרת.

○ עדכון: כתיבה מתבצעת בשיטת Partial Update (עדכון שדה ספציפי) כדי לחסוך ברוחב פס ולמנוע דריסת נתונים אחרים במסמך.

- הצגה בפורמט מסמך (JSON-like) (NoSQL):

▼ cosmetic

▼ availableFrames

0 "FRAME_00"

1 "FRAME_22"

coinsLeft: 1

currentFrame: "FRAME_22"

**מחלקת LoginActivity**

מיקום: com.example.tutorialgame.ui.activities.authentication.LoginActivity

תפקיד המחלקה: מחלקה זו מהווה את שער הכניסה למשתמשים רשומים. היא מנהלת את תהליך ההזדהות מול Firebase Authentication, מבצעת אימות קלט בזמן אמת, ומנהלת את שרשרת טעינת הנתונים הראשונית מהענן (פרופיל משתמש וסלוט משחק אחרון). המחלקה מבטיחה שרק משתמשים עם אימייל מאומת יוכלו להיכנס למשחק, תוך מתן מענה לשכחת סיסמה וניווט להרשמה.

היא אחראית על:

1. אימות משתמשים (Authentication): ביצוע התחברות מאובטחת באמצעות אימייל וסיסמה.
2. ולידציה בזמן אמת: שימוש ב-TextWatcher כדי למנוע ניסיונות התחברות עם קלט לא תקין.
3. שרשר טעינה (Data Fetching Chain): טעינה אטומית של מסמכי הענן מיד לאחר ההתחברות כדי להכין את מצב המשחק.
4. ניהול הרשאות: בקשת הרשאות מערכת נדרשות (כגון READ_PHONE_STATE) בעת עליית המסך.

הסבר על תכונות המחלקה:

- `private static final int REQ_READ_PHONE_STATE = 1234`
 תחום הכרה: `private static final`.
 תפקיד ושימוש: קוד בקשה (Request Code) ייחודי המשמש את מערכת ההרשאות של אנדרואיד כדי לזהות את התשובה לבקשת הרשאת מצב הטלפון.
- `private FirebaseAuth mAuth`
 תחום הכרה: `private`.
 תפקיד ושימוש: מופע של שירות האימות של Firebase. זהו הכלי המרכזי שדרכו מתבצעות פעולות ההתחברות והחלפת הסיסמה מול השרת.
- `private TextInputEditText etEmail, etPassword`
 תחום הכרה: `private`.
 תפקיד ושימוש: רכיבי ממשק המשתמש לקלט טקסטואלי. הם מחזיקים את האימייל והסיסמה שהמשתמש מזין.
- `private Button btnLogin`
 תחום הכרה: `private`.
 תפקיד ושימוש: כפתור ההתחברות. הוא נשאר במצב כבוי (Disabled) עד ששני שדות הקלט עוברים את בדיקות התקינות.
- `private ProgressBar progressBar`
 תחום הכרה: `private`.
 תפקיד ושימוש: רכיב ויזואלי המעיד על פעולה ברקע. מוצג בזמן ההתחברות וטעינת הנתונים כדי לספק משוב למשתמש שהמערכת עובדת.

הסבר פעולות מרכזיות:

- `private void initTextWatcher()`
 פעולה זו מגדירה מאזין לשינויי טקסט בשדות הקלט. בכל פעם שהמשתמש מקליד, הפעולה בודקת את תקינות הפורמט של האימייל והסיסמה (בעזרת ValidationUtils). אם הקלט לא תקין, מוצגת שגיאה ויזואלית על השדה. הכפתור `btnLogin` מופעל רק כאשר כל התנאים מתקיימים, מה שמונע קריאות מיותרות לשרת.
- קוד:

```
private void initTextWatcher() {
    TextWatcher afterTextChangedListener = new TextWatcher() {
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
    };
}
```



```

@Override
public void onTextChanged(CharSequence s, int start, int before, int count) {}
@Override
public void afterTextChanged(Editable s) {
    String email = Objects.requireNonNull(etEmail.getText()).toString().trim();
    String password = Objects.requireNonNull(etPassword.getText()).toString().trim();

    if (!TextUtils.isEmpty(email) && !ValidationUtils.isEmailValid(email))
        setError(etEmail, getString(R.string.invalid_email), R.drawable.king);
    else etEmail.setError(null);

    if (!TextUtils.isEmpty(password) && !ValidationUtils.isPasswordValid(password))
        setError(etPassword, getString(R.string.invalid_password), R.drawable.king);
    else etPassword.setError(null);

    btnLogin.setEnabled(ValidationUtils.isEmailValid(email) &&
        ValidationUtils.isPasswordValid(password));
}
};

etEmail.addTextChangedListener(afterTextChangedListener);
etPassword.addTextChangedListener(afterTextChangedListener);
}

```

● private void loginUser()

- הפעולה המרכזית המפעילה את תהליך ההתחברות. היא שולחת את הפרטים ל-Firebase וממתינה לתשובה.
- **תרשים זרימה (סדר פעולות):**
 - i. אימות מול Firebase Authentication.
 - ii. בדיקה האם המשתמש אימת את כתובת האימייל שלו (isEmailVerified).
 - iii. אם כן: אתחול ה-CloudManager וטעינת נתוני הפרופיל.
 - iv. שליפת מזהה הסלוט האחרון שבו השחקן השתמש וטעינת נתוני המשחק שלו (World State).
 - v. מעבר למסך ה-LauncherActivity אם כל השלבים הצליחו.

○ קוד:

```

private void loginUser() {
    progressBar.setVisibility(View.VISIBLE);
    String email = Objects.requireNonNull(etEmail.getText()).toString().trim();
    String password = Objects.requireNonNull(etPassword.getText()).toString().trim();
    mAuth.signInWithEmailAndPassword(email, password)
        .addOnCompleteListener(task -> {
            if (!task.isSuccessful()) {
                progressBar.setVisibility(View.GONE);
                SoundManager.getInstance(this).playSfx(R.raw.sfx_error);
                showToast(getString(R.string.error) +
                    Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_SHORT);
            }
            else {
                FirebaseUser user = mAuth.getCurrentUser();
                if (user != null && user.isEmailVerified()) {
                    showToast(getString(R.string.loading_data), Toast.LENGTH_SHORT);
                    MyApp.initializeCloudManager();
                    MyApp.startLoadingAccountData(new
                        UserDataManager.OnDataLoadedListener() {
                            @Override
                            public void onDataLoadSuccess() {
                                int lastSlotId = MyApp.getProfile().getLastSelectedSlot();
                                MyApp.getCloudManager().selectSlot(lastSlotId, new UserDataManager.OnDataLoadedListener() {
                                    @Override
                                    public void onDataLoadSuccess() {
                                        progressBar.setVisibility(View.GONE);

```



```
SoundManager.getInstance(getContext()).playSfx(R.raw.sfx_success4);
showToast(getString(R.string.login_successful),
Toast.LENGTH_SHORT);

startActivity(new Intent(getContext(),
LauncherActivity.class));

finish();
}
@Override
public void onDataLoadFailed() {
progressBar.setVisibility(View.GONE);
AlertDialogUtils.showErrorDialogAndRestart(LoginActivity.this);
}
});
}
@Override
public void onDataLoadFailed() {
progressBar.setVisibility(View.GONE);
AlertDialogUtils.showErrorDialogAndRestart(LoginActivity.this);
}
});
} else if (user != null) {
progressBar.setVisibility(View.GONE);
user.sendEmailVerification();
showToast(getString(R.string.verify_email), Toast.LENGTH_LONG);
}
});
}
}
```

private void resetPassword() •

- מאפשרת לשלוח אימייל לשחזור סיסמה. הפעולה מוודא ששדה האימייל אינו ריק לפני פנייה ל-Firebase, ומספקת משוב למשתמש (Toast) על הצלחה או כישלון של שליחת המייל.

○ קוד:

```
private void resetPassword() {
String email = Objects.requireNonNull(etEmail.getText()).toString().trim();
if (email.isEmpty()) showToast(getString(R.string.enter_email), Toast.LENGTH_SHORT);
else {
FirebaseAuth.getInstance()
.sendPasswordResetEmail(email)
.addOnCompleteListener(task -> {
if (task.isSuccessful())
showToast(getString(R.string.reset_password_email_sent),
Toast.LENGTH_SHORT);
else
showToast(getString(R.string.error) +
Objects.requireNonNull(task.getException()).getMessage(), Toast.LENGTH_LONG);
});
}
}
```

בסיס נתונים (Firebase Authentication & Firestore)

המחלקה מהווה את נקודת החיבור הראשונית בין המשתמש למידע שלו בענן.

- סקירת המידע: המשתמש מנוהל ב-Firebase Auth. לאחר מכן, נתוני נשלפים מ-Firebase משני מקורות: אוסף ה-Profiles (נתוני חשבון כלליים) ואוסף ה-Slots (נתוני משחק ספציפיים).
- פעולות:

- טעינה: לאחר Login מוצלח, מתבצעות שתי קריאות Read עוקבות ל-Firebase כדי לסנכרן את מצב האפליקציה המקומי עם המידע האחרון שנשמר בענן.



מחלקת RegisterActivity

מיקום: com.example.tutorialgame.ui.activities.authentication.RegisterActivity

תפקיד המחלקה: מחלקה זו אחראית על תהליך הרישום של משתמשים חדשים למערכת. היא משלבת בין יצירת חשבון אימות (Authentication) לבין אתחול נתוני המשתמש במסד הנתונים (Firestore). המחלקה מבצעת ולידציה מקיפה על פרטי המשתמש, מנהלת את יצירת פרופיל השחקן וסלוט המשחק הראשון שלו באופן אוטומטי, ומבטיחה את אבטחת החשבון באמצעות שליחת אימייל לאימות הזהות.

היא אחראית על:

1. רישום משתמשים (User Provisioning): יצירת חשבון חדש ב-Firebase Auth.
2. אתחול נתונים בענן: יצירת מסמך פרופיל (ProfileDoc) ומסמך מצב עולם (WorldStateDoc) ראשוני עבור השחקן החדש.
3. אימות קלט (Validation): בדיקת תקינות אימייל, סיסמה וכינוי (Nickname) בזמן אמת כדי למנוע שגיאות שרת.
4. אבטחה: ניהול שליחת Verification Email וניתוק המשתמש עד לביצוע האימות.

הסבר על תכונות המחלקה:

- private TextInputEditText etEmail, etPassword, etNickname
 - תחום הכרה: private.
 - תפקיד ושימוש: רכיבי קלט עבור נתוני הרישום. השימוש ב-TextInputEditText מאפשר הצגת שגיאות ויזואליות (Errors) בצורה מובנית בתוך שדה הקלט.
- private Button btnRegister
 - תחום הכרה: private.
 - תפקיד ושימוש: כפתור לביצוע הרישום. הכפתור נמצא במצב כבוי (Disabled) ומופעל רק לאחר שכל השדות עוברים את בדיקת הוולידציה של ה-TextWatcher.
- private FirebaseAuth mAuth
 - תחום הכרה: private.
 - תפקיד ושימוש: מופע של שירות האימות. משמש ליצירת המשתמש בשרתי Firebase וניהול השליחה של אימייל האימות.
- private ProgressBar progressBar
 - תחום הכרה: private.
 - תפקיד ושימוש: חיווי ויזואלי למשתמש בזמן שתהליך הרישום מתבצע מול השרת (פעולה אסינכרונית).

הסבר פעולות מרכזיות:

- private void initTextWatcher()
 - פעולה זו מגדירה לוגיקת ולידציה בזמן אמת. היא משתמשת ב-ValidationUtils כדי לבדוק כל תו שהמשתמש מקליד. אם הקלט אינו תקין (למשל אימייל ללא @ או כינוי קצר מדי), היא מציגה הודעת שגיאה מותאמת לתקלה ואיקון ייחודי בתוך השדה. פעולה זו חוסכת מהמשתמש ללחוץ על "הרשמה" ורק אז לגלות שיש טעות.
- קוד:

```
private void initTextWatcher() {
    TextWatcher afterTextChangedListener = new TextWatcher() {
        @Override
        public void beforeTextChanged(CharSequence s, int start, int count, int after) {}
        @Override
        public void onTextChanged(CharSequence s, int start, int before, int count) {}
        @Override
        public void afterTextChanged(Editable s) {
            String email = Objects.requireNonNull(etEmail.getText()).toString().trim();
            String password = Objects.requireNonNull(etPassword.getText()).toString().trim();
        }
    };
}
```



```
String nickname = Objects.requireNonNull(etNickname.getText()).toString().trim();

if (!TextUtils.isEmpty(email) && !ValidationUtils.isEmailValid(email))
    setError(etEmail, getString(R.string.invalid_email), R.drawable.king);
else etEmail.setError(null);

if (!TextUtils.isEmpty(password) && !ValidationUtils.isPasswordValid(password))
    setError(etPassword, getString(R.string.invalid_password), R.drawable.king);
else etPassword.setError(null);

if (!TextUtils.isEmpty(nickname) && !ValidationUtils.isNicknameValid(nickname))
    setError(etNickname, getString(R.string.invalid_nickname), R.drawable.king);
else etNickname.setError(null);

btnRegister.setEnabled(ValidationUtils.isEmailValid(email)
    && ValidationUtils.isPasswordValid(password)
    && ValidationUtils.isNicknameValid(nickname));
}

};

etEmail.addTextChangedListener(afterTextChangedListener);
etPassword.addTextChangedListener(afterTextChangedListener);
etNickname.addTextChangedListener(afterTextChangedListener);
}
```

• private void registerUser()

- הפעולה המרכזית המתחילה את שרשרת הרישום. היא שואבת את הנתונים מהשדות ומבצעת קריאה ל-`createUserWithEmailAndPassword`. אם הפעולה נכשלת (למשל, המשתמש כבר קיים), היא קוראת ל-`handleRegisterFailure`. אם הצליחה, היא עוברת לשלב בניית הנתונים ב-`handleRegisterSuccess`.

○ קוד:

```
private void registerUser() {
    progressBar.setVisibility(View.VISIBLE);

    String email = Objects.requireNonNull(etEmail.getText()).toString().trim();
    String password = Objects.requireNonNull(etPassword.getText()).toString().trim();
    String nickname = Objects.requireNonNull(etNickname.getText()).toString().trim();

    mAuth.createUserWithEmailAndPassword(email, password)
        .addOnCompleteListener(task -> {
            progressBar.setVisibility(View.GONE);
            if (!task.isSuccessful()) {
                handleRegisterFailure(task.getException());
                return;
            }
            handleRegisterSuccess(email, nickname, password);
        });
}
```

• private void handleRegisterSuccess(String email, String nickname, String password)

- זוהי ה"פעולה המוחלטת" של אתחול המשתמש.
- **תרשים זרימה (סדר פעולות):**
 - i. אתחול ה-`CloudManager` עבור המשתמש החדש.
 - ii. יצירת מסמך פרופיל גלובלי עם הכינוי שנבחר.
 - iii. צירת "סלוט" משחק ראשון (1 Slot) המכיל את נתוני ברירת המחדל של המשחק (חיים, מיקום התחלתי וכו').
 - iv. שליחת אימייל אימות.
 - v. ביצוע `signOut` - השחקן נדרש להתחבר מחדש רק לאחר אימות המייל, כדי להבטיח שהחשבון תקין.



.vi מעבר למסך ההתחברות עם פרטים מוזנים מראש.

○ קוד:

```
private void handleRegisterSuccess(String email, String nickname, String password) {
    MyApp.initializeCloudManager();
    if (MyApp.getCloudManager() == null) return;

    // גלובלי פרופיל יצירת 1.
    MyApp.getProfile().createProfile(nickname, email);

    // חדש למשתמש אוטומטית ראשון סלוט יצירת 2.
    MyApp.getCloudManager().createNewSlot(1, new UserDataManager.OnDataLoadedListener() {
        @Override
        public void onDataLoadSuccess() {
            sendVerificationEmail();
            FirebaseAuth.getInstance().signOut();
            MyApp.clearCloudManager();
            SoundManager.getInstance(RegisterActivity.this).playSfx(R.raw.sfx_success4);

            startActivity(getSuccessIntent(email, password));
            finish();
        }

        @Override
        public void onDataLoadFailed() {
            showToast("Failed to initialize game data.", Toast.LENGTH_LONG);
            SoundManager.getInstance(RegisterActivity.this).playSfx(R.raw.sfx_error);
        }
    });
}
```

● private void sendVerificationEmail()

- פעולת עזר השולחת בקשה לשרת לשלוח אימייל אימות לכתובת שנרשמה. זוהי שכבת אבטחה קריטית שמונעת הרשמה עם כתובות פיקטיביות.

○ קוד:

```
private void sendVerificationEmail() {
    if (mAuth.getCurrentUser() == null) return;
    mAuth.getCurrentUser().sendEmailVerification()
        .addOnCompleteListener(t -> {
            if (t.isSuccessful()) showToast(getString(R.string.verification_email_sent),
                Toast.LENGTH_SHORT);
        });
}
```

בסיס נתונים (Firebase Authentication & Firestore)

המחלקה משמשת כיוצרת (Creator) של המבנה הראשוני של המשתמש בענן.

- סקירת המידע: המשתמש נרשם ב-Auth. במקביל, נוצרים שני מסמכים ב-Firestore: מסמך ב-collection Profiles ומסמך ב-collection Slots תחת ה-UID של המשתמש.
- פעולות:

- כתיבה: שימוש ב-set() (דרך מחלקות ה-Doc) ליצירת הנתונים הראשוניים. פעולה זו קורית פעם אחת בלבד במחזור החיים של המשתמש - ברגע ההרשמה.



תשתית וליבת המנוע (Core Engine & Lifecycle)

המחלקות האחראיות על מחזור החיים של המשחק, לולאת הריצה והחיבור למערכת אנדרואיד.

מחלקת GameLoop

מיקום: com.example.tutorialgame.engine.core.GameLoop

תפקיד המחלקה: מחלקה זו היא ה"לב הפועם" של מנוע המשחק כולו. היא פועלת על Thread ייעודי ונפרד מה-UI Thread הראשי של אנדרואיד. תפקידה הבלעדי והקריטי הוא להריץ לולאה מתמשכת ואמינה אשר קוראת באופן עקבי לשתי הפעולות המרכזיות של המשחק: `game.update()` (לעדכון הלוגיקה) ו-`game.render()` (לציור הגרפיקה).

היא אחראית על:

1. **עקביות:** להבטיח שהמשחק ירוץ באותה מהירות על כל מכשיר, ללא תלות בכוח המעבד שלו, באמצעות שימוש ב-delta time.
2. **יעילות:** למנוע שימוש מיותר במשאבי המעבד והסוללה על ידי הגבלת קצב הפריימים (FPS Cap).
3. **יציבות:** לספק מנגנון בטוח להתחלה ועצירה של הלולאה, המנוע קריסות וקפיאות של האפליקציה (שגיאות ANR).

הסבר על תכונות המחלקה:

- `private static final int TARGET_FPS = 60`
 - תחום הכרה: `private static final` - זהו קבוע, פרטי למחלקה, שערכו משותף לכל המופעים שלה (אף שבפועל יהיה רק מופע אחד).
 - תפקיד ושימוש: מגדיר את קצב הפריימים (פריימים לשנייה) הרצוי שהמשחק ישאף אליו. הערך 60 נחשב לסטנדרט המספק חוויה חלקה. כל חישובי התזמון של הלולאה מתבססים על ערך זה.
- `private static final double OPTIMAL_TIME = 1_000_000_000.0 / TARGET_FPS`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: קבוע זה מחשב מראש את משך הזמן האידיאלי, בננו-שניות, שכל פריים בודד צריך לארוך כדי לעמוד בקצב היעד (כ-16.66 מיליון ננו-שניות לפריים). חישוב מראש זה מונע פעולת חילוק שחוזרת על עצמה בתוך הלולאה, ותורם ליעילות.
- `private final Game game`
 - תחום הכרה: `private final` - הפניה (reference) לאובייקט ה-Game הראשי, הנגישה רק מתוך מחלקה זו. היא `final` מכיוון שלולאת משחק קשורה למופע Game יחיד לכל אורך חייה.
 - תפקיד ושימוש: מחזיק את האובייקט שעבורו יופעלו המתודות `update()` ו-`render()` בכל הרצה של הלולאה.
- `private Thread gameThread`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: מחזיק את ההפניה ל-Thread שעליו הלולאה רצה בפועל. משמש כדי להתחיל ולהפסיק את פעולת ה-Thread.
- `private volatile boolean active`
 - תחום הכרה: `private volatile`.
 - תפקיד ושימוש: זהו הדגל המרכזי ששולט על ריצת הלולאה (`while (active)`). המילה `volatile` היא קריטית כאן: היא מבטיחה ששינויים בערך המשתנה שנעשים מ-Thread אחד (למשל, ה-UI Thread שקורא ל-`stopGameLoop()`) יהיו גלויים ומידיים ל-Thread השני (ה-`gameThread`). זה מונע מצב שבו הלולאה תרוץ עוד סיבוב אחד בטעות לאחר שקיבלה הוראה לעצור.



● private int FPS

- תחום הכרה : private.
- תפקיד ושימוש : מאחסן את קצב הפריימים המחושב מהשנייה האחרונה. ניתן לשלוף ערך זה לצורכי ניפוי באגים (debug) או להצגה על המסך.

הסבר פעולות מרכזיות:

● public void run()

- זוהי המתודה המרכזית של המחלקה, אשר מורצת על ידי ה-gameThread. היא מכילה את לולאת המשחק הראשית, שרצה כל עוד הדגל active הוא true. המתודה מתוכננת בקפידה להיות גם בלתי-תלויה בקצב הפריימים וגם חסכונית במשאבים.
- **תרשים זרימה (סדר פעולות):**
 - i. התחלת לולאה : מתחילה לולאת (while (active).
 - ii. חישוב Delta Time : נמדד הזמן המדויק שחלף בנו-שניות מאז הריצה הקודמת של הלולאה. ערך זה מומר לחלק של שנייה (delta). שימוש ב-delta מבטיח שלוגיקת המשחק (כמו תנועה) תהיה מבוססת על זמן אמיתי ולא על מהירות המעבד. כך, דמות שתנוע במהירות $speed * delta$ תעבור את אותו מרחק בשנייה, בין אם המכשיר מריץ את המשחק ב-20 פריימים לשנייה ובין אם ב-60.
 - iii. עדכון לוגיקה : מופעלת המתודה game.update(delta). כל הלוגיקה של המשחק (בינה מלאכותית, פיזיקה, קלט מהשחקן) מתבצעת כאן.
 - iv. ציור גרפיקה : מופעלת המתודה game.render() כדי לצייר את המצב החדש של המשחק על המסך.
 - v. חישוב FPS : מועלה מונה פריימים. פעם בשנייה, ערך המונה מתעדכן לתוך המשתנה FPS.
 - vi. קביעת קצב (Pacing) : זהו המפתח ליעילות. הקוד מחשב כמה זמן ארכו פעולות ה-update וה-render, ומפחית אותו מהזמן האופטימלי לפריימים (OPTIMAL_TIME). התוצאה היא משך הזמן שהלולאה צריכה "לנוח".
 - vii. שינה (Sleep) : מופעלת Thread.sleep(sleepTime). פקודה זו מורה למערכת ההפעלה להכניס את ה-gameThread למצב שינה, ובכך לשחרר לחלוטין את המעבד. זה פותר את בעיית ה-"Busy-Waiting" (לולאה עסוקה), חוסך סוללה ומונע התחממות של המכשיר.
 - viii. טיפול בהפרעה (Interrupt) : אם Thread אחר קורא ל-gameThread.interrupt() (מה שקורה ב-stopGameLoop), פעולת ה-sleep נקטעת ונזרקת חריגת InterruptedException. בלוק ה-catch תופס אותה ומשתמש בה כסימן לצאת מהלולאה באמצעות break, ובכך מבטיח כיבוי נקי ומיידי.
 - ix. חזרה : הלולאה ממשיכה לשלב 1.
- קוד :

```
@Override
public void run() {
    long lastFPScheck = System.currentTimeMillis();
    int fpsCounter = 0;

    long lastLoopTime = System.nanoTime();

    while (active) {
        long now = System.nanoTime();
        long updateLength = now - lastLoopTime;
        lastLoopTime = now;

        double delta = updateLength / 1_000_000_000.0;
        Log.d("GameLoop", "Delta : " + delta);

        // רינדור עדכון
        game.update(delta);
    }
}
```



```
game.render();

// ספירת FPS
fpsCounter++;
if (System.currentTimeMillis() - lastFPScheck >= 1000) {
    FPS = fpsCounter;
    fpsCounter = 0;
    lastFPScheck += 1000;
}

try { // הפריימים קצב הגבלת
    long sleepTime = (long) ((lastLoopTime - System.nanoTime() + OPTIMAL_TIME) /
1_000_000);
    if (sleepTime > 0) {
        Thread.sleep(sleepTime);
    }
} catch (InterruptedException e) {
    break;
}
}
```

• public void startGameLoop()

- מתודה ציבורית פשוטה להתנעת לולאת המשחק. היא קובעת את הדגל active ל-true, יוצרת אובייקט Thread חדש (עם this בתור ה-Runnable), ומתחילה את ריצתו.

○ קוד:

```
public void startGameLoop() {
    if (active) return;
    active = true;
    gameThread = new Thread(this);
    gameThread.start();
}
```

• public void stopGameLoop()

- זוהי המתודה הבטוחה והאמינה לעצירת לולאת המשחק. היא מיועדת לקריאה מה-UI Thread מבלי לגרום לאפליקציה לקפוא. היא קובעת את הדגל active ל-true, ולאחר מכן, באופן קריטי, קוראת ל-`gameThread.interrupt()`. פעולה זו "מעירה" את ה-Thread אם הוא במצב שינה, גורמת לו להיכנס לבלוק ה-`catch` ולצאת מהלולאה. המתודה נמנעת בכוונה מקריאה ל-`gameThread.join()`, פעולה מסוכנת שיכולה לגרום לנעילה הדדית (Deadlock) ולקריסת האפליקציה.

○ קוד:

```
public void stopGameLoop() {
    active = false;
    if (gameThread != null) {
        gameThread.interrupt();
    }
}
```

מחלקת **GamePanel**מיקום: `com.example.tutorialgame.engine.core.GamePanel`

תפקיד המחלקה: מחלקה זו היא רכיב הליבה אשר מגשר בין עולם האנדרואיד הסטנדרטי (מערכת ה-View וה-UI) לבין מנוע המשחק הייעודי שפועל על `Canvas`. היא יורשת מ-`SurfaceView`, רכיב אנדרואיד המותאם במיוחד לציור מהיר ויעיל על המסך מ-`Thread` נפרד, מה שהופך אותה לבחירה האידיאלית למנוע משחק.

היא אחראית על:

1. ניהול מחזור החיים של ה-`Surface`: היא מממשת את ממשק ה-`SurfaceHolder.Callback` כדי "להקשיב" לאירועים החשובים של מערכת ההפעלה, כמו יצירת משטח הציור (`surfaceCreated`) והריסתו (`surfaceDestroyed`).
2. ניהול מחזור החיים של לולאת המשחק: היא מתרגמת את אירועי ה-`Surface` לפקודות עבור מנוע המשחק. כשהמשטח נוצר, היא מפעילה את לולאת המשחק (`startGameLoop`); כשהמשטח נהרס (למשל, כשהמשתמש יוצא מהאפליקציה), היא עוצרת אותה (`stopGameLoop`). זה מבטיח שהמשחק רץ רק כשיש לו איפה לצייר, וחוסך במשאבים.
3. העברת קלט מגע: היא דורסת את מתודת `onTouchEvent` כדי לקלוט את כל אירועי המגע מהמשתמש, ומעבירה אותם ישירות אל מנוע המשחק (`game.touchEvent`) לעיבוד וטיפול.

הסבר על תכונות המחלקה:

- `private final Game game`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: זהו הלב הפועם של הפאנל. הוא מחזיק את המופע היחיד של אובייקט ה-`Game` הראשי, המכיל את כל הלוגיקה, המצבים והאובייקטים של המשחק. כל הפעולות של ה-`GamePanel` (ציור, עדכון, מגע) מואצלות (delegated) למעשה לאובייקט זה.

הסבר פעולות מרכזיות:

- `public GamePanel(Context context)` (הבנאי)
 - הבנאי מבצע שתי פעולות חיוניות:
 - i. `holder.addCallback(this)`: רושם את המחלקה הנוכחית כ"מאזינה" לאירועי מחזור החיים של ה-`Surface`. כך, המתודות `surfaceCreated`, `surfaceChanged` ו-`surfaceDestroyed` ייקראו בזמן הנכון.
 - ii. `game = new Game(holder, context)`: יוצר את מופע המשחק הראשי. הוא מעביר לו שני פרמטרים קריטיים:
 - `holder`: ה-`SurfaceHolder` המאפשר למנוע המשחק לקבל `Canvas` ולצייר עליו.
 - `context`: ה-`Context` של ה-`Activity` המארחת. נעשה שימוש מודע ב-`Context` זה (ולא ב-`ApplicationContext`) מכיוון שבארכיטקטורה הנוכחית, רכיבים עמוקים בתוך מנוע המשחק עשויים להזדקק להציג רכיבי UI (כמו `Toast` או `Dialog`), פעולה הדורשת `Context` של `Activity`. מכיוון שהאפליקציה נעולה למצב אופקי, הסיכון לדליפת זיכרון מסיבוב מסך מנוטרל, והגישה הזו מאפשרת את תפקוד האפליקציה כמתוכנן.

○ קוד:

```
public GamePanel(Context context) {
    super(context);
    SurfaceHolder holder = getHolder();
    holder.addCallback(this);
    game = new Game(holder, context);
}
```



- `public boolean onTouchEvent(MotionEvent event)`

- מתודה זו נקראת על ידי מערכת אנדרואיד בכל פעם שהמשתמש נוגע במסך. תפקידה היחיד הוא להעביר את אובייקט ה-`MotionEvent` הגולמי, ללא שום שינוי, ישירות למתודת `touchEvent` של אובייקט ה-`Game`. שם, במנוע המשחק, מתבצע כל העיבוד המורכב של קלט המגע.

○ קוד:

```
@SuppressWarnings("ClickableViewAccessibility")
@Override
public boolean onTouchEvent(MotionEvent event) {
    return game.touchEvent(event);
}
```

- `public void surfaceCreated(@NonNull SurfaceHolder surfaceHolder)`

- מתודה זו נקראת על ידי מערכת אנדרואיד ברגע שמשטח הציור (`Surface`) מוכן לשימוש. זהו האות הבטוח להתחיל את המשחק. המתודה קוראת ל-`resumeGame()`, אשר בתורו מפעיל את `game.startGameLoop()`.

○ קוד:

```
@Override
public void surfaceCreated(@NonNull SurfaceHolder surfaceHolder) {
    resumeGame();
}
```

- `public void surfaceDestroyed(@NonNull SurfaceHolder surfaceHolder)`

- מתודה זו נקראת רגע לפני שמשטח הציור נהרס (למשל, כשהמשתמש יוצא מהאפליקציה או עובר למסך אחר). זהו הזמן לעצור את המשחק כדי למנוע קריסות ולחסוך במשאבים. המתודה קוראת ל-`pauseGame()`, אשר מפעיל את `game.stopGameLoop()` ועוצר את הלולאה בצורה בטוחה.

○ קוד:

```
@Override
public void surfaceDestroyed(@NonNull SurfaceHolder surfaceHolder) {
    pauseGame();
}
```

**מחלקת `GameActivity`**מיקום: `com.example.tutorialgame.ui.activities.GameActivity`

תפקיד המחלקה: מחלקה זו היא ה-`Activity` שבה מתרחש המשחק בפועל. היא משמשת כ"מארחת" (Host) עבור ה-`GamePanel` ומנהלת את הקשר הקריטי בין מחזור החיים של אפליקציית אנדרואיד לבין לולאת המשחק (`Game Loop`).

היא אחראית על:

1. אירוח ה-`GamePanel`: היא יוצרת את משטח הציור של המשחק ומגדירה את ה-`Layout` שבו הוא יוצג (ממורכז ובגודל הנכון).
2. ניהול מחזור חיים: היא מתרגמת את אירועי מחזור החיים של אנדרואיד (`onResume`, `onPause`) לפקודות עבור מנוע המשחק (חידוש או עצירת לולאת המשחק).
3. טיפול בהפרעות חיצוניות: היא מנהלת מנגנון להאזנה לאירועי מערכת, כגון שיחות טלפון נכנסות, כדי להבטיח שהמשחק יעצור בצורה בטוחה ויעבור למצב תפריט (`MENU`) במקרה של הפרעה.

הסבר על תכונות המחלקה:

- `private GamePanel gamePanel`
 - תחום הכרה: `private GamePanel`
 - תפקיד ושימוש: מופע של ה-`View` הראשי של המשחק. דרכו ה-`Activity` ניגשת למנוע המשחק כדי לשלוט בעצירה וחידוש של הלולאה.
- `private BroadcastReceiver receiveCall`
 - תחום הכרה: `private BroadcastReceiver`
 - תפקיד ושימוש: אובייקט מסוג "מאזין מערכת". הוא אחראי לזהות שינויים במצב הטלפון (שיחה נכנסת) כדי להורות למשחק לעבור למצב "תפריט" באופן אוטומטי.

הסבר פעולות מרכזיות:

- `protected void onCreate(Bundle savedInstanceState)`
 - הסבר לוגי: זוהי הפעולה הראשונה שמתבצעת בעת יצירת ה-`Activity`. היא אחראית על בניית התשתית: אתחול ה-`GamePanel`, הגדרת ה-`Layout` (סידור רכיבים על המסך) כך שהמשחק יוצג במרכז ובמידות הנכונות, והגדרת ה-`Receiver` לטיפול בשיחות.
 - כיצד `onCreate` עובד (כללי): מתודה זו היא חלק מ"מחזור החיים" של אנדרואיד. המערכת קוראת לה פעם אחת כשהמסך נוצר. היא מקבלת אובייקט `Bundle` המכיל מצב שמור (אם קיים). זהו המקום לבצע את כל ה-`Setup` הראשוני: קישור ל-`XML` (או יצירת `Views` בקוד), מציאת רכיבים (`findViewById`) ואתחול משתנים.

○ קוד:

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    this.gamePanel = new GamePanel(this);

    FrameLayout.LayoutParams params = new FrameLayout.LayoutParams(
        SCREEN_WIDTH,
        SCREEN_HEIGHT,
        Gravity.CENTER
    );

    FrameLayout layout = new FrameLayout(this);
    layout.addView(gamePanel, params);
    layout.setBackgroundColor(Color.BLACK);
    setContentView(layout);

    // טלפון לשיחות המאזין יצירת
```



```

receiveCall = new BroadcastReceiver() {
    @Override
    public void onReceive(Context context, Intent intent) {
        if (gamePanel == null || gamePanel.getGame() == null) return;

        // מצב לשנות טעם אין, מוות במסך כבר השחקן אם
        if (gamePanel.getGame().getCurrentGameState() == Game.GameState.DEATH_SCREEN)
            return;

        if (intent != null &&
            TelephonyManager.ACTION_PHONE_STATE_CHANGED.equals(intent.getAction())) {
            String state = intent.getStringExtra(TelephonyManager.EXTRA_STATE);
            if (TelephonyManager.EXTRA_STATE_RINGING.equals(state) ||
                TelephonyManager.EXTRA_STATE_OFFHOOK.equals(state)) {
                Game.requestStateChange(Game.GameState.MENU);
            }
        }
    }
};
}

```

• protected void onResume()

- נקראת כשהמשחק חוזר למסך. היא מחדשת את לולאת המשחק ב-GamePanel.
- ורושמת את ה-receiveCall במערכת כדי שיתחיל להאזין לאירועים.

קוד: ○

```

@Override
protected void onResume() {
    super.onResume();
    gamePanel.resumeGame();

    // לפעילות חוזר שהמסך ברגע המאזין רישום
    IntentFilter filter = new IntentFilter(TelephonyManager.ACTION_PHONE_STATE_CHANGED);
    registerReceiver(receiveCall, filter);
}

```

• protected void onPause()

- נקראת כשהמסך מאבד פוקוס (שיחה, יציאה מהאפליקציה). היא עוצרת את לולאת המשחק כדי לחסוך במשאבים, מבטלת את רישום ה-Receiver, ומבצעת "בדיקת בטיחות": אם השחקן יצא מהאפליקציה באמצע משחק, היא מעבירה אותו למצב MENU כדי שלא יופתע כשחוזר.

קוד: ○

```

@Override
protected void onPause() {
    super.onPause();
    gamePanel.pauseGame();

    // מהאפליקציה רגילה ביציאה לתפריט למעבר גיבוי לוגיקה
    if (!isFinishing()) {
        Game.GameState currentState = gamePanel.getGame().getCurrentGameState();
        if (currentState != Game.GameState.DEATH_SCREEN && currentState != Game.GameState.MENU)
        {
            Game.requestStateChange(Game.GameState.MENU);
        }
    }

    // זיכרון דליפות למניעת המאזין רישום ביטול
    try {
        unregisterReceiver(receiveCall);
    } catch (IllegalArgumentException ignored) {}
}

```



• `protected void onDestroy()`

- נקראת רגע לפני שה-Activity נמחק מהזיכרון. היא מוודאת עצירה סופית של ה-Thread של המשחק כדי למנוע דליפות זיכרון (Memory Leaks).

○ קוד:

```
@Override
protected void onDestroy() {
    super.onDestroy();
    if (gamePanel != null && gamePanel.getGame() != null) {
        gamePanel.getGame().stopGameLoop();
    }
}
```

**מחלקת GameStateInterface (ממשק)**

מיקום: com.example.tutorialgame.engine.interfaces.GameStateInterface

תפקיד המחלקה: ממשק זה מגדיר את ה"חוזה" (Contract) היסודי עבור כל מצב משחק במערכת. הוא מבטיח אחידות פונקציונלית בין כלל המצבים (כמו תפריטים, שלבי משחק, סינמטיקה וכו'), ומאפשר למנוע המשחק הראשי (Game.java) לנהל אותם בצורה פולימורפית. בזכות הממשק, המנוע יכול להריץ את לולאת המשחק (Update & Render) מבלי להכיר את הלוגיקה הפנימית של המצב הספציפי שנמצא כרגע בשימוש.

היא אחראית על:

1. הגדרת מחזור החיים: קביעת הפעולות ההכרחיות לעדכון לוגי ורינדור גרפי.
2. אחידות בקלט: הבטחה שכל מצב משחק ידע לקבל ולפענח אירועי מגע (Touch Events).
3. תמיכה בפולימורפיזם: אפשרור למנוע להחזיק משתנה מסוג הממשק ולהפעיל עליו מתודות מבלי לדעת את זהות המחלקה המממשת.

הסבר על תכונות המחלקה:

הממשק אינו מחזיק תכונות (Fields), שכן הוא נועד להגדיר התנהגות בלבד.

הסבר פעולות מרכזיות:

- void update(double delta)
 - פעולה האחראית על עדכון הלוגיקה הפנימית של המצב. היא מקבלת את ה-delta (הזמן שעבר מאז הפריים הקודם) כדי להבטיח תנועה ואנימציה עקביות ללא תלות במהירות המעבד.
- void render(Canvas c)
 - פעולה האחראית על ציור המצב על גבי המסך. היא מקבלת את ה-Canvas של המשחק ומבצעת עליו את כל פעולות הרינדור הגרפיות הרלוונטיות לאותו רגע.
- void touchEvents(MotionEvent event)
 - פעולה האחראית על קבלת אירועי מגע מהמשתמש. כל מצב משחק מממש אותה בצורה שונה (למשל: לחיצה על כפתור בתפריט לעומת הזזת השחקן במשחק).

מחלקת SplashActivity

מיקום: com.example.tutorialgame.ui.activities.SplashActivity

תפקיד המחלקה: מחלקה זו מהווה את נקודת הכניסה הראשונית לאפליקציה (Entry Point). תפקידה הוא להכין את תשתית המשחק לפני תחילת הפעילות: היא מחשבת את מידות המסך עבור מנוע הגרפיקה, מאתחלת את הגדרות השפה, ומנהלת את תהליך ה-"Deep Loading" - בדיקת סטטוס המשתמש בענן וטעינת נתוני השחקן והמפה ברקע תוך הצגת אנימציות מיתוג (Branding).

היא אחראית על:

1. אתחול מידות עולם: חישוב והגדרת קבועי הגרפיקה (GameConstants) בהתאם לרזולוציית המכשיר לשמירה על יחס גובה-רוחב קבוע.
2. ניהול אנימציות פתיחה: הרצת אנימציות Sprite של לוגו המשחק ותנועה וקטורית על פני המסך.
3. בקרת הזדהות אוטומטית: בדיקה מול Firebase האם קיים משתמש מחובר ומאומת.
4. שרשור טעינה (Loading Chain): טעינה מדורגת של פרופיל המשתמש, סלוט המשחק והמפה ההתחלתית בטרם הכניסה למשחק.

הסבר על תכונות המחלקה:

- private static final int SPLASH_DELAY = 4000
 - תחום הכרה: private static final



- תפקיד ושימוש : זמן ההמתנה המינימלי (במילישניות) להצגת מסך הפתיחה. מבטיח שהמשתמש ייחשף למיתוג ושהאנימציות יספיקו לרוץ לפני המעבר למסך הבא.
- `private ObjectAnimator logoAnimator`
 - תחום הכרה : `private`.
 - תפקיד ושימוש : אובייקט האחראי על תנועת הלוגו על ציר ה-X. נשמר כמשתנה מחלקה כדי לאפשר את עצירתו ב-`onDestroy` למניעת זליגת משאבים.
- `private final Handler splashHandler`
 - תחום הכרה : `private final`.
 - תפקיד ושימוש : מנהל תזמון המשימות על ה-`Main Thread`. משמש להפעלת בדיקת סטטוס המשתמש לאחר תום הדיליי המוגדר.
- `private float lastVol`
 - תחום הכרה : `private`.
 - תפקיד ושימוש : משתנה עזר השומר את עוצמת השמע שהייתה מוגדרת לפני כניסת ה-Splash, כדי להחזירה למצב הקודם בעת המעבר למסך הבא.

הסבר פעולות מרכזיות:

- `private void initDimensions()`
 - פעולה קריטית לסנכרון הגרפי. היא מזהה את מידות המסך הפיזיות של המכשיר ומחשבת את ה-`Scaling` הנדרש עבור המנוע (לפי יחס 9:16). המידע מוזן ל-`GameConstants` כדי שכל החישובים המתמטיים של ה-`Hitboxes` והציור יהיו אחידים בכל המכשירים.
 - קוד :

```
private void initDimensions() {
    DisplayMetrics dm = new DisplayMetrics();
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.R) {
        WindowManager wm = (WindowManager) getSystemService(Context.WINDOW_SERVICE);
        WindowMetrics windowMetrics = wm.getCurrentWindowMetrics();
        Rect bounds = windowMetrics.getBounds();
        dm.heightPixels = bounds.height();
        dm.widthPixels = bounds.width();
    } else {
        getWindowManager().getDefaultDisplay().getRealMetrics(dm);
    }

    int screenHeight = dm.heightPixels;
    // Logic for game-specific aspect ratio scaling (16:9 aspect ratio)
    int screenWidth = (int) (screenHeight / 0.5625);

    // Notify game engine of screen dimensions for coordinate mapping
    GameConstants.View.initScreenDimensions(screenWidth, screenHeight);
}
```

- `private void checkUserStatus()`
 - הליכה הלוגית של תהליך הכניסה. הפעולה בודקת אם המשתמש מחובר ל-Firebase Auth.
 - **תרשים זרימה (סדר פעולות):**
 - i. אם אין משתמש או שהאימייל לא מאומת -> מעבר ל-`LoginActivity`.
 - ii. אם קיים משתמש -> התחלת טעינת פרופיל (`Account Data`).
 - iii. לאחר הצלחת הפרופיל -> טעינת הסלוט האחרון שבו השחקן שיחק.
 - iv. לאחר טעינת הסלוט -> הרצת טעינת המפה ב-`Thread` נפרד למניעת תקיעה ב-UI.
 - v. בסיום הכל -> מעבר ל-`LauncherActivity`.



קוד: ○

```
private void checkUserStatus() {
    FirebaseUser currentUser = FirebaseAuth.getInstance().getCurrentUser();
    if (currentUser == null || !currentUser.isEmailVerified()) {
        navigateToLogin();
        return;
    }
    if (MyApp.getCloudManager() == null) MyApp.initializeCloudManager();
    // Deep Loading Chain: Profile -> Slot -> Map -> Launcher
    MyApp.startLoadingAccountData(new UserDataManager.OnDataLoadedListener() {
        @Override
        public void onDataLoadSuccess() {
            int savedSlotId = MyApp.getProfile().getLastSelectedSlot();

            MyApp.getCloudManager().selectSlot(savedSlotId, new
            UserDataManager.OnDataLoadedListener() {
                @Override
                public void onDataLoadSuccess() {
                    // Preload map on a background thread to prevent UI stutter
                    new Thread() -> {
                        MapManager.initStartingWorld();
                        runOnUiThread(SplashActivity.this::navigateToMain);
                    }.start();
                }
                @Override
                public void onDataLoadFailed() { handleLoadError(); }
            });
        }
        @Override
        public void onDataLoadFailed() { handleLoadError(); }
    });
}
```

● public void startTransition()

- מנהלת את הצד הויזואלי. היא מפעילה AnimationDrawable (אנימציה פריים-ביי-פריים) על רקע הלוגו ויוצרת תנועה אופקית (Translation) מחלקו השמאלי של המסך לימינו.

קוד: ○

```
public void startTransition() {
    ivLogo.setBackgroundResource(R.drawable.logo_animation);
    AnimationDrawable animationDrawable = (AnimationDrawable) ivLogo.getBackground();
    animationDrawable.start();

    int width = getResources().getDisplayMetrics().widthPixels;
    logoAnimator = ObjectAnimator.ofFloat(ivLogo, "translationX", -width / 3f, width);
    logoAnimator.setDuration(2300);
    logoAnimator.setRepeatCount(ObjectAnimator.INFINITE);
    logoAnimator.setRepeatMode(ObjectAnimator.RESTART);
    logoAnimator.start();
}
```

בסיס נתונים (Firebase Authentication & Firestore)

המחלקה משמשת כ"מנוע הטעינה" שמסנכרן את האפליקציה עם הענן מיד עם פתיחתה.

- סקירת המידע: המידע נקרא מ-Firebase Auth (אימות) ומ-Firebase (נתוני פרופיל וסלוטים).
- פעולות:

- טעינה: ביצוע קריאות Read ראשוניות לענן כדי לאכלס את אובייקטי ה-Singleton של הנתונים (MyApp.getProfile(), MyApp.getCloudManager()) לפני שהשחקן מתחיל לשחק.

**מחלקת BaseState**

מיקום: com.example.tutorialgame.gamestates.BaseState

תפקיד המחלקה: מחלקה זו משמשת כמחלקת בסיס (Parent Class) אבסטרקטית לכל מצבי המשחק (Game States) במערכת. היא מגדירה את המבנה המשותף ואת מחזור החיים של המצבים השונים, ומספקת כלי עזר לניהול מעברים בין מסכים, זיהוי קלט וניקוי משאבים. השימוש במחלקה אבסטרקטית מאפשר ליישם את עקרון הפולימורפיזם, שכן מחלקת ה-Game יכולה לנהל רשימת מצבים מבלי לדעת את המימוש הספציפי של כל אחד מהם.

היא אחראית על:

1. הגדרת מחזור חיים: קביעת מתודות הכניסה (onEnter) והיציאה (onExit) עבור כל המצבים היורשים.
2. ניהול מעברים: אספקת מנגנון בטוח לסגירת פעילות המשחק וחזרה למסך הראשי (Launcher).
3. תשתית שירותים: ריכוז רפרנסים לאובייקטי הליבה (Game, Context) עבור כל תתי-המערכות.

הסבר על תכונות המחלקה:

- `protected Game game`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: רפרנס למחלקת הניהול הראשית של המנוע. מאפשר למצבים השונים לבקש שינוי מצב (State Change) או לגשת ללופ המשחק.
- `protected Context context`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: הקשר האפליקציה המשמש לגישה למשאבים, קבצים והפעלת `Activities`.
- `private final Intent intent`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: אובייקט כוונה המוגדר מראש למעבר אל ה-`LauncherActivity`. הגדרתו בבנאי חוסכת הקצאת זיכרון בכל פעם שהמשתמש יוצא מהמשחק.

הסבר פעולות מרכזיות:

- `protected void returnLauncher()`
 - פעולה זו מבצעת סגירה מסודרת והדרגתית של המשחק וחזרה לתפריט הראשי של האפליקציה. כדי למנוע דליפות זיכרון או שגיאות ב-Thread של הציור, הפעולה בודקת תחילה אם קיימת מפה פעילה ומנקה ממנה את השחקן, ולאחר מכן עוצרת את ה-Game Loop ועוברת ל-Activity החדש.
 - קוד:

```
protected void returnLauncher() {
    // Clean up map resources
    GameMap current = MapManager.getCurrentMap();
    if (current != null) {
        current.removePlayer();
    }

    // Finalize state and switch activity
    game.stopGameLoop();
    context.startActivity(intent);
    finishGameActivity();
}
```

מחלקת **MenuManager**מיקום: `com.example.tutorialgame.gamestates.menu.MenuManager`

תפקיד המחלקה: מחלקה זו משמשת כמנהל (Orchestrator) המרכזי של מערכת התפריטים במשחק. היא אחראית על ניהול מצבי המשנה (Sub-States) בתוך תפריט המשחק, המעברים ביניהם, וניתוב אירועי המערכת (עדכון לוגיקה, רינדור וקלט) לתפריט הפעיל. המחלקה מיישמת את תבנית העיצוב State Pattern, המאפשרת למשתמש לנווט בין מסכי הגדרות שונים מבלי לצאת מהקשר התפריט הראשי.

היא אחראית על:

1. ניהול מצבי תפריט: מעקב אחר התפריט הנוכחי והחלפתו בבטחה תוך שמירה על עקביות ויזואלית.
2. אתחול משאבים: יצירת מופעים של כל תתי-התפריטים מראש בזיכרון לביצועים מהירים ומניעת השהיות (Lags) בזמן המעברים.
3. ניהול מחזור חיים: הפעלת מתודות הכניסה והיציאה של תפריטי המשנה להבטחת רענון נתונים (כמו טעינת הגדרות שמורות).
4. ניתוב אירועים: העברת פקודות ה-`update`, `render` ו-`touchEvents` מהמנוע הראשי אל המימוש הספציפי של התפריט המוצג.

הסבר על תכונות המחלקה:

- `private Main main, Options options, MusicSounds musicSounds, VideoSettings videoSettings`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: רפרנסים קבועים לכל אחד מתפריטי המשנה הקיימים במערכת. המופעים נוצרים פעם אחת בבנאי של המנהל ונשמרים בזיכרון, מה שמאפשר מעבר מיידי ביניהם ללא צורך בטעינה מחדש של גרפיקה.
- `private MenuState currentState`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה מטיפוס `Enum` המציין את המצב הנוכחי של המערכת (איזה תפריט מוצג ופעיל כרגע).

הסבר פעולות מרכזיות:

- `public void setCurrentMenuState(MenuState newState)`
 - פעולה זו מנהלת את המעבר הלוגי והבטוח בין תפריט אחד למשנהו. כדי להבטיח מעבר מסונכרן, הפעולה שולפת תחילה את המופע של התפריט הנוכחי וקוראת למתודת ה-`onExit` שלו (למשל לצורך ניקוי זמני או שמירה). לאחר מכן, היא מעדכנת את ה-`State` למצב החדש, שולפת את המופע המבוקש וקוראת למתודת ה-`onEnter` שלו (המפעילה טעינת נתונים טרייה).
- קוד:

```
public void setCurrentMenuState(MenuState newState) {
    BaseMenu old = getMenuInstance(currentMenuState);
    if (old != null) old.onExit();

    this.currentState = newState;

    BaseMenu now = getMenuInstance(currentMenuState);
    if (now != null) now.onEnter();
}
```

- `private BaseMenu getMenuInstance(MenuState state)`
 - פעולת עזר (Helper) למיפוי בין ערכי ה-`Enum` למופעי המחלקות הפיזיים. משתמשת במבנה של `switch-case` כדי להחזיר את הרפרנס הנכון לאובייקט התפריט בהתבסס על



המצב הלוגי. פעולה זו מרכזת את כל "צמתי הניווט" של המערכת במקום אחד, מה שמאפשר תחזוקה קלה והוספת תפריטים עתידיים ללא שינוי הלוגיקה העוטפת.

קוד: ○

```
private BaseMenu getMenuInstance(MenuState state) {  
    switch (state) {  
        case Main: return main;  
        case Options: return options;  
        case MusicSounds: return musicSounds;  
        case VideoSettings: return videoSettings;  
        default: return null;  
    }  
}
```

**מחלקת BaseMenu**

מיקום: com.example.tutorialgame.gamestates.menu.BaseMenu

תפקיד המחלקה: מחלקת בסיס אבסטרקטית המגדירה את ה"שלד" הויזואלי והלוגי המשותף לכל מסכי התפריטים באפליקציה. היא משמשת כתבנית (Template) המבטיחה אחידות עיצובית, ומספקת למחלקות היורשות כלים מוכנים לרינדור הרקע, המסגרת וכו' המסך. השימוש במחלקה אבסטרקטית זו מאפשר ריכוז של לוגיקת הציוור והחישובים הגיאומטריים של התפריט במקום אחד, ובכך מונע כפל קוד ומשפר את יציבות הממשק.

היא אחראית על:

1. הגדרת שפה עיצובית אחידה: ניהול הנכסים הגרפיים המרכיבים את חלונות התפריט (רקע ומסגרת).
2. אופטימיזציית רינדור טקסט: חישוב ומיקום כותרות המסך בצורה ממורכזת ומדויקת.
3. תשתית לניווט: החזקת רפרנס למנהל התפריטים לצורך מעברים בין מסכי המשנה.

הסבר על תכונות המחלקה:

- `protected final NinePatchDrawable background, frame`
 - תחום הכרה: `protected final` (מוגן וקבוע).
 - תפקיד ושימוש: אובייקטים לניהול הגרפיקה של תיבת התפריט והמסגרת האדומה. השימוש בפורמט Nine-Patch מאפשר למנוע למתוח את התמונות לכל גודל מסך מבלי לעוות את הפינות או לפגוע באיכות המרקם.
- `protected final TextRenderer titlePaint`
 - תחום הכרה: `protected final` (מוגן וקבוע).
 - תפקיד ושימוש: רכיב רינדור טקסט מתקדם (מבוסס `TextPaint`) המשמש לציוור כותרות. הוא מוגדר מראש עם גופן המשחק, צל ויישור מתאים.
- `protected final MenuManager menuManager`
 - תחום הכרה: `protected final`.
 - תפקיד ושימוש: רפרנס למנהל המצבים של התפריט, המאפשר לכל תפריט ספציפי (כמו הגדרות וידאו) פקוד על מעבר למסך אחר.
- `private int lastRes`
 - תחום הכרה: `private` (פרטי).
 - תפקיד ושימוש: משתנה עזר השומר את מזהה משאב הטקסט (Resource ID) האחרון שצויר. משמש כמנגנון "זיכרון" למניעת חישובים גיאומטריים חוזרים ומיותרים בכל פריים אם הכותרת לא השתנתה.

הסבר פעולות מרכזיות:

- `protected void drawBackground(Canvas c, @StringRes int stringResId)`
 - הפעולה הלוגית המרכזית האחראית על רינדור ה"מעטפת" של כל תפריט. הפעולה מציירת את שכבות הרקע והמסגרת. החלק המתוחכם בה הוא ניהול הכותרת: כדי לחסוך במשאבי מעבד, הפעולה בודקת אם הכותרת הנוכחית שונה מזו שצוירה בפריים הקודם. רק בעת שינוי, היא מודדת את רוחב הטקסט ומחשבת את נקודת המרכז המדויקת שלו. בסיום, היא מבצעת ציוור משולב של הטקסט והצל שלו בפקודה אחת דרך `TextRenderer`.
 - **תרשים זרימה (סדר פעולות):**
 - i. ציוור המסגרת (`frame`) על הקנבס.
 - ii. ציוור רקע התפריט (`background`) מעל המסגרת.
 - iii. בדיקה: האם המחרוזת המבוקשת (`stringResId`) שונה מהמחרוזת הקודמת (`lastRes`)?



● אם כן:

- a. עדכון lastRes למזהה החדש.
- b. מדידת רוחב הטקסט בשפה הנוכחית.
- c. חישוב וקביעת מיקום ה-X וה-Y של ה-Renderer (מרכז אופקי בשליש העליון).
- iv. רינדור הכותרת עם אפקט הצל המובנה.

○ קוד:

```
protected void drawBackground(Canvas c, @StringRes int stringResId) {  
    if (frame != null) frame.draw(c);  
    if (background != null) background.draw(c);  
  
    if (lastRes != stringResId) {  
        lastRes = stringResId;  
        float titleWidth = titlePaint.measureText(context.getString(stringResId));  
        titlePaint.setPosition(SCREEN_WIDTH / 2f - titleWidth / 2f, SCREEN_HEIGHT / 5f);  
    }  
    // Single call handles both text and its shadow using the custom TextRenderer  
    titlePaint.drawWithShadow(context.getString(stringResId), c);  
}
```

**מחלקת BitmapManager**

מיקום: com.example.tutorialgame.managers.BitmapManager

תפקיד המחלקה: מחלקה זו משמשת כמנהל המשאבים הגרפיים המרכזי והבלעדי של המשחק. היא פועלת כ"מחסן חכם" (Cache) המיישם את תבנית העיצוב Flyweight. תפקידה המרכזי הוא להבטיח שכל תמונה (Bitmap), רצועת אנימציה (D Array1), מטריצת דמויות (D Array2) או תת-אזור מתוך אטלס (Atlas Region) ייטענו מהדיסק, יעברו פענוח (Decoding) ועיבוד (Scaling/Smoothing) בדיוק פעם אחת במהלך מחזור חיי האפליקציה. על ידי ניהול ריכוזי של הזיכרון הגרפי, המחלקה מונעת בזבז משמעותי של זיכרון RAM, מפחיתה את העומס על המעבד (CPU) ומונעת השהיות (Stuttering) בלולאת המשחק הנובעות מפעולות קלט/פלט (I/O).

היא אחראית על:

1. מניעת כפילויות: הבטחה שמשאב גרפי זהה לא ייטען לזיכרון יותר מפעם אחת עבור סוג שימוש נתון.
2. ניהול מטמון רב-שכבתי: החזקת מפות נתונים (Maps) נפרדות המותאמות לסוגי משאבים שונים (בודדים, מערכים ומטריצות).
3. אופטימיזציית טעינה (Lazy Loading): שימוש במטמון זמני לקבצי מקור גולמיים כדי למנוע פענוח חוזר של קבצי אטלס גדולים.
4. גמישות רינדור: מתן שליטה מלאה על רמת ההחלקה (Smoothing) ומכפילי גודל (Multiplier) לכל טעינה בנפרד.
5. שימוש ב-Object Reuse: צמצום הקצאות זיכרון ב-Runtime דרך החזרת רפרנסים קיימים מהמטמון.

הסבר על תכונות המחלקה:

- `private static final Map<String, Bitmap> bitmapCache`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מאגר השומר Bitmaps בודדים מעובדים וכן אזורי חתוכים מאטלסים. המפתח (Key) הוא מחרוזת ייחודית המורכבת ממזהה המשאב והפרמטרים שלו.
- `private static final Map<String, Bitmap[]> sheetCache`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מטמון למערכי תמונות (D1), המשמשים לאנימציות פשוטות של חפצים בעולם (כגון מטבעות או אפקטים).
- `private static final Map<String, Bitmap[][]> sheet2DCache`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מטמון למטריצות של תמונות (D2), המשמשות לניהול דמויות בעלות כיווני מבט ומצבי אנימציה מרובים (כגון השחקן והאויבים).
- `private static final Map<Integer, Bitmap> rawAtlasCache`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מטמון זמני השומר את תמונות המקור הלא-חתוכות. הוא מונע את הפעולה היקרה של `decodeResource` כאשר נדרשים מספר חיתוכים מאותו קובץ גרפי.
- `private static final BitmapMethods helper`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מופע אנונימי של הממשק `BitmapMethods`. הוא מאפשר למחלקה הסטטית להשתמש בלוגיקת ה-Scaling וה-Smoothing המוגדרת בממשק ללא צורך ביצירת מופעים של המחלקה.

הסבר פעולות מרכזיות:



- static { ... } (בלוק אתחול סטטי)

- פעולה זו מתבצעת פעם אחת בלבד עם עליית האפליקציה. היא מגדירה את `options.inScaled = false`, מה שמבטל את ה-Scaling האוטומטי של אנדרואיד ומאפשר למנוע לבצע את התאמת הרזולוציה ידנית בדיוק של פיקסל (Pixel-Perfect). (Scaling)

- קוד:

```
static {
    options.inScaled = false;
}
```

- private static String getCacheKey(int resId, double multiply, boolean smooth)

- פעולת עזר המייצרת מזהה ייחודי לכל משאב מעובד. המפתח מורכב מה-Resource ID, המכפיל וסוג הסינון. טכניקה זו מאפשרת למערכת להחזיק בזיכרון גרסה קטנה וגרסה גדולה של אותה תמונה בו-זמנית מבלי שיתנגשו במטמון.

- קוד:

```
private static String getCacheKey(int resId, double multiply, boolean smooth) {
    return resId + "_" + multiply + "_" + (smooth ? "s" : "c");
}
```

- public static Bitmap getBitmapRegion(int resId, int x, int y, int w, int h, double multiply, boolean smooth)

- זוהי הפעולה המאפשרת שימוש חסכוני באטלסים (Atlases).

- תרשים זרימה (סדר פעולות):

- יצירת מפתח: הפקת מזהה ייחודי הכולל את הקואורדינטות (x,y) והממדים (w,h).
- בדיקת מטמון: אם האזור המבוקש כבר נחתך בעבר, הוא נשלף מ-`bitmapCache` ומוחזר מיד.
- טעינת אטלס: שליפת התמונה המלאה דרך `getRawAtlas` (שבעצמו בודק אם האטלס כבר פוענח).
- חיתוך (Cropping): יצירת `Bitmap` חדש המכיל רק את האזור הרלוונטי בעזרת `Bitmap.createBitmap`.
- עיבוד: שליחת ה-`Bitmap` הגולמי ל-`helper` לביצוע Scaling וסינון לפי הצורך.
- אחסון: שמירת התוצאה הסופית במטמון והחזרתה לקוד המבקש.

- קוד:

```
public static Bitmap getBitmapRegion(int resId, int x, int y, int w, int h, double multiply, boolean smooth) {
    String key = getCacheKey(resId, multiply, smooth) + "_r_" + x + "_" + y + "_" + w + "_" + h;
    if (bitmapCache.containsKey(key)) return bitmapCache.get(key);

    Bitmap atlas = getRawAtlas(resId);
    if (atlas == null) return null;

    Bitmap rawRegion = Bitmap.createBitmap(atlas, x, y, w, h);
    Bitmap processed = smooth ? helper.getMultiplyBitmapSmth(rawRegion, multiply) :
    helper.getMultiplyBitmapClean(rawRegion, multiply);

    bitmapCache.put(key, processed);
    return processed;
}
```

- public static Bitmap[][] getSpritesheet2D(int resId, int frameW, int frameH, int rows, int cols, double multiply, boolean smooth)



- הפעולה המרכזית לטעינת דמויות המשחק. הפעולה חותכת גיליון דמויות שלם למטריצה (מערך דו-מימדי) של פריימים. היא מבצעת לולאה כפולה: הלולאה החיצונית עוברת על השורות (כיוונים/מצבים) והלולאה הפנימית עוברת על העמודות (פריימים של אנימציה). כל פריים נחתך ומעובד בנפרד, ובסיום התהליך המטריצה כולה נשמרת במטמון תחת מפתח יחיד.

○ קוד:

```
public static Bitmap[][] getSpritesheet2D(int resId, int frameW, int frameH, int rows, int cols, double multiply, boolean smooth) {
    String key = getCacheKey(resId, multiply, smooth) + "_m_" + rows + "x" + cols;
    if (sheet2DCache.containsKey(key)) return sheet2DCache.get(key);

    Bitmap sheet = getRawAtlas(resId);
    if (sheet == null) return null;

    Bitmap[][] frames = new Bitmap[rows][cols];
    for (int row = 0; row < rows; row++) {
        for (int col = 0; col < cols; col++) {
            Bitmap rawFrame = Bitmap.createBitmap(sheet, col * frameW, row * frameH, frameW, frameH);
            frames[row][col] = smooth ? helper.getMultiplyBitmapSmoth(rawFrame, multiply) : helper.getMultiplyBitmapClean(rawFrame, multiply);
        }
    }

    sheet2DCache.put(key, frames);
    return frames;
}
```

דוגמאות לשימוש (Usage Examples)

- טעינת דמות ב-Enum של דמויות (GameCharacters.java): במקום לבצע לולאות טעינה בבנאי של ה-Enum, פשוט פונים למנהל:

```
sprites = BitmapManager.getSpritesheet2D(resID, width, height, 7, 4, 1, false);
```

- טעינת אובייקט מתוך אטלס חפצים:

```
objectImg = BitmapManager.getBitmapRegion(R.drawable.atl_world_objects, x, y, width, height, 1.0, false);
```



מערכת הישויות והרכיבים (Entity System & Components) השלב של כל האובייקטים הפיזיים הקיימים בעולם המשחק.

מחלקת Entity

מיקום: com.example.tutorialgame.entities.Entity

תפקיד המחלקה: מחלקה זו היא מחלקת הבסיס האבסטרקטית (abstract) עבור כל האובייקטים הפיזיים הקיימים בעולם המשחק. היא מייצגת את המכנה המשותף הנמוך והבסיסי ביותר של "ישות" - כל דבר שיש לו מיקום וגודל בעולם, ויכול פוטנציאלית להיות חלק מאינטראקציה פיזית. היא אינה ניתנת ליצירה ישירה, אלא חייבים לרשת ממנה. תפקידה הוא לספק את התכונות וההתנהגויות היסודיות ביותר.

היא אחראית על:

1. מיקום וגודל: כל ישות מחזיקה RectF המשמש כ-hitBox, המגדיר את גבולותיה הפיזיים בעולם.
2. קיום: לכל ישות יש מצב active, המאפשר "לכבות" ולהפעיל אותה מבלי להסיר אותה מהזיכרון.
3. התנגשות: לכל ישות יש מצב collider, הקובע אם יש להתייחס אליה בבדיקות התנגשות.
4. יכולת מיון: המחלקה מממשת את הממשק Comparable, מה שמאפשר למיין רשימה של ישויות. יכולת זו קריטית עבור "אלגוריתם הצייר" (Painter's Algorithm), כדי להבטיח שישויות הממוקמות נמוך יותר על המסך יצוירו מעל ישויות גבוהות יותר, ובכך ליצור אשליה של עומק ופרספקטיבה נכונה.

הסבר על תכונות המחלקה:

• protected RectF hitBox

- תחום הכרה: protected RectF
- תפקיד ושימוש: זוהי התכונה המרכזית של הישות. זהו מלבן המגדיר את המיקום (x,y) של הפינה השמאלית-עליונה והממדים (רוחב וגובה) של הישות בעולם המשחק. הוא משמש לכל החישובים הפיזיקליים: בדיקות התנגשות, חישוב מיקום לציור, וקביעת סדר הציור. תחום ההכרה מאפשר למחלקות היורשות (כמו Character או Building) גישה ישירה אליו לצורך הזזה או שינוי גודל.

• protected boolean active

- תחום הכרה: protected boolean
- תפקיד ושימוש: זהו דגל (flag) הקובע אם הישות "קיימת" ופעילה בעולם המשחק. ישויות שאינן פעילות (active = false) בדרך כלל לא מעודכנות ולא מצוירות. זהו מנגנון יעיל להסרת ישויות מהעולם (למשל, אויב שמת) באופן זמני או קבוע, מבלי לגרום לשינויים מורכבים במבני הנתונים שמחזיקים אותן.

• protected boolean collider

- תחום הכרה: protected boolean
- תפקיד ושימוש: דגל זה קובע אם הישות צריכה להשתתף בבדיקות התנגשות פיזיות. למשל, לדמות או לקיר יהיה collider = true, אך למטבע או לאפקט ויזואלי יכול להיות collider = false, מה שמאפשר לדמויות לעבור דרכם. זה מאפשר לייעל את בדיקות ההתנגשות על ידי התעלמות מישויות שאינן רלוונטיות.

הסבר פעולות מרכזיות:

• Entity(PointF pos, float width, float height, boolean collider) (הבנאי)

- הבנאי מאתחל את הישות. הוא מקבל מיקום התחלתי, רוחב וגובה, ויוצר את ה-hitBox בהתאם. הוא גם קובע את מצב ה-collider הראשוני ומגדיר את הישות כ-active כברירת מחדל.

○ קוד:

```
public Entity(PointF pos, float width, float height, boolean collider) {
    this.hitBox = new RectF(pos.x, pos.y, pos.x + width, pos.y + height);
    this.active = true;
}
```



```
this.collider = collider;  
}
```

• compareTo(Entity other)

- זוהי המימוש של הממשק Comparable<Entity>, והיא מהווה את לב לוגיקת המיון לציור. המתודה משווה בין שתי ישויות על בסיס הערך של הקצה התחתון של ה-hitBox שלהן (hitBox.bottom).
- חשיבות לוגית: כאשר מציירים את עולם המשחק, יש צורך למיין את כל הישויות כדי לקבוע את סדר הציור. ישות עם ערך bottom גבוה יותר נמצאת "נמוך יותר" על המסך ולכן צריכה להיות מצוירת אחרי (כלומר, "מעל") ישות עם ערך bottom נמוך יותר. מיון באמצעות מתודה זו מבטיח שהסצנה תצויר עם תחושת עומק נכונה, ודמויות לא ייראו כאילו הן "מרחפות" דרך אובייקטים שהן אמורות להיות מאחוריהם.

○ קוד:

```
@Override  
public int compareTo(Entity other) {  
    return Float.compare(hitBox.bottom, other.hitBox.bottom);  
}
```

**מחלקת GameCharacters (Enum)**

מיקום: com.example.tutorialgame.entities.characters.GameCharacters

תפקיד המחלקה: מחלקה זו היא Enum מרכזי המגדיר את כל סוגי הדמויות הקיימות במשחק (שחקן, חברים, אויבים ו-NPCs). היא משמשת כ"מרכז הנתונים" (Configuration Hub) שבו מוגדרים המשאבים הגרפיים, נתוני הקול, ממדי הגוף, ומרחקי הראייה של כל סוג דמות. המחלקה מיישמת טעינה חכמה וריכוזית של משאבים דרך ה-`BitmapManager`, ובכך מבטיחה יעילות מקסימלית בניהול זיכרון ה-RAM.

היא אחראית על:

1. הגדרת ישויות: ריכוז כל סוגי הדמויות והמאפיינים הייחודיים שלהן במקום אחד.
2. ניהול משאבים גרפיים: טעינה וחיתוך של מטריצות אנימציה (Spritesheets) ופרצופים לדיאלוגים (Facesets) מהאטלסים הגלובליים.
3. לוקליזציה: קישור בין סוג הדמות לבין מזהה השם שלה (String Resource) לצורך תרגום.
4. פיזיקה ותצוגה: החזקת נתונים קבועים כמו רוחב, גובה ומרחק אינטראקציה (View Distance).

הסבר על תכונות המחלקה:

- `private final Bitmap[] sprites`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מטריצה דו-מימדית המכילה את כל פריימי האנימציה של הדמות. כל עמודה מייצגת מצב/כיוון (למשל: הליכה למטה, הליכה למעלה) וכל שורה מייצגת פריים בתוך האנימציה. נטען פעם אחת עבור כל סוג דמות. השורה האחרונה מייצגת פריימים ייחודיים, כמו מוות, התקפות מיוחדות וכדומה.
- `private final Bitmap faceSet`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: תמונת הדיוקן של הדמות המופיעה בחלון הדיאלוג. התמונה נגזרת מתוך אטלס הפרצופים הכללי (`atl_faceset`) לפי קואורדינטות השורה והעמודה המוגדרות ב-Enum.
- `private float viewDistance`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: קובע את המרחק (ביחידות של `TILE_SIZE`) שבו הדמות "מזהה" את השחקן או מאפשרת להתחיל איתה שיחה.
- `private final int nameId, voiceRes`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מזהי משאבים (Resource IDs) לשם הדמות (לצורכי תרגום) ולקובץ הקול הייחודי שלה המושמע בזמן דיאלוג.
- `private final int width, height`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: הממדים הפיזיים של הדמות בעולם המשחק (בפיקסלים גולמיים לפני Scaling).

הסבר פעולות מרכזיות:

- `GameCharacters(int resID, int voiceRes, float viewDistance, int charName, int width, int height, int faceCol, int faceRow)` - הבנאי
 - הבנאי אחראי על אתחול כל המשאבים של סוג הדמות ברגע טעינת האפליקציה.
 - **תהליך האתחול:**
 1. טעינת מטריצת האנימציה: קריאה ל-`BitmapManager.getSpritesheet2D` המבצעת פענוח, חיתוך ו-Scaling של גיליון הדמות למטריצה של 4x7.



- ii. טעינת הדיוקן (Faceset): בדיקה האם הוגדרו קואורדינטות פנים (faceCol/Row != -1). אם כן, חיתוך האזור המדויק מתוך אטלס הפרצופים הגלובלי דרך `BitmapManager.getBitmapRegion`.
- iii. אתחול נתוני המטא: שמירת מזהי הקול, השם והממדים בזיכרון המופע.

קוד: ○

```
GameCharacters(int resID, int voiceRes, float viewDistance, int charName, int width, int height, int faceCol, int faceRow) {
    // Loads the full 2D movement matrix (7 states, 4 directions)
    this.sprites = BitmapManager.getSpritesheet2D(resID, width, height, 7, 4, 1, false);

    // Loads the UI faceset from the shared atlas only if coordinates are provided
    if (faceCol != -1 && faceRow != -1)
        this.faceSet = BitmapManager.getBitmapRegion(R.drawable.atl_faceset, 38 * faceCol, 38 * faceRow, 38, 38, 1, false);
    else this.faceSet = null;

    this.viewDistance = viewDistance;
    this.nameId = charName;
    this.voiceRes = voiceRes;
    this.width = width;
    this.height = height;
}
```

● `public String getName()`

- הפעולה מחזירה את השם התצוגתי של הדמות. הפעולה מבצעת הבחנה בין השחקן לבין דמויות NPC. במידה ומדובר בשחקן (`nameId == -1`), השם נשלף דינמית מתוך פרופיל המשתמש בעזרת `(MyApp.getProfile().getNickname())`. עבור כל שאר הדמויות, השם נשלף ממערכת המשאבים של אנדרואיד בהתאם לשפה שנבחרה.

קוד: ○

```
public String getName() {
    if (nameId == -1)
        return MyApp.getProfile().getNickname();
    return BaseActivity.getContext().getString(nameId);
}
```

בסיס נתונים (Firebase Firestore)

המחלקה קשורה באופן עקיף למסמך ה-`ProfileDoc` השמור בענן.

- סקירת המידע: השם המוצג עבור ישות ה-`PLAYER` נשמר בשדה `nickname` בתוך מסמך הפרופיל של המשתמש.

● פעולות:

- טעינה: בזמן קריאת השם (`getName()`), המערכת פונה לאובייקט ה-`Cached` של הפרופיל כדי להציג את השם המעודכן שהמשתמש בחר.

דוגמאות לשימוש (Usage Examples)

● השחקן:

```
PLAYER(R.drawable.spr_player, R.raw.sfx_voice_player, 1.2f, -1, DEFAULT_SIZE, DEFAULT_SIZE, 0, 0)
```

● שלד:

```
SKELETON(R.drawable.spr_skeleton, -1, 5, R.string.skeleton, DEFAULT_SIZE, DEFAULT_SIZE, -1, -1) // No faceset
```



מחלקת `AnimatedObject`

מיקום: `com.example.tutorialgame.entities.foregrounds.animated.AnimatedObject`

תפקיד המחלקה: מחלקה זו מייצגת ישויות סביבתיות בעלות אנימציה מחזורית קבועה (כגון לפידים, מזרקות או מטבעות רקע). תפקידה הוא לספק תשתית גרית לניהול אובייקטים שאינם דמויות אך דורשים תצוגה דינמית. המחלקה מפרידה בין הגדרת סוג האובייקט (`AnimatedType`) לבין המופע שלו בעולם, ובכך מאפשרת טעינה מרוכזת וחסכונית של משאבים גרפיים.

היא אחראית על:

1. שימוש ברכיב `AnimationComponent` לעדכון הפריימים בקצב המוגדר לכל סוג אובייקט.
2. ציור הפריימים המתאימים במיקומי העולם, תוך התבססות על משאבים שנטענו מראש.
3. הגדרת גבולות התנגשות (`Collision`) במידת הצורך עבור אובייקטים דקורטיביים.

הסבר על תכונות המחלקה:

- `protected final AnimationComponent animation`
 - תחום הכרה: `protected final` (מוגן וקבוע).
 - תפקיד ושימוש: רכיב האחראי על תזמון החלפת הפריימים. הוא שומר על אינדקס הפריימים הנוכחי ומקדם אותו בכל עדכון.
- `protected final Bitmap[] sprites`
 - תחום הכרה: `protected final` (מוגן וקבוע).
 - תפקיד ושימוש: מערך המכיל את כל שלבי האנימציה (`Frames`) של האובייקט. המידע נשלף מתוך ה-`Enum` הסטטי כדי למנוע טעינה חוזרת מהדיסק.

הסבר פעולות מרכזיות:

- `public void draw(Canvas c)`
 - מציירת את האובייקט במיקומו הנוכחי במפה. הפעולה שולפת את אינדקס הפריימים הנוכחי מה-`animation` ומציירת את ה-`Bitmap` המתאים מתוך מערך ה-`sprites`. הציור מתבצע בפינה השמאלית-עליונה של ה-`hitBox`, המייצגת את מיקום האובייקט בעולם.
- קוד:

```
@Override
public void draw(Canvas c) {
    if (sprites == null || sprites.length == 0) return;
    c.drawBitmap(sprites[animation.getAniIndex()], hitBox.left, hitBox.top, null);
}
```

מחלקת `AnimatedType (Enum)`

מיקום: `com.example.tutorialgame.entities.foregrounds.animated.AnimatedType`

תפקיד המחלקה: `Enum` מרכזי המשמש כ"מרכז נתונים" עבור כל סוגי האובייקטים המונפשים. הוא מגדיר את נכסי הגרפיקה, מהירות האנימציה, הגודל הפיזי והאם האובייקט מהווה מכשול (`Collision`).

הסבר פעולות מרכזיות:

- הבנאי (`Constructor`): משתמש ב-`BitmapManager.getSpritesheet` כדי לטעון ולבצע `Scaling` של כל הפריימים פעם אחת בלבד בעת טעינת האפליקציה.
- `fromString(String type)`: פעולת שירות המאפשרת למנוע המפות (`MapManager`) ליצור אובייקטים דינמיים בהתבסס על מחרוזות טקסט המגיעות מקובץ המפה (`TMX`).

**מחלקת BreakableEntity**

מיקום: com.example.tutorialgame.entities.foregrounds.breakable.BreakableEntity

תפקיד המחלקה: מחלקה זו מייצגת אובייקטים סביבתיים אינטראקטיביים הניתנים לשבירה (כמו כדים, ארגזים או שיחים). תפקידה הוא לנהל את מחזור החיים המלא של האובייקט: ממצב "שלם", דרך רגע השבירה הכולל פיזיקת חלקיקים, ועד לשלב ההמתנה והחזרה למצב פעיל (Respawn). המחלקה פועלת כמכונת מצבים (State Machine) ומיישמת אופטימיזציה של זיכרון באמצעות ניהול "בריכת אובייקטים" (Object Pooling) עבור השברים.

היא אחראית על:

1. ניהול מצבי ישות: בקרה על המעברים בין מצב שלם (INTACT), מצב התפרקות (PIECES_FLYING) ומצב המתנה לחידוש (WAITING_TO_RESPAWN).
2. זיהוי אינטראקציית לחימה: ניטור פגיעות מצד תיבת התקיפה של השחקן.
3. ניהול חלקיקים חסכוני: יצירה ומחזור של שברי האובייקט דרך רכיב ה-ParticlePool למניעת עומס על ה-Garbage Collector.
4. סימולציה מבוססת זמן: ניהול הפיזיקה והטיימרים באמצעות זמן אמת (delta) להבטחת התנהגות אחידה בכל מכשיר.

הסבר על תכונות המחלקה:

- private State currentState
 - תחום הכרה: private.
 - תפקיד ושימוש: המשתנה המגדיר באיזה שלב של מחזור החיים נמצא האובייקט.
- private final BreakableType type
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט נתונים (Enum) המכיל את כל המידע הספציפי לסוג האובייקט: תמונה, קול שבירה, זמן התחדשות וסוג החלקיקים שיעופו ממנו.
- private final List<BrokenParticle> activeParticles
 - תחום הכרה: private final.
 - תפקיד ושימוש: רשימה המנהלת את כל השברים הנמצאים כרגע בתנועה על המסך. הרשימה מתעדכנת דינמית - חלקיקים מתווספים אליה בעת שבירה ומוחזרים ל-Pool בסיום האנימציה.
- private float respawnTimer
 - תחום הכרה: private.
 - תפקיד ושימוש: מונה זמן (בשניות) המחושב מתוך נתוני ה-Type. הוא דועך בכל פריים עד להגעה ל-0, אז האובייקט חוזר למצבו השלם.

הסבר פעולות מרכזיות:

- public void handleHit()
 - פעולה זו מנהלת את רגע המעבר ממצב שלם למצב שבור. הפעולה משנה את המצב הפנימי של האובייקט ומבטלת את יכולת ההתנגשות שלו. היא מחשבת מספר אקראי של חלקיקים שייווצרו, שולפת אותם מתוך ה-ParticlePool (תוך הזרקת התמונה המתאימה לסוג האובייקט) ומעניקה להם מהירות התחלתית. בסיום היא מפעילה צליל שבירה ומחשבת את זמן ההתחדשות.
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקה: האם האובייקט כבר שבור? (אם כן - יציאה).
 - ii. שינוי מצב ל-PIECES_FLYING וביטול ה-collider.
 - iii. לולאת יצירת חלקיקים:
 1. שליפת חלקיק פנוי מה-ParticlePool.
 2. מיקומו במרכז האובייקט והפעלת הפיזיקה שלו.



3. הוספה לרשימת ה-activeParticles.

הפעלת צליל שבירה ב-pitch אקראי. iv

הגרלת זמן התחדשות (respawnTimer). v

קוד: ○

```
public void handleHit() {
    if (currentState != State.INTACT) return;

    currentState = State.PIECES_FLYING;
    this.collider = false;

    // Visual Feedback: Break into pieces using the Pool
    int pieces = 4 + rnd.nextInt(4);
    for (int i = 0; i < pieces; i++) {
        activeParticles.add(ParticlePool.acquire(
            hitBox.centerX(),
            hitBox.centerY(),
            type.getParticles().getRandomParticle()
        ));
    }

    // Audio and Timing
    SoundManager.getInstance(BaseActivity.getContext()).playRndPitchSfx(type.getParticles().getSfxId());
    this.respawnTimer = (type.getRespawnTime() + rnd.nextInt(10000)) / 1000f;
}
```

● public void update(double delta, Player player)

○ מנהלת את לוגיקת המצבים של האובייקט בכל פריים. הפעולה משתמשת ב-switch על ה-currentState. במצב שלם, היא בודקת התנגשות מול התקפת השחקן. במצב שבירה, היא מעדכנת את תנועת השברים. במצב המתנה, היא מקטינה את הטיימר לקראת חזרה למפה.

קוד: ○

```
public void update(double delta, Player player) {
    switch (currentState) {
        case INTACT:
            if (player.isAttacking() && player.getAttackBox().intersect(hitBox))
                handleHit();
            break;
        case PIECES_FLYING:
            updateParticles(delta);
            if (activeParticles.isEmpty()) {
                if (type.canRespawn()) {
                    currentState = State.WAITING_TO_RESPAWN;
                } else active = false; // Final death
            }
            break;
        case WAITING_TO_RESPAWN:
            respawnTimer -= (float) delta;
            if (respawnTimer <= 0) respawn();
            break;
    }
}
```

**מחלקת BrokenParticle**

מיקום: com.example.tutorialgame.entities.foregrounds.breakable.BrokenParticle

תפקיד המחלקה: מחלקה זו מייצגת חלקיק בודד (שבר) הנוצר בעת הרס של אובייקט סביבתי. תפקידה הוא לדמות את הפיזיקה של השברים - תנועה בליסטית, סיבוב ודעיכה ויזואלית (Fade) - כדי ליצור אפקט פיצוץ ריאליסטי. המחלקה תוכננה כ"ישות רב-פעמית" הנתמכת במנגנון Object Pooling, המאפשר לה לאתחל את מצבה מחדש עבור סוגים שונים של אובייקטים (כגון חרס, עץ או אבן) ללא צורך בהקצאות זיכרון חדשות.

היא אחראית על:

1. סימולציה קינמטית: חישוב המיקום, המהירות וכוח הכבידה של השבר בזמן אמת.
2. ניהול אוריינטציה: עדכון זווית הסיבוב של השבר בהתאם למהירות הזוויתית שהוגרלה לו.
3. אפקט דעיכה: ניהול רמת השקיפות (Alpha) של החלקיק לאורך זמן עד להעלמותו המוחלטת.
4. אדפטיביות לרזולוציה: התאמת מהירויות התנועה וכוח הכבידה לגודל המסך הפיזי באמצעות קבוע הרזולוציה המותאמים לכל מסך.

הסבר על תכונות המחלקה:

- private float x, y, velX, velY
 - תחום הכרה: private.
 - תפקיד ושימוש: משתני המיקום והמהירות בצירים X ו-Y. ערכי המהירות מחושבים בפיסקלים לשנייה ומותאמים לקנה המידה של המנוע.
- private float alpha, fadeSpeed
 - תחום הכרה: private.
 - תפקיד ושימוש: alpha שומרת את רמת השקיפות הנוכחית (0-255). fadeSpeed קובעת את קצב ההיעלמות, מה שמאפשר לחלקיקים שונים "להתפוגג" בקצבים שונים ליצירת מראה טבעי.
- private static final float GRAVITY
 - תחום הכרה: private static final.
 - תפקיד ושימוש: קבוע התאוצה האנכית המושך את החלקיק כלפי מטה. הערך מחושב יחסית ל-TILE_SIZE כדי להבטיח שהנפילה תיראה זהה בכל גודל מסך.
- private boolean active
 - תחום הכרה: private.
 - תפקיד ושימוש: דגל מצב המציין אם החלקיק נמצא בתנועה. כאשר הערך הוא false, החלקיק נחשב ל"פנוי" בתוך ה-Pool ויכול לשמש שוב.

הסבר פעולות מרכזיות:

- public void init(float startX, float startY, Bitmap img)
 - פעולה המאתחלת את החלקיק לשימוש חוזר. במקום ליצור אובייקט חדש, היא "מנקה" את המשתנים הקודמים ומזריקה נתונים חדשים: מיקום התחלתי, תמונת השבר (Bitmap) המתאימה לאובייקט שנשבר, וערכי פיזיקה אקראיים. השימוש ב-SCALE_MULTIPLIER מבטיח שהמהירויות ההתחלתיות יתאימו לרזולוציית המכשיר.

תרשים זרימה (סדר פעולות):

- i. הגדרת מיקום התחלה ושינוי הסטטוס ל-active.
- ii. הגרלת מהירות אופקית (velX) ואנכית (velY).
- iii. הגרלת מהירות סיבוב (rotVel) וקצב דעיכה (fadeSpeed).
- iv. איפוס שקיפות מלאה (255) וטיימר החיים.

קוד:

```
public void init(float startX, float startY, Bitmap img) {
```



```
this.x = startX;
this.y = startY;
this.img = img;
this.active = true;
this.alpha = 255;
this.rotation = r.nextFloat() * 360;
// Randomized velocities scaled by the engine's multiplier
this.velX = (r.nextFloat() - 0.5f) * 100f * SCALE_MULTIPLIER;
this.velY = (r.nextFloat() - 0.8f) * 100f * SCALE_MULTIPLIER;
this.rotVel = (r.nextFloat() - 0.5f) * 720f;

this.fadeSpeed = 300f + r.nextInt(200);
this.timeBeforeFade = 0.15f + r.nextFloat() * 0.1f;
this.lifeTimer = 0;
this.paint.setAlpha(255);
}
```

● public void update(double delta)

- הפעולה מיישמת חוקי תנועה שוות תאוצה מבוססי זמן (delta). היא מעדכנת את המיקום לפי המהירות, ואת המהירות האנכית לפי כוח הכבידה. השימוש ב-delta מבטיח שהנפילה והסיבוב ייראו חלקים ובאותה מהירות בכל קצב פריימים (FPS independent). בסיום, היא מחשבת את הירידה בשקיפות ומכבה את החלקיק כשהוא הופך לשקוף לחלוטין.

○ קוד:

```
public void update(double delta) {
    if (!active) return;
    float dt = (float) delta;
    lifeTimer += dt;
    x += velX * dt;
    y += velY * dt;
    velY += GRAVITY * dt;
    rotation += rotVel * dt;
    // Fade logic
    if (lifeTimer > timeBeforeFade) {
        alpha -= fadeSpeed * dt;
        if (alpha <= 0) {
            alpha = 0;
            active = false;
        }
        paint.setAlpha((int) alpha);
    }
}
```

● public void draw(Canvas c)

- הפעולה משתמשת ב-c.save() ו-c.restore() כדי לבצע סיבוב של החלקיק סביב צירו מבלי להשפיע על שאר האלמנטים במפה. הציור מתבצע עם היסט של חצי מרוחב/גובה התמונה כדי להבטיח שהסיבוב יקרה סביב מרכז השבר.

○ קוד:

```
public void draw(Canvas c) {
    if (!active || img == null) return;
    c.save();
    c.translate(x, y);
    c.rotate(rotation);
    c.drawBitmap(img, -img.getWidth() / 2f, -img.getHeight() / 2f, paint);
    c.restore();
}
```



מחלקת StaticObject

מיקום: com.example.tutorialgame.entities.foregrounds.statics.StaticObject

תפקיד המחלקה: היא מהווה את הייצוג הגנרי והמאוחד של כל האובייקטים הסטטיים והדוממים בעולם המשחק (בניינים, גדרות, פריטים, עציצים וכו'). היא יורשת מ-Entity (ולכן יש לה קיום פיזי בעולם) ומממשת את StaticEntity (ולכן ניתן לצייר אותה). תפקידה המרכזי הוא ליישם את עקרון ההפרדה בין לוגיקה לנתונים:

- **הלוגיקה:** המחלקה מכילה, במקום אחד בלבד, את כל הלוגיקה הקשורה לחישוב מיקום ה-hitbox (בהתחשב בגובה הגג) וחישוב מיקום הציור (בהתחשב במצלמה).
 - **הנתונים:** היא אינה יודעת דבר על האובייקט הספציפי שהיא מייצגת. במקום זאת, היא מקבלת בבנאי כל אובייקט המקיים את חוזה הממשק StaticObjectData (למשל, Buildings.HOUSE_ONE או Fences.PILLAR_SMALL) ושואבת ממנו את הנתונים הדרושים לה (תמונה, מידות וכו').
- ארכיטקטורה זו, המכונה עיצוב מונחה-נתונים (Data-Driven Design), מפשטת דרמטית את הקוד, מונעת שכפולים, ומקלה על הוספת סוגי אובייקטים חדשים בעתיד.

הסבר על תכונות המחלקה:

- `private final StaticObjectData objectData`
- תחום הכרה: `private final StaticObjectData`
- תפקיד ושימוש: זוהי התכונה המרכזית. היא מחזיקה רפרנס לאובייקט ה-enum הספציפי (Buildings.HOUSE_ONE למשל) שממנו נוצר ה-StaticObject. הרפרנס הוא מסוג הממשק (StaticObjectData), ולכן המחלקה יכולה להחזיק כל סוג של enum שמקיים את החוזה. היא משתמשת בו כדי לקבל את כל הנתונים הדרושים לה: תמונה, מידות hitbox, וגובה הגג.

הסבר פעולות מרכזיות:

- `(public StaticObject(PointF pos, IStaticObjectData objectData, boolean isCollider` (בנאי)
- הבנאי מקבל את מיקום האובייקט במפה (pos), את אובייקט הנתונים מה-enum (objectData), ודגל הקובע אם יש לו התנגשות. הוא קורא לבנאי של Entity (מחלקה האב) ומחשב את מיקום ה-hitbox האמיתי. החישוב לוקח את מיקום האובייקטים כפי שהוא מופיע בקובץ המפה (Tiled), ומוסיף לו את ההיסט של גובה הגג (hitboxRoof) כדי שה-hitbox יתחיל רק מהבסיס של האובייקט, ולא מהקצה העליון של התמונה.

○ קוד:

```
public StaticObject(PointF pos, StaticObjectData objectData, boolean isCollider) {
    super(new PointF(pos.x, pos.y + objectData.getHitboxRoof()),
          objectData.getHitboxWidth(),
          objectData.getHitboxHeight(),
          isCollider);
    this.objectData = objectData;
}
```

- `public StaticObjectData getObjectData()`

- זוהי מתודת "פתח מילוט" (escape hatch) חשובה. אף על פי שאין בה שימוש בתוך המחלקה, היא חיונית לקוד החיצוני. היא מאפשרת לקוד שמחזיק StaticObject "לשבור" את הגנריות ולקבל גישה חזרה ל-enum המקורי. זה נחוץ כדי להשתמש במתודות ייחודיות של enum ספציפי, כמו `getDoorwayPoint()` של Buildings, שאינן חלק מהממשק הכללי.

○ קוד:

```
public StaticObjectData getObjectData() {
    return objectData;
}
```



}

● • `getDrawX()` ו-`getDrawY()`

- אלו הן המתודות המממשות את חוזה הציור של הממשק `StaticEntity`. תפקידן הוא לחשב את קואורדינטת ה-X וה-Y המדויקת שבה יש להתחיל לצייר את התמונה של האובייקט על המסך. החישוב לוקח את מיקום ה-Hitbox בעולם המשחק ומתאים אותו למערכת הצירים של המסך על ידי הוספת ההיסט (offset) של המצלמה.
- **לוגיקה ייחודית ב-`getDrawY()`** מתודה זו גם מפחיתה את ערך `hitboxRoof` מהתוצאה. זהו תיקון קריטי שמבטיח שהתמונה המלאה של האובייקט (כולל הגג) תצויר במקום הנכון, בעוד שה-Hitbox עצמו מתחיל רק מבסיס האובייקט. זה מה שמאפשר לדמויות להיראות כאילו הן "עוברות מאחורי" האובייקט.

○ קוד:

```
@Override
public float getDrawX() {
    return getHitBox().left;
}
@Override
public float getDrawY() {
    return getHitBox().top - objectData.getHitboxRoof();
}
```



מחלקת Coin

מיקום: com.example.tutorialgame.entities.foregrounds.Coin

תפקיד המחלקה: המחלקה מייצגת מטבע איסוף בעולם המשחק. תפקידה לנהל את מחזור החיים המלא של המטבע: מהופעתו במפה עם אפקט קפיצה אקראי, דרך זיהוי אינטראקציית איסוף על ידי השחקן, ועד לאנימציות "טיסה" קולנועית לעבר מונה המטבעות בממשק המשתמש (HUD).

היא אחראית על:

1. ניהול משאבים חסכוני: שימוש בתבנית Flyweight לטעינת משאבים גרפיים פעם אחת בלבד בזיכרון הסטטי עבור כלל המטבעות.
2. סימולציה פיזיקלית: ניהול גובה המטבע (קפיצה וריחוף) ואנימציות סיבוב מבוססת רכיבים.
3. ניהול מרחבי: ביצוע המרה מתמטית בין קואורדינטות "עולם" (מיקום במפה) לקואורדינטות "מסך" (מיקום פיקסלי אבסולוטי) בזמן האיסוף לצורך רינדור ממשק מדויק

הסבר על תכונות המחלקה:

- `private static final Bitmap[] staticCoinSprites`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מערך סטטי המאחסן את פריימי האנימציה של המטבע בזיכרון ה-RAM. בזכות היותו סטטי, כל המטבעות חולקים את אותן התמונות לחיסכון בזיכרון.
- `private static final Bitmap staticShadowSprite`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: תמונת הצל המשותפת לכל המטבעות. נטענת ומעובדת פעם אחת בלבד בעת טעינת המחלקה לראשונה דרך ה-`BitmapManager`.
- `private final AnimationComponent animation`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רכיב האחראי על תזמון החלפת הפריימים ליצירת אשליה של מטבע מסתובב.
- `private final JumpComponent jumpComponent`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רכיב המנהל את ציר הגובה (Elevation) של המטבע. משמש לקפיצה הראשונית ולאפקט הריחוף לאחר מכן.
- `private PointF screenPosition`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: שומר את המיקום המדויק על המסך (בפיקסלים) ברגע האיסוף. משמש כנקודת הייחוס לתנועה לעבר הממשק ללא תלות במצלמה.
- `private boolean isHovering`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: דגל המציין אם המטבע סיים את קפיצת ה"לידה" שלו ועבר למצב ריחוף עדין מעל הקרקע.

הסבר פעולות מרכזיות:

- `public void pickCoin(Player player)`
 - הפעולה מנהלת את רגע המעבר הלוגי מעולם המשחק למערכת הממשק. היא בודקת התנגשות, ובמידה והתרחשה, היא דוגמת את מיקום המסך הנוכחי ומעבירה את המטבע למצב "טיסה" לעבר ה-HUD.
 - קוד:

```
public void pickCoin(Player player) {
    if (!active || !RectF.intersects(player.getProjectedHitBox(), hitBox)) return;
```



```

active = false;
animation.setSpeed(Math.max(1, animation.getSpeed() - 2));

// Capture current SCREEN position including camera offset and zoom
float zoom = CameraManager.getTempZoom();
this.screenPosition = new PointF(
    (hitBox.centerX() + CameraManager.getOffsetX()) * zoom,
    (hitBox.centerY() + CameraManager.getOffsetY()) * zoom
);

SoundManager.getInstance(BaseActivity.getContext()).playRndPitchSfx(R.raw.sfx_coin_collected);
MyApp.getCosmetic().addCoin();
}

```

● private void drawScreenCord(Canvas c)

- מציירת את המטבע במיקום אבסולוטי ביחס למסך המשתמש בעת הטיסה לעבר ה-UI. מכיוון שכל הציור בעולם המשחק מתבצע תחת התמרת המצלמה (זום והזזה), כדי לצייר אובייקט במיקום מסך קבוע, הפעולה מבצעת "חישוב הפוך" המבטל את השפעת המצלמה על הקנבס ומאפשר למטבע לטוס ליעדו בצורה חלקה מול עיני השחקן.

○ קוד:

```

private void drawWorldCord(Canvas c) {
    drawShadow(c);
    c.drawBitmap(staticCoinSprites[animation.getAniIndex()],
        hitBox.left,
        hitBox.top - jumpComponent.getElevation(),
        null);
}

```

בסיס נתונים (Firebase Firestore)

המחלקה משתמשת במסמך ה-CosmeticDoc השמור בענן כדי לנהל את רכוש השחקן.

- סקירת המידע: המידע נשמר בשדה coins (מסוג מספר) בתוך מסמך הקוסמטיקה האישי של המשתמש ב-Firebase. פעולות:
- טעינה: המידע נטען בעת עליית האפליקציה דרך ה-UserDataManager וזמין לגישה מהירה דרך MyApp.getCosmetic().
- שמירה/עדכון: בעת איסוף מטבע, נקראת הפעולה addCoin() המעדכנת את הערך המקומי בזיכרון ומבצעת פקודת עדכון (Update) לשרתי Firebase לסנכרון קבוע.

**מחלקת HealthComponent**

מיקום: com.example.tutorialgame.components.HealthComponent

תפקיד המחלקה: מחלקה זו מהווה רכיב לוגי האחראי על ניהול משאבי החיים (Health) של הישויות במשחק. היא מרכזת את כל החישובים הקשורים לקבלת נזק, ריפוי ומצב חיות (חי/מת), ומספקת משוב ויזואלי מיידי דרך אפקט "הבהוב" (Flash) בעת פגיעה. המחלקה פועלת בצורה מנותקת מהגרפיקה הממשית של הדמות, אך מספקת את הנתונים הנדרשים לרכיבי הרינדור וה-UI כדי להציג את מצב השחקן וכל דמות אחרת בצורה מדויקת.

היא אחראית על:

1. ניהול משאבים: מעקב אחר ערכי החיים הנוכחיים והמקסימליים של הישות.
2. לוגיקת פגיעה וריפוי: חישוב שינויי חיים תוך הבטחת עקביות (מניעת חריגה מהמקסימום או מתחת לאפס).
3. סימולצית משוב ויזואלי: ניהול ערך השקיפות (Alpha) של אפקט הפלאש המופעל בעת פגיעה.

הסבר על תכונות המחלקה:

- `private int maxHealth`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: הגדרת קיבולת החיים המקסימלית של הישות.
- `private int currentHealth`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: המשתנה הלוגי המרכזי המציין את המצב הבריאותי הרגעי של הישות.
- `private int prevHealth`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: שומר את ערך החיים שהיה לפני קבלת הנזק האחרון, משמש לחישובי עזר.
- `private int lastFlashHeartIndex`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: שומר את האינדקס של ה"לב" הספציפי שנפגע (בחישוב של יחידות 100). תכונה זו קריטית עבור ה-UI כדי לדעת איזה לב אמור להבהב במסך השחקן.
- `private float flashAlpha`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: ערך בטווח 0-1 המייצג את עוצמת אפקט הפלאש הלבן על הדמות. הערך דועך אוטומטית לאורך זמן.
- `private static final float FLASH_DURATION_SEC`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: קבוע המגדיר את משך הזמן (בשניות) של אפקט ההבהוב הוויזואלי.

הסבר פעולות מרכזיות:

- `public void takeDamage(int damage)`
 - הפעולה מפחיתה מכמות החיים של הישות ומפעילה את אפקטי הפגיעה.
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקת תקינות: מוודא שהדמות אינה מתה והנזק שהתקבל חיובי.
 - ii. שמירת מצב קודם: עדכון `prevHealth` לערך הנוכחי לפני השינוי.
 - iii. עדכון חיים: הפחתת הנזק מהחיים ושימוש ב-`Math.max` כדי למנוע ירידה מתחת לערך 0.



- iv. חישוב אינדקס לב: קביעת המיקום היחסי של הלב שנפגע (עבור תצוגת ה-HUD) על ידי חילוק ב-100.
- v. הפעלת משוב: קריאה ל-`startFlash` להתחלת האנימציה הוויזואלית.

קוד: ○

```
public void takeDamage(int damage) {
    if (isDead() || damage <= 0) return;

    this.prevHealth = this.currentHealth;
    this.currentHealth = Math.max(0, this.currentHealth - damage);

    lastFlashHeartIndex = (prevHealth - 1) / 100;
    startFlash();
}
```

● `public void heal(int amount)`

- הפעולה מוסיפה נקודות חיים לישות עד למקסימום המוגדר. בדומה ללוגיקת הנוק, הפעולה מוודאת שהדמות בחיים והערך חיובי. השימוש ב-`Math.min(maxHealth, ...)` מבטיח שהריפוי לעולם לא יעבור את רף החיים המקסימלי שהוגדר לישות בבנאי או בהגדרות השחקן.

קוד: ○

```
public void heal(int amount) {
    if (isDead() || amount <= 0) return;
    this.currentHealth = Math.min(maxHealth, this.currentHealth + amount);
}
```

● `public void update(double delta)`

- מנהלת את הדעיכה של אפקט הפלאש הוויזואלי לאורך זמן (Frame-rate independent). הפעולה מחסירה מה-`flashAlpha` ערך יחסי לזמן שעבר (`delta`) חלקי משך האנימציה הכולל. בכך היא מייצרת דעיכה חלקה וליניארית של ההבהוב הלבן, ללא תלות בקצב הפריימים של המכשיר עליו המשחק רץ.

קוד: ○

```
public void update(double delta) {
    if (flashAlpha > 0f) {
        flashAlpha -= (float) (delta / FLASH_DURATION_SEC);
        if (flashAlpha < 0f) flashAlpha = 0f;
    }
}
```



מחלקת StaminaComponent

מיקום: com.example.tutorialgame.components.StaminaComponent

תפקיד המחלקה: מחלקה זו מהווה רכיב לוגי האחראי על ניהול משאב הסטמינה (אנרגיה) של הישויות במשחק. היא מנהלת את מאזן הכוחות של הדמות על ידי הגבלת פעולות מאמץ (כמו ריצה, קפיצה או תקיפה) וביצוע סימולציה של התחדשות אוטומטית לאורך זמן. המחלקה מיישמת מנגנון "נעילת תשישות" (Exhaustion Lock) המונע מהשחקן לבצע פעולות עד להתאוששות מינימלית, ובכך מוסיפה רובד אסטרטגי לניהול הקרב והתנועה.

היא אחראית על:

1. בקרת צריכה: ניהול הפחתת אנרגיה עבור פעולות שונות ומניעת חריגה מתחת לאפס.
2. תחדשות אוטומטית (Regeneration): חישוב העלאת רמת האנרגיה בזמן מנוחה בהתבסס על זמן אמת (Delta-Time).
3. יהול מצב תשישות: הפעלת דגל חסימה כאשר האנרגיה אוזלת, ושחרורו רק לאחר הגעה לסף התאוששות מוגדר (50%).
4. משוב ויזואלי: ניהול ערך שקיפות (Alpha) עבור אפקט "הבהוב" המדווח ל-UI על צריכת משאבים.

הסבר על תכונות המחלקה:

- `private int maxStamina`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: מגדיר את קיבולת האנרגיה המקסימלית של הדמות.
- `private float currentStamina`
 - תחום הכרה: `private`.
 - פקיד ושימוש: המשתנה הלוגי המרכזי המייצג את רמת האנרגיה הרגעית. המשתנה הוא מסוג `float` כדי לאפשר דיוק מרבי בחישובי התחדשות זעירים המתרחשים בכל פריים.
- `private float cooldownTimer, cooldownDuration`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: ה-`Duration` קובע כמה זמן (בשניות) על הדמות להמתין לאחר פעולה לפני שהאנרגיה תתחיל להתחדש. ה-`Timer` עוקב אחר הזמן שחלף מאז השימוש האחרון.
- `private boolean staminaLock`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: דגל בטיחות (Exhaustion). כאשר הוא פעיל, השחקן אינו יכול לבצע פעולות הצורכות סטמינה, גם אם קיימת כמות קטנה של אנרגיה במיכל.
- `private float flashAlpha`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: ערך דועך (1.0 עד 0.0) המשמש את רכיבי `ui` כדי להציג אנימציות מתאימות.

הסבר פעולות מרכזיות:

- `public void useStamina(int amount)`
 - הפעולה מחסירה כמות מוגדרת של אנרגיה ומאפסת את זמני ההמתנה. הפעולה בודקת תחילה אם הישות אינה נמצאת במצב "נעול" (תשישות). אם הפעולה מאושרת, היא מעדכנת את ערך ה-`prevStamina` (לצורך חישובי UI), מפחיתה את האנרגיה, ומפעילה את אפקט הפלאש הויזואלי. בסיום, היא מאפסת את ה-`cooldownTimer` כדי להשהות את ההתחדשות האוטומטית.
 - **תרשים זרימה (סדר פעולות):**



- i. בדיקת תקינות: האם כמות הצריכה חיובית והסטמינה אינה נעולה?
- ii. שמירת מצב נוכחי ל-`prevStamina`.
- iii. ביצוע החסרה: עדכון `currentStamina` תוך הגנה על ערך מינימום 0.
- iv. הפעלת משוב: השמת ערך 1.0 ב-`flashAlpha`.
- v. איפוס השהייה: השמת ערך 0 ב-`cooldownTimer`.

קוד: ○

```
public void useStamina(int amount) {
    if (amount <= 0 || staminaLock) return;

    this.prevStamina = (int) this.currentStamina;
    this.currentStamina = Math.max(0, this.currentStamina - amount);

    this.flashAlpha = 1f;
    this.cooldownTimer = 0f; // Reset cooldown timer on use
}
```

● `public void update(double delta)`

- מנהלת את מצב הרכיב וסימולציית ההתחדשות בכל פריים. הפעולה מחולקת לשלושה שלבים.
- **תרשים זרימה (סדר פעולות):**
 - i. ניהול תשישות: אם האנרגיה הגיעה ל-0, הרכיב ננעל. השחרור מהנעילה יתבצע רק כאשר האנרגיה תתחדש ותעבור את רף ה-50%.
 - ii. דעיכת פלאש: עדכון ערך ה-`flashAlpha` כלפי מטה לפי הזמן שחלף.
 - iii. התחדשות: קידום טיימר ההמתנה. אם עבר זמן הצינון, האנרגיה תעלה בהדרגה לפי קבוע ה-`REGEN_RATE` כפול ה-`delta`.

קוד: ○

```
public void update(double delta) {
    // 1. Manage Stamina Lock (Exhaustion Logic)
    if (currentStamina <= 0) {
        staminaLock = true;
    } else if (staminaLock && currentStamina >= maxStamina * 0.5f) {
        // Unlock only when recovered to 50%
        staminaLock = false;
    }

    // 2. Handle Visual Flash Decay
    if (flashAlpha > 0f) {
        flashAlpha -= (float) (delta / FLASH_DECREASE_DURATION);
        if (flashAlpha < 0f) flashAlpha = 0f;
    }

    // 3. Handle Cooldown and Regeneration
    if (currentStamina < maxStamina) {
        cooldownTimer += (float) delta;
        if (cooldownTimer >= cooldownDuration) {
            this.currentStamina = Math.min(maxStamina, currentStamina + (float) (REGEN_RATE *
delta));
        }
    }
}
```

מחלקת **JumpComponent**

מיקום: com.example.tutorialgame.components.JumpComponent

תפקיד המחלקה: מחלקה זו מהווה רכיב פיזיקלי האחראי על ניהול התנועה בציר הגובה (Elevation) עבור ישויות במשחק. היא מדמה כוח כבידה ותנע אנכי, ומאפשרת לישויות לבצע קפיצות, נפילות ואפקטים של ריחוף בצורה ריאליסטית. הרכיב מפריד את חישובי הגובה מהמיקום הדו-ממדי (X,Y) של הישות על המפה, ובכך מאפשר יצירת אשליה של עומק וגובה בעולם דו-ממדי.

היא אחראית על:

1. סימולצית כבידה: חישוב שינוי המיקום האנכי לאורך זמן בהתבסס על קבוע גרביטציה.
2. ניהול מצבי קפיצה: מעקב אחר סטטוס הישות (באוויר או על הקרקע) ומניעת כפל קפיצות.
3. חישוב פרמטרים קינמטיים: המרת דרישות עיצוביות (גובה רצוי וזמן הגעה לשיא) לערכים פיזיקליים של מהירות התחלתית ותאוצה.
4. סנכרון ויזואלי: אספקת נתונים על אחוז השלמת הקפיצה (Fraction) לצורך סנכרון אפקטים אחרים, כגון שינוי שקיפות הצל ב-CharacterRenderer.

הסבר על תכונות המחלקה:

- private float elevation
 - תחום הכרה: private.
 - תפקיד ושימוש: המשתנה המייצג את המרחק הנוכחי של הישות מהקרקע (בפיקסלים). משמש את ה-Renderer להסטת הציוור של הדמות כלפי מעלה.
- private float vz
 - תחום הכרה: private.
 - תפקיד ושימוש: המהירות האנכית הרגעית של הישות (Vertical Velocity). ערך חיובי מסמל עלייה, וערך שלילי מסמל נפילה.
- private float gravity
 - תחום הכרה: private.
 - תפקיד ושימוש: קבוע התאוצה המושך את המהירות האנכית כלפי מטה בכל פריים.
- private float maxElevation
 - תחום הכרה: private.
 - תפקיד ושימוש: גובה השיא של הקפיצה (בפיקסלים). משמש לחישוב יחסי ההתקדמות של הישות באוויר.
- private long jumpStartTime
 - תחום הכרה: private.
 - תפקיד ושימוש: שומר את זמן תחילת הקפיצה האחרונה (במילישניות).

הסבר פעולות מרכזיות:

- public void update(double deltaSeconds)
 - מבצעת את האינטגרציה הפיזיקלית של התנועה בכל פריים של המשחק. הפעולה בודקת אם הישות באוויר. אם כן, היא מעדכנת את המהירות האנכית לפי כוח הכבידה ואז מקדמת את הגובה לפי המהירות החדשה. הפעולה כוללת "מנגנון בלימה" בעת נחיתה המאפס את המהירות והגובה ברגע שהישות נוגעת בקרקע (`elevation <= 0f`).
 - קוד:

```
public void update(double deltaSeconds) {
    if (!isAirborne()) return;

    vz += gravity * (float) deltaSeconds;
    elevation += vz * (float) deltaSeconds;

    if (elevation <= 0f) {
```



```

    elevation = 0f;
    vz = 0f;
}
}

```

• public void startJump()

- פעולה המפעילה את כוח הזינוק הראשוני ליצירת קפיצה. הפעולה מאתחלת את המהירות האנכית (vz) לערך המהירות המחושב מראש. היא כוללת בדיקת הגנה קריטית: הקפיצה תתבצע רק אם הישות נמצאת כרגע על הקרקע, ובכך מונעת את תופעת ה"קפיצה הכפולה" באוויר.

○ תרשים זרימה (סדר פעולות):

i. בדיקת תנאי: האם הגובה הנוכחי שווה או קטן מ-0?

- אם כן: השמת jumpVelocity לתוך vz ועדכון jumpStartTime.
- אם לא: התעלמות מהקריאה.

○ קוד:

```

public void startJump() {
    if (elevation <= 0f) {
        this.vz = jumpVelocity;
        this.elevation = 0f;
        jumpStartTime = System.currentTimeMillis();
    }
}

```

• public float getJumpFraction()

- מחזירה את אחוז ההתקדמות של הישות במסלול הקפיצה שלה. הפעולה מחשבת את היחס שבין הגובה הנוכחי לבין גובה השיא. הערך המוחזר הוא מנורמל (בין 0.0 ל-1.0), מה שמאפשר שימוש בטוח עבור רכיבי גרפיקה.

○ קוד:

```

public float getJumpFraction() {
    if (maxElevation <= 0f) return 0f;
    float perc = elevation / maxElevation;
    return Math.max(0f, Math.min(1f, perc));
}

```

• private void initFromTiles(float desiredTiles, float timeToApexSec)

- פעולת עזר המתרגמת דרישות עיצוביות לנוסחאות פיזיקליות. הפעולה מחשבת את קבועי הפיזיקה הנדרשים, כך שהישות תגיע בדיוק לגובה האריחים המבוקש בזמן המדויק שהוגדר להגעה לשיא. החישוב מבוסס על נוסחאות תנועה שוות תאוצה.

○ קוד:

```

private void initFromTiles(float desiredTiles, float timeToApexSec) {
    float h = desiredTiles * TILE_SIZE;
    // Formula: h = (v0^2) / (2g) and t = v0 / g => g = 2h / t^2 and v0 = gt
    float g = 2f * h / (timeToApexSec * timeToApexSec);
    float v = 2f * h / timeToApexSec;

    this.gravity = -g;
    this.jumpVelocity = v;
    this.maxElevation = h;
}

```

**מחלקת MovementComponent**

מיקום: com.example.tutorialgame.components.MovementComponent

תפקיד המחלקה: חלקה זו מהווה את ה"מנוע הפיזיקלי" של הדמות. היא אחראית על ניהול התנועה במרחב הדו-ממדי, זיהוי התנגשויות (Collision Detection) וסימולציה של כוחות הדף (Knockback). המחלקה מפרידה את חוקי הפיזיקה והמרחב מהלוגיקה של הדמות, ובכך מאפשרת התנהגות עקבית ואמינה של תנועה, קפיצה והשפעות קרב לכלל הישויות במשחק. הודות למנגנוני "החלקה" מתקדמים, הרכיב מבטיח תנועה חלקה גם בסביבות צפופות ומורכבות.

היא אחראית על:

1. בקרת תנועה חכמה (Wall Sliding): יישום לוגיקה המאפשרת לדמות להחליק לאורך קירות אם התנועה האלכסונית חסומה.
2. זיהוי התנגשויות רב-שכבתי: בדיקה מול גבולות המפה, אובייקטים סטטיים, דמויות אחרות ואריחים חסומים דרך CollisionUtils.
3. מנוע הדף פיזיקלי (Knockback Engine): חישוב דעיכת תנע המבוסס על חיכוך, מסת היעד ועוצמת המכה.
4. אינטגרציית גובה: אישור מעבר "מעל" אובייקטים כאשר הדמות נמצאת בגובה מסוים (קפיצה).

הסבר על תכונות המחלקה:

- private float kbX, kbY
 - תחום הכרה: private.
 - תפקיד ושימוש: וקטור המהירות הנוכחי של ההדף. ערכים אלו מציינים כמה פיקסלים הדמות נדחפת בכל פריים בעקבות פגיעה.
- private static final float KB_FRICTION = 0.85f
 - תחום הכרה: private static final.
 - תפקיד ושימוש: קבוע החיכוך המדמה התנגדות של הקרקע. קובע כמה מהר תדעך מהירות ההדף עד לעצירה מוחלטת.
- private final Character self
 - תחום הכרה: private final.
 - תפקיד ושימוש: רפרנס לדמות המחזיקה ברכיב. משמש לשליפת נתוני מסה, כיוון וגובה בזמן אמת.
- private final Point[] tileCords
 - תחום הכרה: private final.
 - תפקיד ושימוש: מערך נקודות המייצג את ארבע פינות הדמות במונחי אריחים (Tiles). השימוש במערך קבוע מונע הקצאות זיכרון חוזרות בכל פריים (GC optimization).

הסבר פעולות מרכזיות:

- public void applyMovement(float dx, float dy, GameMap gameMap)
 - הפעולה המרכזית המבצעת את הזזת הדמות בפועל. פעולה זו מיישמת את לוגיקת ה-Sliding. תחילה היא מנסה להזיז את הדמות בוקטור המלא (X ו-Y יחד). אם התנועה חסומה, היא מנסה להזיז את הדמות רק בציר X ואז רק בציר Y בנפרד. טכניקה זו מונעת את תופעת ה"היתקעות" בקירות בזמן הליכה באלכסון ומייצרת תחושת שליטה חלקה ואינטואיטיבית.
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקה: האם קיים הדף פעיל? (אם כן - ביטול תנועה ידנית).
 - ii. ניסיון תנועה מלאה (dx, dy):
 1. הצליח: הזזת ה-Hitbox וסיום.
 2. נכשל: ניסיון תנועה אופקית (0, dx).
 3. נכשל: ניסיון תנועה אנכית (dy, 0).



קוד: ○

```
public void applyMovement(float dx, float dy, GameMap gameMap) {
    if (isKnockbackActive() || gameMap == null) return;

    if (canMoveTo(dx, dy, gameMap)) {
        selfHitbox.offset(dx, dy);
    } else {
        if (dx != 0 && canMoveTo(dx, 0, gameMap)) selfHitbox.offset(dx, 0);
        if (dy != 0 && canMoveTo(0, dy, gameMap)) selfHitbox.offset(0, dy);
    }
}
```

public float applyKnockback(float weaponForce, float attackerMass, int totalDamage, float sourceX, float sourceY) •

- מפעילה כוח פיזיקלי על הדמות ממקור חיצוני. הפעולה מיישמת נוסחת דחף משוקללת. היא מחשבת את יחס המסות בין התוקף ליעד (massRatio) ומשקללת את עוצמת הנוק (damageFactor). ככל שהנוק גבוה יותר והתוקף כבד יותר מהיעד, כך ההדף יהיה חזק יותר. המהירות מוזרקת לוקטור ה-kbX/Y ומתחילה דעיכה ב-update.

קוד: ○

```
public float applyKnockback(float weaponForce, float attackerMass, int totalDamage, float sourceX, float sourceY) {
    if (weaponForce <= 0) return 0;

    float targetMass = self.getGameCharType().getMass();
    float finalForce = weaponForce * (1f + (totalDamage / 100f)) * (attackerMass / targetMass);
    float dx = selfHitbox.centerX() - sourceX;
    float dy = selfHitbox.centerY() - sourceY;
    float dist = (float) Math.hypot(dx, dy);

    if (dist > 0) {
        float startingVelocity = finalForce * (1f - KB_FRICTION);
        this.kbX += (dx / dist) * startingVelocity;
        this.kbY += (dy / dist) * startingVelocity;
    }
    return finalForce;
}
```

public void update(double delta, GameMap gameMap) •

- מנהלת את תנועת ההדף והחיכוך בזמן אמת. אם קיים הדף פעיל, הפעולה מזיזה את הדמות תוך בדיקת התנגשויות. לאחר מכן, היא מכפילה את המהירות ב-frictionFactor המחושב לפי delta. שימוש ב-Math.pow מבטיח שההדף ידעך באותו קצב בדיוק בכל מכשיר, ללא תלות בקצב הפריימים (FPS independent).

קוד: ○

```
public void update(double delta, GameMap gameMap) {
    if (isKnockbackActive()) {
        float dx = (float) (kbX * delta * 60);
        float dy = (float) (kbY * delta * 60);

        if (canMoveTo(dx, dy, gameMap)) selfHitbox.offset(dx, dy);
        else cancelKnockback();

        float frictionFactor = (float) Math.pow(KB_FRICTION, delta * 60);
        kbX *= frictionFactor;
        kbY *= frictionFactor;
    }
}
```

**מחלקת CombatComponent**

מיקום: com.example.tutorialgame.components.CombatComponent

תפקיד המחלקה: מחלקה זו היא הרכיב האחראי על ריכוז וניהול כל היבטי הלחימה של הדמות. היא משמשת כיחידה עצמאית המנהלת את מיקום הנשק, חישובי טווח התקיפה (Hitboxes), הפעלת נזק משולב, ויישום חוקי פיזיקה בעת פגיעה (Knockback). בנוסף, הרכיב מתזמר את האפקטים הויזואליים והקוליים המלווים את הקרב, ובכך מפריד את המורכבות של עולם הלחימה ממחלקת הדמות הכללית.

היא אחראית על:

1. ניהול גיאומטריית לחימה: חישוב דינמי של מיקום וזווית הנשק במרחב הדו-ממדי בהתאם לכיוון המבט וגובה הדמות.
2. בקרה על טווח פגיעה: עדכון תיבת התקיפה (attackBox) בזמן אמת כדי להבטיח דיוק בבדיקת התנגשויות מול מטרות.
3. חישוב פיזיקה ונזק: שקלול עוצמת הדמות יחד עם נתוני הנשק ליצירת מכה אפקטיבית הכוללת הדף המושפע ממסה.
4. ניהול אפקטים וויזואליים (VFX): תזמון והצגה של אפקטי "הנפה" (Swing) או "פגיעה" (Impact) המסונכרנים עם תנועת הנשק.
5. משוב קולי: הפעלת צלילי קרב מרחביים המשתנים בהתאם לסוג הנשק ומיקום הפעולה.

הסבר על תכונות המחלקה:

- private final Character owner
 - תחום הכרה: private final.
 - תפקיד ושימוש: רפרנס לדמות המחזיקה ברכיב. משמש לשליפת נתונים קריטיים כמו כיוון המבט, גובה הקפיצה ומיקום ה-Hitbox הבסיסי.
- private Weapons weapon
 - תחום הכרה: private.
 - תפקיד ושימוש: אובייקט הנתונים של הנשק המצויד (חרב, פטיש וכו'). קובע את נזק הבסיס, עוצמת ההדף וסוג האפקט הויזואלי.
- private final RectF attackBox
 - תחום הכרה: private final.
 - תפקיד ושימוש: מלבן המגדיר את השטח הפיזי שבו הנשק יכול לפגוע במטרות. גודלו ומיקומו משתנים בכל פריים לפי כיוון הנשק.
- private final AnimationComponent effectAnimator
 - תחום הכרה: private final.
 - תפקיד ושימוש: רכיב פנימי המנהל את קצב החלפת הפריימים של האפקט הויזואלי (למשל הבזק הנפת החרב).
- private boolean effectActive
 - תחום הכרה: private.
 - תפקיד ושימוש: דגל המציין אם אפקט ויזואלי נמצא כרגע בתהליך ניגון פעיל.
- private float weaponRotation, effectRotation
 - תחום הכרה: private.
 - תפקיד ושימוש: זוויות הסיבוב (במעלות) המחושבות עבור ציור הנשק והאפקט הויזואלי, בהתאמה לכיוון הדמות.

הסבר פעולות מרכזיות:

- public void attack(Character target)
 - הפעולה המרכזית המבצעת את לוגיקת הפגיעה ביעד. כאשר מזוהה פגיעה, הפעולה מחשבת את הנזק הסופי על ידי חיבור כוח הדמות עם נזק הבסיס של הנשק. לאחר מכן,



היא מעבירה את הנזק ליעד ומפעילה את מערכת ההדף הפיזיקלית (`applyKnockback`). בחישוב ההדף, הפעולה שולחת את מסת התוקף ואת עוצמת המכה, מה שמאפשר ליעד לחשב את עוצמת הזינוק לאחור בצורה ריאליסטית.

קוד: ○

```
public void attack(Character target) {
    if (weapon == null || target == null || target.isDead()) return;

    onImpact();

    // 1. Total Damage = Base + Weapon
    int totalDamage = owner.getAttackDamage() + weapon.getBaseDamage();
    target.takeDamage(totalDamage, owner);

    // 2. Physics-based knockback
    target.getMovementComponent().applyKnockback(
        weapon.getKnockBackForce(),
        owner.getGameCharType().getMass(),
        totalDamage,
        owner.getHitBox().centerX(),
        owner.getHitBox().centerY()
    );
}
```

private void updateWeaponTransform() •

○ מחשבת את המיקום הגיאומטרי והזווית של הנשק בכל פריים. הפעולה משתמשת במבנה של switch-case על כיוון המבט של הדמות. היא מחשבת את ה-Offset המדויק (הסטה) של הנשק יחסית ליד הדמות, כולל התחשבות בגובה הקפיצה (`elevation`) כדי שהנשק יעלה וירד יחד עם הישות. בסיס, היא קובעת את זווית הסיבוב (0, 90, 180, 270 מעלות) לצורך ציור נכון של הנשק.

קוד: ○

```
private void updateWeaponTransform() {
    RectF ownerHitbox = owner.getHitBox();
    float elevation = owner.getElevation();

    switch (owner.getFaceDir()) {
        case UP:
            weaponPos.set(ownerHitbox.left - 0.5f * SCALE_MULTIPLIER, ownerHitbox.top -
                weapon.getHeight() - Y_DRAW_OFFSET - elevation);
            weaponRotation = 180;
            break;
        case DOWN:
            weaponPos.set(ownerHitbox.left + 0.5f * SCALE_MULTIPLIER, ownerHitbox.bottom -
                elevation);
            weaponRotation = 0;
            break;
        case LEFT:
            weaponPos.set(ownerHitbox.left - weapon.getHeight() - X_DRAW_OFFSET,
                ownerHitbox.bottom - weapon.getWidth() - 0.75f * SCALE_MULTIPLIER - elevation);
            weaponRotation = 90;
            break;
        case RIGHT:
            weaponPos.set(ownerHitbox.right + X_DRAW_OFFSET, ownerHitbox.bottom -
                weapon.getWidth() - 0.75f * SCALE_MULTIPLIER - elevation);
            weaponRotation = 270;
            break;
    }
}
```

public void draw(Canvas c) •



- רינדור שכבות הקרב על המסך. הפעולה מציירת את הנשק תוך שימוש ב-`c.rotate` לפי הזווית שחושבה ב-`update`. לאחר מכן, אם האפקט הויזואלי פעיל, היא משתמשת ב-Matrix כדי לבצע טרנספורמציה מורכבת: מירכוז האפקט, סיבובו לפי כיוון התקיפה, והצבתו בדיוק במרכז ה-`attackBox`.

○ קוד:

```
public void draw(Canvas c) {
    if (weapon == null || owner.isDead() || !owner.isAttacking()) return;
    update();

    // Draw weapon
    c.save();
    c.rotate(weaponRotation, attackBox.left, attackBox.top);
    c.drawBitmap(weapon.getWeaponInHandImg(),
        attackBox.left + wepAdjustLeft() + wepAdjustBottom(),
        attackBox.top + wepAdjustTop(), null);
    c.restore();

    // Draw visual effect (Swing/Impact)
    if (effectActive && weapon.getEffectType() != null) {
        Bitmap frame = weapon.getEffectType().getSprite(effectAnimator.getAniIndex());
        if (frame != null) {
            effectMatrix.reset();
            effectMatrix.postTranslate(-frame.getWidth() / 2f, -frame.getHeight() / 2f);
            effectMatrix.postRotate(effectRotation);
            effectMatrix.postTranslate(attackBox.centerX(), attackBox.centerY());
            c.drawBitmap(frame, effectMatrix, null);
        }
    }
}
```

**מחלקת EmoteComponent**

מיקום: com.example.tutorialgame.components.EmoteComponent

תפקיד המחלקה: מחלקה זו מהווה רכיב ויזואלי האחראי על ניהול והצגת "בועות רגש" (Emotes) מעל דמויות המשחק. תפקידה הוא להוסיף רובד נרטיבי ומשוב ויזואלי מיידי למצבי הרוח של הדמויות (כגון כעס, הפתעה או חיבה). המחלקה מנהלת את מחזור החיים של הבועה, החל מאנימציה הופעה מסוג Pop-in, דרך ריחוף עדין מעל ראש הדמות, ועד להעלמות אוטומטית לאחר זמן קצוב. הרכיב תוכנן לעבודה בסנכרון מלא עם הפיזיקה של הדמות, כולל התחשבות בגובה הקפיצה (Elevation) ובקנה המידה של המסך.

היא אחראית על:

1. ניהול תזמון (Timing): בקרה על משך הצגת הבועה בהתבסס על זמן אמת (Delta-Time).
2. אנימציה כניסה (Pop-in Effect): ביצוע Scaling מהיר ודינמי ברגע ההופעה ליצירת מראה מלוטש ומודרני.
3. סימולציה ריחוף: יישום פונקציית סינוס ליצירת תנועה אנכית עדינה המדמה ציפה של בועת דיבור.
4. דיוק מרחבי: חישוב ומיקום הבועה במרכז ה-Hitbox של הדמות תוך התאמה לרזולוציית המסך (SCALE_MULTIPLIER).

הסבר על תכונות המחלקה:

- private Emotes activeEmote
 - תחום הכרה: private.
 - תפקיד ושימוש: סוג הבועה המוצגת כרגע (נשלף מתוך ה-Enum הסטטי). אם הערך הוא null, הרכיב נחשב ללא פעיל.
- private float lifeTimer, displayDuration
 - תחום הכרה: private.
 - תפקיד ושימוש: מוני זמן בשניות. ה-lifeTimer עוקב אחר משך הופעת הבועה הנוכחית, וה-displayDuration קובע את הרף לסגירתה.
- private float currentScale
 - תחום הכרה: private.
 - תפקיד ושימוש: ערך ה-Scale הנוכחי של הבועה (0.0 עד 1.0). משמש ליצירת אפקט ה-"Pop" ברגע ההפעלה.
- private final Matrix drawMatrix
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט עזר לביצוע טרנספורמציות מורכבות (שינוי גודל והזזה) בבת אחת על הקנבס, מה שמשפר את יעילות הצירור.

הסבר פעולות מרכזיות:

- public void showEmote(Emotes emote, long durationMillis)
 - מפעילה בועת רגש חדשה מעל הדמות. הפעולה מאתחלת את מצב הרכיב: היא מאפסת את ה-Scale ל-0 (כדי להתחיל את האנימציה), קובעת את משך התצוגה המבוקש ומגדירה את הבועה הפעילה. קריאה חוזרת לפעולה זו תדרוס את הבועה הקודמת ותתחיל את התהליך מחדש.

○ קוד:

```
public void showEmote(Emotes emote, long durationMillis) {
    this.activeEmote = emote;
    this.lifeTimer = 0;
    this.displayDuration = durationMillis / 1000f;
    this.currentScale = 0f; // Reset for pop-in effect
    this.offsetY = targetOffsetY;
```



}

- `public void update(double delta)`

- מעדכנת את לוגיקת הזמן והאנימציה בכל פריים. הפעולה מקדמת את ה-lifeTimer. אם הבועה צריכה להופיע, היא מבצעת שני תהליכים במקביל: 1. קידום ליניארי מהיר של ה-currentScale עד להגעה לגודל מלא (1.0). 2. חישוב תנועת ריחוף גלית (Floating) באמצעות `Math.sin` סביב גובה המטרה. בסיום הזמן המוגדר, היא מאפסת את activeEmote להפסקת הרינדור.

- קוד:

```
public void update(double delta) {
    if (activeEmote == null) return;

    lifeTimer += (float) delta;

    if (lifeTimer >= displayDuration) {
        activeEmote = null;
    } else {
        // Smooth pop-in animation using scaling
        if (currentScale < 1.0f) {
            currentScale = Math.min(1.0f, currentScale + (float) (delta * 8.0));
        }

        // Gentle floating movement
        offsetY = targetOffsetY + (float) Math.sin(lifeTimer * 4.0) * (2 * SCALE_MULTIPLIER);
    }
}
```

- `public void drawEmote(Canvas c)`

- רינדור הבועה על המסך. הפעולה מחשבת את המיקום הסופי תוך התחשבות ב-Elevation של הדמות (כדי שהבועה "תקפוץ" יחד עם הדמות). היא משתמשת ב-Matrix כדי לבצע Scaling ממרכז הבועה (ולא מהפינה), מה שמעניק לאנימציה ה-Pop מראה טבעי. בסיום היא מציירת את ה-Bitmap המבוקש במיקום המחושב.

- קוד:

```
public void drawEmote(Canvas c) {
    if (!isActive() || activeEmote.getEmote() == null) return;

    float drawX = character.getHitBox().left + offsetX;
    float drawY = character.getHitBox().top - offsetY - character.getElevation() -
        SCALE_MULTIPLIER*3;

    drawMatrix.reset();
    // Scale from the center of the emote
    drawMatrix.postScale(currentScale, currentScale,
        activeEmote.getEmote().getWidth() / 2f,
        activeEmote.getEmote().getHeight() / 2f);
    drawMatrix.postTranslate(drawX, drawY);

    c.drawBitmap(activeEmote.getEmote(), drawMatrix, null);
}
```



מחלקת **AnimationComponent**

מיקום: `com.example.tutorialgame.components.AnimationComponent`

תפקיד המחלקה: מחלקה זו היא רכיב (Component) יסודי, אוניברסלי ורב-פעמי במנוע המשחק. תפקידה הבלעדי הוא לנהל את המצב והתזמון של רצף אנימציה. היא פועלת כ"מטרונום" או "טיימר" עבור כל ישות הזקוקה לאנימציה מבוססת-פריימים.

היא אחראית על:

1. הלוגיקה של קידום אינדקס הפריים (`aniIndex`) בקצב מבוקר (`speed`).

הסבר על תכונות המחלקה:

- `private int aniTick`
 - תחום הכרה: `private int`
 - תפקיד ושימוש: זהו מונה פנימי ("טיקים"). בכל פעם שמתודת ה-`update` נקראת (כלומר, בכל פריים של המשחק), המונה גדל ב-1. הוא משמש כדי להאט את קצב שינוי פריימים של האנימציה.
- `private int aniIndex`
 - תחום הכרה: `private int`
 - תפקיד ושימוש: זהו הערך החשוב ביותר שהרכיב חושף. הוא מייצג את האינדקס של הפריים הנוכחי ברצף האנימציה (למשל, מ-0 עד 3 באנימציה הליכה). קוד הרינדור ישתמש במספר זה כדי לבחור את התמונה הנכונה מה-Sprite Sheet.
- `private final int frameCount`
 - תחום הכרה: `private final int`
 - תפקיד ושימוש: מייצג את המספר הכולל של פריימים ברצף האנימציה. הוא מוגדר כ-`final` מכיוון שאורך האנימציה נקבע מראש ואינו משתנה בזמן ריצה. הוא משמש כדי לדעת מתי לאפס את `aniIndex` חזרה ל-0 ולהתחיל את הלופ מחדש.
- `private int speed`
 - תחום הכרה: `private int`
 - תפקיד ושימוש: קובע את מהירות האנימציה. זהו למעשה ערך סף - רק כאשר `aniTick` מגיע לערך `speed`, אינדקס הפריים `aniIndex` מתקדם. ערך `speed` גבוה יותר משמעותו אנימציה איטית יותר, מכיוון שידרשו יותר "טיקים" של המשחק כדי לעבור לפריים הבא.

הסבר פעולות מרכזיות:

- `public void update()`
 - זוהי הפעולה המרכזית והיחידה המכילה לוגיקה משמעותית. היא נובעת מלולאת המשחק הראשית ומסונכרנת איתה, מה שנקרא "מונעת על ידי הלולאה" (Loop-driven).
 - **תרשים זרימה (סדר פעולות):**
 - i. `aniTick` גדל ב-1.
 - ii. נבדק התנאי: `if (aniTick >= speed)`.
 - iii. אם התנאי מתקיים (הגיע הזמן להחליף פריים):
 1. `aniTick` מאופס ל-0.
 2. `aniIndex` גדל ב-1.
 3. נבדק תנאי נוסף - `if (aniIndex >= frameCount)`
 - a. אם התנאי מתקיים (הגענו לסוף האנימציה), `aniIndex` מאופס ל-0 והאנימציה מתחילה מהתחלה.



○ קוד:

```
public void update() {
    aniTick++;
    if (aniTick >= speed) {
        aniTick = 0;
        aniIndex++;
        if (aniIndex >= frameCount)
            aniIndex = 0;
    }
}
```

● public void resetAnimation()

○ מתודת עזר פשוטה המאפסת את מצב האנימציה להתחלה (aniTick ו-aniIndex מתאפסים). שימושית מאוד כאשר יש שינוי מצב של הדמות, למשל, כאשר היא עוצרת הליכה, ורוצים שהאנימציה הבאה תתחיל מהפריים הראשון.

○ קוד:

```
public void resetAnimation() {
    aniTick = 0;
    aniIndex = 0;
}
```



ממשק משתמש וגרפיקה (User Interface & Graphics) מערכת ה-UI המותאמת אישית שבנית מאפס עבור המשחק.

מחלקת **BitmapMethods** (ממשק)

מיקום: `com.example.tutorialgame.engine.interfaces.BitmapMethods`

תפקיד המחלקה: ממשק זה (Interface) מרכז פעולות עזר גנריות לעיבוד ושינוי גודל של תמונות (Bitmaps) בזמן ריצה. הוא מספק מימושי ברירת מחדל (Default methods) המבטיחים שכל תמונה שנטענת למנוע תעבור התאמה מדויקת לקנה המידה של המשחק (Scaling). הממשק מאפשר למחלקות המשתמשות בו לבצע פעולות הגדלה וצמצום תוך שמירה על איכות התמונה או שמירה על מראה "פיקסלי" נקי (Hard Edges), בהתאם לצורך האמנותי של האלמנט.

היא אחראית על:

1. התאמת קנה מידה גלובלי: סנכרון אוטומטי של תמונות גולמיות עם ה-`SCALE_MULTIPLIER` של המנוע.
2. שינוי גודל דינמי: מתן כלים להגדלה או הקטנה של תמונות לפי מקדם הכפלה (Multiplier) משתנה.
3. בקרת איכות הרינדור (Filtering): אפשרות לבחירה בין החלקת תמונה (Smooth) לבין שמירה על חדות פיקסלים (Clean) בעת שינוי גודל.

הסבר על תכונות המחלקה:

- `public static final BitmapFactory.Options options` : תחום הכרה: `public static final` (ברירת מחדל בממשק).
- תפקיד ושימוש: אובייקט הגדרות המשמש את ה-`BitmapFactory` בעת פענוח (Decoding) של קבצי תמונה. הוא מאפשר למנוע לשלוט באופן הטעינה של התמונות לזיכרון ה-RAM.

הסבר פעולות מרכזיות:

- `default Bitmap getScaledBitmap(Bitmap bitmap)` : פעולה בסיסית המבצעת הגדלה של ה-`Bitmap` שהתקבל לפי קבוע המערכת. היא מחזירה אובייקט תמונה חדש שגודלו הפיזי מותאם לרזולוציית התצוגה שחושה ב-`SplashActivity`.

קוד: ○

```
default Bitmap getScaledBitmap(Bitmap bitmap) {
    return Bitmap.createScaledBitmap(bitmap, bitmap.getWidth() *
GameConstants.Sprite.SCALE_MULTIPLIER, bitmap.getHeight() *
GameConstants.Sprite.SCALE_MULTIPLIER, false);
}
```

- `default Bitmap getMultiplyBitmapSmoth(Bitmap bitmap, double multiply)` : פעולה לשינוי גודל התמונה תוך הפעלת סינון Bilinear. השיטה משתמשת ב-Matrix כדי לבצע את ההכפלה. הפלט הוא תמונה מוחלקת, מה שמתאים בעיקר לאלמנטים גרפיים שאינם "פיקסל-ארט" או לאפקטים שזקוקים למעברים רכים.

קוד: ○

```
default Bitmap getMultiplyBitmapSmoth(Bitmap bitmap, double multiply) {
    Matrix matrix = new Matrix();
    matrix.setScale((float) multiply * GameConstants.Sprite.SCALE_MULTIPLIER, (float) multiply
* GameConstants.Sprite.SCALE_MULTIPLIER);
    // פנימי לעיגול שהורג העשורני בגודל חדש Bitmap לנו 'יוצר'
    return Bitmap.createBitmap(
        bitmap,
        0, 0,
        bitmap.getWidth(),

```



```
        bitmap.getHeight(),  
        matrix,  
        true // filter = true => 'פעיל' bilinear  
    );  
}
```

• default Bitmap getMultiplyBitmapClean(Bitmap bitmap, double multiply)

- זוהי הפעולה המועדפת עבור אלמנטים של Pixel-art. היא מבצעת את אותה הכפלה מתמטית, אך עם פרמטר `filter = false`. כתוצאה מכך, המנוע לא מנסה להחליק את הקצוות, והתמונה נשארת חדה וברורה גם לאחר הגדלה משמעותית, תוך שמירה על ה"שלמות" של כל פיקסל מקורי.

○ קוד:

```
default Bitmap getMultiplyBitmapClean(Bitmap bitmap, double multiply) {  
    Matrix matrix = new Matrix();  
    matrix.setScale((float) multiply * GameConstants.Sprite.SCALE_MULTIPLIER, (float) multiply  
* GameConstants.Sprite.SCALE_MULTIPLIER);  
    // פנימי לעיגול שהומר העשרוני בגודל חדש Bitmap לנו 'וצר'  
    return Bitmap.createBitmap(  
        bitmap,  
        0, 0,  
        bitmap.getWidth(),  
        bitmap.getHeight(),  
        matrix,  
        false // filter = true => 'פעיל' bilinear  
    );  
}
```




מחלקת CustomDialog

מיקום: com.example.tutorialgame.ui.dialogs.CustomDialog

תפקיד המחלקה: מחלקה אבסטרקטית זו מהווה את התשתית המרכזית ליצירת תיבות דו-שיח (Dialogs) מותאמות אישית. היא מנהלת את הנראות והלוגיקה של חלונות צפים, ותומכת בשלושה מצבי עבודה שונים: קבלת קלט מהמשתמש, אישור פעולה (כן/לא), והצגת התראות (OK בלבד). המחלקה מבטיחה אחידות ויזואלית בכל רחבי האפליקציה ומפרידה בין עיצוב ה-UI לבין הלוגיקה העסקית הממומשת במחלקות היורשות.

היא אחראית על:

1. ניהול מצבי תצוגה: הצגה דינמית של רכיבים (כותרת, הודעה, שדה קלט) בהתאם להגדרות ה-DialogKeys.
2. תמיכה בדיאלוגים מסוג "התראה": הוספת אפשרות להצגת כפתור אישור בודד (btn_ok) עבור הודעות מערכת או שגיאות.
3. בקרת אינטראקציה: ניהול מאזינים (Listeners) המבצעים ניקוי קלט, השמעת אפקטים קוליים וסגירה מאובטחת של התיבה.
4. אטומיות הנתונים: הבטחה ששדה הקלט יאופס בכל סגירה כדי למנוע "זיהום" של מידע בין דיאלוגים שונים.

הסבר על תכונות המחלקה:

- private final Dialog customDialog
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט הדיאלוג הבסיסי של אנדרואיד. הוא משמש כקונטיינר לכל ה-View של הדיאלוג ומנהל את מחזור החיים שלו (Show/Dismiss).
- private final EditText etInput
 - תחום הכרה: private final.
 - תפקיד ושימוש: שדה קלט טקסטואלי. מוצג רק כאשר הדיאלוג דורש מהמשתמש להזין נתון (כמו שם דמות או סיסמה).
- private final Button btnCancel, btnContinue, btnOk
 - תחום הכרה: private final.
 - תפקיד ושימוש: כפתורי הפעולה של הדיאלוג. ה-btnOk משמש כאלטרנטיבה במצב של כפתור בודד, בעוד ה-btnCancel וה-btnContinue משמשים לבחירה בינארית.
- private final LinearLayout layoutDualButtons
 - תחום הכרה: private final.
 - תפקיד ושימוש: מיכל (Container) המאגד את כפתורי ה"ביטול" וה"המשך". הוא מוסתר באופן אוטומטי כאשר הדיאלוג עובר למצב "התראה" (כפתור בודד).

הסבר פעולות מרכזיות:

- private void setVisibles()
 - זוהי "פעולת התצורה" של הדיאלוג. היא מנתחת את ה-dialogKey ומחליטה אילו רכיבים להציג.
 - **תרשים זרימה (סדר פעולות):**
 - i. הסתרת כותרת/הודעה אם ערכן הוא 1-
 - ii. הגדרת סוג המקלדת וה-Hint של שדה הקלט.
 - iii. מעבר למצב "Alert" (כפתור בודד) במידה וערך ה-Hint מוגדר 2-.
 - קוד:

```
private void setVisibles() {
    // 1. Title and Message handling
    if (dialogKey.getTitle() == -1) tvTitle.setVisibility(View.GONE);
```



```
else tvTitle.setText(dialogKey.getTitle());

if (dialogKey.getMessage() == -1) tvMessage.setVisibility(View.GONE);
else tvMessage.setText(dialogKey.getMessage());

if (dialogKey.getInputType() == -1) {
    etInput.setVisibility(View.GONE);
} else {
    etInput.setVisibility(View.VISIBLE);
    etInput.setInputType(dialogKey.getInputType());
    if (dialogKey.getHint() != -1) etInput.setHint(dialogKey.getHint());
}

if (dialogKey.getHint() == -2) {
    layoutDualButtons.setVisibility(View.GONE);
    btnOk.setVisibility(View.VISIBLE);
} else {
    layoutDualButtons.setVisibility(View.VISIBLE);
    btnOk.setVisibility(View.GONE);
}
}
```

• private void initListeners()

- מגדירה את התנהגות הכפתורים. כל לחיצה מפעילה שרשרת פעולות: הפעלת הלוגיקה הייחודית של הדיאלוג (onClick), איפוס שדה הטקסט, השמעת צליל משוב וסגירת החלון.

○ קוד:

```
private void initListeners() {
    btnCancel.setOnClickListener(v -> {
        dismiss();
        clearInput();
        playSound();
    });
    btnContinue.setOnClickListener(v -> {
        onClick();
        clearInput();
        playSound();
        dismiss();
    });
    btnOk.setOnClickListener(v -> {
        onClick();
        clearInput();
        playSound();
        dismiss();
    });
}
```

• public void onClick() {}

- פעולה המהווה את ה"הוק" (Hook) ללוגיקה העסקית. כל מחלקה שיורשת מ-CustomDialog יכולה לממש פעולה זו כדי להגדיר מה קורה כשהמשתמש מאשר את הדיאלוג. במקרה של דיאלוגי התראה אין שום פעולה שצריכה להתבצע לאחר האישור.

מחלקת **TextRenderer**מיקום: `com.example.tutorialgame.engine.renderer.TextRenderer`

תפקיד המחלקה: מחלקה זו היא הכלי המרכזי לרינדור טקסט מותאם אישית במנוע המשחק. היא מרחיבה את מחלקת `TextPaint` של אנדרואיד ומוסיפה יכולות מתקדמות שאינן קיימות כברירת מחדל, כגון ניהול שכבות צל (`Shadow Rendering`), תמיכה בגופני `Pixel-Art`, גלישת שורות אוטומטית (`Word Wrap`), ושינוי צבע דינמי המבוסס על ערכים לוגיים.

היא אחראית על:

1. עיצוב אחיד: הבטחה שכל הטקסטים במשחק ישתמשו באותו גופן ושפה עיצובית.
2. ניהול צל: יצירת עומק ויזואלי על ידי ציור שכבה נוספת מתחת לטקסט הראשי בהיסט (`Offset`) מוגדר.
3. טיפול בטקסט ארוך: שימוש ב-`StaticLayout` כדי לאפשר לטקסט (כמו דיאלוגים) להתפרס על מספר שורות באופן אוטומטי מבלי לחרוג מגבולות המסך.
4. אינטרפולציית צבעים: חישוב צבע הטקסט על סקאלה (למשל מאדום לירוק) בהתאם למשתני משחק (כמו מספר הפריימים בשנייה).

הסבר על תכונות המחלקה:

- `private final TextRenderer shadowRenderer`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: זוהי תכונה ייחודית המשתמשת ברקורסיה מבוקרת. המחלקה מחזיקה מופע נוסף של עצמה שמשמש כ"צל". אובייקט הצל מוגדר עם `shadowRenderer = null` כדי לעצור את הרקורסיה.
- `private float x, y`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: קואורדינטות הציור הנוכחיות של הטקסט על הקנבס.
- `private float xOffset, yOffset`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: מגדירים את המרחק של הצל מהטקסט הראשי. כברירת מחדל, ה-`yOffset` נקבע כעשירית מגודל הגופן כדי לשמור על פרופורציה.
- תכונות מורשות (`TextPaint`): המחלקה יורשת תכונות כמו `color`, `textSize` ו-`typeface` המנוהלות דרך מתודות ה-`Set` של מחלקת האב.

הסבר פעולות מרכזיות:

- `public TextRenderer(float size, int color)` (בנאי)
 - הבנאי מאתחל את מאפייני הגופן והצבע. הוא טוען את הגופן הייחודי של המשחק (`pixel_font`) מהמשאבים, מגדיר את הגודל, ויוצר את אובייקט ה-`shadowRenderer` (הצל) עם הגדרות צבע שחור כברירת מחדל. לבסוף, הוא מחשב את היסט הצל הראשוני.
- קוד:

```
public TextRenderer(float size, int color) {
    super();
    this.setTextSize(size);
    this.setColor(BaseActivity.getContext().getColor(color));
    this.setTypeface(ResourcesCompat.getFont(BaseActivity.getContext(), R.font.pixel_font));
    // הפרטי הבנאי באמצעות הצל את יוצר
    this.shadowRenderer = new TextRenderer(size, Color.BLACK, true);
    this.yOffset = this.getTextSize() / 10f;
    // הצל של המיקום את מיד עדכן
    updateShadowPosition();
}
```



● `public void drawWithShadow(String text, Canvas c)`

- זוהי הפעולה המרכזית לציור טקסט מוצלל.

○ קוד:

```
public void drawWithShadow(String text, Canvas c) {
    shadowRenderer.drawText(text, c);
    drawText(text, c);
}
```

● `public void drawWrappedText(Canvas c, String text, float x, float y, int maxWidth)`

- פעולה לוגית מורכבת לטיפול בטקסט רב-שורות. מכיוון שמתודת `drawText` הרגילה של `StaticLayout.Builder` היא מקבלת רוחב מקסימלי (`maxWidth`), מחשבת היכן לשבור את המילים כדי שלא יחתכו, ומציירת "בלוק" של טקסט מסודר במיקום המבוקש.

○ קוד:

```
public void drawWrappedText(Canvas c, String text, float x, float y, int maxWidth) {
    StaticLayout staticLayout = StaticLayout.Builder.obtain(text, 0, text.length(), this,
maxWidth)
        .setAlignment(Layout.Alignment.ALIGN_NORMAL)
        .setLineSpacing(0f, 1.0f)
        .setIncludePad(false)
        .build();
    c.save();
    c.translate(x, y);
    staticLayout.draw(c);
    c.restore();
}
```

● `public void updateColorBasedOnValue(int value, int minValue, int maxValue)`

- פעולה לחישוב צבע דינמי (אינטרפולציה). הפעולה מנרמלת את הערך שהתקבל לטווח של 0 עד 1. היא משתמשת בערך זה כדי לחשב יחס בין ערוץ הצבע האדום (Red) לירוק (Green). ככל שהערך גבוה יותר, הטקסט יהיה ירוק יותר, וככל שיהיה נמוך (קרוב למינימום), הוא יהיה אדום יותר. זהו כלי ויזואלי רב עוצמה להצגת סטטוסים לשחקן.

○ קוד:

```
public void updateColorBasedOnValue(int value, int minValue, int maxValue) {
    float normalizedValue = (float) (value - minValue) / (maxValue - minValue);
    // Clamp the normalized value to ensure it's within 0 and 1
    normalizedValue = Math.max(0f, Math.min(1f, normalizedValue));
    int redComponent = (int) (255 * (1 - normalizedValue)); // Decreases as value increases
    int greenComponent = (int) (255 * normalizedValue); // Increases as value increases

    // Ensure components are within 0-255 range
    redComponent = Math.max(0, Math.min(255, redComponent));
    greenComponent = Math.max(0, Math.min(255, greenComponent));
    int interpolatedColor = Color.rgb(redComponent, greenComponent, 0); // Blue is 0
    this.setColor(interpolatedColor);
}
```



מחלקת UpgradeState

מיקום: com.example.tutorialgame.gamestates.UpgradeState

תפקיד המחלקה: מחלקה זו מנהלת את מסך שדרוג הדמות במשחק. היא משמשת כממשק ויזואלי ולוגי המאפשר לשחקן להשקיע "נקודות שדרוג" (Upgrade Points) שצבר כדי לשפר מדדים כמו חיים, כוח, מהירות התקפה וסטמינה. המחלקה מספקת תצוגה מקדימה (Preview) בזמן אמת של השינויים על גבי דגם הדמות, ומנהלת מערכת גלילה אופקית עבור רשימת השדרוגים, כל זאת תוך שמירה על סנכרון מלא מול ה-StatsDoc בענן.

היא אחראית על:

1. ניהול ממשק שדרוגים דינמי: יצירה ורינדור של אובייקטי CustomUpgrade הכוללים אנימציות של "בלונים" עם מידע מפורט.
2. מערכת תצוגה מקדימה (Visual Preview): הצגת ההשפעה הצפויה של השדרוג (למשל, הוספת לבבות שקופים למחצה) לפני האישור הסופי.
3. ניהול גלילה ואינטראקציה: מימוש לוגיקת גלילה אופקית (Horizontal Scrolling) עם אפקט "דעיכה" (Fade) בקצוות.
4. סנכרון מול הענן: ביצוע עדכון אטומי של כלל השדרוגים שנבחרו בלחיצה אחת (applyChanges), כולל עדכון ה-XP, המטבעות והסטטיסטיקות.

הסבר על תכונות המחלקה:

- private static final BlurMaskFilter[] BLUR_FILTERS
 - תחום הכרה: private static final.
 - תפקיד ושימוש: מערך מוגדר מראש של פילטרים ליצירת אפקט "טשטוש" (Blur) הדרגתי. השמירה במערך סטטי מונעת יצירת פילטרים יקרים בזמן ריצה ומשפרת משמעותית את הביצועים.
- private CustomUpgrade[] upgrades
 - תחום הכרה: private.
 - תפקיד ושימוש: מערך המחזיק את כל רכיבי השדרוג המוצגים במסך. כל איבר מייצג סוג שדרוג שונה (HEALTH, STRENGTH וכו').
- private float scrollX
 - תחום הכרה: private.
 - תפקיד ושימוש: משתנה השומר את מיקום הגלילה הנוכחי של אזור השדרוגים. משמש לחישוב ההסטה (Offset) בזמן הצירור והטיפול באירועי מגע.
- private int pointsLeft
 - תחום הכרה: private.
 - תפקיד ושימוש: מונה נקודות השדרוג הזמינות לשחקן בתוך המופע הנוכחי. הוא מתעדכן בכל פעם שהשחקן "מוסיף" שדרוג זמני לפני הלחיצה על Apply.
- private final Paint previewPaint
 - תחום הכרה: private final.
 - תפקיד ושימוש: מברשת (Paint) המוגדרת עם פילטר צבע תכלת שקוף. משמשת לצירור ה"בונוס" הצפוי על מדדי החיים והסטמינה כתצוגה מקדימה.

הסבר פעולות מרכזיות:

- public void update(double delta)
 - מעדכנת את לוגיקת המסך. היא מחשבת את ה-Stagger (הסטה מדורגת) של שורות השדרוגים בזמן גלילה כדי לתת תחושה של עומק, ומנהלת את טיימר הבלר כדי להחליף פילטרים בקצב קבוע.
 - קוד:



```
@SuppressWarnings("IntegerDivisionInFloatingPointContext")
@Override
public void update(double delta) {
    lineAnimation.update();
    updateBlurEffect();

    for (int i = 0; i < upgrades.length; i++) {
        int row = i % 3;
        float rowStagger = row * (-scrollX * STAGGER_FACTOR);
        float baseX = startX + (i / 3) * spacingX;
        float baseY = (lineY - TILE_SIZE) - row * spacingY;

        upgrades[i].setCenter(baseX + rowStagger, baseY);
        upgrades[i].update(pointsLeft, delta);
    }

    btnApplyChanges.setEnabled(prevPointsLeft > pointsLeft);
}
```

● private void applyChanges()

- זוהי הפעולה המבצעת את הרישום הסופי. היא עוברת על כל השדרוגים, אוספת את אלו שסומנו, ומעדכנת את ה-StatsDoc בבת אחת. בסיום, היא מרעננת את נתוני הדמות בעולם המשחק ומשמיעה צליל הצלחה.

○ קוד:

```
private void applyChanges() {
    StatsDoc stats = MyApp.getPlayerStats();
    boolean changed = false;

    for (CustomUpgrade u : upgrades) {
        if (u.getCountToApply() > 0) {
            stats.updateStat(u.getStatField(), u.getCountToApply() * u.getUpgradeValue(),
                u.getTotalPendingPrice(), u.getCountToApply());
            u.resetCountToApply();
            changed = true;
        }
    }

    if (changed) {
        MapManager.getCurrentMap().getPlayer().refreshStats();
        SoundManager.getInstance(context).playSfx(R.raw.sfx_success4);
        prevPointsLeft = pointsLeft;
    }
}
```

בסיס נתונים (Firestore) (Firebase)

המחלקה משמשת כממשק כתיבה מורכב מול מסמכי המשתמש.

- סקירת המידע: הנתונים נשלחים למסמך ה- stats (עבור המדדים הפיזיים) ולמסמך ה-progress (עבור ניצול נקודות השדרוג).
- פעולות:

- טעינה: ב-onEnter, נשלף מספר הנקודות הזמין מה-ProgressDoc.
- שמירה: ב-applyChanges, מתבצעת כתיבה מרוכזת לענן המעדכנת את רמות השדרוג לצמיתות.

**מחלקת MainMenu**

מיקום: com.example.tutorialgame.gamestates.menu.menustates.MainMenu

תפקיד המחלקה: מחלקה זו מהווה את שער הניווט הראשי של המשתמש בעת עצירת המשחק (Pause). תפקידה הוא לנהל את מסך הבחירה המרכזי המאפשר לשחקן לחזור למשחק, לגשת להגדרות או לצאת מהמשחק. המחלקה מיישמת אנימציות "החלקה" (Sliding) קולנועיות בכניסה הראשונה, ומבטיחה חוויית מעבר חלקה ורציפה בין תפריטי המשנה ללא אנימציות מיותרות בעת ניווט פנימי.

היא אחראית על:

1. ניהול אנימציות פריסה: ביצוע החלקה הדרגתית של פאנל התפריט מחלקו העליון של המסך למרכז.
2. בקרה על מעברים רציפים: מניעת הפעלה חוזרת של אנימציות בעת חזרה מתפריטי הגדרות.
3. ניווט ראשי: קישור המשתמש לתפריט האפשרויות (OptionsMenu) או יציאה ל- Launcher.
4. ניהול חזרה למשחק: הפעלת אנימציות יציאה מסודרת לפני החזרת השליטה למנוע המשחק.

הסבר על תכונות המחלקה:

- private boolean isEntering, isExiting
 - תחום הכרה: private.
 - תפקיד ושימוש: דגלים לוגיים המציניים האם התפריט נמצא כרגע בתהליך תנועה. הם משמשים לחסימת קלט בזמן האנימציה ולני הול כיוון ההחלקה.
- private final RectButton btnBack, btnOptions, btnExit
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקטי הכפתורים הגרפיים המרכיבים את התפריט. מיקומם מחושב יחסית לרוחב וגובה המסך להבטחת התאמה לכל מכשיר.
- private float offsetY
 - תחום הכרה: private.
 - תפקיד ושימוש: משתנה המנהל את ההיסט האנכי של כלל התפריט. הוא קובע כמה הצויר "מוסט" ביחס לקצה העליון של המסך.
- private static final float SLIDE_SPEED
 - תחום הכרה: private static final.
 - תפקיד ושימוש: קבוע המגדיר את מהירות ההחלקה, מבוסס על ה-TILE_SIZE להבטחת ביצועים אחידים במכשירים שונים.

הסבר פעולות מרכזיות:

- public void update(double delta)
 - מחשבת את המיקום החדש של התפריט בכל פריים. הפעולה מקדמת את ה-offSetY בהתאם לדגלים הפעילים. במצב כניסה, היא מגדילה את ההיסט עד להגעה ל-0 (מרכז המסך). במצב יציאה, היא מקטינה אותו עד לה סתרה מוחלטת. השימוש ב-delta מבטיח שהתנועה תיראה חלקה ומהירה באותה מידה ללא קשר לעומס על המעבד.

קוד:

```
@Override
public void update(double delta) {
    if (isEntering) {
        offsetY += (float) (SLIDE_SPEED * delta);
        if (offsetY >= 0) {
            offsetY = 0;
            isEntering = false;
        }
    } else if (isExiting) {
        offsetY -= (float) (SLIDE_SPEED * delta * 1.5f);
        if (offsetY <= -SCREEN_HEIGHT) {
```



```
        offsetY = -SCREEN_HEIGHT;
        isExiting = false;
        Game.setNextGameState(Game.GameState.PLAYING);
    }
}
```

• public void onEnter()

- מתודה הנקראת בכל פעם שהתפריט הופך לגלוי. כדי למנוע את ה"קפיצות" והאנימציות החוזרות שנוצרות בעת ניווט פנימי, הפעולה מבצעת בדיקה קריטית: היא מאתחלת את האנימציה (resetAnimation) רק אם התפריט נמצא כרגע מחוץ למסך. אם השחקן רק חזר ממסך ה-OptionsMenu, התפריט יישאר סטטי במיקומו הנוכחי.

○ קוד:

```
@Override
public void onEnter() {
    super.onEnter();
    // Only reset animation if the panel is currently hidden (e.g., coming from the game)
    if (offsetY <= -SCREEN_HEIGHT) {
        resetAnimation();
    }
}
```

• public void render(Canvas c)

- מבצעת את הרינדור של התפריט כיחידה אחת. הפעולה משתמשת ב-c.translate(0, -offsetY) כדי להזיז את כל מערכת הצירים של הקנבס. המשמעות היא שכל פעולות הציור של ה-BaseMenu (רקע וכוותרת) ושל הכפתורים מתבצעות במיקום המוסט, מה שיוצר את אפקט ההחלקה המאוחד.

○ קוד:

```
@Override
public void render(Canvas c) {
    c.save();
    c.translate(0, offsetY);

    drawBackground(c, R.string.game_menu);
    btnBack.draw(c);
    btnOptions.draw(c);
    btnExit.draw(c);

    c.restore();
}
```




מחלקת OptionsMenu

מיקום: com.example.tutorialgame.gamestates.menu.menustates.OptionsMenu

תפקיד המחלקה: מחלקה זו מהווה את צומת הניווט המרכזי עבור כל הגדרות המשחק. תפקידה הוא לרכז ולארגן את הגישה לתפריטי המשנה השונים (וידאו, שמע, שפות וקרדיטים). המחלקה מנהלת את סידור הכפתורים במבנה של רשת (Grid), ומבטיחה חוויית ניווט חלקה בתוך מערכת התפריטים.

היא אחראית על:

- ניהול תצוגת ניווט: יצירה וסידור של כפתורי פקודה במבנה טבלאי המותאם לרזולוציית המסך.
- ניתוב תפריטים: קישור המשתמש למצבי התפריט הספציפיים (MusicSounds, VideoSettings) דרך ה- `MenuManager`.

הסבר על תכונות המחלקה:

- `private final List<RectButton> buttons`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רשימה המאגדת את כל הכפתורים בתפריט. שימוש ברשימה מאפשר להריץ פעולות גורפות (כמו ציור או בדיקת נגיעה) בלולאה אחת, מה שמונע כפל קוד ומשפר את יציבות המערכת.
- `private final RectButton btnDone, btnVideoSettings, btnMusicSounds, btnLanguage, btnCredits`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מופעים של כפתורים מלבניים המייצגים את האפשרויות השונות. כל כפתור מקושר לתמונה ייחודית מתוך `RectImages`.

הסבר פעולות מרכזיות:

- `public void render(Canvas c)`
 - מבצעת את הציור של כל מרכיבי התפריט. הפעולה קוראת תחילה למתודת ה- `drawBackground` של מחלקת הבסיס כדי לצייר את המסגרת והכותרת. לאחר מכן, היא רצה בלולאה על רשימת הכפתורים המרוכזת ומציירת כל אחד מהם במיקומו המוגדר.
- קוד:

```
@Override
public void render(Canvas c) {
    drawBackground(c, R.string.options);
    for (RectButton btn : buttons)
        btn.draw(c);
}
```

- `public void onClick(BaseButton button)`
 - מגיבה לחיצות המשתמש ומבצעת את הניווט הלוגי.
 - קוד (לדוגמה):

```
@Override
public void onClick(BaseButton button) {
    if (button == btnDone) {
        menuManager.setCurrentMenuState(MenuManager.MenuState.Main);
    } else if (button == btnMusicSounds) {
        menuManager.setCurrentMenuState(MenuManager.MenuState.MusicSounds);
    } else if (button == btnVideoSettings) {
        menuManager.setCurrentMenuState(MenuManager.MenuState.VideoSettings);
    }
}
```

// ועוד

מחלקת **VideoSettingsMenu**מיקום: `com.example.tutorialgame.gamestates.menu.menustates.VideoSettingsMenu`

תפקיד המחלקה: מחלקה זו מנהלת את תפריט המשנה המיועד להתאמה אישית של הגדרות התצוגה, המצלמה וכלי הניפוי (Debug) של המשחק. היא מאפשרת למשתמש לשלוט ברמת הקירוב של המצלמה (Zoom), במהירות העקיבה שלה, בסוג אפקט המגע המועדף, ובהצגת מדדי ביצועים בזמן אמת.

היא אחראית על:

1. בקרת מצלמה: סנכרון רמת ה-Zoom והחלקה (Lerp Factor) מול ה-CameraManager.
2. ניהול אפקטים ויזואליים: בחירת סוג אפקט הנגיעה (ImpactEffectType) דרך פקד בחירה (CustomRadioGroup).
3. תשתית ניפוי וביצועים: הפעלה או השבתה של תצוגת ה-FPS ותיבות הפגיעה (Hitboxes) למטרות בדיקה.
4. משוב חזותי בזמן אמת: ביצוע "הצצה" (Preview) של שינויי הזום על ידי מיקוד המצלמה על השחקן בעת הזזת הסליידר.

הסבר על תכונות המחלקה:

- `private final CustomSeekBar sbZoom, sbCameraSpeed`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: פקדי בחירה רציפים השולטים בפרמטרים של המצלמה. הם מחזיקים ערכים נורמליים (0.0-1.0) שמתורגמים לערכים מבצעיים בתוך מנהל המצלמה.
- `private final CustomRadioGroup radioTapEffects`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: פקד בחירה מרובה המאפשר למשתמש לבחור את סגנון האנימציה שמופיעה בעת נגיעה במסך.
- `private final CustomSwitch swFPS, swHitbox`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מתגים בינאריים להפעלה/השבתה של תצוגות מיוחדות מעל עולם המשחק.

הסבר פעולות מרכזיות:

- `(VideoSettingsMenu(Game game, MenuManager menuManager))` - הבנאי
 - מנהל את האתחול והקשר הלוגי בין הממשק למנוע. בבנאי מתבצע תהליך ה-Binding: קשירת ה-Listeners של הסליידרים והמתגים לפעולות המנוע. פעולה מיוחדת מתבצעת ב-sbZoom: בעת הזזה, המערכת לא רק מעדכנת את הערך, אלא גם פוקדת על המצלמה לבצע מיקוד מיידי (lookAt) על השחקן כדי להדגים למשתמש את רמת הזום החדשה בזמן אמת.
- `public void onSelectionChanged(CustomRadioGroup group, int selectedIndex)`
 - מגיבה לשינוי בבחירת אפקט המגע. הפעולה משתמשת ב-switch-case על האינדקס שנבחר בקבוצת הרדיו ומעדכנת את אובייקט ה-Game בסוג האפקט החדש. המידע נשמר מיידיית ב-SharedPreferences לצורך רציפות בין הפעלות.

○ קוד:

```
@Override
public void onSelectionChanged(CustomRadioGroup group, int selectedIndex) {
    if (group == radioTapEffects) {
        switch (selectedIndex) {
            case 0: game.setTapEffectType(ImpactEffectType.WHITE_CIRCLE); break;
            case 1: game.setTapEffectType(ImpactEffectType.ORANGE_CIRCLE); break;
            case 2: game.setTapEffectType(ImpactEffectType.SPARK); break;
            case 3: game.setTapEffectType(ImpactEffectType.SPARK2); break;
        }
    }
}
```



```
}
}
```

• public void render(Canvas c)

- מנהלת את הרינדור של כל פקדי ההגדרות. הפעולה קוראת תחילה ל-drawBackground כדי לייצר את מעטפת התפריט, ולאחר מכן מציירת את כל הפקדים (כפתורים, סליידרים ומתגים) במיקומם המחושב. סדר הציור מבטיח שהפקדים יופיעו מעל הרקע והמסגרת.

○ קוד:

```
@Override
public void render(Canvas c) {
    drawBackground(c, R.string.video_settings);
    btnDone.draw(c);
    radioTapEffects.render(c);
    swFPS.draw(c);
    swHitbox.draw(c);
    sbZoom.drawSeekBar(c);
    sbCameraSpeed.drawSeekBar(c);
}
```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת בקובץ ההגדרות המקומי "settings" לצורך סנכרון העדפות המשתמש.

- סקירת המידע: נשמרים ערכי float עבור zoom ו-lerpFactor, וערכי boolean עבור תצוגת FPS ו-Hitbox.
- פעולות:

- טעינה: מתבצעת בבנאי דרך CameraManager.getNormalizedZoom() ודומיו, כדי להציב את הסליידרים במיקום הנכון בעת פתיחת התפריט.
- עדכון: מתבצע אוטומטית ברגע שהמשתמש משנה ערך, דרך המתודות ב-CameraManager וב-Game.

**מחלקת MusicSoundsMenu**

מיקום: com.example.tutorialgame.gamestates.menu.menustates.MusicSoundsMenu

תפקיד המחלקה: מחלקה זו מנהלת את תפריט המשנה הייעודי להגדרות השמע והפידבק במשחק. תפקידה הוא לספק למשתמש ממשק ויזואלי לשליטה בעוצמת המוזיקה, האפקטים הקוליים (SFX) והפעלת/השבתת המשוב התחושי (Haptics).

היא אחראית על:

1. סנכרון נתונים בזמן אמת: עדכון עוצמת הקול בנגיני ה-Media וה-SoundPool מיד עם שינוי הערך על ידי המשתמש.
2. ניהול מצב (State Persistence): טעינת הערכים השמורים בעת הכניסה לתפריט והבטחת שמירתם לשימוש עתידי.

הסבר על תכונות המחלקה:

- private final CustomSeekBar sbMusic, sbSound
 - תחום הכרה: private final.
 - תפקיד ושימוש: פקדי הסליידר השולטים בעוצמת המוזיקה ובעוצמת האפקטים, בהתאמה. הם מחזיקים את הערך הנורמלי (0.0 עד 1.0).
- private final CustomSwitch swHaptics
 - תחום הכרה: private final.
 - תפקיד ושימוש: פקד מתג המאפשר למשתמש להפעיל או לכבות את הרטט בכפתורים ובאירועי משחק.
- private final RectButton btnDone
 - תחום הכרה: private final.
 - תפקיד ושימוש: כפתור אישור המסיים את עריכת ההגדרות ומחזיר את המשתמש לתפריט הקודם.

הסבר פעולות מרכזיות:

- public void onProgressChanged(CustomSeekBar seekBar, float progress)
 - כאשר המשתמש מזיז את אחד הסליידרים, פעולה זו מזהה איזה סוג שמע הושפע ומעבירה את הערך החדש למנהל המתאים. המנהלים מעדכנים את הווליום בנגנים הפעילים ודואגים לשמור את הערך החדש בדיסק.
- קוד:

```
@Override
public void onProgressChanged(CustomSeekBar seekBar, float progress) {
    if (seekBar == sbMusic) MusicManager.getInstance(context).setVolume(progress);
    else if (seekBar == sbSound) SoundManager.getInstance(context).setVolume(progress);
}
```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת בבסיס הנתונים המקומי של אנדרואיד כדי לשמור על רציפות הגדרות המשתמש.

- סקירת המידע: המידע נשמר בקובץ הגדרות בשם "settings". נשמרים ערכים מסוג float עבור ווליום וערך boolean עבור רטט.
- פעולות:
 - טעינה: מתבצעת בתוך refreshSettings באמצעות sp.getFloat. הערכים נשלפים עבור המפתחות "music_volume" ו-"sound_volume".
 - עדכון: השמירה מתבצעת באופן אסינכרוני דרך מנהלי השמע ברגע שמופעלת הפעולה setVolume.

מחלקת **NpcHealthBar**מיקום: `com.example.tutorialgame.engine.ui.NpcHealthBar`

תפקיד המחלקה: מחלקה זו מייצגת רכיב גרפי מסוג "פס חיים מיניאטורי" המוצג מעל ראשן של דמויות שאינן השחקן (NPCs ואויבים). תפקידה לספק לשחקן משוב ויזואלי מיידי על מצב בריאותם של היצורים בעולם המשחק, תוך שימוש באפקטים דינמיים המדגישים אובדן חיים בזמן אמת. המחלקה מתוכננת לעבודה ביעילות מרבית, כך שגם כאשר מופיעים עשרות אויבים על המסך בו-זמנית, צריכת המשאבים נשארת מינימלית.

היא אחראית על:

1. רינדור פס חיים עילי: ציור רכיבי הפס במיקום מדויק יחסית לתיבת הפגיעה של הדמות (Projected HitBox).
2. ניהול תצוגה דינמית: חישוב המילוי היחסי של הפס בהתאם ליחס שבין החיים הנוכחיים למקסימליים.
3. אפקט פלאש נזק (Damage Flash): יצירת שכבה ויזואלית זמנית המהבהבת באזור שבו נגרעו חיים, כדי להעניק תחושת עוצמה לפגיעות.
4. אופטימיזציית משאבים: שימוש במשאבים סטטיים (Bitmaps ו-Paints) המשותפים לכל הישויות, למניעת בזבז זיכרון RAM.

הסבר על תכונות המחלקה:

- `private static final Bitmap lifeBarEmpty, lifeBarProgressFull, frame`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: שלוש תמונות המקור המרכיבות את הפס (רקע, מילוי ומסגרת). בזכות היותן סטטיות, הן נטענות לזיכרון פעם אחת בלבד דרך ה-`BitmapManager` ומשרתות את כל האויבים במשחק.
- `private static final Paint flashPaint`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מברשת ציור המוגדרת עם פילטר צבע (Tint) בגוון ורדרד-לבן. היא משמשת לציור אפקט ה"נזק הטרי" שנגרם לדמות.
- `private final Character npc`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רפרנס לדמות הספציפית לה שייך הפס. דרך אובייקט זה המחלקה שולפת נתוני חיים ומיקום בכל פריים.
- `private final Rect srcRect, dstRect` (ועוד)
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מלבני הגדרה המשמשים לביצוע חיתוך (Clipping) של תמונת המילוי. השימוש במופעים קבועים מונע יצירת אובייקטים חדשים בלולאת הציור.

הסבר פעולות מרכזיות:

- `public void drawBar(Canvas c)`
 - הפעולה המרכזית המבצעת את הרינדור השכבתי של פס החיים. הפעולה מחשבת תחילה את מיקום הציור (מעל ראש הדמות). לאחר מכן היא מציירת את השכבות בסדר הבא: רקע ריק, מילוי חיים יחסי (באמצעות חיתוך רוחבי), ואת אפקט הפלאש במידה וקיים. בסיום היא מציירת את המסגרת המעוצבת מעל הכל.
 - קוד:

```
public void drawBar(Canvas c) {
    if (npc.isDead() || !npc.isActive()) return;

    // Position the bar above the NPC's head
    float y = npc.getProjectedHitBox().top - 8 * SCALE_MULTIPLIER;
```



```
float x = npc.getProjectedHitBox().left - 2.5f * SCALE_MULTIPLIER;

// 1. Draw the background track
c.drawBitmap(lifeBarEmpty, x, y, null);

// 2. Calculate and draw the current health progress
float currentHealth = npc.getCurrentHealth();
int maxHealth = npc.getMaxHealth();
float pct = currentHealth / maxHealth;
int cutW = Math.round(barWidth * Math.max(0, Math.min(1, pct)));

if (cutW > 0) {
    srcRect.set(0, 0, cutW, barHeight);
    dstRect.set((int) x, (int) y, (int) (x + cutW), (int) (y + barHeight));
    c.drawBitmap(lifeBarProgressFull, srcRect, dstRect, null);
}

// 3. Render damage flash and the containing frame
drawFlash(c, x, y, cutW, currentHealth, maxHealth);
c.drawBitmap(frame, x, y, null);
}
```

private void drawFlash(Canvas c, float x, float y, int currentCutW, float currentHealth, int maxHealth) •

○ פעולה האחראית על המשוב הויזואלי לנוק. הפעולה בודקת דרך ה-HealthComponent האם קיימת אנימציה פלאש פעילה (נוצרת מיד לאחר פגיעה). היא מחשבת את ההפרש בין הבריאות הקודמת לנוכחית, וצובעת את האזור ה"חסר" בפס בצבע לבן דועך. זה מאפשר לשחקן לראות בדיוק כמה נזק נגרם במכה האחרונה.

○ קוד:

```
private void drawFlash(Canvas c, float x, float y, int currentCutW, float currentHealth, int maxHealth) {
    float alpha = npc.getHealthComponent().getFlashAlpha();
    if (alpha <= 0) return;

    int prevHealth = npc.getHealthComponent().getPrevHealth();
    if (prevHealth <= currentHealth) return;

    // Calculate the width of the "health lost" section that should flash
    float diffPct = (prevHealth - currentHealth) / maxHealth;
    int flashWidth = (int) (diffPct * barWidth);

    if (flashWidth > 0) {
        flashSrcRect.set(currentCutW, 0, currentCutW + flashWidth, barHeight);
        flashDstRect.set((int) (x + currentCutW), (int) y, (int) (x + currentCutW + flashWidth), (int) (y + barHeight));

        flashPaint.setAlpha((int) (alpha * 255));
        c.drawBitmap(lifeBarProgressFull, flashSrcRect, flashDstRect, flashPaint);
    }
}
```



מחלקת DialoguePanel

מיקום: com.example.tutorialgame.engine.ui.DialoguePanel

תפקיד המחלקה: מחלקה זו מנהלת את הממשק היוזאלי של הדיאלוגים במשחק באמצעות מכונת מצבים (State Machine) ומערכת רינדור אופטימלית. היא אחראית על הצגת תיבת הטקסט, דיוקן הדובר (Faceset), שמו, והרצת שורות הדיאלוג עם אפקט "מכונת כתיבה" (Typewriter Effect) מתקדם. המחלקה תוכננה בדגש על ניהול זיכרון חסכוני (Garbage-Free) וביצועים גבוהים, תוך הבטחת חוויית RPG קולנועית וחלקה.

היא אחראית על:

1. ניהול מצבי ממשק: מעבר מסונכרן בין מצבי הופעה, הקלדה, המתנה ויציאה של הפאנל.
2. אופטימיזציות מחרוזות: שימוש במערכי תווים (char[]) ו-StringBuilder למניעת הקצאות זיכרון מיותרות (Object Allocation) בכל פריים בזמן ההקלדה.
3. טיפוגרפיה דינמית: ניהול קצב הופעת התווים בהתאם לסימני פיסוק (השהיות ארוכות לנקודות, קצרות לפסיקים).
4. ניהול קלט יעיל: טיפול באירועי מגע באמצעות חישובי היסט (Offset) ידניים למניעת שכפול אובייקטים של מערכת ההפעלה.

הסבר על תכונות המחלקה:

- `private enum PanelState { HIDDEN, ENTERING, TYPING, WAITING, EXITING }`
 תחום הכרה: `private` (פנימי).
 תפקיד ושימוש: הגדרת כלל המצבים האפשריים של הפאנל. מונע "מצבים מתנגשים" (Illegal States) ומבטיח לוגיקה ליניארית וברורה.
- `private char[] fullLineChars`
 תחום הכרה: `private`.
 תפקיד ושימוש: מאחסן את שורת הדיאלוג כמערך תווים גולמי. מאפשר גישה מהירה ובניית מחרוזת תצוגה ללא שימוש בפעולת `substring` היקרה בזיכרון.
- `private String visibleText`
 תחום הכרה: `private`.
 תפקיד ושימוש: המחרוזת הנבנית בהדרגה המוצגת למשתמש. היא מתעדכנת רק כאשר תו חדש נוסף, ולא בכל פריים.
- `private float timeSinceLastChar`
 תחום הכרה: `private`.
 תפקיד ושימוש: מונה זמן המבוסס על ה-`delta` של המנוע, המשמש לסנכרון קצב ההקלדה עם מהירות המשחק.
- `private final Bitmap dialogBox`
 תחום הכרה: `private final`.
 תפקיד ושימוש: תמונת הרקע של תיבת הדיאלוג, נטענת בצורה אופטימלית דרך ה-`BitmapManager`.

הסבר פעולות מרכזיות:

- `private void updateTypewriter(double delta)`
 מנהלת את הלוגיקה של אפקט ההקלדה. הפעולה מחשבת את הזמן הנדרש להצגת התו הבא על ידי קריאה ל-`getDelayForChar`. היא משתמשת ב"עודף זמן" (Overflow) כדי לשמור על קצב הקלדה עקבי גם אם קצב הפריימים משתנה. יצירת אובייקט ה-String לתצוגה מתבצעת רק בנקודה שבה נוסף תו חדש, מה שמונע יצירת עשרות אובייקטים זמניים בשנייה.
- **תרשים זרימה (סדר פעולות):**



- i. בדיקה אם הטקסט הסתיים:
 - 1. אם כן: העברת המצב ל-WAITING.
 - ii. צבירת זמן ב-timeSinceLastChar.
 - iii. חישוב השהייה נדרשת לפי התו האחרון שנדפס.
 - iv. אם חלף הזמן:
 - 1. קידום אינדקס התו.
 - 2. בניית visibleText חדש.
 - 3. השמעת צליל דיבור (אם התו אינו רווח או סימן פיסוק).

קוד: ○

```
private void updateTypewriter(double delta) {
    if (fullLineChars == null || charIndex >= fullLineChars.length) {
        currentState = PanelState.WAITING;
        return;
    }

    timeSinceLastChar += (float) delta;
    float requiredDelay = getDelayForChar(fullLineChars[charIndex > 0 ? charIndex - 1 : 0]);

    if (timeSinceLastChar >= requiredDelay) {
        timeSinceLastChar -= requiredDelay;
        charIndex++;
        // String creation only happens when a new character is added, not every frame
        visibleText = new String(fullLineChars, 0, charIndex);

        char current = fullLineChars[charIndex - 1];
        if (current != ' ' && !isPunctuation(current)) {
            SoundManager.getInstance(BaseActivity.getContext()).playSfx(voiceRes);
        }
    }
}
```

● public void eventHandler(MotionEvent event)

- מטפלת באינטראקציית המשתמש עם הדיאלוג. הפעולה מיישמת אופטימיזציית קלט על ידי הסטת ה-y של הנגיעה כך שתתאים למיקום ה-hitbox של הכפתור. הפעולה מאפשרת למשתמש "לדלג" על ההקלדה ולהציג את השורה המלאה בלחיצה ראשונה, ולעבור לשורה הבאה בלחיצה שנייה.

קוד: ○

```
public boolean eventHandler(MotionEvent event) {
    if (currentState == PanelState.HIDDEN || currentState == PanelState.EXITING) return false;
    event.offsetLocation(0, -offsetY); // לב שים: offsetY מ'נוס זה כלל בדרך: לב שים
    הפאנל של המקומי

    try {
        if (btnNext.eventHandler(event)) {
            if (currentState == PanelState.TYPING)
                finishTyping();
            else if (currentState == PanelState.WAITING)
                advanceDialogue();
            return true; // נבלע - נבלע
        }
        return true;
    } finally {
        event.offsetLocation(0, offsetY);
    }
}
```


**מחלקת StaminaDisplay**

מיקום: com.example.tutorialgame.engine.ui.displays.StaminaDisplay

תפקיד המחלקה: מחלקה זו מייצגת רכיב תצוגה עילית (HUD) האחראי על שיקוף רמת האנרגיה (Stamina) של השחקן בזמן אמת. תפקידה לספק משוב ויזואלי ברור על כמות המשאבים הזמינה לביצוע פעולות (כמו ריצה או התקפה) ועל קצב צריכתם, תוך שימוש בשורת אייקונים דינמית המשלבת אפקטי חיתוך (Clipping) והבהוב (Flash).

היא אחראית על:

1. רינדור שורת סטמינה: יצירת תצוגה מודולרית של אייקוני ברק המייצגים את קיבולת האנרגיה המקסימלית.
2. ניהול מצב דינמי: חישוב המילוי היחסי של כל אייקון בודד בהתאם לרמת הסטמינה הנוכחית של השחקן.
3. אפקטים ויזואליים (Flash Effect): הצגת "שארית" לבנה דועכת המציינת את כמות האנרגיה שנצרכה זה עתה, לצורך מתן משוב אינטואיטיבי למשתמש.

הסבר על תכונות המחלקה:

- `private static final Bitmap progressBmp, emptyIconBmp`
 תחום הכרה: `private static final`.
 תפקיד ושימוש: תמונות המקור המרכיבות את האייקון (רקע ריק ומילוי אנרגיה). בזכות היותן סטטיות, הן נטענות לזיכרון פעם אחת בלבד ומשמשות את כל חלקי המערכת.
- `private final Rect srcRect, dstRect`
 תחום הכרה: `private final`.
 תפקיד ושימוש: מלבני עזר המשמשים לביצוע Clipping (חיתוך) של תמונת המילוי, המאפשרים להציג אייקון "חצי מלא" בדיוק של פיקסל.
- `private final Paint flashPaint`
 תחום הכרה: `private final`.
 תפקיד ושימוש: מברשת ייעודית המוגדרת עם `PorterDuff.Mode.SRC_ATOP` וצבע לבן, המשמשת לציור אפקט הניצול של הסטמינה.
- `private static final int MAX_PER_ICON = 50`
 תחום הכרה: `private static final`.
 תפקיד ושימוש: קבוע המגדיר כמה יחידות לוגיות של סטמינה מייצג כל אייקון ברק אחד בתצוגה.
- `private final float xOffset, yOffset`
 תחום הכרה: `private final`.
 תפקיד ושימוש: נקודת העוגן של הרכיב על המסך, ממנה מתחילה להיפרס שורת האייקונים.

הסבר פעולות מרכזיות:

- `public void draw(Canvas c)`
 זוהי הפעולה המרכזית המבצעת את רינדור כל מערך הסטמינה על המסך. הפעולה רצה בלולאה על מספר האייקונים הנדרש (המחושב לפי `maxStamina`). עבור כל אייקון, היא מציירת תחילה את הרקע הריק. לאחר מכן, היא מחשבת את כמות הסטמינה השייכת לאותו אינדקס (ערך Clamp בין 0 ל-50). אם הערך חיובי, היא מחשבת את רוחב החיתוך ומציירת את המילוי. לבסוף, אם יש אפקט פלאש פעיל (לאחר צריכת אנרגיה), היא מחשבת את המיקום והרוחב של ה"חלק החסר" ומציירת אותו בלבן דועך.
- **תרשים זרימה (סדר פעולות):**
 - i. חישוב מיקום X אופקי לפי אינדקס הלולאה והמרווח המוגדר.



- ii. ציור אייקון הרקע הריק במיקום המחושב.
- iii. בדיקת מילוי:
- אם כמות הסטמינה לאייקון זה < 0 :
 - חישוב אחוז המילוי וחיתוך ה-`srcRect` (`Bitmap`).
 - ציור המילוי במיקום המדויק מעל הרקע.
- בדיקת פלאש (משוב ויזואלי):
 - אם קיים הפרש בין הסטמינה הקודמת לנוכחית ואפקט ה-`Alpha` פעיל:
 - חישוב האזור שנצרך זה עתה.
 - הגדרת שקיפות המברשת וציור ה"ניצוץ" הלבן מעל האזור הריק.

○ קוד לוגיקת החיתוך והפלאש:

```
// (עד 50) הזה לאייקון השייכת הסטמינה כמות חישוב
int staminaInThisIcon = Math.max(0, Math.min(MAX_PER_ICON, currentStamina - (MAX_PER_ICON * i)));

if (staminaInThisIcon > 0) {
    float pct = (float) staminaInThisIcon / MAX_PER_ICON;
    int cutW = Math.round(ICON_WIDTH * pct);

    srcRect.set(0, 0, cutW, ICON_HEIGHT);
    float drawX = xPos + progressXAdjustment;
    dstRect.set((int) drawX, (int) yOffset, (int) (drawX + cutW), (int) (yOffset + ICON_HEIGHT));

    c.drawBitmap(progressBmp, srcRect, dstRect, null);
}
```

בסיס נתונים (Firebase Firestore)

המחלקה אינה פונה ישירות לענן, אך היא מסתמכת על נתונים המנוהלים ומסונכרנים על ידי רכיב הסטמינה.

- סקירת המידע: הנתונים מגיעים ממסמך ה-`StatsDoc` דרך השדות הקשורים לקיבולת הסטמינה המקסימלית.
- פעולות:
- טעינה: בבנאי ובכל פריים, המחלקה ניגשת ל-`player.getStaminaComponent()` המכיל את הערכים שסונכרנו מהענן בעת עליית המשחק או שדרוג הדמות. שינויים בסטמינה במהלך המשחק (כמו שימוש בכישורים) משתקפים מיידית בתצוגה זו.



מחלקת XpDisplay

מיקום: com.example.tutorialgame.engine.ui.XpDisplay

תפקיד המחלקה: מחלקה זו מייצגת רכיב תצוגה עילית (HUD) האחראי על שיקוף התקדמות השחקן ברמת הניסיון (XP) והדרגה (Level). תפקידה הוא לספק משוב ויזואלי מיידי על המרחק שנותר לשחקן עד לעליית הרמה הבאה, תוך שימוש בטכניקות רינדור יעילות המדמות פס התקדמות מתמלא.

היא אחראית על:

1. ניהול תצוגת רמה: שליפת דרגת השחקן הנוכחית מהענן והצגתה עם אפקט צל לשיפור הקריאות.
2. חישוב יחסי התקדמות: המרת ערכי ה-XP הגולמיים לאחוזים (0-100%) ביחס ליעד הנדרש.
3. רינדור גרפי חכם (Clipping): שימוש במלבני חיתוך כדי להציג רק חלק יחסי מתמונת הפס, מה שיוצר אפקט "מילוי" ללא צורך באנימציות מורכבות.
4. אופטימיזציית משאבים: שימוש ב-BitmapManager לטעינה אחת של הנכסים הגרפיים עבור כל חלקי המשחק.

הסבר על תכונות המחלקה:

- private static final Bitmap xpImg, containerBg, containerFrame, progress
 - תחום הכרה: private static final.
 - תפקיד ושימוש: אוסף נכסי הגרפיקה המרכיבים את הפקד (אייקון, מיכל, מסגרת ופס). בזכות היותם סטטיים, הם נטענים פעם אחת בלבד לזיכרון ה-RAM ומשרתים את כל מופעי המחלקה באפליקציה.
- private final Rect srcRect, dstRect
 - תחום הכרה: private final.
 - תפקיד ושימוש: מלבני הגדרה לציור. srcRect קובע איזה חלק מהתמונה המקורית "ייחתך", ו-dstRect קובע היכן הוא יצויר על המסך.
- private double currentXP
 - תחום הכרה: private.
 - תפקיד ושימוש: שומרת את ערך הניסיון האחרון שרונדר. משמשת למניעת חישובים חוזרים בכל פריים במידה ולא חל שינוי בנתוני השחקן.
- private final TextRenderer lv
 - תחום הכרה: private final.
 - תפקיד ושימוש: רכיב עזר לרינדור טקסט הדרגה מעל האייקון.

הסבר פעולות מרכזיות:

- private void updateProgress()
 - זוהי הפעולה הלוגית המרכזית האחראית על חישוב מצב הפס. הפעולה בודקת אם חל שינוי בערך ה-XP בענן. אם כן, היא שולפת את היעד לעליית רמה (neededXpForLevelUp) ומחשבת את היחס ביניהם. על בסיס יחס זה, היא מעדכנת את רוחב מלבני החיתוך (srcRect ו-dstRect). פעולה זו מבטיחה שפס ההתקדמות יציג בדיוק את מצבו האמיתי של השחקן בכל רגע נתון.
- קוד:

```
private void updateProgress() {
    double xp = MyApp.getProgress().getXp();
    if (xp == currentXP) return;

    currentXP = xp;
    float needed = MyApp.getProgress().neededXpForLevelUp();
    float pct = (float) (currentXP / needed);

    int cutW = Math.round(progWidth * Math.min(1f, pct));

    srcRect.set(0, 0, cutW, progHeight);
}
```



```
dstRect.set(progressOffset, (int) y, progressOffset + cutW, (int) (y + progHeight));
}
```

● public void draw(Canvas c)

- מנהלת את הרינדור השכבתי של רכיבי התצוגה. הפעולה קוראת ל-drawContainer שמציירת את הרקע, את הפס החתוך (לפי ה-Rects שחושבו ב-update) ואת המסגרת מעל. לאחר מכן היא מציירת את אייקון ה-XP ואת מספר הדרגה. שימוש בסדר ציור זה (שכבות) מבטיח שהפס יראה כאילו הוא נמצא "בתוך" המיכל.

○ קוד:

```
private void drawContainer(Canvas c) {
    updateProgress();
    c.drawBitmap(containerBg, containerOffset, y, null);
    c.drawBitmap(progress, srcRect, dstRect, null);
    c.drawBitmap(containerFrame, containerOffset, y, null);
}
```

בסיס נתונים (Firebase Firestore)

המחלקה משתמשת בנתונים המסונכרנים מול מסד הנתונים כדי להציג מידע עדכני.

- סקירת המידע: המידע נשמר בשדות xp (מסוג double) ו-level (מסוג int) בתוך מסמך ה-ProgressDoc.
- פעולות:

- טעינה: בכל פריים, המחלקה ניגשת לאובייקט ה-Cached של ההתקדמות דרך MyApp.getProgress(). ברגע שפעולה אחרת במשחק (כמו הבסת אויב) מעדכנת את ה-XP בענן, רכיב זה משקף את השינוי באופן אוטומטי בפריים הבא.



מחלקת CoinDisplay

מיקום: com.example.tutorialgame.engine.ui.CoinDisplay

תפקיד המחלקה: המחלקה מהווה רכיב HUD (Heads-Up Display) האחראי על הצגת כמות המטבעות שבידי השחקן בזמן אמת. תפקידה המרכזי הוא לספק משוב ויזואלי דינמי על ידי "החלקה" (Sliding) של פאנל המידע לתוך המסך בעת איסוף מטבע, והסתתרותו באופן אוטונומי לאחר זמן קצר. בנוסף, היא משמשת כנקודת ייחוס גיאומטרית (Global Destination) עבור ישויות המטבע בעולם, ומכוונת אותן ליעד הטיסה המדויק בממשק המשתמש.

היא אחראית על:

1. ניהול תצוגת מונים: מעקב אחר כמות המטבעות במסמך הקוסמטיקה ורינדור הטקסט המתאים.
2. ניהול אנימציה: ביצוע מעברי תנועה חלקים של פאנל התצוגה (כניסה ויציאה מחלקו העליון של המסך).
3. סנכרון מערכות צירים: אספקת קואורדינטות מסך אבסולוטיות עבור אנימציות ה"טיסה" של המטבעות שנאספו.
4. ניהול זמן תצוגה: שליטה על משך הזמן שהפאנל נשאר גלוי למשתמש לאחר העדכון האחרון.

הסבר על תכונות המחלקה:

- private final Bitmap coinImg
 - תחום הכרה: private final.
 - תפקיד ושימוש: שומרת את האייקון הגרפי של המטבע שיוצג בפאנל. המידע נטען דרך ה- BitmapManager לחיטון בזיכרון.
- private final NinePatchDrawable background
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט גרפי מסוג Patch-9 המשמש כרקע לפאנל. סוג זה מאפשר הרחבה של הרקע בהתאם לאורך המספר מבלי לאבד את איכות הפינות.
- private float offset
 - תחום הכרה: private.
 - תפקיד ושימוש: מייצג את מיקום ה-Y הנוכחי של הפאנל יחסית לקצה המסך. משמש ליצירת אפקט ההחלקה.
- public static float xDestination, yDestination
 - תחום הכרה: public static.
 - תפקיד ושימוש: קואורדינטות היעד הגלובליות. כל מטבע שנאסף בעולם פונה למשתנים אלו כדי לדעת לאן עליו "לעוף" על המסך. ה-yDestination מתעדכן דינמית לפי גובה הפאנל.
- private long lastUpdate
 - תחום הכרה: private.
 - תפקיד ושימוש: חותמת זמן של השינוי האחרון במאזן המטבעות, משמשת לחישוב הזמן שנותר עד להסתרת הפאנל.

הסבר פעולות מרכזיות:

- public void update(double delta)
 - מנהלת את הלוגיקה של הפאנל בכל פריים של המשחק. הפעולה מבצעת שלוש פעולות:
 - i. האם כמות המטבעות בענן השתנתה? (אם כן, מעדכנת מיקומים וחותמת זמן).
 - ii. מפעילה את חישובי האנימציה.
 - iii. מעדכנת את יעד הטיסה הסטטי (yDestination) כך שיתאים לגובה הנוכחי של הפאנל המונפש.

קוד: ○

```
public void update(double delta) {
```



```
int currentCoins = MyApp.getCosmetic().getCoinsLeft();
if (coinsCount != currentCoins) {
    coinsCount = currentCoins;
    setPositions();
}
slideAnimation(delta);
yDestination = (4.5f * SCALE_MULTIPLIER) + offset;
}
```

- private void slideAnimation(double delta)
 - אחראית על התנועה הפיזיקלית של הפאנל.
 - **תרשים זרימה (סדר פעולות):**
 - i. חישוב מהירות תנועה יחסית ל-delta.
 - ii. בדיקת זמן: האם עברו פחות מ-3 שניות מהעדכון האחרון?
 - כן: קידום ה-offset כלפי מטה עד להגעה ל-0 (מצב גלוי).
 - לא: קידום ה-offset כלפי מעלה עד להסתרה מוחלטת מעל גבולות המסך.

○ קוד:

```
private void slideAnimation(double delta) {
    float speed = (float) (400 * delta); // Pixels per second adjusted by delta
    if (System.currentTimeMillis() - lastUpdate <= 3000) {
        if (offset < 0) offset += speed;
        else offset = 0;
    } else {
        if (offset > -background.getBounds().height()) offset -= 1.2f * speed;
        else offset = -background.getBounds().height();
    }
}
```

- public void draw(Canvas c)
 - כדי שהפאנל ינוע כיחידה אחת, הפעולה משתמשת ב-c.translate המזיז את כל מערכת הצירים של הקנבס לפי ה-offset הנוכחי. לאחר מכן מצוירים הרקע, האייקון והטקסט במיקומים היחסיים שלהם. השימוש ב-save() ו-restore() מבטיח שהזזת הקנבס לא תשפיע על אלמנטים אחרים ב-UI.

○ קוד:

```
public void draw(Canvas c) {
    if (offset <= -background.getBounds().height()) return;
    c.save();
    c.translate(0, offset);
    background.draw(c);
    c.drawBitmap(coinImg, xDestination, 4.5f * SCALE_MULTIPLIER, null);
    textPaint.drawText(String.valueOf(coinsCount), c);
    c.restore();
}
```

בסיס נתונים (Firestore)

המחלקה אינה כותבת ישירות למסד הנתונים, אך היא "מאזינה" לשינויים בו.

- סקירת המידע: המידע מגיע ממסמך ה-CosmeticDoc ותלוי בערך השדה coins.
- פעולות:
 - טעינה: בבנאי ובכל קריאה ל-update, המחלקה ניגשת ל-MyApp.getCosmetic().getCoinsLeft(). ברגע שפעולת ה-addCoin (במחלקה אחרת) מעדכנת את הענן, רכיב זה מזהה את השינוי ומפעיל את האנימציה באופן אוטומטי.

**מחלקת TapEffect**מיקום: `com.example.tutorialgame.engine.ui.effects.tapeffects.TapEffect`

תפקיד המחלקה: מחלקה זו מנהלת אפקט ויזואלי קצר-מועד (Visual Feedback) המופעל בעקבות אינטראקציית מגע של המשתמש במסך או אירועי מערכת. תפקידה הוא לחזק את תחושת ה"היענות" (Responsiveness) של המשחק לקלט של השחקן. המחלקה מחשבת בזמן אמת את הפריים המתאים להצגה על פי הזמן שחלף מרגע הלחיצה, ומבטיחה שהאפקט יסתיים בדיוק לאחר פרק זמן מוגדר מראש.

היא אחראית על:

1. ניהול תזמון עצמאי: חישוב משך החיים של האפקט מרגע ההפעלה ועד להיעלמותו המוחלטת.
2. חישוב אנימציה מבוסס זמן: גזירת אינדקס התמונה הנכון מתוך ה-Sprite-Sheet בהתבסס על היחס שבין הזמן שעבר לבין משך האפקט הכולל.
3. רינדור גרפי ממוקד: ציור האפקט במיקום המדויק של המגע תוך ביצוע "מרכז" אוטומטי של התמונה.
4. ורסטיליות עיצובית: תמיכה בסוגים שונים של אפקטים (עיגולים, ניצוצות וכו') דרך אובייקט `TapEffects` הנתונים.

הסבר על תכונות המחלקה:

- `private static final long TOTAL_DURATION_MS = 250`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: קבוע המגדיר את משך הזמן המקסימלי (רבע שנייה) שבו האפקט יוצג על המסך.
- `private TapEffects tapType`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: רפרנס לסוג האפקט הנבחר (Enum). הוא מכיל את אוסף התמונות ואת כמות הפריימים הייחודית לכל סוג.
- `private long startTime`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה השומר את זמן המערכת (Timestamp) ברגע שבו האפקט הופעל. משמש כנקודת ייחוס לכל חישובי האנימציה. ערך של -1 מצוין שהאפקט אינו פעיל כרגע.
- `private long elapsed`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה השומר את הזמן שעבר (במילישניות) מאז תחילת האפקט.

הסבר פעולות מרכזיות:

- `public void show(Canvas c, float x, float y)`
 - זוהי הפעולה המרכזית המשלבת לוגיקה ורינדור בכל פריים.
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקת סטטוס: אם האפקט לא הופעל (`startTime < 0`) או שסוג האפקט לא הוגדר, הפעולה יוצאת ללא ביצוע.
 - ii. חישוב זמן: חישוב ה-`elapsed` (הזמן הנוכחי פחות זמן ההתחלה).
 - iii. בדיקת סיום: אם הזמן שעבר עולה על `TOTAL_DURATION_MS`, הפעולה מאפסת את ה-`startTime` ל-1 ומפסיקה את הציור.
 - iv. ישוב אינדקס פריים: המרת הזמן שעבר לאינדקס תמונה (0 עד סוף המערך) באמצעות הנוסחה: `(elapsed * frameCount) / duration`.
 - v. רינדור: שליפת ה-`Bitmap` המתאים וציורו במרכז נקודת המגע (תוך שימוש ב-`SCALE_MULTIPLIER` להתאמה לרזולוציה).



קוד: ○

```
public void show(Canvas c, float x, float y) {
    if (startTime < 0 || tapType == null) return;

    elapsed = System.currentTimeMillis() - startTime;
    if (elapsed > TOTAL_DURATION_MS) {
        startTime = -1;
        return;
    }

    int index = (int) ((elapsed * tapType.getFrameCount()) / TOTAL_DURATION_MS);
    if (index >= tapType.getFrameCount()) index = tapType.getFrameCount() - 1;
    Bitmap sprite = tapType.getSprite(index);
    c.drawBitmap(sprite,
        x - (tapType.getWidthHeight() / 2f * SCALE_MULTIPLIER),
        y - (tapType.getWidthHeight() / 2f * SCALE_MULTIPLIER),
        null);
}
```

public void resetEffect() ●

○ זוהי פעולת ה"איפוס והפעלה" של האפקט. הפעולה מעדכנת את ה-startTime לזמן הנוכחי של המערכת. פעולה זו גורמת למתודת ה-show להתחיל בחישובי האנימציה ובציוור בפריים הבא. היא נקראת בדרך כלל מתוך ה-onTouchEvent ברגע זיהוי לחיצה חדשה.

קוד: ○

```
public void resetEffect() {
    this.startTime = System.currentTimeMillis();
}
```


**מחלקת CharacterRenderer**

מיקום: com.example.tutorialgame.engine.renderer.CharacterRenderer

תפקיד המחלקה: מחלקה זו משמשת כ"מנוע הצیור" הייעודי של הישויות החיות (Characters) במשחק. תפקידה הוא ליישם את עקרון הפרדת הרשויות: בעוד שמחלקת ה-Character מנהלת את הלוגיקה (מיקום, חיים, נשק), ה-CharacterRenderer מנהלת אך ורק את אופן התצוגה שלהן. היא מטפלת בציור השכבות השונות של הדמות (צל, גוף, נשק, ממשק) תוך שימוש בטכניקות לסימולציית עומק וגובה בעולם דו-ממדי.

היא אחראית על:

1. רינדור רב-שכבתי (Depth Rendering): ציור רכיבי הדמות בסדר הנכון (צל -> גוף -> נשק -> UI) כדי לשמור על עקביות ויזואלית.
2. סימולציית גובה ומרחק: יצירת אפקט תלת-ממדי על ידי הזזת גוף הדמות כלפי מעלה בזמן קפיצה, בעוד הצל מתרחק מהדמות ומשנה את שקיפותו בהתאם.
3. ניהול נשק ואנימציה: חישוב זווית הסיבוב והמיקום המדויק של הנשק בהתאם לכיוון המבט ומצב הדמות.
4. אופטימיזציית ביצועים: טעינת הגדרות גלובליות (כמו הצגת Hitboxes) פעם אחת בלבד לכלל המופעים באמצעות בלוק אתחול סטטי.

הסבר על תכונות המחלקה:

- private final Character character
 - תחום הכרה: private final.
 - תפקיד ושימוש: רפרנס לדמות אותה המחלקה מציירת. משמש לשליפת נתונים בזמן אמת כמו קואורדינטות ה-Hitbox, פריים האנימציה הנוכחי וגובה הקפיצה.
- private static boolean SHOW_HITBOX
 - תחום הכרה: private static.
 - תפקיד ושימוש: דגל גלובלי השולט על הצגת תיבות הפגיעה. השימוש במשתנה סטטי מאפשר שינוי של תצוגת כל הדמויות במשחק בו-זמנית דרך הגדרות הוידאו.
- private final Paint shadowPaint, hitboxPaint
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקטי "מברשת" המגדירים את סגנון הציור. shadowPaint מוגדרת עם שקיפות משתנה, ו-hitboxPaint מוגדרת בסגנון קווי (Stroke) לצורכי ניפוי באגים.
- private final Bitmap shadowBitmap
 - תחום הכרה: private final.
 - תפקיד ושימוש: תמונת הצל הקבועה המצוירת בבסיס הדמות.

הסבר פעולות מרכזיות:

- static { ... } (בלוק אתחול סטטי)
 - בלוק זה מתבצע פעם אחת בלבד עם טעינת המחלקה. הוא ניגש לקובץ ה-SharedPreferences ושולף את הגדרת המשתמש לגבי הצגת תיבות פגיעה. פעולה זו במיקום זה מבטיחה שכל מופעי הדמויות יכירו את ההגדרה מיד עם היווצרותן, מבלי לבצע קריאות חוזרות ומיותרות לדיסק (I/O).

○ קוד:

```
static {
    SharedPreferences sp = BaseActivity.getContext().getSharedPreferences("settings",
Context.MODE_PRIVATE);
    SHOW_HITBOX = sp.getBoolean("hitbox", false);
}
```



- `public void drawFullCharacter(Canvas c)`

- זוהי מתודת הניהול הראשית המפעילה את כל שלבי הציור. כדי שהדמות תראה נכון, הציור חייב להתבצע בסדר שכבות מדויק מהנמוך לגבוה (אלגוריתם הצייר). פעולה זו מבטיחה שכל רכיבי הדמות יציירו במקומם היחסי הנכון.

○ קוד:

```
public void drawFullCharacter(Canvas c) {
    drawShadow(c); // שכבה 1: צל
    drawCharacter(c); // שכבה 2: גוף
    drawWeapon(c); // שכבה 3: נשק
    drawEmote(c); // שכבה 4: דיבור/רגש
    drawHealthBar(c); // שכבה 5: ממשק ממשק (HP)
}
```

- `private void drawShadow(Canvas c)`

- מציירת את הצל הדינמי מתחת לדמות. הפעולה מבצעת סימולציה של מרחק מהקרקע. ככל שהדמות עולה (קופצת), הצל הופך לשקוף יותר ומוזז מעט כלפי מטה יחסית לדמות. שילוב זה יוצר אשליה אופטית של גובה.

○ קוד:

```
private void drawShadow(Canvas c) {
    // Shadow transparency based on jump height
    shadowPaint.setAlpha(Math.round(200 * (1 - character.getJumpFraction())));

    // Shadow moves down slightly as the character ascends to simulate distance
    float shadowY = character.getHitBox().bottom - 5 * SCALE_MULTIPLIER +
        (character.getElevation() / SCALE_MULTIPLIER);

    c.drawBitmap(shadowBitmap, character.getHitBox().left, shadowY, shadowPaint);
}
```

- `private void drawCharacter(Canvas c)`

- מציירת את גוף הדמות ואת תיבת הפגיעה שלה. הפעולה משתמשת ב-`c.save()` וב-`c.translate()` כדי להזיז את הדמות אנכית (קפיצה) מבלי להשפיע על שאר הסביבה. היא שולפת את ה-`Bitmap` הנכון מה-`SpriteSheet` של הדמות לפי כיוון המבט ופריים האנימציה הנוכחי.

○ קוד:

```
private void drawCharacter(Canvas c) {
    c.save();
    // Visual vertical displacement for jumping
    c.translate(0, -character.getElevation());

    Bitmap sprite = character.getGameCharType().getSprite(character.getAniIndex(),
        character.getFaceDir());
    float drawX = character.getHitBox().left - X_DRAW_OFFSET;
    float drawY = character.getHitBox().top - Y_DRAW_OFFSET;

    c.drawBitmap(sprite, drawX, drawY, null);

    if (SHOW_HITBOX)
        c.drawRect(character.getHitBox(), hitboxPaint);
    c.restore();
}
```

- `private void drawWeapon(Canvas c)`

- מטפלת בציור הנשק וסיבובו בזמן התקפה. הפעולה מוודאת שהדמות חיה ומבצעת התקפה. היא משתמשת ב-`c.rotate()` כדי לסובב את הנשק סביב נקודת האחיזה המדויקת של היד (Pivot Point), ומחשבת את הסטייה הנדרשת כדי שהנשק ייראה מוחזק בצורה טבעית.

○ קוד:



```
private void drawWeapon(Canvas c) {
    if (character.isDead() || !character.isAttacking() || character.getWeapon() == null)
        return;
    character.updateWepHitbox();

    c.save();
    c.rotate(character.getWepRot(),
             character.getAttackBox().left,
             character.getAttackBox().top);

    c.drawBitmap(character.getWeapon().getWeaponInHandImg(),
                 character.getAttackBox().left + character.wepAdjustLeft() +
character.wepAdjustBottom(),
                 character.getAttackBox().top + character.wepAdjustTop(),
                 null);

    c.restore();

    if (SHOW_HITBOX)
        c.drawRect(character.getAttackBox(), hitboxPaint);
}
```

● (private void drawEmote(Canvas c) ו-drawHealthBar(Canvas c)

- פעולות המציירות שכבות ממשק מעל הדמות. drawEmote בודקת אם קיים רכיב רגשות פעיל ומציירת אותו. drawHealthBar מציירת את מד החיים רק עבור דמויות NPC שאינן בשיא חייהן, מה שמונע עומס וזיזואלי על המסך כאשר הכל תקין.

○ קוד:

```
private void drawEmote(Canvas c) {
    if (character.getEmoteComponent() != null) {
        character.getEmoteComponent().drawEmote(c);
    }
}

private void drawHealthBar(Canvas c) {
    if (character.getCurrentHealth() < character.getMaxHealth() &&
        character.getFaction() != GameConstants.Faction.PLAYER)
        healthBar.drawBar(c);
}
```

**מחלקת BaseButton**מיקום: `com.example.tutorialgame.engine.ui.customviews.buttons.BaseButton`

תפקיד המחלקה: מחלקה אבסטרקטית זו מהווה את השלד והמוח עבור כל הכפתורים המותאמים אישית במשחק. היא אינה ניתנת ליצירה ישירה, אלא מספקת את כל הלוגיקה המורכבת של ניהול אירועי מגע (`MotionEvent`), ומשאירה למחלקות היורשות (`RectButton`, `CircleButton`) לממש רק את החלקים הייחודיים להן: צורת ה-`hitbox` ופעולת הציור.

היא אחראית על:

1. ניהול מגע מתקדם: היא מכילה `eventHandler` חזק המטפל באירועי `multi-touch`, עוקב אחר `pointerId` ספציפי, ומבדיל בין מצבי לחיצה, גרירה ושחרור.
2. בעלות בלעדית (`Exclusive Ownership`): היא מממשת מנגנון סטטי חכם (`exclusiveOwner`) המונע מכפתורים שאינם תומכים ב-`multi-touch` להפריע זה לזה. ברגע שכפתור כזה נלחץ, הוא הופך ל"בעלים הבלעדי" של הקלט עד לשחרורו.
3. מנגנון תגובה (`Callback`): היא חושפת ממשק `OnClickListener` ומאפשרת לקוד חיצוני להירשם ולהגיב לאירוע הלחיצה, ובכך מפרידה בצורה נקייה בין זיהוי הלחיצה לבין הפעולה שמתבצעת כתוצאה ממנה.
4. גמישות תגובה: היא תומכת בשני סוגי תגובה (`PressType`): תגובה מיידית בלחיצה (`ON_DOWN`), או תגובה רק לאחר שחרור האצבע מהכפתור (`ON_UP`), שהיא ברירת המחדל.

הסבר על תכונות המחלקה:

- `private static BaseButton exclusiveOwner`
 - תחום הכרה: `private static BaseButton`
 - תפקיד ושימוש: זוהי תכונה סטטית ומרכזית. היא מחזיקה רפרנס לכפתור (שאינו `multi-touch`) שנלחץ כרגע. כל עוד היא אינה `null`, כפתורים אחרים שאינם `multi-touch` יתעלמו מאירועי קלט, ובכך נמנע "גניבת" קלט ביניהם. זהו פתרון אלגנטי לניהול קלט יחיד על פני מספר כפתורים.
- `protected final RectF hitbox`
 - תחום הכרה: `protected final RectF`
 - תפקיד ושימוש: מגדיר את האזור המלבני של הכפתור המגיב ללחיצות. תחום ההכרה `protected` מאפשר למחלקות היורשות גישה לצורך חישובי `isIn`.
- `protected boolean pushed`
 - תחום הכרה: `protected boolean`
 - תפקיד ושימוש: במערכות `multi-touch`, לכל אצבע על המסך יש ID ייחודי (`pointerId`). משתנה זה שומר את ה-ID של האצבע שלחצה ראשונה על הכפתור, ומבטיח שהכפתור יגיב רק לתנועות ושחרור של אותה אצבע ספציפית.
- `protected PressType pressType`
 - תחום הכרה: `protected PressType`
 - תפקיד ושימוש: `enum` הקובע את אופן התגובה של הכפתור: האם הפעולה תתבצע מיד עם הלחיצה (`ON_DOWN`) או רק לאחר שהאצבע שוחררה מעל הכפתור (`ON_UP`).
- `private OnClickListener onClickListener`
 - תחום הכרה: `private OnClickListener`
 - תפקיד ושימוש: שומר רפרנס לאובייקט המאזין שהוגדר לכפתור. כאשר `eventHandler` מזהה לחיצה חוקית, הוא יקרא למתודת `onClick` של אובייקט זה.

הסבר פעולות מרכזיות:

- `public boolean eventHandler(MotionEvent event)`



○ זוהי הפעולה המרכזית והמורכבת ביותר במחלקה, המהווה את לב מנגנון הקלט. היא מקבלת `MotionEvent` ממערכת ההפעלה ומפרקת אותו כדי לנהל את כל ההיבטים של אינטראקציה עם הכפתור.

○ **תרשים זרימה (סדר פעולות):**

i. `ACTION_DOWN / ACTION_POINTER_DOWN` (לחיצה ראשונית):

1. בודקת אם הכפתור נמצא תחת האצבע (`isIn`).
2. אם כן, ולכפתור אין בעלות בלעדית, היא הופכת ל-`exclusiveOwner`.
3. שומרת את ה-`pointerId` של האצבע שלחצה ומגדירה את `pushed` ל-`true`.
4. אם סוג הלחיצה הוא `ON_DOWN`, היא מפעילה את ה-`onClickListener` באופן מיידי.

ii. `ACTION_MOVE` (תנועה):

1. עוקבת אחר האצבע עם ה-`pointerId` השמור.
2. אם האצבע זזה אל מחוץ לגבולות הכפתור (`isIn` מחזיר `false`), היא משחררת את מצב הלחיצה (`pushed = false`).

iii. `ACTION_UP / ACTION_POINTER_UP` (שחרור):

1. בודקת אם האצבע ששחררה היא זו שלחצה במקור (`pointerId == pid`).
2. בודקת אם האצבע עדיין נמצאת בתוך גבולות הכפתור (`isIn`).
3. אם כל התנאים מתקיימים וסוג הלחיצה הוא `ON_UP`, היא מפעילה את ה-`onClickListener`.
4. משחררת את ה-`pointerId` ואת הבעלות הבלעדית (`exclusiveOwner`).

○ **קוד:**

```
public boolean eventHandler(MotionEvent event) {
    if (!enabled) return false;

    int action = event.getActionMasked();
    int actionIndex = event.getActionIndex();
    int pid = event.getPointerId(actionIndex);

    switch (action) {
        case MotionEvent.ACTION_DOWN:
        case MotionEvent.ACTION_POINTER_DOWN: {
            float x = event.getX(actionIndex), y = event.getY(actionIndex);

            if (!this.multitouch && exclusiveOwner != null && exclusiveOwner != this)
                return false;

            if (isIn(x, y)) {
                if (!this.multitouch && exclusiveOwner == null)
                    exclusiveOwner = this;
                setPushed(true, pid);

                if (vibe) BaseActivity.ButtonPressVibe();
                if (pressType == PressType.ON_DOWN) {
                    // --- לוגית הפעלה ---
                    if (onClickListener != null) {
                        onClickListener.onClick(this);
                    }
                }
                SoundManager.getInstance(BaseActivity.getContext()).playSfx(R.raw.sfx_button_pressed);
            }
            return true;
        }
        case MotionEvent.ACTION_MOVE:
            // ... (code for ACTION_MOVE) ...
            break;
        case MotionEvent.ACTION_UP:
        case MotionEvent.ACTION_POINTER_UP:
            // ... (code for ACTION_UP) ...
            break;
    }
}
```



```
case MotionEvent.ACTION_MOVE: {
    if (pointerId != -1) {
        int idx = findPointerIndex(event, pointerId);
        if (idx >= 0) {
            if (!isIn(event.getX(idx), event.getY(idx))) pushed = false;
        } else {
            pushed = false;
            if (exclusiveOwner == this) exclusiveOwner = null;
            pointerId = -1;
        }
    }
    break;
}

case MotionEvent.ACTION_CANCEL: {
    pushed = false;
    if (exclusiveOwner == this) exclusiveOwner = null;
    pointerId = -1;
    break;
}

case MotionEvent.ACTION_UP:
case MotionEvent.ACTION_POINTER_UP: {
    float ux = event.getX(actionIndex), uy = event.getY(actionIndex);
    boolean wasPushed = (pointerId == pid) && pushed && isIn(ux, uy);

    if (wasPushed && pressType == PressType.ON_UP) {
        if (onClickListener != null) {
            onClickListener.onClick(this);
        }
    }

    SoundManager.getInstance(BaseActivity.getContext()).playSfx(R.raw.sfx_button_pressed);

    if (pointerId == pid) {
        pushed = false;
        pointerId = -1;
    }

    if (exclusiveOwner == this) exclusiveOwner = null;
    if (pressType == PressType.ON_UP) return wasPushed;
    else return false;
}

return false;
}
```

• public static void releaseExclusiveOwner()

- מתודה סטטית חשובה שנועדה ל"ניקוי" חיצוני. במצבים מסוימים (כמו מעבר בין מסכים), יש צורך לוודא שאף כפתור לא נשאר במצב "תפוס". קריאה למתודה זו מאפסת את ה-exclusiveOwner ומבטיחה שהמערכת חוזרת למצב נקי.

○ קוד:

```
public static void releaseExclusiveOwner() {
    if (exclusiveOwner != null) {
        exclusiveOwner.pushed = false;
        exclusiveOwner.pointerId = -1;
        exclusiveOwner = null;
    }
}
```



מחלקת CircleButton

מיקום: com.example.tutorialgame.engine.ui.customviews.buttons.circles.CircleButton.java

תפקיד המחלקה: מחלקה זו היא מימוש קונקרטי (Concrete Implementation) של BaseButton ומייצגת כפתור בעל צורה עגולה. היא מקבלת את כל לוגיקת ניהול המגע, הבעלות הבלעדית והפעלת ה-ClickListner בירושה מ-BaseButton.

היא אחראית על:

1. הגדרת אזור הלחיצה: היא דורסת (override) את המתודה isIn ומממשת לוגיקה המגדירה את האזור הלחיצה שלה כעיגול מושלם, ולא כמלבן ה-hitbox הכללי.
2. ציור הכפתור: היא דורסת את המתודה draw ומממשת את לוגיקת הציור, הכוללת בחירה והצגה של התמונה הנכונה (normal, pressed, disabled) בהתאם למצב הכפתור הנשלט על ידי BaseButton.

הסבר על תכונות המחלקה:

- private final Bitmap normal, pressed, disabled
 תחום הכרה: private final Bitmap
 תפקיד ושימוש: אלו הם שלושת ה-Bitmap המייצגים את המצבים היוזואליים של הכפתור. הם מאותחלים פעם אחת בבנאי מתוך אובייקט ה-CircleImages enum שמועבר אליו, ונשמרים לשימוש מהיר במתודות הציור.

הסבר פעולות מרכזיות:

- protected boolean isIn(float x, float y)
 זהו מקבלת קואורדינטות של לחיצה ובודקת אם הן נמצאות בתוך העיגול. בניגוד לבדיקה מלבנית פשוטה, היא משתמשת בנוסחת פיתגורס (Math.hypot) כדי לחשב את המרחק בין נקודת הלחיצה למרכז הכפתור. רק אם המרחק קטן מרדיוס הכפתור, המתודה מחזירה true. זה מה שמבטיח שהפינוט השקופות של התמונה המרובעת לא יגיבו ללחיצה.

קוד:

```
@Override
protected boolean isIn(float x, float y) {
    float cx = hitbox.centerX();
    float cy = hitbox.centerY();
    float dx = x - cx;
    float dy = y - cy;
    float dist = (float) Math.hypot(dx, dy);
    float radius = Math.min(hitbox.width(), hitbox.height()) / 2f;
    return dist <= radius;
}
```

- public void draw(Canvas c)

- זוהי המימוש של חובת הציור מ-BaseButton. היא קוראת למתודות העזר הפנימיות getImg() כדי לקבוע איזה Bitmap להציג (רגיל, לחוץ או מושבת) בהתבסס על הדגלים enabled ו-pushed שירשה מ-BaseButton, ולאחר מכן מציירת את התמונה הנבחרת על הקנבס במיקום המדויק של ה-hitbox.

קוד:

```
@Override
public void draw(Canvas c) {
    c.drawBitmap(getImg(), hitbox.left, hitbox.top, null);
}
```



מחלקת RectButton

מיקום: com.example.tutorialgame.engine.ui.customviews.buttons.rects.RectButton.java

תפקיד המחלקה: מחלקה זו היא מימוש קונקרטי של BaseButton המייצגת כפתור בעל צורה מלבנית. היא נועדה לשמש ככפתור הסטנדרטי בממשקי המשתמש של המשחק (למשל, בתפריטים). בניגוד ל-CircleButton, היא מנצלת את העוצמה של NinePatchDrawable, מה שמאפשר לה להימתח לכל גודל רצוי מבלי לעוות את פינותיה או את המסגרת שלה.

היא אחראית על:

1. הגדרת אזור לחיצה מלבני: היא מממשת את המתודה isIn באמצעות בדיקה מלבנית פשוטה ויעילה.
2. ציור הכפתור והטקסט: היא אחראית על ציור הרקע של הכפתור באמצעות NinePatchDrawable, ובנוסף, היא מטפלת בציור טקסט על גבי הכפתור. לוגיקת הציור שלה כוללת אפקט צל לטקסט, והזזה קלה של הטקסט כלפי מטה כאשר הכפתור לחוץ, כדי לספק פידבק ויזואלי נוסף למשתמש.

הסבר על תכונות המחלקה:

- private NinePatchDrawable normal, pressed
 - תחום הכרה: private NinePatchDrawable
 - תפקיד ושימוש: אלו הם שני אובייקטי ה-NinePatchDrawable המייצגים את המצבים הויזואליים של רקע הכפתור (רגיל ולחוץ). הם מאותחלים בבנאי מתוך אובייקט RectImages ומותאמים לגבולות ה-hitbox של הכפתור.
- private RectImages rectImages
 - תחום הכרה: private RectImages
 - תפקיד ושימוש: זהו רפרנס לאובייקט ה-enum ששימש ליצירת הכפתור. ה-enum RectImages משמש כ"מחסן נתונים" מרכזי עבור הכפתור, ומספק לא רק את תמונות הרקע, אלא גם את כל המידע הקשור לטקסט: תוכן הטקסט, מיקומו, וסגנון הציור (Paint) שלו ושל הצל שלו.

הסבר פעולות מרכזיות:

- public void draw(Canvas c)
 - זוהי הפעולה המממשת את חובת הציור מ-BaseButton. היא מורה ל-RectImages לעדכן את מיקום הטקסט שלו בהתבסס על מצב הלחיצה הנוכחי (pushed), ולאחר מכן מציירת את הרקע והטקסט (כולל הצל) באמצעות קריאות פשוטות לאובייקט rectImages. כל מורכבות הציור מוסתרת כעת בתוך RectImages ו-TextRenderer.
 - קוד:

```
@Override
public void draw(Canvas c) {
    rectImages.setTextPos(pushed);
    this.getBtnImg().draw(c);
    rectImages.getTextPaint().drawWithShadow(rectImages.getText(), c);
}
```

- protected boolean isIn(float x, float y)
 - מימוש בדיקת הפגיעה מ-BaseButton. מפני שהכפתור מלבני, הוא משתמש במתודה המובנית hitbox.contains(x, y) כדי לבדוק אם נקודת הלחיצה נמצאת בתוך גבולותיו.
 - קוד:

```
@Override
protected boolean isIn(float x, float y) {
    return hitbox.contains(x, y);
}
```




מחלקת CustomSeekBar

מיקום: com.example.tutorialgame.engine.ui.customviews.CustomSeekBar

תפקיד המחלקה: מחלקה זו מייצגת פקד בחירה על ציר (SeekBar) מותאם אישית. היא משמשת בממשקי המשתמש של המשחק (כמו מסך הגדרות השמע) כדי לאפשר למשתמש לבחור ערך על סקאלה רציפה. המחלקה מיישמת את תבנית העיצוב "צופה" (Observer Pattern) המאפשרת לחלקי מערכת אחרים להגיב לשינויים בזמן אמת, ומנהלת באופן עצמאי את כל היבטי הקלט, הלוגיקה והרינדור של הפקד.

היא אחראית על:

1. ניהול ערך התקדמות (Progress): חישוב ושמירה של ערך נורמלי (בין 0.0 ל-1.0) המייצג את מצב הפקד.
2. דיווח אירועים: חשיפת ממשק Callback המאפשר לקוד חיצוני (כמו MusicSounds) להתעדכן בשינוי הערך מבלי לבצע בדיקות חוזרות (Polling).
3. רינדור גרפי מתקדם: שימוש ב-NinePatchDrawable למתיחה איכותית של הפס ושימוש ב-Caching של צבעים לשיפור ביצועי הציור בתוך לולאת המשחק.
4. משוב חוויית משתמש (UX): יצירת אפקטים של רטט וצלילי "טיק" בעת שינוי הערך, ושימוש בטכניקת Clipping ליצירת אפקט ויזואלי של טקסט משתנה.

הסבר על תכונות המחלקה:

- private OnProgressChangeListener listener
 - תחום הכרה: private.
 - תפקיד ושימוש: רפרנס לאובייקט המממש את הממשק (Interface) של המאזין. דרכו המחלקה מדווחת החוצה על שינויי ערך.
- private final RectF bgHitbox
 - תחום הכרה: private.
 - תפקיד ושימוש: מגדיר את גבולות הפקד על המסך. משמש לזיהוי לחיצות ולמיקום רכיבי הגרפיקה והטקסט.
- private final NinePatchDrawable normal, pressed, background
 - תחום הכרה: private.
 - תפקיד ושימוש: אובייקטי הגרפיקה של הפס. background הוא הרקע, normal הוא הפס המלא, ו-pressed הוא הפס כשהוא לחוץ. השימוש ב-NinePatch מבטיח שהפינות לא יתעוותו במתיחה.
- private final int colorPushedText, colorPushedShadow, colorNormalText, colorNormalShadow
 - תחום הכרה: private.
 - תפקיד ושימוש: משתנים אלו מאחסנים את ערכי הצבעים שנשלפו מה-Resources בזמן האתחול. שמירתם (Caching) מונעת קריאות יקרות למערכת בכל פריים של הציור, ובכך תורמת לביצועי ה-FPS של המשחק.
- private float progress
 - תחום הכרה: private.
 - תפקיד ושימוש: המשתנה הלוגי המרכזי. ערך עשרוני בין 0 ל-1 המייצג את מצב ה-SeekBar.
- private boolean pushed
 - תחום הכרה: private.
 - תפקיד ושימוש: דגל המציין האם המשתמש לוחץ כרגע על הפקד. משמש לקביעת מצב הציור (נורמלי או לחוץ).



- `private int currentPercent, lastPercent`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משמשים למעקב אחר אחוז ההתקדמות (0-100) לצורך הפעלת אפקטים קוליים רק כאשר האחוז משתנה בפועל.
- `private final TextRenderer percentRenderer, nameRenderer`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מופעים של מחלקת עזר לציור טקסט. `percentRenderer` אחראי על הצגת האחוזים ו-`nameRenderer` על שם הפקד.
- `private final float percentY, nameX, nameY`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: משתני מיקום (Coordinates) המחושבים פעם אחת בבנאי. הם קובעים את נקודת הציור המדויקת של הטקסטים ביחס למלבן הפקד, מה שחוסך חישובים חוזרים בכל פריים.
- `private final String name`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מחרוזת המכילה את שם הפקד (למשל "MUSIC").
- `private String percentText`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה "מטמון" (Cache) השומר את המחרוזת של האחוז הנוכחי (למשל "50%"), כדי למנוע יצירת אובייקטי String חדשים בכל פריים של הציור.
- `private float percentX`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: מיקום ה-X של טקסט האחוזים, המחושב מחדש רק כשהערך משתנה כדי לשמור על מרכז הטקסט.

הסבר פעולות מרכזיות:

- `public float eventHandler(MotionEvent event)`
 - פעולה זו מנהלת את אירועי המגע של המשתמש עם הפקד.
 - **תרשים זרימה (סדר פעולות):**
 - i. זיהוי סוג האירוע:
 1. `ACTION_DOWN`: בודקת אם הלחיצה בוצעה בתוך ה-`hitbox`. אם כן, מסמנת `pushed = true`, מפעילה רטט ומעדכנת את הערך.
 2. `ACTION_MOVE`: אם הפקד במצב לחוץ, מעדכנת את הערך באופן רציף לפי תנועת האצבע ומפעילה אפקטים.
 3. `ACTION_UP` / `ACTION_CANCEL`: משחררת את מצב הלחיצה (`pushed = false`).
 - ii. עדכון הערך: במקרה של לחיצה או תנועה, קוראת ל-`setProgressFromX` שממירה את מיקום ה-X לאחוז לוגי.
 - iii. החזרה: מחזירה את ערך ה-`progress` הנוכחי.
 - קוד:

```
public float eventHandler(MotionEvent event) {
    int action = event.getActionMasked();
    switch (action) {
        case MotionEvent.ACTION_DOWN:
            if (isIn(event)) {
                pushed = true;
                BaseActivity.ButtonPressVibe();
                setProgressFromX(event.getX());
            }
    }
}
```



```
        } else pushed = false;
        break;
    case MotionEvent.ACTION_MOVE:
        if (pushed) {
            setProgressFromX(event.getX());
            seekBarEffects();
        }
        break;
    case MotionEvent.ACTION_CANCEL:
    case MotionEvent.ACTION_UP:
        pushed = false;
        break;
    }
    return progress;
}
```

● public void setProgressNormalized(float p)

- מעדכנת את מצב הפקד ומודיעה למאזינים. הפעולה מבצעת clamping לערך, מעדכנת את גבולות הגרפיקה (bounds), מעדכנת את הטקסט המוצג, ומפעילה את ה-listener עם הרפרנס של המחלקה הנוכחית (this) והערך החדש.

○ קוד:

```
public void setProgressNormalized(float p) {
    float clamped = Math.max(0f, Math.min(1f, p));
    if (Math.abs(clamped - this.progress) < 0.0005f) return;

    this.progress = clamped;
    updateDrawableBoundsFromProgress();
    updatePercentCache();

    // Notify listener of the change
    if (listener != null) {
        listener.onProgressChanged(this, this.progress);
    }
}
```

● private void drawText(Canvas c)

- אחראית על הציור הדינמי והמתקדם של הטקסט. הפעולה משתמשת בטכניקת Clipping. היא מחשבת את קו הסיום של הפס המלא. היא מציירת את הטקסט פעמיים: פעם אחת תחת "מסכה" (ClipRect) של הפס המלא (עם הגדרות צל כהות), ופעם שנייה על כל השאר. זה יוצר את האפקט שבו הטקסט משנה סגנון בדיוק בקו המעבר של ה-SeekBar.

○ קוד:

```
private void drawText(Canvas c) {
    float filledRight = bgHitbox.left + (progress * bgHitbox.width());
    drawSeekBarName(c, filledRight);
    drawPercentsText(c, filledRight);
}

private void drawSeekBarName(Canvas c, float filledRight) {
    if (filledRight > nameX) {
        c.save();
        c.clipRect(nameX, bgHitbox.top, filledRight, bgHitbox.bottom);
        nameRenderer.drawWithShadow(name, c);
        c.restore();
    }
    nameRenderer.drawText(name, c);
}
```

**מחלקת CustomRadioGroup**

מיקום: com.example.tutorialgame.engine.ui.customviews.radiogroup.CustomRadioGroup

תפקיד המחלקה: מחלקה זו מייצגת קבוצת כפתורי בחירה (Radio Buttons). היא מאפשרת למשתמש לבחור אפשרות אחת בלבד מתוך רשימה. המחלקה מנהלת באופן אוטונומי את לוגיקת ה"בחירה היחידה" (Single Selection), טעינת מצב קודם מה-SharedPreferences, ודיווח על שינויים לרכיבים חיצוניים באמצעות ממשק Callback.

היא אחראית על:

1. ניהול בחירה בלעדית: הבטחה שרק כפתור אחד יהיה נבחר בכל זמן נתון.
2. סנכרון מול בסיס נתונים מקומי: שלפת המצב השמור מתוך ה-SharedPreferences בעת היצירה.
3. רינדור דינמי וצפוף: חישוב אוטומטי של גובה הרקע והריווח בין הכפתורים בצורה אנכית וחסיכונית במקום.
4. דיווח אירועים: שימוש בממשק OnSelectionChangeListener כדי לעדכן את המשחק על בחירה חדשה.

הסבר על תכונות המחלקה:

- private OnSelectionChangeListener listener
 - תחום הכרה: private.
 - תפקיד ושימוש: רפרנס לממשק המאזין. מאפשר הפרדה בין רכיב ה-UI לבין הלוגיקה שמגיבה לבחירה.
- private final CircleButton[] radioButtons
 - תחום הכרה: private final.
 - תפקיד ושימוש: מערך של כפתורים עגולים המהווים את רכיבי הבחירה הפיזיים בקבוצה.
- private CircleButton selectedButton
 - תחום הכרה: private.
 - תפקיד ושימוש: שומר רפרנס לכפתור שנבחר כרגע. משמש לביטול הבחירה הקודמת כאשר נבחר כפתור חדש.
- private final TextRenderer labelRenderer, titleRenderer
 - תחום הכרה: private final.
 - תפקיד ושימוש: רכיבי עזר לרינדור טקסט. ה-titleRenderer משמש לכותרת הקבוצה וה-labelRenderer משמש לתוויות לצד כל כפתור.
- private final SharedPreferences settingsPreferences
 - תחום הכרה: private final.
 - תפקיד ושימוש: ממשק לשמירת נתונים קטנים. משמש לקריאת ההגדרה השמורה של המשתמש.
- private final RadioGroupList radioGroupEnum
 - תחום הכרה: private final.
 - פקיד ושימוש: אובייקט נתונים (Enum) המכיל את המידע עבור הקבוצה: כותרת, שמות האופציות, והמפתח (Key) לשמירה ב-Preferences.
- private final NinePatchDrawable background
 - תחום הכרה: private final.
 - תפקיד ושימוש: גרפיקת הרקע של הקבוצה.
- private final float spacing



- תחום הכרה: private final
- תפקיד ושימוש: מרחק אנכי קבוע בין מרכזי הכפתורים, המחושב בבנאי לפי גובה הכפתור בתוספת ריווח מינימלי, להבטחת מראה צפוף ועקבי.

הסבר פעולות מרכזיות:

- public void eventHandler(MotionEvent event)
 - פעולה זו מנהלת את אינטראקציית המגע עם כל הכפתורים בקבוצה.
 - **תרשים זרימה (סדר פעולות):**
 - i. מעבר על מערך הכפתורים: הלולאה עוברת על כל ה-`radioButtons`.
 - ii. בדיקת פגיעה: קריאה ל-`eventHandler` של כל כפתור בנפרד.
 - iii. זיהוי לחיצה: אם כפתור מסוים החזיר `true` (נלחץ):
 1. ביטול כפתור קודם: אם היה `selectedButton`, הוא מוחזר למצב פעיל.
 2. סימון חדש: הכפתור שנלחץ הופך ל-`selectedButton` ועובר למצב "מושבת" ויזואלית.
 3. דיווח: הפעלת ה-`listener` לעדכון המערכת ושליחת האינדקס החדש.
 4. משוב: השמעת צליל לחיצה.
 - iv. עצירה: יציאה מהלולאה לאחר זיהוי הלחיצה.
 - קוד:

```
public void eventHandler(MotionEvent event) {
    for (int i = 0; i < radioButtons.length; i++) {
        CircleButton button = radioButtons[i];
        if (button.eventHandler(event)) {
            if (selectedButton != null)
                selectedButton.setEnabled(true);

            selectedButton = button;
            selectedIndex = i;
            selectedButton.setEnabled(false);

            if (listener != null) {
                listener.onSelectionChanged(this, selectedIndex);
            }

            SoundManager.getInstance(context).playSfx(R.raw.sfx_bloop);
            break;
        }
    }
}
```

- private void setBackground(float x, float y)
 - פעולה זו מחשבת את גבולות הרקע באופן דינמי בהתאם לתוכן. הפעולה מודדת את אורך הטקסט הארוך ביותר מבין האופציות. גובה הרקע מחושב לפי הנוסחה: (מספר כפתורים - 1) * spacing. זה מבטיח שהרקע יקיף בדיוק את קבוצת הכפתורים ללא שטח מת מיותר.

○ קוד:

```
private void setBackground(float x, float y) {
    int[] namesID = radioGroupEnum.getNamesID();
    float width = 0;
    for (int i = 0; i < namesID.length; i++) {
        width = Math.max(width, labelRenderer.measureText(radioGroupEnum.getName(i)));
    }

    // Add padding for the radio buttons and text
    width += radioButtons[0].getHitbox().width() + 10 * SCALE_MULTIPLIER;
}
```



```
float height = (radioButtons.length - 1) * spacing + radioButtons[0].getHitbox().height() +  
7 * SCALE_MULTIPLIER;  
  
float bgX = x - radioButtons[0].getHitbox().width() / 2 - 5 * SCALE_MULTIPLIER;  
float bgY = y - radioButtons[0].getHitbox().height() / 2 - 5 * SCALE_MULTIPLIER;  
  
background.setBounds((int) bgX, (int) bgY, (int) (bgX + width), (int) (bgY + height));  
}
```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת בקובץ ההגדרות של האפליקציה ("settings") כדי לשמור ולטעון את בחירות המשתמש.

- סקירת המידע: המידע נשמר כזוג של מפתח (String) וערך (Integer - האינדקס שנבחר).
- פעולות:

○ טעינה: במתודה `setInitialButton`, המחלקה שולפת את האינדקס השמור לפי ה-key הייחודי ומסמנת את הכפתור הנכון כנבחר מיד עם עליית המסך.

**מחלקת CustomSwitch**

מיקום: com.example.tutorialgame.engine.ui.customviews.switches.CustomSwitch

תפקיד המחלקה: מחלקה זו מייצגת פקד בחירה דו-מצבי (Switch/Toggle). היא משמשת לניהול הגדרות בינאריות (On/Off) באפליקציה. המחלקה מנהלת באופן אוטונומי את שמירת המצב ב-SharedPreferences, את הרינדור הגרפי של מצבי ה-Switch, ואת אנימציות מעבר הצבעים המתוחכמות של הטקסט הנלווה.

היא אחראית על:

1. ניהול מצב (Checked State): מעקב אחר המצב הנוכחי ושמירתו באופן קבוע בזיכרון המכשיר לסנכרון בין הפעלות האפליקציה.
2. אנימציות צבעים (Color Interpolation): ביצוע מעבר הדרגתי וחלק (Fade) בין צבע המצב הכבוי לצבע המצב הפעיל עבור הטקסט, תוך שימוש במתמטיקה של אינטרפולציה.
3. דיווח אירועים: שימוש בממשק `OnCheckedChangeListener` לעדכון רכיבים אחרים במערכת על שינוי המצב באופן אסינכרוני.

הסבר על תכונות המחלקה:

- `private OnCheckedChangeListener listener`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: רפרנס לממשק המאזין לשינויים. מאפשר לקוד חיצוני להגיב ללחיצה על ה-Switch.
- `private final RectF hitbox`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מגדיר את האזור הלחיצה של הפקד על המסך לצורך זיהוי אינטראקציות מגע.
- `private final String key`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: המפתח הייחודי תחתיו נשמר מצב ה-Switch בתוך ה-SharedPreferences של המערכת.
- `private final TextRenderer paint`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רכיבי עזר לרינדור טקסט.
- `private boolean checked`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: המשתנה הלוגי המרכזי המציין אם הפקד במצב פעיל או כבוי.
- `private int baseColorFrom, baseColorTo` ... (משתני אנימציה)
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנים אלו שומרים את ערכי ה-ARGB של צבע ההתחלה וצבע היעד במהלך מעבר האנימציה.
- `private long colorAnimStart`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: שומר את זמן תחילת האנימציה (במילישניות) המשמש לחישוב אחוז התקדמות האנימציה בזמן אמת.
- `private boolean colorAnimating`
 - תחום הכרה: `private`.



- תפקיד ושימוש: דגל בטיחות המציין האם האנימציה פעילה כרגע, כדי לחסוך במשאבי מעבד כאשר הצבעים סטטיים.

הסבר פעולות מרכזיות:

- `public boolean eventHandler(MotionEvent event)`
 - פעולה זו מנהלת את הקלט מהמשתמש ומשנה את מצב הפקד.
 - **תרשים זרימה (סדר פעולות):**
 - i. זיהוי שחרור (`ACTION_UP`): הפעולה מתבצעת רק כאשר המשתמש מרים את האצבע מהמסך.
 - ii. בדיקת פגיעה: אם נקודת השחרור נמצאת בתוך ה-`hitbox`:
 1. היפוך מצב: המשתנה `checked` מקבל את הערך ההפוך ממצבו הנוכחי.
 2. הפעלת אנימציה: חישוב צבעי היעד החדשים (למשל: מאפור לכתום במצב פעיל, אפור לכבוי) וקריאה ל-`startColorTransition`.
 3. שמירה: עדכון ה-`SharedPreferences` בערך החדש.
 4. משוב: הפעלת רטט והשמעת צליל לחיצה.
 5. דיווח: קריאה למתודת ה-`listener` לעדכון המערכת.
 - iii. החזרה: מחזירה את המצב החדש של ה-`Switch`.
 - **קוד:**

```
public boolean eventHandler(MotionEvent event) {
    if (event.getActionMasked() == MotionEvent.ACTION_UP) {
        if (isIn(event)) {
            checked = !checked;

            // Determine target colors
            int newBase = checked ?
                BaseActivity.getContext().getColor(R.color.text_color_pressed) :
                BaseActivity.getContext().getColor(R.color.dark_charcoal);

            int newShadow = checked ?
                BaseActivity.getContext().getColor(R.color.black) :
                BaseActivity.getContext().getColor(R.color.dark_moon);

            startColorTransition(newBase, newShadow);

            // Save state and provide feedback
            BaseActivity.getSpSettings().edit().putBoolean(key, checked).apply();
            SoundManager.getInstance(BaseActivity.getContext()).playSfx(R.raw.sfx_bloop);
            BaseActivity.ButtonPressVibe();

            // Notify listener
            if (listener != null) {
                listener.onCheckedChanged(this, checked);
            }
        }
    }
    return checked;
}
```

- `private void updateAnimatedColors()`
 - פעולה זו אחראית על חישוב המעבר ההדרגתי של הצבעים בכל פריים של המשחק.
 - הפעולה מחשבת את היחס (t) בין הזמן שעבר למשך האנימציה המוגדר ($ms180$). היא משתמשת בפונקציית החלקה ($Smoothstep - t * t * (3 - 2t)$) כדי ליצור תנועה רכה



וטבעית יותר לעין. בסיום, היא מעדכנת את הצבע של ה-`TextRenderer` באמצעות פונקציית ה-`lerpColor`.

○ קוד:

```
private void updateAnimatedColors() {
    if (!colorAnimating) return;

    long now = System.currentTimeMillis();
    long colorAnimDuration = 180L;
    float t = (now - colorAnimStart) / (float) colorAnimDuration;

    if (t >= 1f) {
        paint.setColor(baseColorTo);
        paint.setShadowColor(shadowColorTo);
        colorAnimating = false;
        return;
    }

    float smoothT = t * t * (3f - 2f * t);
    paint.setColor(lerpColor(baseColorFrom, baseColorTo, smoothT));
    paint.setShadowColor(lerpColor(shadowColorFrom, shadowColorTo, smoothT));
}
```

● `private int lerpColor(int fromColor, int toColor, float t)`

○ מבצעת אינטרפולציה ליניארית (Linear Interpolation) בין שני צבעי ARGB. הפעולה מפרקת את שני הצבעים לארבעת הערוצים המרכיבים אותם (Alpha, Red, Green, Blue). עבור כל ערוץ בנפרד, היא מחשבת ערך ביניים המבוסס על המרחק ביניהם וההתקדמות `t`. לבסוף, היא מחברת את הערוצים בחזרה למספר שלם המייצג את הצבע החדש.

○ קוד:

```
private int lerpColor(int fromColor, int toColor, float t) {
    int a1 = (fromColor >> 24) & 0xff, r1 = (fromColor >> 16) & 0xff, g1 = (fromColor >> 8) & 0xff, b1 = fromColor & 0xff;
    int a2 = (toColor >> 24) & 0xff, r2 = (toColor >> 16) & 0xff, g2 = (toColor >> 8) & 0xff, b2 = toColor & 0xff;
    return (Math.round(a1 + (a2 - a1) * t) & 0xff) << 24 | (Math.round(r1 + (r2 - r1) * t) & 0xff) << 16 | (Math.round(g1 + (g2 - g1) * t) & 0xff) << 8 | (Math.round(b1 + (b2 - b1) * t) & 0xff);
}
```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת בבסיס הנתונים המקומי של אנדרואיד כדי לשמור על רציפות הגדרות המשתמש.

- סקירת המידע: המידע נשמר בפורמט Key-Value, כאשר המפתח הוא ה-`key` של ה-`Switch` והערך הוא Boolean.
- פעולות:

○ טעינה: בבנאי (Constructor), המחלקה קוראת את הערך השמור מה-`SharedPreferences`. אם לא קיים ערך, היא משתמשת בערך ברירת המחדל המוגדר ב-`SwitchList`.



מחלקת CustomUpgrade

מיקום: com.example.tutorialgame.engine.ui.customviews.upgrade.CustomUpgrade

תפקיד המחלקה: מחלקה זו מייצגת רכיב אינטראקטיבי מתקדם לשדרוג תכונות הדמות (Stats). היא משמשת כרכיב שדרוג ומאפשרת למשתמש להעלות או להוריד רמות של יכולות שונות (חיים, כוח, מהירות) תוך הצגת משוב ויזואלי וקולי. המחלקה מיישמת טכניקות רינדור מתקדמות כדי ליצור ממשק עגול ומעוצב, ומנהלת באופן אוטונומי את חישובי המחירים והשינויים הזמניים לפני שמירתם בענן.

היא אחראית על:

1. רינדור גרפי מתקדם (Masking): שימוש במצבי מיזוג (PorterDuff) כדי "לחתוך" אייקונים בצורה עגולה שתתאים למסגרות המשחק.
2. ניהול לוגיקת שדרוגים: מעקב אחר רמת השדרוג הנוכחית, חישוב עלויות דינמי וניהול מונה שינויים זמני (countToApply).
3. אופטימיזציית זיכרון: שימוש בבלוק אתחול סטטי לטעינת משאבים גרפיים ומברשות (Paints) המשותפים לכל מופעי השדרוגים.
4. חוויית משתמש (UX): הצגת בועות מידע (Balloons) עם מחירי השדרוג ומתן משוב קולי ורטט.
5. דיווח אירועים (Event Driven): הודעה למערכת (UpgradeState) על שינויים בנקודות דרך ממשק ממשיק Listener.

הסבר על תכונות המחלקה:

- private OnUpgradeListener listener
 - תחום הכרה: private.
 - תפקיד ושימוש: רפרנס למאזין החיצוני. משמש לדיווח על שינוי בעלות השדרוג כדי לעדכן את כמות הנקודות שנותרו לשחקן.
- private final static Bitmap container
 - תחום הכרה: private static final.
 - תפקיד ושימוש: תמונת הרקע של לוחית המידע. נטענת פעם אחת בלבד בבלוק הסטטי כדי לחסוך בזיכרון RAM.
- private final static Paint cutPaint
 - תחום הכרה: private static final.
 - תפקיד ושימוש: מברשת המוגדרת עם PorterDuff.Mode.SRC_IN. זהו הלב של אפקט ה-Masking, המאפשר לצייר את אייקון השדרוג רק בתוך תחומי העיגול.
- private final Balloon upgradeBalloon
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט תצוגה חיצוני המשמש להצגת בועת מחיר מרחפת מעל השדרוג בעת לחיצה.

הסבר פעולות מרכזיות:

- private void drawUpgradeIcon(Canvas c)
 - זוהי הפעולה המבצעת את אפקט ה-Clipping העגול. כדי למנוע מהאייקון של השדרוג לחרוג מגבולות המסגרת העגולה, הפעולה משתמשת ב-Offscreen Buffer (שכבה זמנית). היא מציירת עיגול רקע מלא, ואז מציירת את אייקון השדרוג תוך שימוש ב-cutPaint. מצב ה-SRC_IN גורם לכך שרק הפיקסלים של האייקון שחופפים לעיגול הרקע יישמרו. לבסוף, היא מציירת את המסגרת המעוטרת מעל הכל.
 - קוד:

```
private void drawUpgradeIcon(Canvas c) {
    int saveId = c.saveLayer(bounds, null);
    c.drawBitmap(CircleFrames.BACKGROUND.getCircleFrame(), bounds.left, bounds.top, null);
    c.drawBitmap(upgradeImg.getUpgradeImg(),
        upgradeCenter.x - upgradeImg.getUpgradeImg().getWidth() / 2f,
```



```
upgradeCenter.y - upgradeImg.getUpgradeImg().getHeight() / 2f,
    cutPaint);
c.restoreToCount(saveId);
c.drawBitmap(circleFrame.getCircleFrame(), bounds.left, bounds.top, null);
}
```

- `public void onClick(BaseButton button)`

- מטפלת בלוגיקה של הוספת או הסרת שדרוג.

- **תרשים זרימה (סדר פעולות):**

- i. זיהוי כפתור: בדיקה אם נלחץ כפתור השדרוג (`btnUpgrade`) או הביטול (`btnDowngrade`).
- ii. עדכון רמות: שינוי ה-`currentLevel` וה-`countToApply` בהתאם.
- iii. חישוב עלות: חישוב ה-`costChange` (כמה נקודות נוספו או הוחזרו).
- iv. דיווח: קריאה למתודת ה-`listener` כדי לעדכן את מסך השדרוגים הראשי.
- v. משוב: השמעת צליל לחיצה (SFX).

- **קוד:**

```
@Override
public void onClick(BaseButton button) {
    int costChange = 0;
    if (button == btnUpgrade) {
        costChange = price;
        countToApply++;
        currentLevel++;
    } else if (button == btnDowngrade) {
        countToApply--;
        currentLevel--;
        costChange = -(price - 2);
    }

    setPrice(); // Update price for next level
    if (listener != null) {
        listener.onUpgradeChanged(this, costChange);
    }
    SoundManager.getInstance(MyApp.getAppContext()).playSfx(R.raw.sfx_bloop);
}
```

- `static { ... }` (בלוק אתחול סטטי)

- פעולה זו מתבצעת פעם אחת בלבד עם טעינת האפליקציה. היא טוענת את גרפיקת ה-`Container`, מבצעת לה `Scaling` מותאם לרזולוציה, ומאתחלת את כל ה-`Paints` המשמשים לציור טקסט וחיתוך גרפי. זהו שיפור קריטי בביצועים המונע טעינת משאבים כפולה עבור כל שדרוג.

**מחלקת Joystick**

מיקום: joystick.Joystick.engine.ui.tutorialgame.example.com

תפקיד המחלקה: מחלקה זו מייצגת בקר תנועה אנלוגי (Virtual Analogue Joystick). היא משמשת ככלי הקלט המרכזי של השחקן לשליטה בדמות בעולם המשחק. המחלקה מיישמת עקרונות של כמיסה (Encapsulation) על ידי ריכוז כל לוגיקת המגע, החישובים הגיאומטריים של הגבלת התנועה והרינדור הגרפי בתוך רכיב עצמאי ורב-פעמי.

היא אחראית על:

1. ניהול מגע (Multi-touch): מעקב אחר פוינטר ספציפי (pointerId) כדי לאפשר לחיצה על כפתורים אחרים במקביל לתנועה.
2. חישובים גיאומטריים: שימוש במשפט פיתגורס לחישוב מרחקים ובאלגוריתמי חיתוך מעגלים להגבלת תנועת הכפתור.
3. נרמול וקטורי (Vector Normalization): הפקת וקטור תנועה מנורמל (בין 1- ל-1) המבטיח מהירות תנועה עקבית בכל הכיוונים.
4. משוב ויזואלי: רינדור רכיבי הבקר כולל אפקטי צל (Blur) למתן תחושת עומק.

הסבר על תכונות המחלקה:

- private final PointF centerPos, knobPos
 - תחום הכרה: private final.
 - תפקיד ושימוש: centerPos מייצג את נקודת המרכז הקבועה של הבקר על המסך.
 - knobPos מייצג את המיקום המשתנה של הכפתור הפנימי הנגרר על ידי המשתמש.
- private final PointF movementVector
 - תחום הכרה: private final.
 - תפקיד ושימוש: ה"פלט" הלוגי של הבקר. וקטור המכיל את כיוון ועוצמת התנועה, המשמש את מנוע הפיזיקה להזזת הדמות.
- private int pointerId
 - תחום הכרה: private.
 - תפקיד ושימוש: שומר את המזהה הייחודי של האצבע שתפסה את הגיויסטיק. זהו מרכיב קריטי במערכות Multi-touch המבטיח שהגיויסטיק יגיב רק לאצבע הנכונה גם אם יש נגיעות נוספות במסך.
- private final float radius
 - תחום הכרה: private final.
 - תפקיד ושימוש: מגדיר את גבול הגזרה המקסימלי של הבקר בפיסקלים.
- private final Paint paintOuter, paintBg, paintKnob...
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקטי הגדרות הצירוף (צבע, סגנון, עובי קו ואפקטים).

הסבר פעולות מרכזיות:

- public void eventHandler(MotionEvent event)
 - מנהלת את כל מחזור החיים של אינטראקציית המגע.
 - **תרשים זרימה (סדר פעולות):**
 - i. ACTION_DOWN / ACTION_POINTER_DOWN: בדיקה האם האצבע נחתה בתוך רדיוס הגיויסטיק (isInside). אם כן, היא הופכת לבעלים של הבקר ושומרת את ה-pointerId.
 - ii. ACTION_MOVE: חיפוש בתוך רשימת הנגיעות הפעילות אחר האצבע המשויכת ל-ID השמור. אם נמצאה, נשלחות הקואורדינטות שלה לעדכון המיקומים (updatePositions).



iii. ACTION_UP / CANCEL : כאשר האצבע הרלוונטית עוזבת את המסך, הבקר מבצע איפוס (reset) למצב התחלתי.

○ קוד:

```
public void eventHandler(MotionEvent event) {
    final int action = event.getActionMasked();
    final int actionIndex = event.getActionIndex();
    final int pid = event.getPointerId(actionIndex);

    switch (action) {
        case MotionEvent.ACTION_DOWN:
        case MotionEvent.ACTION_POINTER_DOWN:
            if (isInside(event.getX(actionIndex), event.getY(actionIndex))) {
                pushed = true;
                pointerId = pid;
                updatePositions(event.getX(actionIndex), event.getY(actionIndex));
            }
            break;
        case MotionEvent.ACTION_MOVE:
            if (pushed) {
                for (int i = 0; i < event.getPointerCount(); i++) {
                    if (event.getPointerId(i) == pointerId) {
                        updatePositions(event.getX(i), event.getY(i));
                        break;
                    }
                }
            }
            break;
        case MotionEvent.ACTION_UP:
        case MotionEvent.ACTION_POINTER_UP:
        case MotionEvent.ACTION_CANCEL:
            if (pid == pointerId) {
                reset();
            }
            break;
    }
}
```

● private void updatePositions(float touchX, float touchY)

○ זוהי הפעולה הלוגית המרכזית המחשבת את תנועת הבקר.

○ תרשים זרימה (סדר פעולות):

- i. חישוב מרחק: הפעולה מחשבת את המרחק (distance) בין האצבע למרכז הבקר באמצעות $\text{Math.hypot}(dx, dy)$.
- ii. הגבלת תנועה: במידה והאצבע מחוץ לרדיוס המותר, נעשה שימוש בפעולת עזר המחשבת את הנקודה הקרובה ביותר על היקף המעגל (Intersection).
- iii. נרמול: המרחק מהמרכז מחולק ברדיוס כדי לקבל ערך יחסי. התוצאה היא וקטור שבו הערך המקסימלי הוא 1.0 (תנועה מלאה) והמינימלי הוא 0.0 (עצירה).

○ קוד:

```
private void updatePositions(float touchX, float touchY) {
    float dx = touchX - centerPos.x;
    float dy = touchY - centerPos.y;
    float distance = (float) Math.hypot(dx, dy);

    if (distance <= radius) {
        knobPos.set(touchX, touchY);
    } else {
        // Keep knob on the edge of the circle
        Other.IntersectCircle(centerPos.x, centerPos.y, radius, touchX, touchY, knobPos);
    }

    // Calculate normalized movement vector
    movementVector.set(dx / radius, dy / radius);
}
```



```
// Clamp vector to unit circle length
float mag = (float) Math.hypot(movementVector.x, movementVector.y);
if (mag > 1.0f) {
    movementVector.x /= mag;
    movementVector.y /= mag;
}
```



ניהול עולם ותצוגה (View & World Management)

פרק זה מרכז את המחלקות האחראיות על החוויה הסביבתית, ניהול המצלמה והתצוגה הדינמית של עולם המשחק. מחלקות אלו יוצרות את ה"אטמוספירה" הויזואלית, מטפלות בסימולציות פיזיקליות של מזג אוויר ותומכות בתחושת העומק והמרחב של השחקן.

מחלקת CameraManager

מיקום: com.example.tutorialgame.managers.CameraManager

תפקיד המחלקה: מחלקה זו היא ה"עין" של המשתמש בעולם המשחק. היא משמשת כגשר חישובי המתרגם קואורדינטות בעולם המשחק לקואורדינטות ציור על המסך הפיזי. המחלקה מנהלת באופן אוטונומי את עקיבת המצלמה אחרי השחקן בצורה חלקה (Smoothing), את רמות הקירוב (Zoom) הקבועות והזמניות, אפקטים של רעידת מסך (Screen Shake), והגבלת תנועת המצלמה בתוך גבולות המפה (Clamping) כדי למנוע הצגת שטחים שחורים מחוץ לעולם המשחק.

היא אחראית על:

1. חישוב היסט (Offsets): קביעת ערכי ה-X וה-Y שיש להחסיר מכל אובייקט בעולם כדי שיוצג במיקום הנכון ביחס למסך.
2. עקיבה חלקה (Lerp): שימוש באינטרפולציה כדי שהמצלמה תעקוב אחרי היעד ב"רכות" ולא בצורה נוקשה.
3. ניהול זום כפול: הפרדה בין ה-Zoom הקבוע של המשתמש לבין ה-Zoom הזמני (למשל לצורכי דיאלוגים קולנועיים).
4. חסימת גבולות: מניעת חריגת המצלמה מגבולות המפה הפיזיים.
5. משוב חזותי: ניהול רעידות מסך בזמן אמת להגברת האפקט של פגיעות או אירועים.

הסבר על תכונות המחלקה:

- private static float offsetX, offsetY
 - תחום הכרה: private static.
 - תפקיד ושימוש: אלו הם ערכי ההיסט הסופיים המשמשים את מנוע הרינדור (בדרך כלל דרך c.translate). הם מציינים כמה הציור מוזז מהפינה השמאלית-עליונה של המסך.
- private static float lookAtX, lookAtY
 - תחום הכרה: private static.
 - תפקיד ושימוש: נקודת המיקוד הנוכחית של המצלמה בעולם המשחק (בדרך כלל מרכז השחקן).
- private static float finalZoom, tempZoom
 - תחום הכרה: private static.
 - תפקיד ושימוש: finalZoom הוא הערך שנבחר בהגדרות ונשמר בדיסק. tempZoom הוא הערך המבצעי שמשמש לציור, מה שמאפשר לבצע זום-אין זמני בדיאלוגים מבלי לאבד את הגדרת המשתמש.
- private static float lerpFactor
 - תחום הכרה: private static.
 - תפקיד ושימוש: קובע את מהירות ה"היצמדות" של המצלמה ליעד. ערך נמוך יוצר תחושה "כבדה" וחלקה יותר.
- private static float mapWidth, mapHeight
 - תחום הכרה: private static.
 - תפקיד ושימוש: מידות המפה הנוכחית בפיקסלים. משמשות כערכי סף לפעולת ה-Clamping.
- private static float shakeOffsetX, shakeOffsetY
 - תחום הכרה: private static.



- תפקיד ושימוש: ההיסט הזמני שנוצר בעקבות אפקט רעידה. ערכים אלו מתווספים ל-X ו-Y הסופיים בכל פריים של רעידה.

הסבר פעולות מרכזיות:

● `public static void lookAt(float targetX, float targetY, double delta)`

- זוהי הפעולה המרכזית המעדכנת את מיקום המצלמה בכל פריים.
- **תרשים זרימה (סדר פעולות):**
 - החלקה (Smoothing): המצלמה מחשבת את המרחק בינה לבין היעד (`targetX/Y`) ומקרבת את נקודת המיקוד (`lookAtX/Y`) בצעד המבוסס על ה-`lerpFactor` ו-`delta`.
 - עדכון רעידה: במידה ויש רעידה פעילה, מחושב "רעש" אקראי שמוזרק למיקום.
 - חסימת גבולות: הפעולה מוודאת שנקודת המיקוד לא קרובה מדי לקצה המפה (באמצעות `clampLookAt`).
 - חישוב היסט סופי: המרת נקודת המיקוד לערכי ה-`Offsets` תוך התחשבות במרכז המסך וברמת ה-`Zoom` הנוכחית.

○ קוד:

```
public static void lookAt(float targetX, float targetY, double delta) {
    float lerpStep = (float) (1.0 - Math.pow(lerpFactor, delta * 10));
    lookAtX += (targetX - lookAtX) * lerpStep;
    lookAtY += (targetY - lookAtY) * lerpStep;

    updateShake(delta);
    clampLookAt();

    offsetX = (SCREEN_WIDTH / (2f * tempZoom)) - lookAtX + (shakeOffsetX / tempZoom);
    offsetY = (SCREEN_HEIGHT / (2f * tempZoom)) - lookAtY + (shakeOffsetY / tempZoom);
}
```

● `private static void clampLookAt()`

- פעולה לוגית המגבילה את תחום הראייה. הפעולה מחשבת מהו גודל ה"חלון" הנראה על המסך במונחים של עולם המשחק (למשל, בזום 1.9 רואים קצת יותר מחצי מרוחב המסך בפיסקלים של עולם). היא מוודאת שנקודת המיקוד לא תהיה קרובה לקצה המפה פחות ממחצית מרוחב ה"חלון", ובכך מבטיחה שהמשתמש לעולם לא יראה את ה"ריק" שמעבר למפה, אך אם השחקן בוחר לבחור בזום נמוך מספיק ביחס לגודל המפה, הוא יוכל לראות מבעד לגבולותיה.

○ קוד:

```
private static void clampLookAt() {
    if (mapWidth == 0 || mapHeight == 0) return;
    float viewW = SCREEN_WIDTH / tempZoom;
    float viewH = SCREEN_HEIGHT / tempZoom;

    if (mapWidth <= viewW) {
        lookAtX = mapWidth / 2f;
    } else {
        float minX = viewW / 2f;
        float maxX = mapWidth - (viewW / 2f);
        lookAtX = Math.max(minX, Math.min(maxX, lookAtX));
    }

    if (mapHeight <= viewH) {
        lookAtY = mapHeight / 2f;
    } else {
        float minY = viewH / 2f;
        float maxY = mapHeight - (viewH / 2f);
        lookAtY = Math.max(minY, Math.min(maxY, lookAtY));
    }
}
```




בסיס נתונים (SharedPreferences)

המחלקה משתמשת בקובץ ההגדרות "settings" כדי לשמור ולטעון את העדפות הצפייה של המשתמש.

- סקירת המידע: נשמרים הערכים (float) zoom ו-(float) lerpFactor.
- פעולות:
- טעינה: מתבצעת בבלוק ה-static מיד עם טעינת המחלקה לראשונה.
- שמירה: מתבצעת בתוך setZoomFromProgress ו-setLerpFactorFromProgress בכל פעם שהמשתמש משנה את הסליידר בתפריט.



מחלקת SceneManager

מיקום: com.example.tutorialgame.gamestates.cutscenes.SceneManager

תפקיד המחלקה: מחלקה זו משמשת כ"מנהל הפקה" נרטיבי, האחראי על תזמור והפעלת קטעים קולנועיים (Cutscenes) בתוך המשחק. תפקידה הוא לגשר בין זרימת המשחק הרגילה לבין רגעי השיא בעלילה, תוך בחירה אוטומטית של הסצנה הרלוונטית ביותר בהתאם להתקדמות המשתמש (Checkpoints). המחלקה פועלת כמנהל מצבים (State Manager) פנימי, המעביר את פקודות המערכת לסצנה הפעילה ומנהל את החזרה למשחקיות בסיומה.

היא אחראית על:

1. ניהול רצף עלילתי: החזקת רשימה כרונולוגית של כל אירועי הסיפור המרכזיים במשחק.
2. סינון דינמי: זיהוי הסצנה הבאה שיש להפעיל על ידי הצלבת רשימת האירועים מול המידע השמור בענן.
3. בקרה על מחזור חיים: ניהול המעבר בין מצבי "קאט-סין" ל"משחק פעיל" ותיאום התחלת דיאלוגים משלימים בסיום סצנה.
4. ניתוב אירועים: העברת פקודות הציור והעדכון מהמנוע אל הסצנה המורצת כרגע.

הסבר על תכונות המחלקה:

- `private final List<BaseScene> scenePlaylist`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רשימה מאורגנת של אובייקטי סצנות (כגון `Intro` או `GettingSword`). בניית הרשימה פעם אחת בבנאי מבטיחה ביצועים מהירים בעת בדיקת התקדמות.
- `private Base Scene currentScene`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: רפרנס לסצנה המנוגנת כרגע על המסך. אם הערך הוא `null`, המשמעות היא שאין סצנה פעילה.
- `private boolean isStartingDialogueAfter`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה בוליאני המוגדר בעת סיום כל סצנה המודיע למנהל המצבים הכללי האם להפעיל דיאלוג המשך משלים לאחר סיום הסצנה.

הסבר פעולות מרכזיות:

- `public void playNextRelevantScene()`
 - פעולה זו מהווה את ה"מנוע הנרטיבי" של המשחק, האחראי לקבוע מהו האירוע העלילתי הבא שיש להציג למשתמש. במקום להפעיל סצנות בצורה ידנית ומפוזרת בקוד, הפעולה סורקת את רשימת ההשמעה (`scenePlaylist`) בסדר כרונולוגי. עבור כל סצנה, היא מבצעת בדיקה אלגוריתמית מול נתוני ה-`Checkpoints` השמורים בענן (דרך `BaseScene`). הסצנה הראשונה שמוגדרת כ"טרם הושלמה" נבחרת כסצנה הפעילה ומופעלת מיידית. גישה זו מבטיחה שהשחקן לעולם לא יפספס אירוע קריטי ושהסיפור יתקדם תמיד לפי הסדר המתוכנן.
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקת הגנה: וידוא שאין סצנה שרצה כרגע (מניעת הפרעה לאירוע פעיל).
 - ii. סריקה כרונולוגית: מעבר בולואה על כל הסצנות שהוגדרו ב-`buildPlaylist`.
 - iii. שאילתת התקדמות: בדיקת סטטוס הסצנה בענן (האם דגל ה-`Checkpoint` קיים?).
 - iv. הפעלה: אם וכאשר נמצאה סצנה חדשה - הגדרתה כ-`currentScene`, הפעלת מתודת ה-`onEnter` שלה, ויציאה מהלולאה.
 - v. נסיגה (Fallback): אם כל הסצנות הושלמו, החזרת מצב המשחק ל-`PLAYING` (משחקיות רגילה).



קוד: ○

```
public void playNextRelevantScene() {
    if (currentScene != null) return; // Guard: Don't interrupt active scene

    for (BaseScene scene : scenePlaylist) {
        if (!scene.checkCheckPoint()) {
            this.currentScene = scene;
            this.currentScene.onEnter();
            return;
        }
    }

    // Fallback: If no scenes are pending, resume normal gameplay
    Game.setNextGameState(Game.GameState.PLAYING);
}
```

● public void onSceneFinished()

- פעולת Callback (קריאה חוזרת) המשמשת כנקודת הסיום הטכנית של קטע קולנועי. כאשר סצנה (כמו ה-Intro) מסיימת את תפקידה, היא מודיעה למנהל דרך פעולה זו. התפקיד של המנהל כאן הוא כפול: ראשית, עליו לדגום מהסצנה נתונים לגבי העתיד (האם יש צורך בשיחת המשך?), ושנית, עליו לנקות את המשאבים. הפעולה מאפסת את הרפרנס ל-currentScene (מה שמאפשר ל-Garbage Collector לשחרר זיכרון) ומעבירה את המנוע חזרה למצב משחק פעיל.

קוד: ○

```
public void onSceneFinished() {
    if (currentScene != null) {
        isStartingDialogueAfter = currentScene.isDialogueAfter();
    }
    this.currentScene = null;
    Game.setNextGameState(Game.GameState.PLAYING);
}
```

● public void onExit()

- מתודה זו מנהלת את ה"גשר" הנרטיבי שנוצר בין סרטון לבין אינטראקציה חיה. לעיתים קרובות, סצנה קולנועית מסתיימת ודורשת שהשחקן יתחיל מיד בשיחה עם דמות שהופיעה בה. המנהל משתמש בדגל isStartingDialogueAfter שנלכד בסיום הסצנה. בעת היציאה ממצב "קאט-סיין", המנהל פוקד על ה-PlayingManager להפעיל את הדיאלוג המתוזמן. פעולה זו מייצרת רצף עלילתי חלק שבו השחקן עובר מ"צופה" ל"משתתף" ללא קטיעה של חוויית המשחק.

קוד: ○

```
@Override
public void onExit() {
    super.onExit();
    // Trigger post-scene dialogue if the completed scene required it
    if (isStartingDialogueAfter) {
        game.getPlayingManager().continueDialog();
        isStartingDialogueAfter = false;
    }
}
```

**מחלקת BaseScene**

מיקום: com.example.tutorialgame.gamestates.cutscenes.BaseScene

תפקיד המחלקה: מחלקה זו מהווה את תשתית האב לכל הסצנות הסינמטיות (Cutscenes) במשחק. היא מנהלת רצף של תמונות (Frame Sequencing), תזמון מעברים אוטומטיים, אפקטים ויזואליים של טקסט (Fade-in), וסינמטיקה מול מערכת השמע והענן. המחלקה מאפשרת יצירת רגעי עלילה המופרדים מהמשחקיות השוטפת, תוך הבטחה שהשחקן לא יצטרך לצפות שוב בתוכן שכבר נצפה (באמצעות בדיקת Checkpoints).

היא אחראית על:

1. ניהול רצף פרימיים: הצגת תמונות בסדר כרונולוגי וניהול המעבר ביניהן (אוטומטי או ידני).
2. בקרת קלט והגנה (Input Guarding): מניעת דילוג מהיר מדי או בטעות על קטעי עלילה.
3. סנכרון מול מערכת השמע: הפעלת מוזיקת רקע ייחודית לכל סצנה.
4. ניהול התקדמות בענן: עדכון ה-Database בסיום הסצנה כדי למנוע כפילות תוכן.

הסבר על תכונות המחלקה:

- private final TextRenderer nextLabel
○ תחום הכרה: private final.
○ תפקיד ושימוש: אובייקט האחראי על רינדור טקסט ה-"Next" המופיע בפינת המסך. הוא משמש לניהול השקיפות (Alpha) של הטקסט כדי לייצר אפקט הופעה הדרגתית.
- private final Bitmap[] frames
○ תחום הכרה: private final.
○ תפקיד ושימוש: מערך המכיל את כל התמונות (Bitmaps) המרכיבות את הסצנה. התמונות נשלפות מתוך אובייקט הנתונים Scenes ומוצגות אחת אחרי השנייה.
- private double frameTimer
○ תחום הכרה: private.
○ תפקיד ושימוש: מונה זמן המצטבר בכל פריים של המשחק. כאשר הוא עובר את הסף המוגדר ב-SECONDS_PER_FRAME, המערכת תעבור אוטומטית לתמונה הבאה.
- private double inputGuardTimer
○ תחום הכרה: private.
○ תפקיד ושימוש: משתנה בקרת קלט. הוא מבטיח שגם אם השחקן לוחץ על המסך, הלחיצה תתקבל רק אם עבר זמן מינימלי מסוים (INPUT_DELAY_SEC), מה שמונע "ספאם" של לחיצות.
- private final String checkPointKey
○ תחום הכרה: private final.
○ תפקיד ושימוש: מזהה (Key) ייחודי לסצנה הנוכחית. משמש כקישור למסמך ה-WorldState בענן כדי לבדוק או לעדכן אם הסצנה הושלמה.
- private boolean isSkipping
○ תחום הכרה: private.
○ תפקיד ושימוש: דגל לוגי המסמן שהסצנה בתהליך דילוג (אם כבר נצפתה בעבר). כשהוא פעיל, פעולות ה-Update וה-Render נעצרות כדי לחסוך במשאבים עד ליציאה מהמצב.

הסבר פעולות מרכזיות:

- public void onEnter()
○ פעולה זו מאתחלת את מצב הסצנה. תחילה היא בודקת מול הענן (באמצעות checkCheckPoint) האם השחקן כבר ראה את הסצנה הזו. אם כן - היא מבצעת דילוג מיידי. אם לא - היא מפעילה את המוזיקה המתאימה, מאפסת את אינדקס התמונה והטיימרים, ומכינה את טקסט ה-"Next" במצב שקוף.



○ קוד:

```
@Override
public void onEnter() {
    super.onEnter();

    // 1. Skip if already completed
    if (checkCheckPoint()) {
        isSkipping = true;
        onExit();
        return;
    }

    // 2. Setup audio
    MusicManager.getInstance(context).play(musicRes);

    // 3. Reset state
    this.currentFrameIndex = 0;
    this.frameTimer = 0;
    this.inputGuardTimer = 0;
    this.nextLabel.setAlpha(0);
}
```

● public void update(double delta)

- הפעולה מעדכנת את לוגיקת הזמן של הסצנה. היא מקדמת את טיימר הפריים ואת טיימר הגנת הקלט. בנוסף, היא מבצעת חישוב אינטרפולציה לערך ה-Alpha של ה-nextLabel כדי ליצור Fade-in ויזואלי. אם נגמר הזמן המוקצב לתמונה, היא קוראת לפעולת הקידום או הסיום.

○ קוד:

```
@Override
public void update(double delta) {
    if (isSkipping) return;

    frameTimer += delta;
    inputGuardTimer += delta;

    // Visual Fade-in logic for the "Next" prompt
    int currentAlpha = nextLabel.getAlpha();
    if (currentAlpha < 255) {
        int newAlpha = currentAlpha + (int) (255 * delta * 1.5);
        nextLabel.setAlpha(Math.min(newAlpha, 255));
    }

    // Automatic frame progression
    if (frameTimer >= SECONDS_PER_FRAME) {
        if (currentFrameIndex < frames.length - 1) {
            advanceFrame();
        } else {
            finishScene();
        }
    }
}
```

● public void touchEvents(MotionEvent event)

- מנהלת את האינטראקציה הידנית. בעת לחיצה, היא בודקת אם עבר מספיק זמן (לפי ה-inputGuardTimer). אם כן, היא מאפשרת לשחקן לקדם את הסצנה ידנית לתמונה הבאה, מה שמאפס את הטיימרים מחדש.

○ קוד:

```
@Override
public void touchEvents(MotionEvent event) {
    if (isSkipping) return;
```



```
if (event.getAction() == MotionEvent.ACTION_DOWN) {  
    // Prevent accidental double-taps using input guard  
    if (inputGuardTimer > INPUT_DELAY_SEC) {  
        if (currentFrameIndex < frames.length - 1) {  
            advanceFrame();  
            inputGuardTimer = 0; // Reset guard for next frame  
        } else finishScene();  
    }  
}
```

• private void finishScene()

- הפעולה המסיימת את רצף העלילה. היא ניגשת ל-WorldStateDoc ומעדכנת את ה-Checkpoint הרלוונטי כ-"נצפה". פעולה זו קריטית לשמירת רציפות הסיפור גם אם השחקן סוגר את האפליקציה ופותח אותה מחדש.

○ קוד:

```
private void finishScene() {  
    // Mark as completed in cloud storage  
    MyApp.getWorldStateDoc().setCheckpoint(checkPointKey);  
    onExit();  
}
```

בסיס נתונים (Firebase Firestore)

המחלקה מקושרת ישירות למערכת ה-Cloud Save של המשחק כדי לנהל את הליניאריות של הסיפור.

- סקירת המידע: המידע נשמר בתוך מסמך ה-worldState ב-Firebase, תחת Map שנקרא checkpoints המכיל זוגות של מפתח (שם הסצנה) וערך (בוליאני).

• פעולות:

- טעינה: ב-onEnter, המערכת קוראת את הערך מה-Cache של הענן כדי להחליט אם להציג את הסצנה.
- עדכון: ב-finishScene, המערכת כותבת true למפתח הסצנה, מה שמעורר סנכרון מול שרתי Firebase.

**מחלקת DeathScreen**

מיקום: com.example.tutorialgame.gamestates.DeathScreen

תפקיד המחלקה: מחלקה זו מנהלת את מצב המשחק המופעל בעת תבוסת השחקן ("מסך מוות"). תפקידה המרכזי הוא לספק משוב ויזואלי וקולי דרמטי המדגיש את סיום המשחק, תוך הצגת אנימציות פתיחה קולנועיות (Expanding Screen), הפעלת אפקטים של ניצוצות (Sparks) ומתן אפשרות למשתמש לבחור בין ניסיון חוזר לבין יציאה לתפריט הראשי.

היא אחראית על:

1. ניהול מצבי אנימציה: שליטה על המעבר בין שלב הפתיחה לשלב האינטראקטיבי שבו הכפתורים פעילים.
2. אנימציות פריסה גיאומטרית: חישוב שינוי גודל הרקע והמסגרת ממרכז המסך ועד לכיסוי מלא שלו.
3. ניהול אפקטים ויזואליים: הפעלה ותזמון של ארבעה מוקדי ניצוצות עצמאיים המגיבים לשלבי האנימציה.
4. בקרת שמע: הפעלת מוזיקת רקע ייעודית למסך המוות בסיום שלב המעבר.
5. טיפול בקלט משתמש: ניהול אירועי הלחיצה על כפתורי ה-Replay וה-Exit.

הסבר על תכונות המחלקה:

- `private final RectButton btnReplay, btnExit`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: פקדי כפתור גרפיים המאפשרים לשחקן לבחור את המשך הצעדים לאחר המוות.
- `private final NinePatchDrawable background, frame`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: אובייקטי הגרפיקה של הרקע והמסגרת האדומה. השימוש ב-NinePatch מבטיח פריסה חלקה ללא איבוד איכות על פני כל גודל מסך.
- `private int left, top, right, bottom`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנים המגדירים את גבולות המלבן של המסך. משתנים אלו משתנים דינמית במהלך פתיחת המסך ליצירת אפקט הפריסה.
- `private final TapEffect tapLT, tapRT, tapLB, tapRB`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: ארבעה מופעים נפרדים של אפקטי ניצוצות (Sparks) הממוקמים בפינות המסך.
- `private final TextRenderer titlePaint`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רכיב האחראי על רינדור כותרת ה-"YOU DIED" כולל ניהול צל והיסט (Shadow Offset).
- `private boolean isAnimationDone`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: דגל לוגי המציין האם אנימציות הפריסה הסתיימה. משמש לחסימת אינטראקציה עם כפתורים בזמן שלב המעבר.

הסבר פעולות מרכזיות:

- `private void updateOpeningAnimation(double delta)`
 - פעולה זו מחשבת את המידות החדשות של המסך בכל פריים ליצירת אפקט פריסה "סינמטי". הפעולה מקדמת תחילה את הציר האנכי (Top/Bottom) ממרכז המסך.



לקצוות. רק כאשר הגבולות האנכיים מגיעים לקצה המסך, הפעולה מתחילה להרחיב את הציר האופקי (Left/Right) עד למילוי מלא. בסיום כל חישוב, הפעולה מעדכנת את גבולות הציר של ה-Drawables.

○ **תרשים זרימה (סדר פעולות):**

- i. חישוב קצב הקידום (deltaPos) יחסית לזמן שחלף.
- ii. עדכון גבולות ה-Top וה-Bottom.
- iii. בדיקה: האם הציר האנכי הסתיים?
- אם כן: עדכון גבולות ה-Left וה-Right.
- iv. עדכון Bounds לאובייקטי ה-frame ו-background.
- v. אם כל הגבולות הגיעו לקצה: סימון `isAnimationDone = true`.

○ **קוד:**

```
private void updateOpeningAnimation(double delta) {
    int deltaPos = (int) (delta * 7.29 * TILE_SIZE);
    top = Math.max(0, top - deltaPos);
    bottom = Math.min(SCREEN_HEIGHT, bottom + deltaPos);

    if (top == 0 && bottom == SCREEN_HEIGHT) {
        int deltaX = deltaPos * 2;
        left = Math.max(0, left - deltaX);
        right = Math.min(SCREEN_WIDTH, right + deltaX);
    }

    // החדשים המיקומים חישבו לאחר הציר גבולות עדכון
    if (background != null) background.setBounds(left, top, right, bottom);
    if (frame != null) frame.setBounds(left, top, right, bottom);

    if (top == 0 && bottom == SCREEN_HEIGHT && left == 0 && right == SCREEN_WIDTH)
        isAnimationDone = true;
}
```

● **private void drawSparks(Canvas c)**

- מנהלת את תצוגת הניצוצות בשני מצבים שונים של המסך. בזמן שהמסך בתהליך פתיחה (`top != 0`), הפעולה מציירת את הניצוצות כך שהם "רודפים" אחרי הגבולות המתרחבים של הריבוע.

○ **קוד:**

```
private void drawSparks(Canvas c) {
    if (isAnimationDone) return;

    if (top != 0) {
        float midX = (left + right) / 2f;
        tapLT.show(c, midX, top + effectOffset);
        tapLB.show(c, midX, bottom - effectOffset);
    }
    else {
        tapLT.show(c, left + effectOffset, top + effectOffset);
        tapLB.show(c, left + effectOffset, bottom - effectOffset);
        tapRT.show(c, right - effectOffset, top + effectOffset);
        tapRB.show(c, right - effectOffset, bottom - effectOffset);
    }
}
```

● **public void onClick(BaseButton button)**

- מטפלת בבחירות השחקן ותגובת המערכת.
- **תרשים זרימה (סדר פעולות):**
- i. זיהוי הכפתור שנלחץ:
- אם `btnReplay`:
- a. החלפת מוזיקה למוזיקת עולם.



b. קריאה ל-`game.restartGame()` לטעינת נתונים חדשה.

● אם `btnExit`:

a. החלפת מוזיקה למוזיקת תפריט.

b. קריאה ל-`returnLauncher()` לסגירת המשחק.

ii. ביצוע `resetAnimation` להחזרת המסך למצבו ההתחלתי.

○ קוד:

```
@Override
public void onClick(BaseButton button) {
    if (button == btnReplay) {
        MusicManager.getInstance(context).play(R.raw.music_calm_village);
        game.restartGame();
    } else if (button == btnExit) {
        MusicManager.getInstance(context).play(R.raw.music_launcher);
        returnLauncher();
    }
    resetAnimation();
}
```

בסיס נתונים (Firebase Firestore)

המחלקה קשורה באופן עקיף למסד הנתונים בענן דרך פעולת ה-Restart.

● פעולות:

○ סנכרון: בעת לחיצה על כפתור ה-Replay, המערכת מפעילה את `game.restartGame()`.

פעולה זו גורמת ל-`MapManager` לנקות את המטמון ולמשוך מחדש את הנתונים

העדכניים ביותר מה-`StatsDoc` ומה-`WorldStateDoc` ב-Firebase, מה שמבטיח

שהשחקן חוזר לחיים עם הנתונים השמורים האחרונים שלו.

**מחלקת DialogState**מיקום: `com.example.tutorialgame.gamestates.playing.playingstates.DialogState`

תפקיד המחלקה: מחלקה זו מנהלת את מצב המשחק בזמן דיאלוגים בין השחקן לדמויות אחרות (NPCs). היא מתפקדת כמעין "במאי קולנוע" המשנה את התצוגה ואת האווירה כדי להעביר את מוקד השליטה לסיפור. המחלקה אחראית על יצירת אפקט "זום קולנועי" לעבר הדוברים, הצגת שוליים שחורים (Letterbox), הנמכת מוזיקת הרקע (Ducking) וניהול האינטראקציה מול פאנל הדיאלוג.

היא אחראית על:

1. בימוי מצלמה: יצירת תנועת מצלמה חלקה הממקדת את נקודת המבט במרכז שבין השחקן לדובר.
2. אנימציות זום קולנועיות: ניהול מעבר הדרגתי וחלק (Fade-in/out) של רמת הקירוב של המצלמה באמצעות פונקציית החלקה (Easing).
3. אפקטים קולנועיים: הצגת פסים שחורים בחלק העליון והתחתון של המסך לחיזוק החוויה הסיפורית.
4. ניהול סאונד: הפעלת אפקט Ducking במוזיקת הרקע כדי להבטיח שהדיאלוג הוא המוקד המרכזי.
5. סנכרון דמויות: דאגה לכך שהשחקן והדובר יסתכלו זה על זה בזמן השיחה.

הסבר על תכונות המחלקה:

- `private final DialogPanel dialogPanel`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מופע של רכיב ה-UI המציג את הטקסט והתמונות. אליו מועברים פקודות רינדור ואירועי מגע.
- `private float elapsedTime`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנה עזר השומר את הזמן שעבר (בשניות) מתחילת האנימציה. משמש כקלט לפונקציית ההחלקה.
- `private static final float MAX_CINEMATIC_ZOOM = 2.2f`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: קבוע המגדיר את רמת הזום המקסימלית בזמן דיאלוג להשגת אפקט "Close-up".
- `private boolean isEnding`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: דגל המציין אם המשתמש סיים את הדיאלוג והמערכת נמצאת בתהליך של זום-אאוט וחזרה לעולם המשחק.
- `private final Paint letterboxPaint`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מברשת הציור המשמשת לציור הפסים השחורים. שקיפותה (Alpha) משתנה דינמית לפי התקדמות האנימציה.

הסבר פעולות מרכזיות:

- `public void update(double delta)`
 - מנהלת את הלוגיקה והאנימציות בזמן אמת. הפעולה מעדכנת את מיקום המצלמה (LookAt) לעבר נקודת הפוקוס שבין הדמויות. לב המערכת הוא ניהול ה-`animProgress`.
 - **תרשים זרימה (סדר פעולות):**



- i. בכניסה: הערך עולה לעבר 1.0, מה שגורם למצלמה להתקרב ולשוליים השחורים להופיע.
- ii. ביציאה: הערך יורד לעבר 0.0. רק כשהוא מגיע לאפס, המשחק מחליף מצב חזרה ל-OVER_WORLD.
- iii. החלקה: הערך עובר פונקציית easeInOutCubic ליצירת תנועה רכה ויוקרתית.

קוד: ○

```
@Override
public void update(double delta) {
    dialogPanel.update(delta);
    updateFocusPoint();
    CameraManager.lookAt(focusPoint.x, focusPoint.y, delta);

    float startZoom = CameraManager.getFinalZoom();
    float deltaZoom = MAX_CINEMATIC_ZOOM - startZoom;

    if (!isEnding) {
        elapsedTime = Math.min(ZOOM_DURATION, elapsedTime + (float) delta);
    } else {
        elapsedTime = Math.max(0f, elapsedTime - (float) delta);
        if (elapsedTime <= 0f) {
            playingManager.setCurrentPlayingState(PlayingManager.PlayingState.OVER_WORLD);
        }
    }

    // פרמטרים 4 עם ההחלקה בפונקציית שימוש
    float currentZoom = easeInOutCubic(elapsedTime, startZoom, deltaZoom, ZOOM_DURATION);
    CameraManager.setTempZoom(currentZoom);

    // (בהחלקה משתמשים כאן גם) השוליים שקיפות עדכון
    float alphaProgress = easeInOutCubic(elapsedTime, 0, 220, ZOOM_DURATION);
    letterboxPaint.setAlpha((int) alphaProgress);
}
```

● public void setCurrentSpeaker(Character speaker)

- פעולת האתחול של הדיאלוג. הפעולה קובעת מי ה-NPC שאיתו מדברים, מפנה את הדמויות אחת כלפי השנייה, פונה ל-DialogueManager כדי לטעון את קובץ ה-JSON המתאים לשפה הנוכחית, ומפעילה את הצורך ב-dialogPanel.

קוד: ○

```
public void setCurrentSpeaker(Character speaker) {
    lastSpeaker = speaker;
    Character player = playingManager.getOverWorld().getPlayer();

    // Character interactions
    player.turnTowardsTarget(speaker);
    speaker.turnTowardsTarget(player);
    speaker.resetAnimation();
    speaker.setAttacking(false);

    // Resolve and load dialogue lines
    List<String> lines = DialogueManager.resolveDialogue(speaker.getGameCharType().name());
    speaker.getDialogueComponent().updateLines(lines);
    this.currentDialogue = speaker.getDialogueComponent();

    dialogPanel.startDialogue(speaker);
}
```

● public void endDialogue()

- במקום לסגור את המסך בבת אחת, פעולה זו רק "מרימה דגל". זה מאפשר למתודת ה-update לבצע את האנימציה ההפוכה (זום אאוט) בצורה חלקה לפני שהשליטה חוזרת לשחקן.



קוד: ○

```
public void endDialogue() {
    isEnding = true; // Start the cinematic exit sequence
}
```

public void render(Canvas c) ●

- ציור סצנת הדיאלוג. הפעולה מציירת תחילה את עולם המשחק (ללא רכיבי ה-UI הרגילים). לאחר מכן, היא מציירת שני מלבנים שחורים בחלק העליון והתחתון של המסך כדי ליצור את אפקט ה-Letterbox. לבסוף, היא מציירת את פאנל הדיאלוג מעל הכל.

קוד: ○

```
@Override
public void render(Canvas c) {
    // Draw the world with the cinematic camera settings
    playingManager.getOverWorld().renderWithoutUi(c);

    // Draw cinematic black bars (Letterbox)
    if (letterboxPaint.getAlpha() > 0) {
        float barHeight = SCREEN_HEIGHT * 0.12f * easeInOutCubic(animProgress);
        c.drawRect(0, 0, SCREEN_WIDTH, barHeight, letterboxPaint);
        c.drawRect(0, SCREEN_HEIGHT - barHeight, SCREEN_WIDTH, SCREEN_HEIGHT, letterboxPaint);
    }

    // Render the dialogue UI panel on top
    dialogPanel.draw(c);
}
```

private float easeInOutCubic(float t, float b, float c, float d) ●

- זוהי פעולה לוגית-מתמטית המיישמת פונקציית החלקה מסוג Standard Penner Easing. הסבר לוגי: במנועי משחק מקצועיים, תנועות ליניאריות נראות לעין האנושית כ"מכאניות". כדי ליצור אפקט זום רך, הפעולה מבצעת עיוות מתמטי של הזמן. התוצאה היא אנימציה שמתחילה לאט (Ease-in), מאיצה במרכז, ומאיטה שוב לקראת הסוף (Ease-out). הפונקציה מקבלת את הזמן שעבר (t), ערך ההתחלה (b), ההפרש לשינוי (c) ומשך הזמן הכולל (d).

קוד: ○

```
private float easeInOutCubic(float t, float b, float c, float d) {
    t /= d / 2;
    if (t < 1) return c / 2 * t * t * t + b;
    t -= 2;
    return c / 2 * (t * t * t + 2) + b;
}
```

**מחלקת WeatherEffect**

מיקום: WeatherEffect: engine.effects.weathereffects.com.example.tutorialgame

תפקיד המחלקה: מחלקה זו משמשת כשלד, תשתית ותבנית בסיס (Template) עבור כל האפקטים הסביבתיים והאקלימיים המופיעים בעולם המשחק (כגון עלים נופלים, עננים מרחפים, ובעתיד גשם או שלג). מטרתה המרכזית היא לרכז את התכונות והלוגיקה המשותפות לכלל האפקטים - ניהול מיקום, וקטורי מהירות, בקרת שקיפות (Alpha) וניהול מחזור חיים של חלקיקים. שימוש במחלקה אבסטרקטית זו מדגים את עקרון הירושה (Inheritance) בתיכנות מונחה עצמים, שכן היא מונעת שכפול קוד ומאפשרת למחלקות היורשות להתמקד אך ורק ב"רעש" (האקראיות הטבעית) ובגרפיקה הייחודית שלהן.

היא אחראית על:

1. ריכוז תשתית פיזיקלית: החזקת משתני המיקום והמהירות היסודיים המשמשים את כל סוגי האפקטים.
2. ניהול מחזור חיים (Lifecycle): הגדרת דגלי מצב המאפשרים למערכת לדעת מתי אפקט סיים את תפקידו (dead) ויש להחזירו למאגר לשימוש חוזר.
3. בקרת שקיפות (Alpha Management): ניהול ערך ה-Alpha המאפשר יצירת אפקטים של הופעה והיעלמות הדרגתית (Fade in/out).
4. אופטימיזציה משאבים: שימוש באובייקט Paint יחיד לכל מופע לחיסכון בהקצאות זיכרון בתוך לולאת המשחק.

הסבר על תכונות המחלקה:

- `protected final Random random`
 - תחום הכרה: `protected` - נגיש למחלקות היורשות ממנה.
 - תפקיד ושימוש: מחולל מספרים אקראיים המאותחל לפי זמן המערכת. משמש את המחלקות היורשות לקביעת ערכים אקראיים ייחודיים לכל מופע (כמו מהירות או קצב אנימציה) כדי ליצור מראה טבעי ולא אחיד.
- `protected final Paint paint`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: אובייקט הגדרות הציור. משמש לקביעת רמת השקיפות (Alpha) והמסננים הגרפיים של האפקט בעת הרינדור.
- `protected float x, y`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: קואורדינטות המיקום של האפקט במרחב עולם המשחק.
- `protected float speedX, speedY`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: וקטור המהירות המציין כמה פיקסלים האפקט יעבור בכל יחידת זמן.
- `protected int alpha`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: ערך השקיפות (0 עד 255). משמש ליצירת אפקט דעיכה (Fade out) לפני הסרת האפקט מהמסך.
- `protected boolean dead, fading`
 - תחום הכרה: `protected`.
 - תפקיד ושימוש: דגלים לוגיים לניהול מצב האפקט. `fading` מציין שהאפקט נמצא בתהליך היעלמות, ו-`dead` מאותת למנהל המפות (MapManager) שניתן לבצע לאפקט זה "לידה מחדש" (Respawn) במיקום חדש.

הסבר פעולות מרכזיות:

- `protected void applyMovement(double delta)`



- זוהי פעולת תשתית לניהול התנועה הפיזיקלית. הפעולה מעדכנת את מיקום האפקט על בסיס המהירות שלו. היא משתמשת בערך ה-delta (הזמן שעבר מהפריים הקודם) ומוכפלת בקבוע התדר 60. שימוש זה ב-Delta Time מבטיח שהאפקט ינוע באותה מהירות ויזואלית על גבי המסך בכל מכשיר, ללא קשר לעוצמת המעבד או לקצב הפריימים (FPS), ובכך שומרת על עקביות הפיזיקה של עולם המשחק.

○ קוד:

```
protected void applyMovement(double delta) {  
    x += (float) (speedX * (delta * 60));  
    y += (float) (speedY * (delta * 60));  
}
```

● public void respawn(float worldX, float worldY)

- פעולה לניהול מחזור חיים יעיל (Reuse). במקום להשמיד את האובייקט וליצור חדש בזיכרון (פעולה בזבזנית), הפעולה מבצעת "לידה מחדש". היא מקבלת מיקום חדש וקוראת למתודה האבסטרקטית init. כתוצאה מכך, האובייקט הקיים מאתחל את עצמו עם ערכים אקראיים חדשים, מה שחוסך משאבי מערכת ומונע עומס על ה-Garbage Collector. בנוסף מתבצע איפוס הסיד של תכונות האקראיות, מה שתורם לאינדיבידואליות המשתנה, ומונע את מחזוריות התנועה של כל אובייקט.

○ קוד:

```
public void respawn(float worldX, float worldY) {  
    init(worldX, worldY);  
    random.setSeed(System.currentTimeMillis());  
}
```

● protected abstract void init(float startX, float startY)

- מתודה אבסטרקטית המחייבת כל מחלקה יורשת להגדיר את אופן האתחול שלה. בתוך מתודה זו, כל אפקט (כמו עלה או ענף) קובע לעצמו את ה"רעש" והערכים האקראיים הייחודיים לו.



מחלקת Leaf

מיקום: com.example.tutorialgame.engine.ui.effects.weathereffects.leaves.Leaf

תפקיד המחלקה: מחלקה זו היא מימוש קונקרטי של WeatherEffect המייצגת עלה בודד הנופל מעצי הכפר. תפקידה הוא ליצור אפקט ויזואלי טבעי ודינמי המעשיר את עולם המשחק. המחלקה משלבת את תשתית התנועה של מחלקת האם יחד עם לוגיקה ייחודית של "נדנד" (Drift) מבוסס סינוס ואנימציות פריימים אקראית, מה שמבטיח שכל עלה יתנהג בצורה שונה ומונע מראה רובוטי או אחיד.

היא אחראית על:

1. סימולצית נדנד (Swaying): שימוש בנוסחאות טריגונומטריות ליצירת תנועה אופקית מתנדנדת המדמה רוח.
2. ניהול אנימציה אסינכרונית: הרצת רצף פריימים מתוך Sprite-Sheet בקצב משתנה לכל עלה.
3. בקרת מחזור חיים: ניהול מספר מחזורי אנימציה אקראיים לפני תחילת דעיכה והסרה מהמסך.
4. מימוש עקרון הירושה: שימוש במשתנים ובשיטות של מחלקת האם לניהול מיקום, מהירות ושקיפות.

הסבר על תכונות המחלקה:

- private final Leaves leafType
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט נתונים (Enum) המכיל את הפריימים הגרפיים של סוג העלה (ירוק/ורוד).
- private float driftIntensity, driftFrequency
 - תחום הכרה: private.
 - תפקיד ושימוש: משתנים הקובעים את "אופי" הנדנד ברוח. ה-Intensity קובע את רוחב התנועה לצדדים, וה-Frequency קובע את מהירות התנדודה.
- private double randomOffset
 - תחום הכרה: private.
 - תפקיד ושימוש: ערך אקראי המשמש כהיסט זמן. הוא מבטיח שכל עלה יתחיל את גל הסינוס שלו בנקודה אחרת, מה שמונע סנכרון בין העלים.
- private AnimationComponent animator
 - תחום הכרה: private final.
 - תפקיד ושימוש: רכיב האחראי על ניהול מצב האנימציה. הוא עוקב אחר אינדקס הפריים הנוכחי ומדווח למחלקה כאשר הושלם מחזור אנימציה שלם.
- private int cycles
 - תחום הכרה: private.
 - תפקיד ושימוש: מונה הקובע כמה פעמים האנימציה תחזור על עצמה לפני שהעלה יתחיל בתהליך הדעיכה מהמסך.

הסבר פעולות מרכזיות:

- public void update(double delta)
 - זוהי הפעולה הלוגית המרכזית המשלבת פיזיקה ואנימציה בכל פריים.
- **תרשים זרימה (סדר פעולות):**
 - i. חישוב נדנד: הפעולה משתמשת בזמן המערכת וב-randomOffset כדי לחשב ערך סינוס דינמי המדמה רוח.
 - ii. עדכון מיקום: הוספת ערך הנדנד למהירות ה-X הבסיסית ועדכון קואורדינטות העולם (תוך הכפלה ב-delta וביחס תדר של 60).
 - iii. ניהול אנימציה: הפעלת מתודת ה-update של ה-animator.



- .iv בדיקת מחזור: אם ה-`animator` החזיר `true` (סיום לופ), מונה ה-`cycles` יורד ב-1.
- .v מעבר לדעיכה: אם המונה הגיע ל-0, הדגל `fading` מופעל.
- .vi דעיכה: הפחתה הדרגתית של ה-`alpha` עד להגעה ל-0 וסימון האפקט כ-`dead`.

קוד: ○

```
@Override
public void update(double delta) {
    if (dead) return;
    // Combine base movement with sine-wave drift
    double time = System.currentTimeMillis() + randomOffset;
    float drift = (float) Math.sin(time * driftFrequency) * driftIntensity;

    x += (float) ((speedX + drift) * (delta * 60));
    y += (float) (speedY * (delta * 60));

    if (!fading) {
        // Use the animation component and detect cycle completion
        if (animator.update()) {
            cycles--;
            if (cycles <= 0) fading = true;
        }
    } else {
        alpha -= (int) (10 * delta * 60);
        if (alpha <= 0) {
            alpha = 0;
            dead = true;
        }
    }
}
```

● `protected void init(float startX, float startY)`

- פעולה זו דורסת את מתודת האתחול של מחלקת האם. היא מגדירה את ה"אישיות" הייחודית של העלה על ידי הגרלת ערכים אקראיים לכל משתני הפיזיקה והאנימציה. היא מעדכנת את רכיב האנימציה עם מהירות חדשה, ומגדירה את פרמטרי הנדנדו האופקיים. פעולה זו מאפשרת לעלה "להיוולד מחדש" (`Respawn`) עם אופי תנועה שונה לחלוטין ללא יצירת אובייקט חדש בזיכרון.

קוד: ○

```
@Override
protected void init(float startX, float startY) {
    this.x = startX;
    this.y = startY;
    // Randomize leaf-specific physics
    this.speedX = (random.nextBoolean() ? 1 : -1) * (0.5f + random.nextFloat() * 1.5f);
    this.speedY = 0.8f + random.nextFloat() * 1.2f;

    this.driftIntensity = 1.0f + random.nextFloat() * 2.5f;
    this.driftFrequency = 0.002f + random.nextFloat() * 0.003f;
    this.randomOffset = random.nextDouble() * 1000;

    // Initialize the animator component with a randomized speed
    int animationSpeed = GameConstants.Animation.SPEED + random.nextInt(8) - 4;
    this.animator.setSpeed(animationSpeed);
    this.cycles = random.nextInt(5) + 2;
}
```




מחלקת Cloud

מיקום: com.example.tutorialgame.engine.ui.effects.weathereffects.Cloud

תפקיד המחלקה: מחלקה זו היא מימוש קונקרטי של WeatherEffect המייצגת ענן המרחף בשמי עולם המשחק. תפקידה הוא להעניק עומק ויזואלי ואווירה "חיה" לרקע בצורה יעילה וחסכונית במשאבים. המחלקה משלבת את תשתית התנועה והדעיכה של מחלקת האם יחד עם טכניקות של טעינת משאבים סטטית (Static Caching), מה שמואפשר להציג עננים רבים בו-זמנית ללא פגיעה בביצועי המשחק (FPS).

היא אחראית על:

1. ניהול תנועה אטמוספרית: יצירת תנועת ריחוף איטית ורכה בצירי X ו-Y המדמה רוח גבית.
2. בקרת זמן חיים (TTL): ניהול משך זמן הופעה אקראי לכל ענן לפני מעבר אוטומטי לתהליך דעיכה.
3. אופטימיזציית זיכרון: טעינת תמונת הענן ויצירת מסנני הצבע פעם אחת בלבד עבור כל המופעים.
4. סימולציית דעיכה רכה: יישום תהליך הדרגתי של כניסה (Fade-in) ויציאה (Fade-out) של הענן מהמסך.

הסבר על תכונות המחלקה:

- `private static final Bitmap staticCloudBmp`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: תמונת הענן המשותפת לכל המופעים. היא נטענת ומבוצע לה Scaling פעם אחת בלבד בבוקר הסטטי, מה שחוסך זיכרון RAM ומונע פעולות קריאה יקרות מהדיסק בכל פעם שנוצר ענן חדש.
- `private static final PorterDuffColorFilter cloudFilter`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: פילטר צבע המוחל על העננים כדי להעניק להם גוון מותאם לאווירת המשחק (גוון אפרפר-כהה).
- `private long birthTime, lifeDuration`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתני תזמון. `birthTime` שומר את רגע ה"לידה" של הענן, ו-`lifeDuration` קובע כמה זמן הוא ירחף לפני שיתחיל להיעלם.
- תכונות מורשות (WeatherEffect): המחלקה משתמשת בשדות הירושים `x, y, speedX, speedY, alpha` ו-`paint` לניהול המצב הפיזיקלי שלה.

הסבר פעולות מרכזיות:

- `public void update(double delta)`
 - מנהלת את הלוגיקה הפיזיקלית והויזואלית של הענן בכל פריים.
 - **תרשים זרימה (סדר פעולות):**
 - i. תנועה: קריאה ל-`applyMovement` לעדכון המיקום לפי וקטור המהירות וה-Delta Time.
 - ii. שלב הופעה (Fade In): אם הענן בתחילת חייו, ערך ה-`alpha` עולה בהדרגה כדי למנוע "קפיצה" ויזואלית על המסך.
 - iii. בדיקת זמן חיים: הפעולה משווה את הזמן שעבר מאז יצירת הענן לערך ה-`lifeDuration` האקראי. אם הזמן חלף, הענן עובר למצב `fading`.
 - iv. שלב היעלמות (Fade Out): הפחתה הדרגתית של ה-`alpha` (מותאמת זמן).
 - v. סיום: סימון הענן כ-`dead` כדי לאותת למערכת שניתן להחזירו למאגר ולבצע לו Respawn במיקום חדש.

קוד: ○

```
@Override
public void update(double delta) {
```



```
if (dead) return;

applyMovement(delta);

if (!fading) {
    if (alpha < 60) alpha++;
    if (System.currentTimeMillis() - birthTime >= lifeDuration) fading = true;
} else {
    alpha -= (int) (5 * delta * 60);
    if (alpha <= 0) {
        alpha = 0;
        dead = true;
    }
}
paint.setAlpha(alpha);
}
```

● static { ... } (בלוק אתחול סטטי)

- פעולה זו מתבצעת ברגע שהמחלקה נטענת לראשונה. היא מבצעת את כל ה"עבודה הכבדה": טעינת ה-Bitmap מהמשאבים, התאמת הגודל שלו לרזולוציית המכשיר (Scaling), ויצירת ה-ColorFilter. כך, יצירת ענן חדש (Constructor) הופכת לפעולה מהירה ביותר שאינה צורכת משאבי עיבוד משמעותיים.



ניהול תוכן וקוסמטיקה (Content & Customization)

תת-פרק זה יכלול את כל מערכת האחראית על מסגרות העיצוב היפות מאוד שניתן להשיג במהלך המשחק (מסגרות מהממות שמישהו שילם דולר שלם בשבילן)

מחלקת `FrameUnlockCondition`

מיקום: `com.example.tutorialgame.engine.ui.circleframes.FrameUnlockCondition`

תפקיד המחלקה: מחלקה זו מגדירה את הלוגיקה של מערכת פתיחת הנעילות (Unlock System) עבור פריטים קוסמטיים. היא משמשת כ"מנוע חוקים" הקובע מהו הקריטריון הנדרש כדי להשיג פריט מסוים (למשל: הגעה לשלב מסוים, ניצחון על אויבים או רכישה בכסף וירטואלי). המחלקה יודעת לקשר בין סוג התנאי לבין הנתונים האמיתיים השמורים בפרופיל המשתמש.

היא אחראית על:

1. הגדרת סוגי התניות: ניהול רשימת אפשרויות לפתיחת פריטים (Enum).
2. ניהול מחירי מדרוג: החזקת רשימת ערכי סף (Price Array) התואמים לדרגות השונות בתוך סדרה.
3. סנכרון מול נתוני המשחק: שליפת הערכים הנוכחיים של השחקן (כמו רמת שלב או כמות מטבעות) מתוך ה-`SharedPreferences` והענן בזמן אמת.

הסבר על תכונות המחלקה:

- `public enum Condition`
 - תחום הכרה: `public`.
 - תפקיד ושימוש: רשימת הקבועים המייצגת את סוגי המשימות האפשריים (כמו רכישה-PURCHASE, ימי התחברות-DAYS ועוד).
- `private final List<Integer> priceArr`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רשימת ערכי המטרה. כל אינדקס ברשימה תואם לפריט בסדרה. למשל, אם מדובר ברמה, הערכים יכולים להיות `[1, 5, 10, ...]`.
- `private final Condition selectedCondition`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: סוג התנאי הספציפי שהוגדר עבור אובייקט זה.

הסבר פעולות מרכזיות:

- `public int getCurrentAmount()`
 - זוהי הפעולה הלוגית המרכזית המחברת בין החוקים לבין מצב השחקן בפועל. הפעולה בודקת מהו סוג התנאי שנבחר (`selectedCondition`) ופונה למחלקות ניהול המידע הסטטיות (`MyApp.getProgress()` או `MyApp.getCosmetic()`) כדי למשוך את הנתון העדכני של השחקן. ערך זה ישווה מאוחר יותר לערך המטרה כדי לקבוע אם הפריט נפתח.
 - קוד:

```
public int getCurrentAmount() {
    switch (selectedCondition) {
        case DAYS: return MyApp.getProgress().getDaysLoggedIn();
        case LEVEL: return MyApp.getProgress().getLevel();
        case ENEMIES_DEFEATED: return MyApp.getProgress().getEnemiesDefeated();
        case PURCHASE: return MyApp.getCosmetic().getCoinsLeft();
        default: return -9999999;
    }
}
```



מחלקת CircleFrameSeries

מיקום: com.example.tutorialgame.engine.ui.circleframes.CircleFrameSeries

תפקיד המחלקה: מחלקה זו מייצגת "סדרת מסגרות" שלמה במשחק. היא משמשת כיחידת מידע המאגדת תחתיה קבוצה של פריטים קוסמיים בעלי נושא משותף (למשל: סדרת "Dragon Fist"), את שמה המתורגם של הסדרה, ואת מערך החוקים הנדרש לפתיחתם. המחלקה מהווה את הבסיס עבור ה-Adapters המציגים את התוכן בגלריה.

היא אחראית על:

1. איגוד נתונים: ריכוז רשימת הפריימים השייכים לסדרה הספציפית.
2. ניהול תרגום (Localization): החזקת מזהה מחרוזת (StringRes) המאפשר תמיכה בריבוי שפות עבור שמות הסדרות.
3. בדיקת תנאים: מתן תשובה לוגית לשאלה האם פריט מסוים בתוך הסדרה זכאי להיפתח על סמך מצב השחקן הנוכחי.

הסבר על תכונות המחלקה:

- `private final int seriesName`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מזהה משאב (Resource ID) של שם הסדרה. המחלקה שומרת את המספר ולא את הטקסט עצמו כדי לאפשר למערכת להחליף שפה בזמן אמת.
- `private final List<CircleFrames> frames`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: רשימה של אובייקטי Enum המכילים את ה-Bitmaps של המסגרות הכלולות בסדרה זו.
- `private final FrameUnlockCondition condition`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מופע של מחלקת `FrameUnlockCondition`. הוא מגדיר את סוג המשימה והערכים הנדרשים עבור כל פריט ברשימת ה-frames.

הסבר פעולות מרכזיות:

- `public boolean checkCondition(int i)`
 - הפעולה מקבלת אינדקס (i) של פריט בתוך הסדרה. היא פונה לאובייקט ה-`condition` שלה ומבצעת השוואה מתמטית: האם הערך הנדרש באינדקס זה (מתוך ה-`priceArr`) קטן או שווה לערך הנוכחי של השחקן. הפעולה מחזירה `true` אם השחקן עומד בתנאים.
 - קוד:

```
public boolean checkCondition(int i) {
    return (condition.getPriceArr().get(i) <= condition.getCurrentAmount());
}
```

- `public String getSeriesName()`
 - פעולה זו הופכת את מזהה המשאב (int) למחרוזת טקסט אמיתית הקריאה למשתמש. היא משתמשת ב-Context של האפליקציה כדי לשלוף את התרגום הנכון.
 - קוד:

```
public String getSeriesName() {
    return BaseActivity.getContext().getString(seriesName);
}
```

**מחלקת FrameData**

מיקום: com.example.tutorialgame.engine.ui.circleframes.FrameData

תפקיד המחלקה: מחלקה זו משמשת כ"מחסן הנתונים" (Data Repository) הסטטי והמרכזי עבור כל מערך המסגרות הקוסמטיות במשחק. תפקידה הוא לרכז במקום אחד את כל הגדרות הקולקציות (Series), שמות הסדרות, פריטי הגרפיקה הכלולים בהן ותנאי הפתיחה הייחודיים לכל פריט. היא מספקת נקודת גישה אחידה עבור שאר חלקי האפליקציה, ומפרידה בין הגדרת התוכן לבין הלוגיקה של המערכת.

היא אחראית על:

1. איגוד תוכן המשחק: הגדרה ידנית של כל סדרות המסגרות (למשל: "Frost", "Dragon Fist", "Demon").
2. ניהול זיכרון יעיל (Caching): טעינת כל אובייקטי המידע לזיכרון ה-RAM פעם אחת בלבד עם עליית האפליקציה.
3. הבטחת עקביות (Data Integrity): שימוש ברשימה קבועה (final) המונעת שינויים לא רצויים בנתוני המשחק בזמן ריצה.

הסבר על תכונת המחלקה:

● `private static final List<CircleFrameSeries> allSeries`

- תחום הכרה: `private static final`.
- תפקיד ושימוש: הרשימה המרכזית המאחסנת את כל מופעי הסדרות. הגדרתה כ-`final` מבטיחה שהפרנס לרשימה לא ישתנה לעולם, והגדרתה כ-`static` מבטיחה שהנתונים משותפים לכל חלקי האפליקציה ללא צורך ביצירת מופעים של המחלקה.

הסבר פעולה מרכזיות:

● `static { ... }` (בלוק אתחול סטטי)

- זוהי הפעולה המרכזית המאכלסת את מאגר הנתונים של המסגרות. הבלוק מתבצע פעם אחת בלבד ברגע שהמערכת ניגשת למחלקה בפעם הראשונה. בתוך הבלוק, נוצרת רשימה ארוכה של אובייקטי `CircleFrameSeries`. עבור כל סדרה, מוגדרים שלושה רכיבים: מזהה השם (Resource ID), רשימת התמונות (Bitmaps) הכלולות בה, ואובייקט `FrameUnlockCondition` המגדיר את הקריטריונים להשגת כל פריט בסדרה. שיטה זו מאפשרת ניהול תוכן נוח ומרוכז המופרד מקוד ה-UI.

○ קוד (דוגמה למבנה הבלוק):

```
static {
    allSeries.add(new CircleFrameSeries(R.string.series_a_new_start,
        Arrays.asList(
            CircleFrames.FRAME_00,
            CircleFrames.FRAME_10,
            CircleFrames.FRAME_11,
            CircleFrames.FRAME_12,
            CircleFrames.FRAME_42,
            CircleFrames.FRAME_43),
        new FrameUnlockCondition(FrameUnlockCondition.Condition.LEVEL, Arrays.asList(
            0, 3, 9, 27, 36, 72))
    ));
}
```

// כך מוגדרים כל איברי הרשימה, עם הפרטים המתאימים להם

**מחלקת FramesAdapter**

מיקום: com.example.tutorialgame.engine.ui.circleframes.FramesAdapter

תפקיד המחלקה: מחלקה זו היא ה-Adapter הפנימי האחראי על ניהול התצוגה והאינטראקציה של פריטי המסגרות בתוך שורה (סדרה) בודדת. היא מהווה את הלב החווייתי של הגלריה, שכן היא מנהלת את מצבי הנעילה, אנימציות פתיחת הפריטים, והצגת מידע מפורט על התקדמות השחקן באמצעות בועות מידע אינטראקטיביות.

היא אחראית על:

1. ניהול מצבי ויזואליים: קביעת רמת השקיפות וצבע המנעול של כל פריים בהתאם לעמידה בתנאי הפתיחה (למשל: מנעול ירוק כאשר יש מספיק מטבעות).
2. אנימציות פתיחה (Unlock Animation): ביצוע אפקט ויזואלי של נפילת המנעול והיעלמותו בעת רכישה או עמידה ביעד.
3. ממשק מידע (Popups): ניהול ותצוגה של בועות Balloon המפרטות את המשימה הנדרשת לפתיחת כל פריים.
4. סנכרון נתונים מקומי: עדכון ה-CosmeticDoc בעת בחירת פריים חדש או ביצוע רכישה.

הסבר על תכונות המחלקה:

- private final CircleFrameSeries series
 - תחום הכרה: private final.
 - תפקיד ושימוש: אובייקט הנתונים של הסדרה הנוכחית. משמש לבדיקת תנאי הפתיחה עבור כל פריט בשורה.
- private final OnDataChangeListener dataChangeListener
 - תחום הכרה: private final.
 - תפקיד ושימוש: ממשק דיווח המקשר בין האדפטר הפנימי לאדפטר הראשי. מאפשר לעדכן שורות אחרות בגלריה כאשר מתבצעת רכישה (שימוש במטבעות).
- private final String selectedFrameName
 - תחום הכרה: private final.
 - תפקיד ושימוש: מזהה את הפריים שנבחר על ידי השחקן, לצורך הצגת אפקט ה-Glow סביבו.

הסבר פעולות מרכזיות:

- private void setupLockedState(int pos, Context context)
 - פעולה הקובעת את הנראות של פריט נעול. הפעולה מבצעת בדיקה כפולה: האם התנאי הלוגי התקיים (למשל יש מספיק כסף) והאם הפריט הקודם בסדרה כבר נפתח. אם כן, המנעול נצבע בצבע ייחודי (לדוגמה ירוק לרכישה) והשקיפות שלו יורדת, כדי לרמוז לשחקן שהפריט מוכן לפתיחה.
 - קוד:

```
private void setupLockedState(int pos, Context context) {
    lockIcon.setVisibility(View.VISIBLE);
    lockIcon.setImageBitmap(CircleFrames.LOCK.getCircleFrame());

    if (series.checkCondition(pos) && (pos == 0 ||
    MyApp.getCosmetic().isFrameAvailable(frames.get(pos-1).name()))) {
        frameImage.setAlpha(1f);
        lockIcon.setAlpha(0.3f);
        if (series.getCondition().getSelectedCondition() ==
        FrameUnlockCondition.Condition.PURCHASE)
            lockIcon.setColorFilter(Color.GREEN);
        else lockIcon.setColorFilter(ContextCompat.getColor(context,
        R.color.text_color_pressed));
    } else {
```



```
frameImage.setAlpha(0.7f);  
lockIcon.setAlpha(1f);  
lockIcon.clearColorFilter();  
}  
}
```

• private void handleUnlock(int pos)

- מנהלת את תהליך פתיחת הפריים.
- **תרשים זרימה (סדר פעולות):**
 - i. ביצוע רכישה: לו התנאי הוא כספי, המערכת מפחיתה את המטבעות ממאגר השחקן.
 - ii. דיווח גלובלי: הפעלת ה-`dataChangeListener` כדי ששורות אחרות יתעדכנו במצב המטבעות החדש.
 - iii. אנימציה: הפעלת אנימציה `animate()` על אייקון המנעול (סיבוב ב-90 מעלות, נפילה כלפי מטה ו-fade out).
 - iv. ריענון פנימי: בסיום האנימציה, המערכת מעדכנת רק את הפריט שנפתח ואת הפריט הבא אחריו בסדרה.

○ קוד:

```
private void handleUnlock(int pos) {  
    String frameName = frames.get(pos).name();  
    lockIcon.clearColorFilter();  
  
    // 1. (הדיווח לפני חשוב) לוגית הרכישה ביצוע  
    if (series.getCondition().getSelectedCondition() ==  
        FrameUnlockCondition.Condition.PURCHASE) {  
        MyApp.getCosmetic().purchase(series.getCondition().getPriceArr().get(pos));  
        SoundManager.getInstance(itemView.getContext()).playSfx(R.raw.sfx_coin_drop);  
    }  
  
    MyApp.getCosmetic().addAvailableFrame(frameName);  
    SoundManager.getInstance(itemView.getContext()).playSfx(R.raw.sfx_unlock);  
  
    // 3. מקומית אנימציה  
    lockIcon.animate()  
        .rotation(90f)  
        .translationY(100f)  
        .alpha(0f)  
        .setDuration(500)  
        .withEndAction(() -> {  
            lockIcon.setVisibility(View.GONE);  
            notifyItemChanged(pos);  
            dataChangeListener.onDataChanged(seriesPosition);  
            if (pos + 1 < frames.size()) notifyItemChanged(pos + 1);  
        })  
        .start();  
}
```

**מחלקת FrameSeriesAdapter**

מיקום: com.example.tutorialgame.engine.ui.circleframes.FrameSeriesAdapter

תפקיד המחלקה: מחלקה זו היא ה-Adapter הראשי (Parent Adapter) האחראי על ניהול מבנה הגלריה בשיטת ה-RecyclerView המקוון. תפקידה הוא לייצר ולנהל את השורות האופקיות בגלריה, כאשר כל שורה מייצגת סדרה קוסמטית שונה (Series). המחלקה משמשת כ"מתזמרת" (Orchestrator) המנהלת את סנכרון הנתונים בין השורות השונות, עוקבת אחר בחירת המשתמש ברמה הגלובלית, ומיישמת מנגנוני רענון ממוקדים לשיפור ביצועי המערכת.

היא אחראית על:

1. ניהול היררכיית רשימות: יצירת אדפטרים פנימיים (FramesAdapter) עבור כל שורה בגלריה.
2. סנכרון חוצה-שורות (Cross-Row Sync): עדכון סטטוס המנעולים בשורות מבוססות רכישה כאשר מתבצעת קנייה בשורה אחרת, ללא צורך ברענון כללי של המסך.
3. ניהול בחירה בלעדית: מעקב אחר המיקום הדו-ממדי של הפריים הנבחר והבטחה שרק פריט אחד בכל הגלריה יסומן כפעיל.
4. אופטימיזציית זיכרון וביצועים: שימוש ב-SparseArray לניהול יעיל של האדפטרים הפנימיים ומניעת בנייה מחדש של רכיבי UI קיימים.

הסבר על תכונות המחלקה:

- `private final List<CircleFrameSeries> seriesList`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מקור הנתונים המכיל את רשימת הסדרות שנטענו ממחלקת `FrameData`.
- `private final SparseArray<FramesAdapter> childAdapters`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מבנה נתונים אופטימלי השומר רפרנס לכל אדפטר פנימי שנוצר (Child Adapter) לפי מספר השורה שלו. זהו המפתח המאפשר למחלקה "לפקוד" על שורות ספציפיות להתרען מבלי להשפיע על האחרות.
- `private int selectedSeriesPosition, selectedFramePosition`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: משתנים השומרים את המיקום המדויק (אינדקס שורה ואינדקס עמודה) של הפריים שהשחקן "לובש" כרגע.
- `private final Runnable globalRefreshTask`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: פעולת Callback המגיעה מה-Fragment (בדרך כלל עדכון מונה האיסוף העליון), המופעלת בכל פעם שמתבצע שינוי בנתונים.

הסבר פעולות מרכזיות:

- `public void onChanged(int seriesPos)`
 - פעולה המממשת את ממשק ההקשבה לשינויי נתונים בשורות. כאשר מתבצעת רכישה או פתיחת פריים בשורה מסוימת, הפעולה נקראת. היא מפעילה את עדכון המונה הכללי ואז רצה על כל האדפטרים השמורים ב-`childAdapters`. הפעולה מוודאת שהיא מרעננת רק שורות מבוססות רכישה שהן אינן השורה שבה התבצעה הפעולה כרגע. בכך היא מאפשרת לשורות אחרות להתעדכן במחירי המטבעות החדשים מבלי לקטוע את אנימציות המנעול שרצה בשורה הנוכחית.

○ קוד:

```
@Override
public void onChanged(int seriesPos) {
    if (globalRefreshTask != null) globalRefreshTask.run();
}
```




```
for (int i = 0; i < childAdapters.size(); i++) {  
    int key = childAdapters.keyAt(i);  
    if (key != seriesPos) {  
        FramesAdapter adapter = childAdapters.get(key);  
        if (adapter.getConditionType() == FrameUnlockCondition.Condition.PURCHASE) {  
            adapter.refreshLockedItemsStatus();  
        }  
    }  
}  
}
```

• private void findInitialSelection()

- פעולה המתבצעת פעם אחת בלבד בעת יצירת האדפטר. הפעולה סורקת את הרשימה המקוננת (כל הסדרות וכל הפריימים בתוכן) ומשווה אותם לפריימס השמור בנתוני המשתמש בענן. ברגע שהיא מוצאת את הפריימס המצויד, היא שומרת את מיקומו כדי שבעת הרינדור הראשוני המערכת תדע היכן לצייר את אפקט ה-Glow.

○ קוד:

```
private void findInitialSelection() {  
    String currentFrameName = MyApp.getCosmetic().getCurrentFrame();  
    if (currentFrameName == null) return;  
    for (int i = 0; i < seriesList.size(); i++) {  
        List<CircleFrames> frames = seriesList.get(i).getFrames();  
        for (int j = 0; j < frames.size(); j++) {  
            if (frames.get(j).name().equals(currentFrameName)) {  
                selectedSeriesPosition = i;  
                selectedFramePosition = j;  
                return;  
            }  
        }  
    }  
}
```

• public void onBindViewHolder(@NonNull SeriesViewHolder holder, int position)

- מקשרת בין נתוני הסדרה לבין ה-View המייצג אותה. בתוך פעולה זו מוגדר ה-FrameSelectionListener. כאשר פריימס נבחר בתוך שורה, המאזין מעדכן את משתני ה-Selected הגלובליים וקורא ל-notifyItemChanged עבור השורה החדשה והשורה הקודמת בלבד. טכניקה זו מבטיחה שרק שתי שורות ירונדו מחדש במקום כל הגלריה.

○ קוד:

```
@Override  
public void onBindViewHolder(@NonNull SeriesViewHolder holder, int position) {  
    CircleFrameSeries series = seriesList.get(position);  
    holder.bind(series, position, (seriesPos, framePos) -> {  
        int previousSeriesPos = selectedSeriesPosition;  
        selectedSeriesPosition = seriesPos;  
        selectedFramePosition = framePos;  
        notifyItemChanged(seriesPos);  
        if (previousSeriesPos != -1 && previousSeriesPos != seriesPos) {  
            notifyItemChanged(previousSeriesPos);  
        }  
    }, this);  
}
```



כלי עזר ואלגוריתמיקה (Logic Utilities & Algorithms)

פרק זה מרכז את מחלקות השירות והכלים החשובים המשמשים כ"תשתית הלוגית" של המשחק. מחלקות אלו אינן מנהלות מצב ואינן מצוירות על המסך, אלא מספקות פתרונות מתמטיים ואלגוריתמיים מורכבים ליחידות האחרות במנוע, כגון חישובי מרחקים, זיהוי התנגשויות וניהול מערכות צירים.

מחלקת TmxLoaderUtils

מיקום: com.example.tutorialgame.utils.TmxLoaderUtils

תפקיד המחלקה: מחלקה זו משמשת כ"מתרגם" המרכזי של עולם המשחק. תפקידה הוא לקרוא קבצי מפה בפורמט TMX (פורמט סטנדרטי של Tiled Map Editor), לפענח את מבנה ה-XML שלהם ולהמיר אותם לאובייקטי נתונים (MapLoadData) שהמנוע יודע לרנדר. המחלקה מטפלת בטעינת שכבות אריחים (Tiles), אובייקטים אינטראקטיביים, תאורה ומאפיינים גלובליים של המפה, ובכך מאפשרת עיצוב שלבים חיצוני ודינמי ללא שינוי בקוד המקור.

היא אחראית על:

1. פענוח שכבות אריחים (Tile Parsing): המרת נתוני CSV של מזהי אריחים למטריצות דו-ממדיות של מספרים שלמים.
2. ניהול Tilesets: קישור בין מזהי אריחים גלובליים (GIDs) לבין קבצי המקור והמאפיינים הספציפיים שלהם.
3. עיבוד אובייקטים (Object Mapping): חילוץ מיקומי דמויות, בניינים, נקודות תאורה ומעברים בין מפות, כולל קריאת מאפיינים מותאמים אישית (Custom Properties).
4. טעינת הגדרות עולם: שליפת נתוני אווירה כמו רמת חשיכה (ambientDarkness) וסוגי אויבים לשכפול.

הסבר על תכונות המחלקה:

- `private static final String TAG = "TmxLoaderUtils"`
- תחום הכרה: `private static final`
- תפקיד ושימוש: מחרוזת קבועה המשמשת לזיהוי הודעות ב-Logcat. חיוני לאיתור תקלות בקבצי מפה לא תקינים בזמן אמת.

הסבר פעולות מרכזיות:

- `public static MapLoadData loadMapFromTMX(String fileName)`
- הפעולה הראשית שמתחילה את תהליך טעינת השלב. היא פותחת את הקובץ מתיקיית ה-Assets ומפעילה סדרה של פעולות פענוח עבור כל סוג שכבה. בסיום, היא מאגדת את כל המידע לאובייקט MapLoadData יחיד ומחזירה אותו למנוע.
- **תרשים זרימה (סדר פעולות):**
 - i. פתיחת הקובץ כ-InputStream ונירמול מבנה ה-XML.
 - ii. קריאת מאפייני מפה גלובליים (מאפייני אווירה).
 - iii. טעינת ופענוח כל ה-Tilesets המקושרים למפה.
 - iv. קריאת שכבות גרפיות (Ground, Walls, Surface) ושכבות פיזיקליות (Collision).
 - v. מעבר על כל שכבות האובייקטים (דמויות, אורות, דלתות).

○ קוד:

```
public static MapLoadData loadMapFromTMX(String fileName) {
    List<TilesetData> localTilesets = new ArrayList<>();
    int[][] groundLayer = null, wallLayer = null, collisionLayer = null, surfaceLayer = null;

    List<ObjectData> buildingLayer = new ArrayList<>(), fenceLayer = new ArrayList<>(),
        characterLayer = new ArrayList<>(), doorLayer = new ArrayList<>(),
        natureLayer = new ArrayList<>(), itemLayer = new ArrayList<>(),
        objectLayer = new ArrayList<>(), lightLayer = new ArrayList<>(),
        animatedLayer = new ArrayList<>(), breakableLayer = new ArrayList<>();
```



```
int ambientDarkness = 0, minMonsters = 0;
String spawnType = "";

try (InputStream is = BaseActivity.getContext().getAssets().open("maps/" + fileName)) {
    DocumentBuilder docBuilder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
    Document doc = docBuilder.parse(is);
    doc.getDocumentElement().normalize();

    Element mapRoot = doc.getDocumentElement();

    // Parse global map properties
    NodeList mapPropsNodes = mapRoot.getElementsByTagName("properties");
    if (mapPropsNodes.getLength() > 0) {
        NodeList props = ((Element)mapPropsNodes.item(0)).getElementsByTagName("property");
        for (int i = 0; i < props.getLength(); i++) {
            Element p = (Element) props.item(i);
            String pName = p.getAttribute("name");
            String pVal = p.getAttribute("value");
            if ("ambientDarkness".equals(pName)) ambientDarkness = Integer.parseInt(pVal);
            else if ("minMonsters".equals(pName)) minMonsters = Integer.parseInt(pVal);
            else if ("spawnType".equals(pName)) spawnType = pVal;
        }
    }

    // Parse tilesets
    NodeList tilesetNodes = doc.getElementsByTagName("tileset");
    for (int i = 0; i < tilesetNodes.getLength(); i++) {
        TilesetData ts = parseTileset((Element) tilesetNodes.item(i));
        if (ts != null) localTilesets.add(ts);
    }

    // Parse tile layers
    groundLayer = parseTileLayer(doc, "Ground", localTilesets);
    wallLayer = parseTileLayer(doc, "Walls", localTilesets);
    collisionLayer = parseTileLayer(doc, "Collision", localTilesets);
    surfaceLayer = parseTileLayer(doc, "Surface", localTilesets);

    // Parse object layers
    buildingLayer = parseObjectLayer(doc, "Buildings");
    fenceLayer = parseObjectLayer(doc, "Fences");
    characterLayer = parseObjectLayer(doc, "Characters");
    natureLayer = parseObjectLayer(doc, "Natures");
    itemLayer = parseObjectLayer(doc, "Items");
    objectLayer = parseObjectLayer(doc, "Objects");
    doorLayer = parseObjectLayer(doc, "Doors");
    lightLayer = parseObjectLayer(doc, "Lights");
    animatedLayer = parseObjectLayer(doc, "AnimatedObjects");
    breakableLayer = parseObjectLayer(doc, "Breakables");

} catch (Exception e) {
    Log.e(TAG, "Error loading TMX map file: " + fileName, e);
}

MapLoadData loadData = new MapLoadData(
    fileName, groundLayer, wallLayer, collisionLayer, surfaceLayer, localTilesets,
    buildingLayer, fenceLayer, characterLayer, natureLayer,
    itemLayer, objectLayer, doorLayer, lightLayer, animatedLayer, breakableLayer
);
loadData.ambientDarkness = ambientDarkness;
loadData.minMonsters = minMonsters;
loadData.spawnType = spawnType;
return loadData;
}
```



private static int[][] parseTileLayer(Document doc, String layerName, List<TilesetData> tilesets) •

- הפעולה האחראית על המרת הטקסט הגולמי במפה למבנה נתונים לוגי. היא משתמשת ב-Scanner כדי לעבד את הנתונים בצורה יעילה בזיכרון. עבור כל מזהה אריח (GID), היא מבצעת חישוב מול ה-Tilesets כדי להמיר אותו למזהה מקומי שניתן לרנדר.

○ קוד:

```
private static int[][] parseTileLayer(Document doc, String layerName, List<TilesetData>
tilesets) {
    Element layerEl = findLayerElement(doc, layerName);
    if (layerEl == null) return null;

    int width = Integer.parseInt(layerEl.getAttribute("width"));
    int height = Integer.parseInt(layerEl.getAttribute("height"));
    int[][] mapData = new int[height][width];

    NodeList dataNodes = layerEl.getElementsByTagName("data");
    if (dataNodes.getLength() == 0) return null;

    String csvData = dataNodes.item(0).getTextContent();
    try (Scanner scanner = new Scanner(csvData).useDelimiter("[\\s,]+")) {
        for (int row = 0; row < height; row++) {
            for (int col = 0; col < width; col++) {
                if (!scanner.hasNextInt()) break;
                int gid = scanner.nextInt();

                if (gid <= 0) {
                    mapData[row][col] = -1;
                } else {
                    // Calculate local ID by finding the correct tileset's firstGid
                    int firstGid = findFirstGidForGid(gid, tilesets);
                    mapData[row][col] = gid - firstGid;
                }
            }
        }
    }
    return mapData;
}
```

private static List<ObjectData> parseObjectLayer(Document doc, String layerName) •

- פעולה זו סורקת את שכבות האובייקטים הוקטוריים. היא מחלצת עבור כל אובייקט את הקואורדינטות שלו (בתוספת המרה ל-SCALE_MULTIPLIER של המנוע) ומפענחת מאפיינים מיוחדים כמו צבע תאורה, דלת יעד או נקודת שמירה נדרשת.

○ קוד:

```
private static List<ObjectData> parseObjectLayer(Document doc, String layerName) {
    List<ObjectData> objectList = new ArrayList<>();
    NodeList objectGroups = doc.getElementsByTagName("objectgroup");

    for (int i = 0; i < objectGroups.getLength(); i++) {
        Element objectGroup = (Element) objectGroups.item(i);
        if (layerName.equals(objectGroup.getAttribute("name"))) {
            NodeList objects = objectGroup.getElementsByTagName("object");
            for (int j = 0; j < objects.getLength(); j++) {
                try {
                    Element objectEl = (Element) objects.item(j);
                    String name = objectEl.getAttribute("name");
                    float x = Float.parseFloat(objectEl.getAttribute("x"));
                    float y = Float.parseFloat(objectEl.getAttribute("y"));
                    float width = objectEl.hasAttribute("width") ?
Float.parseFloat(objectEl.getAttribute("width")) : 0;
                    float height = objectEl.hasAttribute("height") ?
Float.parseFloat(objectEl.getAttribute("height")) : 0;
                } catch (Exception e) {
                    // ignore
                }
            }
        }
    }
    return objectList;
}
```



```
String type = objectEl.hasAttribute("type") ? objectEl.getAttribute("type") :
:
(objectEl.hasAttribute("class") ? objectEl.getAttribute("class") :
"");

String connectsTo = "", targetDoor = "", requiredCp = "";
int lightColor = Color.WHITE;
LightSource.LightType lightType = LightSource.LightType.STATIC;

// Parse custom properties for the object
NodeList properties = objectEl.getElementsByTagName("property");
for (int k = 0; k < properties.getLength(); k++) {
    Element prop = (Element) properties.item(k);
    String pName = prop.getAttribute("name");
    String pVal = prop.getAttribute("value");
    switch (pName) {
        case "type": type = pVal; break;
        case "connectsTo": connectsTo = pVal; break;
        case "targetDoor": targetDoor = pVal; break;
        case "requiredCheckpoint": requiredCp = pVal; break;
        case "color": try { lightColor = Color.parseColor(pVal); } catch
(Exception ignored) {} break;
        case "lightType": try { lightType =
LightSource.LightType.valueOf(pVal.toUpperCase()); } catch (Exception ignored) {} break;
    }
}

// Centering logic for lights, top-left for other objects
PointF pos = (layerName.equals("Lights")) ?
    new PointF((x + width / 2f) * SCALE_MULTIPLIER, (y + height / 2f) *
SCALE_MULTIPLIER) :
    new PointF(x * SCALE_MULTIPLIER, y * SCALE_MULTIPLIER);

ObjectData data = new ObjectData(name, type, pos, connectsTo, targetDoor,
requiredCp);

if (layerName.equals("Lights")) {
    data.setLightProperties((width / 2f) * SCALE_MULTIPLIER, lightColor,
lightType);
}
objectList.add(data);
} catch (Exception e) {
    Log.w(TAG, "Failed to parse object in layer " + layerName, e);
}
}
break;
}
}
return objectList;
}
```

בסיס נתונים (Assets & TMX Files)

המחלקה משמשת כגשר בין קבצי משאבים חיצוניים לבין המערכת הדינמית.

- סקירת המידע: המידע מאוחסן בפורמט XML הכולל נתוני CSV (עבור אריחים) ומבני Node (עבור אובייקטים).
- פעולות:

○ טעינה: מתבצעת פעם אחת בעת כניסה למפה חדשה. המידע נטען מהדיסק (Assets) לזיכרון ה-RAM בצורה של אובייקטי MapLoadData לשימוש מהיר של ה-Renderer.

**מחלקת ObjectPoolManager**

מיקום: com.example.tutorialgame.managers.ObjectPoolManager

תפקיד המחלקה: מחלקה זו משמשת כמנהל משאבים ריכוזי המיישם את תבנית העיצוב Object Pool. תפקידה הוא לנהל "מאגרי זיכרון" עבור ישויות שנוצרות ומושמדות בתדירות גבוהה (כגון מטבעות, חלקיקי פיצוץ ואפקטים ויזואליים). במקום להקצות זיכרון חדש עבור כל אובייקט, המחלקה ממחזרת מופעים קיימים מתוך מערכים קבועים. גישה זו מונעת את תופעת ה"גמגומים" (Stutters) הנגרמת על ידי מנגנון ה-Garbage Collector של אנדרואיד.

היא אחראית על:

1. מניעת לחץ על הזיכרון (Memory Pressure): שימוש חוזר באובייקטים קיימים למניעת הקצאות RAM מיותרות.
2. ניהול מחסניות אובייקטים: תחזוקת מערכים סטטיים ומצביעים מהירים (particleTop, coinTop וכו') לניהול המלאי הזמין.
3. בטיחות בתהליכים (Thread Safety): סנכרון הגישה למאגרים כדי למנוע התנגשויות בין ה-Game Loop ל-Threads אחרים.
4. אופטימיזציה של זמן ריצה: צמצום פעולות ה-new למינימום ההכרחי בלבד (רק כאשר המאגר ריק).

הסבר על תכונות המחלקה:

- `private static final BrokenParticle[] particlePool`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מערך קבוע בגודלו המשמש כ"מחסנית" (Stack) השומרת חלקיקי שבירה פנויים.
- `private static int particleTop`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: מצביע (Index) המציין את ראש המחסנית. ערכו מייצג את כמות האובייקטים הפנויים כרגע במאגר.
- `private static final int MAX_POOL_SIZE = 100`
 - תחום הכרה: `private static final`.
 - תפקיד ושימוש: קבוע המגדיר את קיבולת המאגר המקסימלית, המאזן בין חיסכון בביצועים לבין תפיסת נפח ב-RAM.

הסבר פעולות מרכזיות:

- (ההסבר על לוגיקת ה-Pool יתמקד במאגר החלקיקים, שכן היא זהה לכלל המאגרים במחלקה)
- `public static BrokenParticle acquireParticle(float x, float y, Bitmap img)`
 - פעולה זו "מזמנת" חלקיק מוכן לשימוש מתוך המאגר. הפעולה פועלת כ"שולפת" ממחסנית. היא נכנסת לבלוק מסונכרן ובודקת את מצביע ה-Top. אם המצביע גדול מ-0, היא שולפת את האובייקט האחרון במערך (הפעולה המהירה ביותר ברמת המעבד), מנקה את הרפרנס במערך למניעת דליפות זיכרון, ומקטינה את המצביע. במידה והמאגר ריק, היא יוצרת מופע חדש. לבסוף, מחוץ לבלוק הנעילה, היא מאתחלת את האובייקט עם הנתונים החדשים דרך מתודת ה-init.
 - **תרשים זרימה (סדר פעולות):**
 - i. כניסה לבלוק מסונכרן.
 - ii. בדיקה: האם המצביע (Top) גדול מ-0?
 - כן: שליפת אובייקט מהמערך במיקום `Top-1` -> ניקוי התא במערך (null) -> הקטנת המצביע.
 - לא: יצירת אובייקט חדש בזיכרון.
 - iii. יציאה מבלוק מסונכרן.



- .iv הפעלת פונקציית האתחול (init) של האובייקט עם המיקום והתמונה המבוקשים.
- .v החזרת האובייקט לשימוש.

קוד: ○

```
public static BrokenParticle acquireParticle(float x, float y, Bitmap img) {
    BrokenParticle p;
    synchronized (particlePool) {
        if (particleTop > 0) {
            p = particlePool[--particleTop];
            particlePool[particleTop] = null; // Clear reference to avoid memory leaks
        } else p = new BrokenParticle();
    }
    p.init(x, y, img);
    return p;
}
```

● public static void releaseParticle(BrokenParticle p)

- הפעולה מחזירה אובייקט שסיים את תפקידו בחזרה למאגר. כאשר חלקיק הופך ל"מת" (למשל, סיים את האנימציה), הוא נשלח לפעולה זו במקום להימחק. הפעולה בודקת אם יש מקום פנוי במערך המאגר. אם כן, היא מכניסה את האובייקט למקום שאליו מצביע ה-Top ומקדמת את המצביע. בכך היא שומרת את המופע "חיי" בזיכרון RAM לשימוש עתידי.

קוד: ○

```
public static void releaseParticle(BrokenParticle p) {
    if (p == null) return;
    synchronized (particlePool) {
        if (particleTop < MAX_POOL_SIZE) {
            particlePool[particleTop++] = p;
        }
    }
}
```

● public static void clearAllPools()

- פעולה לניקוי מוחלט של כלל המאגרים בזיכרון. פעולה זו קריטית בעת מעבר בין עולמות או איפוס המשחק. היא עוברת בלולאה על כל המערכים, מציבה null בכל התאים (כדי לאפשר למערכת ההפעלה לנקות את הזיכרון אם צריך) ומאפסת את כל המצביעים ל-0.

קוד: ○

```
public static void clearAllPools() {
    synchronized (particlePool) {
        for (int i = 0; i < particleTop; i++) particlePool[i] = null;
        particleTop = 0;
    }
    //coins...
    //xpEffects...
}
```



מחלקת DialogueManager

מיקום: com.example.tutorialgame.managers.DialogueManager

תפקיד המחלקה: מחלקה זו משמשת כמנוע הפענוח והניהול של כלל הדיאלוגים במשחק. היא מקבלת מזהה דמות ומחזירה את שורות הטקסט הרלוונטיות ביותר עבורה באותו רגע. המחלקה מבצעת התאמה דינמית בין קבצי תוכן סטטיים (JSON) לבין מצב השחקן בענן, מנהלת מערכת מטמון (Cache) חסינה בריבוי תהליכים, ותומכת בלוקליזציה (תרגום) אוטומטית. זהו אחד הרכיבים המורכבים ביותר במערכת הנרטיבית, המאפשר לעולם המשחק להגיב בצורה אינטליגנטית להתקדמות השחקן.

היא אחראית על:

1. ניהול משאבי תוכן: טעינה, פענוח ודה-סריאליזציה אוטומטית של קבצי JSON באמצעות ספריית GSON.
2. סנכרון שפות (Localization): ניהול מטמון דינמי המתנקה ומתעדכן אוטומטית בעת החלפת שפה בהגדרות האפליקציה.
3. ניהול מטמון חכם (Thread-Safe Caching): שימוש ב-ConcurrentHashMap למניעת קריאות חוזרות מהדיסק ושמירה על יציבות ביצועים.
4. אלגוריתמיקה של הסתעפויות (Branching Logic): ניתוח מבנים מורכבים ב-JSON הכוללים תנאים לוגיים ובחירת ה"ענף" הנכון לשיחה.

הסבר על תכונות המחלקה:

- private static final Gson gson
 - תחום הכרה: private static final.
 - תפקיד ושימוש: מופע יחיד וקבוע של מפענח ה-JSON. שמירתו כסטטי מונעת את העלות היקרה של אתחול המפענח בכל פעם שנטען קובץ דיאלוג חדש.
- private static final Type STRING_LIST_TYPE
 - תחום הכרה: private static final.
 - תפקיד ושימוש: אובייקט הגדרת טיפוס (Type Token) המשמש את ספריית Gson כדי להמיר מערכי JSON ישירות לרשימות של מחרוזות ב-Java בצורה מהירה ובטוחה.
- private static final Map<String, JsonElement> dialogueCache
 - תחום הכרה: private static final.
 - תפקיד ושימוש: מאגר מסוג ConcurrentHashMap השומר את תוכן הקבצים שכבר פוענחו. השימוש במימוש זה מבטיח בטיחות מלאה בריבוי תהליכים (Thread Safety) ומונע "מרוצי תהליכים" כאשר מספר רכיבים מבקשים טקסט בו-זמנית.
- private static volatile String loadedLanguage
 - תחום הכרה: private static volatile.
 - תפקיד ושימוש: משתנה נדיף השומר את קוד השפה שנטענה לאחרונה. השימוש ב-volatile מבטיח שכל שינוי שפה בהגדרות יראה מיד בכל חלקי המנוע, מה שגורם לניקוי המטמון וטעינת תרגומים חדשים.

הסבר פעולות מרכזיות:

- public static List<String> resolveDialogue(String characterId)
 - זוהי פעולת השער המרכזית (Main API) שדרכה המערכת מבקשת טקסט עבור דמות מסוימת. הפעולה מבצעת שתי בדיקות קריטיות:
 - **תרשים זרימה (סדר פעולות):**
 - i. בדיקת התאמת שפה (checkLanguageSync).
 - ii. חיפוש ב-dialogueCache לפי מזהה דמות:
 - נמצא במטמון: המשך לשלב הפענוח.
 - לא נמצא: טעינה פיזית מתיקיית ה-Assets המתאימה לשפה.



- .iii בדיקת תקינות הקלט (Null Check).
- .iv שליחה לפענוח המבנה (parseDialogueElement).
- .v החזרת רשימת השורות הסופית.

○ קוד:

```
public static List<String> resolveDialogue(String characterId) {
    checkLanguageSync();
    String lang = loadedLanguage;

    // ComputeIfAbsent handles "get from cache or load if missing" atomically
    JsonElement element = dialogueCache.computeIfAbsent(characterId,
        id -> loadCharacterFile(id, lang));

    if (element == null || element.isJsonNull()) return Collections.emptyList();

    return parseDialogueElement(element);
}
```

● private static List<String> parseDialogueElement(JsonElement element)

- הלב הלוגי של המחלקה, המנתח את מבנה התוכן של הדיאלוג. הפעולה תומכת בשני סגנונות כתיבה ב-JSON:
 - i. דיאלוג פשוט: אם הקובץ הוא מערך (JSONArray), הוא מומר ישירות לרשימת שורות.
 - ii. דיאלוג מורכב (Branching): אם הקובץ הוא אובייקט המכיל "ענפים" (branches), הפעולה סורקת אותם לפי סדר הופעתם. עבור כל ענף, היא בודקת אם התנאי שלו מתקיים. הענף הראשון שתנאיו חיוביים הוא זה שנבחר, וכל השורות שבו יוחזרו להצגה.
- תרשים זרימה (סדר פעולות):
 - i. זיהוי סוג אובייקט ה-JSON.
 - ii. אם מערך: דה-סריליזציה ישירה והחזרה.
 - iii. אם אובייקט ענפים:
 - 1. מעבר בלולאה על רשימת ה-branches.
 - 2. בדיקת תנאי הענף מול מצב העולם (isConditionMet).
 - 3. במציאת הענף המתאים: חילוץ השורות והחזרה מיידית.
 - iv. אם לא נמצא התאמה: החזרת רשימה ריקה.

○ קוד:

```
private static List<String> parseDialogueElement(JsonElement element) {
    // Case 1: Simple array of strings
    if (element.isJsonArray()) {
        return gson.fromJson(element, STRING_LIST_TYPE);
    }

    // Case 2: Object with branching logic
    if (element.isJsonObject()) {
        JsonObject obj = element.getAsJsonObject();
        if (obj.has("branches") && obj.get("branches").isJsonArray()) {
            JSONArray branches = obj.get("branches").getAsJSONArray();
            for (JsonElement branchElement : branches) {
                if (!branchElement.isJsonObject()) continue;
                JsonObject branch = branchElement.getAsJsonObject();
                // Find the first branch where the condition is met
                if (isConditionMet(branch.get("condition"))) {
                    return gson.fromJson(branch.get("lines"), STRING_LIST_TYPE);
                }
            }
        }
    }
    return Collections.emptyList();
}
```



- `private static boolean isConditionMet(JsonElement conditionElement)`

- מקשרת בין דרישות הדיאלוג ב-JSON לבין הנתונים האמיתיים בענן. הפעולה מבצעת הערכה לוגית של תנאים. היא בודקת מול ה-`WorldStateDoc` האם דגל (Flag) מסוים הופעל. היא תומכת בבדיקות ברירת מחדל (האם דגל קיים) ובבדיקות מפורטות (האם דגל שווה לערך Boolean מסוים). במידה ולא הוגדר תנאי בענן, הוא נחשב כ"תמיד נכון", מה שמאפשר ליצור ענפי Fallback (שורות ברירת מחדל).

- קוד:

```
private static boolean isConditionMet(JsonElement conditionElement) {
    // If no condition is defined, the branch is always valid (e.g., fallback branch)
    if (conditionElement == null || conditionElement.isJsonNull()) return true;
    JsonObject condition = conditionElement.getAsJsonObject();
    WorldStateDoc worldState = MyApp.getWorldStateDoc();
    if (condition.has("flag")) {
        String flagName = condition.get("flag").getString();
        boolean flagValue = worldState.getCheckPoint(flagName);
        if (condition.has("equals")) {
            return flagValue == condition.get("equals").getAsBoolean();
        }
        return flagValue; // Defaults to checking if flag is true
    }
    return false;
}
```

- `public static void checkLanguageSync()`

- פעולת הבקרה של מערכת ה-Localization. כדי למנוע מצב שבו השחקן מחליף שפה אך ממשיך לראות טקסטים ישנים, הפעולה משווה את שפת המערכת לשפה הטעונה. אם יש שינוי, היא מבצעת ניקוי אטומי של המטמון תחת בלוק `synchronized` (מניעת התנגשויות ב-Thread הציור) ומאפשרת טעינה מחדש של הקבצים מהנתיבים החדשים.

- קוד:

```
public static void checkLanguageSync() {
    String currentLang = BaseActivity.getLang();
    if (!currentLang.equals(loadedLanguage)) {
        synchronized (DialogueManager.class) {
            if (!currentLang.equals(loadedLanguage)) {
                dialogueCache.clear();
                loadedLanguage = currentLang;
                Log.i(TAG, "Dialogue cache cleared. Switched to language: " + currentLang);
            }
        }
    }
}
```

בסיס נתונים (Firebase Firestore & Assets JSON)

המחלקה משמשת כמרכז סנכרון בין קבצי טקסט מקומיים לנתוני התקדמות בענן.

- סקירת המידע: המידע המילולי נשמר כקבצי JSON בתיקיית ה-`assets/dialogues/[lang]`, כאשר המבנה כולל שדות `flag`, `lines` ו-`equals`. הלוגיקה מוצלבת מול שדה ה-`storyFlags` (מפה של Key-Value) בתוך מסמך ה-`worldState` ב-Firestore. פעולות:

- קריאה: בזמן ריצה, המחלקה מבצעת Cross-Reference: היא שולפת תנאי מקובץ ה-JSON ומבצעת שאילתה פנימית מול ה-Cache של ה-`WorldStateDoc` כדי להחליט על תוכן הדיאלוג.

- הצגה בפורמט מסמך JSON (דוגמה לדיאלוג האב):

```
{
  "branches": [
    {
      "condition": { "flag": "event_player_talkedToFather", "equals": true },

```



```
"lines": [
  "Have you spoken to the blacksmith yet? Stop wasting time."
],
{
  "lines": [
    "> Good morning, father.",
    "Korato. I'm heading to a meeting with the village elders.",
    "I expect you to make yourself useful today. Go to the blacksmith, he asked for some
help.",
    "> But I wanted to-",
    "No arguments. Just do as you're told."
  ]
}
]
```



מחלקת Quest

מיקום: com.example.tutorialgame.quest.Quest

תפקיד המחלקה: מחלקה זו מייצגת משימה (Quest) בודדת בעולם המשחק. היא משמשת כיחידת מידע המרכזת את דרישות המשימה, התיאור המילולי שלה והלוגיקה שמופעלת בעת השלמתה. המחלקה מקשרת בין פעולות השחקן (כמו איסוף פריט או שיחה) לבין מערכת ההתקדמות בענן, ומנהלת את חלוקת הפרסים (מטבעות ונקודות ניסיון) בצורה אוטומטית.

היא אחראית על:

1. הגדרת יעדים: אחסון סוג המשימה והמטרה הספציפית באמצעות QuestType.
2. ניהול מצב השלמה: בדיקה מול מסמך ה-WorldStateDoc האם המשימה כבר הושלמה בעבר.
3. ניהול פרסים: עדכון אוטומטי של מאזן המטבעות וה-XP של השחקן בעת סיום המשימה.
4. ביצוע פעולות המשך: הרצת קוד ייחודי (Runnable) המוגדר לכל משימה בנפרד (למשל: פתיחת דלת או הפעלת סצנה).

הסבר על תכונות המחלקה:

- `private final @StringRes int taskRes`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מזהה משאב מחרוזת (String Resource) לתיאור המשימה, מה שמאפשר תמיכה מובנית בריבוי שפות (Localization).
- `private final String internalId`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מזהה ייחודי המשמש כדגל (Flag) בבסיס הנתונים. זהו המפתח לפיו המערכת יודעת אם המשימה בוצעה.
- `private final QuestType questType`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: אובייקט המגדיר את הקטגוריה של המשימה (למשל: DIALOGUE) ואת מזהה המטרה הספציפי.
- `private final Runnable onCompleteAction`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: ממשק פונקציונלי המאפשר להזריק לוגיקה מותאמת אישית שתרוץ בסיום המשימה, מה שמאפשר יצירת אירועים משורשרים בעלילה.
- `private final int coins, xp`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: ערכי התגמול (מטבעות ונקודות ניסיון) שהשחקן מקבל בסיום המשימה.

הסבר פעולות מרכזיות:

- `public boolean isCompleted()`
 - בודקת האם המשימה הושלמה על ידי שאילתה ל-WorldStateDoc ובדיקה האם ה-internalId קיים כצ'קפוינט חיובי. פעולה זו חיונית למניעת הצגת משימות ישנות ביומן המשימות.
 - קוד:

```
public boolean isCompleted() {
    return MyApp.getWorldStateDoc().getCheckPoint(internalId);
}
```

- `public void complete()`
 - הפעולה המרכזית לסגירת המשימה.



○ תרשים זרימה (סדר פעולות):

- i. בדיקת הגנה: אם המשימה כבר הושלמה, הפעולה נעצרת (מניעת כפילות פרסים).
- ii. עדכון ה-Flag ל-True ב-WorldStateDoc (סנכרון לענן).
- iii. עדכון מונה המשימות ב-ProgressDoc.
- iv. חלוקת המטבעות וה-XP דרך ה-CosmeticDoc וה-ProgressDoc.
- v. הפעלת ה-onCompleteAction אם הוא הוגדר.

○ קוד:

```
public void complete() {  
    if (isCompleted()) return;  
  
    // Mark as completed in cloud  
    MyApp.getWorldStateDoc().setCheckpoint(internalId);  
  
    // Update player progress and rewards  
    MyApp.getProgress().increaseQuestsCompleted();  
    MyApp.getCosmetic().increaseCoins(coins);  
    MyApp.getProgress().updateXp(xp);  
  
    // Execute post-completion logic  
    if (onCompleteAction != null) {  
        onCompleteAction.run();  
    }  
}
```

בסיס נתונים (Firestore)

המחלקה משמשת כמעדכנת של מספר מסמכים שונים בענן בו-זמנית ברגע השלמת משימה.

- סקירת המידע: המשימה משפיעה על WorldStateDoc (מפת ה-CheckPoints), ProgressDoc (XP ומונה משימות) ו-CosmeticDoc (מאזן מטבעות).
- פעולות:
 - עדכון: בעת קריאה ל-complete(), מתבצעת כתיבה (Write) למספר שדות ב-Firebase, מה שמבטיח שכלל התקדמות השחקן מסונכרנת לצמיתות.



מחלקת CollisionUtils

מיקום: com.example.tutorialgame.utils.CollisionUtils

תפקיד המחלקה: מחלקה זו משמשת כ"מנוע החישוב" הגיאומטרי של המשחק. היא מרכזת פתרונות סטטיים לבעיות של זיהוי התנגשויות, תרגום קואורדינטות וחישובי טווחים. תפקידה הוא להפריד את המתמטיקה המורכבת מהלוגיקה של הדמויות, ובכך לאפשר למנוע לבצע בדיקות ולידציה ומרחק בצורה מהירה, אחידה וחסכונית במשאבי מעבד. המחלקה הוגדרה כ-`final` עם בנאי פרטי כדי להבטיח שימוש סטטי טהור ללא יצירת מופעים (Instances).

היא אחראית על:

1. מיפוי עולם (Tile Mapping): תרגום מיקום רציף של תיבת פגיעה (Hitbox) לאינדקסים בדידים של משבצות במטריצת המפה.
2. בדיקת עבירות (Walkability): אימות של קבוצת נקודות מול נתוני המפה כדי לקבוע אם תנועה מסוימת חסומה.
3. אופטימיזציה מתמטית: ביצוע חישובי מרחקים וטווחים בשיטות חסכוניות (כגון השוואת ריבועים במקום שימוש בשורש ריבועי).
4. אימות גבולות (Bounds Safety): הבטחה שגישה לנתוני המפה לא תגרום לחריגה מגבולות המערך.

הסבר על תכונות המחלקה:

- המחלקה אינה מחזיקה תכונות (Fields), שכן היא מחלקת שירות סטטית בלבד המקבלת את כל הנתונים כפרמטרים בזמן אמת.

הסבר פעולות מרכזיות:

- `public static void setTileCords(RectF hitBox, float deltaX, float deltaY, Point[] resultPoints)`
 - הפעולה הופכת מיקום גיאומטרי (X, Y) למיקום לוגי במפה (Column, Row). כדי לבדוק התנגשות עתידית, הפעולה מחשבת היכן יימצאו ארבעת הקצוות של ה-`hitBox` לאחר התזוזה המתוכננת. היא מחלקת את הקואורדינטות ב-`TILE_SIZE` ומעגלת אותן למטה (int-casting). שימוש במערך קיים (`resultPoints`) מונע יצירת אובייקטים חדשים 60 פעמים בשנייה, מה ששומר על ביצועים יציבים.
 - **תרשים זרימה (סדר פעולות):**
 - i. קבלת תיבת הפגיעה והתזוזה המבוקשת.
 - ii. חישוב ארבעת גבולות המלבן (Left, Right, Top, Bottom) בתוספת ה-Delta.
 - iii. המרת הערכים לאינדקסים של אריחים על ידי חלוקה בגודל האריח הקבוע.
 - iv. עדכון הקואורדינטות בתוך מערך הנקודות שהתקבל.
 - **קוד:**

```
public static void setTileCords(RectF hitBox, float deltaX, float deltaY, Point[]
resultPoints) {
    if (resultPoints == null || resultPoints.length < 4) return;

    int tileSize = GameConstants.Sprite.TILE_SIZE;
    int left = (int) ((hitBox.left + deltaX) / tileSize);
    int right = (int) ((hitBox.right + deltaX) / tileSize);
    int top = (int) ((hitBox.top + deltaY) / tileSize);
    int bottom = (int) ((hitBox.bottom + deltaY) / tileSize);

    resultPoints[0].set(left, top);
    resultPoints[1].set(right, top);
    resultPoints[2].set(left, bottom);
    resultPoints[3].set(right, bottom);
}
```

- `public static boolean isWithinRange(float x1, float y1, float x2, float y2, float range)`



- בדיקת קרבה בין שתי נקודות בצורה אופטימלית לביצועים. במנועי משחק, פעולת השורש הריבועי (`Math.hypot` או `Math.sqrt`) נחשבת ליקרה יחסית למעבד. מכיוון שבדיקות טווח (כגון "האם השחקן קרוב מספיק ל-NPC כדי לדבר") קורות מאות פעמים בשנייה, הפעולה משתמשת בהשוואת ריבועים. התוצאה הלוגית זהה לחלוטין אך החישוב מהיר משמעותית.

○ קוד:

```
public static boolean isWithinRange(float x1, float y1, float x2, float y2, float range) {  
    float dx = x1 - x2;  
    float dy = y1 - y2;  
    return (dx * dx + dy * dy) <= (range * range);  
}
```

- `public static boolean areTilesWalkable(Point[] tileCords, GameMap gameMap)`
 - הפוסק האחרון לגבי יכולת התנועה של ישות. הפעולה מקבלת אוסף נקודות (בדרך כלל את קצוות השחקן) וסורקת אותן מול המפה. היא משתמשת בלוגיקת "Short Circuit" - ברגע שהיא מוצאת אריח אחד שאינו עביר, היא עוצרת ומחזירה `false`. רק אם כל הנקודות נמצאו עבירות, התנועה תאושר.

○ קוד:

```
public static boolean areTilesWalkable(Point[] tileCords, GameMap gameMap) {  
    if (gameMap == null) return false;  
    for (Point p : tileCords) {  
        if (!gameMap.isTileWalkable(p.x, p.y))  
            return false;  
    }  
    return true;  
}
```



מערכת השמע (Audio System)

השכבה האחראית על כל החוויה השמיעתית במשחק.

מחלקת MusicManager

מיקום: com.example.tutorialgame.engine.audio.MusicManager

תפקיד המחלקה: מחלקה זו אחראית על ניהול מוזיקת הרקע של המשחק. היא פועלת כרכיב "חכם" המאפשר מעברים חלקים בין רצועות שמע שונות, ניהול עוצמת קול דינמית (כולל הנמכה אוטומטית בזמן אירועים), ושמירת העדפות המשתמש. המחלקה משתמשת בשני נגני MediaPlayer במקביל כדי לבצע אפקט "מצלב" (Crossfade) המונע קטיעה של החוויה השמיעתית בעת החלפת שיר או מצב במשחק.

היא אחראית על:

1. ניהול מעברים (Crossfading): ביצוע מעבר הדרגתי (Fade-out) לשיר הישן ו-Fade-in לשיר החדש) לאווירה רציפה.
2. ניהול ווליום דינמי (Ducking): הנמכת ווליום המוזיקה באופן זמני (למשל בעת דיאלוג) והחזרתו למצב הקודם בסיום האירוע.
3. סנכרון מצב: שמירת עוצמת המוזיקה הנבחרת ב-SharedPreferences וטעינתה בכל הפעלה של האפליקציה.
4. יעילות ומשאבים: ניהול נכון של משאבי מערכת (Release) למניעת דליפות זיכרון.

הסבר על תכונות המחלקה:

- private MediaPlayer activePlayer, fadingPlayer
 - תחום הכרה: private.
 - תפקיד ושימוש: שני נגני מדיה המשמשים לתמרון בין שירים. ה-activePlayer מנגן את השיר הנוכחי, בעוד ה-fadingPlayer משמש להמשך ניגון השיר הקודם בזמן שהוא נחלש ונעלם.
- private int activeResId
 - תחום הכרה: private.
 - תפקיד ושימוש: שומר את המזהה (ID) של השיר המתנגן כרגע. משמש למניעת טעינה מחדש של שיר שכבר פועל.
- private float musicVolume
 - תחום הכרה: private.
 - תפקיד ושימוש: עוצמת הקול המוגדרת על ידי המשתמש (0.0 עד 1.0).
- private boolean isDucked
 - תחום הכרה: private.
 - תפקיד ושימוש: דגל המציין האם המוזיקה נמצאת כרגע במצב "מונמד" (למשל לצורך הדגשת דיבור).
- private final Handler handler
 - תחום הכרה: private final.
 - תפקיד ושימוש: מנהל תזמון המריץ את לוגיקת ה-Crossfade ב-Thread הראשי של הממשק (UI Thread), מה שמאפשר שינוי ווליום הדרגתי ומדויק.
- private boolean isLooping
 - תחום הכרה: private.
 - תפקיד ושימוש: דגל המשמש לקביעת האם מוזיקת הרקע תפעל בלולאה אינסופית, או שתיעצר בסיום השיר.

הסבר פעולות מרכזיות:

- private MusicManager(Context context) (הבנאי)



- הבנאי מאתחל את המנהל כ-Singleton. הוא טוען את עוצמת הקול האחרונה שנשמרה מההגדרות, יוצר את שני נגני ה-MediaPlayer ומגדיר להם AudioAttributes המתאימים למוזיקת רקע של משחק.

○ קוד:

```
private MusicManager(Context context) {
    this.appContext = context.getApplicationContext();
    this.sp = appContext.getSharedPreferences("settings", Context.MODE_PRIVATE);
    this.musicVolume = clamp(sp.getFloat("music_volume", 0.3f));
    saveRunnable = () -> sp.edit().putFloat("music_volume", musicVolume).apply();

    // הנגנים יצירת
    activePlayer = createMediaPlayer();
    fadingPlayer = createMediaPlayer();
}
```

● public void play(int resId)

- זוהי הפעולה המנהלת את הזרמת המוזיקה והחלפת השירים.
- **תרשים זרימה (סדר פעולות):**
 - בדיקת זהות: אם השיר המבוקש כבר מתנגן, הפעולה נעצרת.
 - בדיקת השהיה: אם השיר המבוקש הוא השיר הנוכחי אך הוא מושהה (Paused), הוא פשוט ימשיך לנגן.
 - הכנה למעבר: אם מתנגן שיר אחר, הנגן הפעיל עובר להיות ה-fadingPlayer.
 - טעינת שיר חדש: הנגן הפנוי (שהופך להיות ה-activePlayer) נטען בשיר החדש בווליום 0.
 - ביצוע Crossfade: הפעלת לוגיקה שמגבירה את החדש ומנמיכה את הישן במקביל למשך שנייה אחת.

○ קוד:

```
public void play(int resId) {
    // לעשות מה אין - מתנגן כבר השיר אם
    if (resId == activeResId && activePlayer != null && activePlayer.isPlaying()) return;
    // (Paused) בהשהיה פשוט הוא אבל הנוכחי השיר הוא המבוקש השיר אם
    if (resId == activeResId && activePlayer != null) {
        activePlayer.start();
        // לחזור לווליום לתת פשוט או אותו לחדש אפשר, שנעצר fade היה אם
        setVolume(musicVolume, false);
        return;
    }
    // --- באמת חדש בשיר טיפול: והלאה מכאן ---
    if (fadeRunnable != null)
        handler.removeCallbacks(fadeRunnable);
    // חדש הפעל פשוט, ישן נגן אין אם
    if (activePlayer == null || !activePlayer.isPlaying()) {
        setupAndPlayNewTrack(resId, musicVolume);
        return;
    }
    // החדש לשיר Crossfade בצע
    MediaPlayer temp = fadingPlayer;
    fadingPlayer = activePlayer;
    activePlayer = temp;
    lastResId = activeResId;
    setupAndPlayNewTrack(resId, 0f);
    startCrossfade();
}
```

● private void startCrossfade()

- פונקציה זו יוצרת Runnable שמתבצע בצעדים קטנים (כל 50 מילישניות). בכל צעד, היא מעלה מעט את הווליום של הנגן החדש ומורידה את הווליום של הנגן הישן. בסיום התהליך (לאחר שנייה), הנגן הישן נעצר לחלוטין כדי לחסוך במשאבים.



○ קוד:

```
private void startCrossfade() {
    final int FADE_DURATION_MS = 1000; // המעבר משך את קצת נקצר
    final int FADE_INTERVAL_MS = 50;
    final int NUMBER_OF_STEPS = FADE_DURATION_MS / FADE_INTERVAL_MS;
    final float STEP_SIZE = 1.0f / NUMBER_OF_STEPS;
    final float[] progress = {0.0f};

    fadeRunnable = new Runnable() {
        @Override
        public void run() {
            progress[0] += STEP_SIZE;
            if (progress[0] > 1.0f) {
                progress[0] = 1.0f;
            }
            float targetVolume = isDucked ? musicVolume * DUCK_VOLUME_MULTIPLIER : musicVolume;
            // החדש הנגן את הגבר
            if (activePlayer.isPlaying())
                activePlayer.setVolume(progress[0] * targetVolume, progress[0] * targetVolume);
            // הישן הנגן את הנמוך
            if (fadingPlayer.isPlaying()) {
                fadingPlayer.setVolume((1.0f - progress[0]) * targetVolume, (1.0f -
progress[0]) * targetVolume);
            }
            if (progress[0] < 1.0f) handler.postDelayed(this, FADE_INTERVAL_MS);
            else {
                // ה-fade סיימה
                if (fadingPlayer.isPlaying()) {
                    fadingPlayer.stop();
                }
                fadeRunnable = null; // חדש fade להתחיל אפשר
            }
        }
    };
    handler.post(fadeRunnable);
}
```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת ב-SharedPreferences כדי לשמור את הגדרות המשתמש.

- סקירת המידע: נשמר ערך עשירי יחיד תחת המפתח "music_volume".
- פעולות:

- שמירה מתבצעת בעיכוב (Debouncing) של ms500 דרך ה-handler כדי למנוע עומס כתיבה לדיסק בזמן שהמשתמש משנה את עוצמת הקול בתפריט.



מחלקת SoundManager

מיקום: com.example.tutorialgame.engine.audio.SoundManager

תפקיד המחלקה: מחלקה זו היא ה"מנהל המרכזי" של כל אפקטי הסאונד (SFX) במשחק. היא משתמשת ב-SoundPool של אנדרואיד כדי לאפשר ניגון בו-זמני של מספר רב של צלילים קצרים בביצועים גבוהים. המחלקה מנהלת את טעינת הצלילים, עוצמת הקול הגלובלית, וכוללת לוגיקה מתקדמת לחישוב סאונד מרחבי (Spatial Audio) המשתנה לפי מיקום השחקן והמצלמה.

היא אחראית על:

1. ניהול משאבים (Singleton): הבטחה שקיים רק מופע אחד של מנהל הסאונד המרכזי את כל המשאבים.
2. סאונד מרחבי (Spatial SFX): חישוב דינמי של עוצמה (Attenuation) ואיזון צדדים (Panning) בהתבסס על המרחק בין מקור הצליל לבין נקודת המבט של המצלמה.
3. גיוון אודיו (Pitch Randomization): שינוי קל ואקראי של גובה הצליל (Pitch) בכל ניגון, כדי למנוע תחושת חזרתיות מונוטונית.
4. בטיחות וביצועים: שימוש במבני נתונים בטוחים לריבוי תהליכים (Concurrent) וניהול חכם של שמירת הגדרות לדיסק (Debouncing).

הסבר על תכונות המחלקה:

- `private final SoundPool soundPool`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: הרכיב המרכזי של אנדרואיד לניגון אפקטים קוליים מהיר. מוגדר לתמוך בעד 10 ערוצי שמע בו-זמנית.
- `private static SoundManager instance`
 - תחום הכרה: `private static`.
 - תפקיד ושימוש: מחזיק את המופע היחיד של המחלקה (Singleton). מאפשר לכל חלק במשחק לגשת למנהל הסאונד ללא צורך ביצירת מופע חדש.
- `private final Map<Integer, Integer> resIdToSoundId`
 - תחום הכרה: `private final ConcurrentHashMap`.
 - תפקיד ושימוש: מפה המקשרת בין מזהה המשאב ב-Raw (למשל `R.raw.jump`) לבין המזהה הפנימי ש-SoundPool הקצה לו. השימוש ב-Concurrent מבטיח יציבות בעבודה מול לולאת המשחק.
- `private final Set<Integer> loadedSoundIds`
 - תחום הכרה: `private final Concurrent Set`.
 - תפקיד ושימוש: רשימת מעקב אחר צלילים שסיימו להיטען בהצלחה מהדיסק לזיכרון. מונעת ניסיון ניגון של צליל שטרם מוכן.
- `private float soundVolume`
 - תחום הכרה: `private`.
 - תפקיד ושימוש: עוצמת הקול הגלובלית של האפקטים (0.0 עד 1.0).
- `private final Handler saveHandler`
 - תחום הכרה: `private final`.
 - תפקיד ושימוש: מנהל תזמון (Handler) המשמש לביצוע פעולות השמירה של עוצמת הקול ל-SharedPreferences בהשהיה (Delay), כדי למנוע עומס על הדיסק בעת שינוי מהיר של ה-SeekBar.

הסבר פעולות מרכזיות:

- `private SoundManager(Context context)` (הבנאי)
 - הבנאי מבצע אתחול מלא של מערך השמע. הוא מגדיר את ה-AudioAttributes (מציין שמדובר בסאונד של משחק לצורך תעדוף במערכת), בונה את ה-SoundPool, ומגדיר



מאזין (OnLoadCompleteListener) שמעדכן את רשימת ה-loadedSoundIds רק כאשר קובץ נטען במלואו לזיכרון. לבסוף, הוא טוען את עוצמת הקול האחרונה שנשמרה מההגדרות.

קוד: ○

```
private SoundManager(Context context) {
    this.appContext = context.getApplicationContext();

    AudioAttributes audioAttributes = new AudioAttributes.Builder()
        .setUsage(AudioAttributes.USAGE_GAME)
        .setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
        .build();

    soundPool = new SoundPool.Builder()
        .setMaxStreams(10)
        .setAudioAttributes(audioAttributes)
        .build();

    soundPool.setOnLoadCompleteListener((pool, sampleId, status) -> {
        if (status == 0) {
            loadedSoundIds.add(sampleId);
        } else {
            Log.w(TAG, "Failed to load soundId = " + sampleId);
        }
    });

    sp = appContext.getSharedPreferences("settings", Context.MODE_PRIVATE);
    this.soundVolume = clamp(sp.getFloat("sound_volume", 0.5f));
}
```

● public void playSpatialSfx(int resId, float sourceX)

○ זוהי הפעולה המורכבת ביותר המדמה שמיעה תלת-ממדית בעולם דו-ממדי.

○ תרשים זרימה (סדר פעולות):

- i. בדיקת מוכנות: וודא שהצליל טעון בזיכרון.
- ii. חישוב מרחק: מדידת המרחק האופקי (distanceX) בין מקור הצליל (sourceX) לבין מרכז המצלמה (listenerX).
- iii. חישוב עוצמה (Attenuation): ככל שהמרחק גדל, עוצמת הקול נחלשת. אם המרחק עולה על פי 1.5 מרוחב המסך, הצליל לא יישמע כלל.
- iv. חישוב איזון (Panning): חישוב היחס בין ימין לשמאל. אם המקור משמאל למצלמה, עוצמת רמקול ימין תיחלש בהתאם למרחק, ולהיפך.
- v. רנדומליזציה: בחירת pitch אקראי קל.
- vi. יגון: שליחת הפרמטרים המחושבים ל-soundPool.play.

קוד: ○

```
public void playSpatialSfx(int resId, float sourceX) {
    if (!isLoading(resId)) return;

    int screenWidth = SCREEN_WIDTH;
    if (screenWidth == 0) {
        playSfx(resId);
        return;
    }

    float listenerX = CameraManager.getLookAtX();
    float distanceX = Math.abs(sourceX - listenerX);
    float maxHearingDistance = screenWidth * 1.5f;

    float attenuationFactor = 1.0f - (distanceX / maxHearingDistance);
    float distanceVolume = Math.max(0, attenuationFactor);

    float finalEffectiveVolume = soundVolume * distanceVolume;
    float pan = (sourceX - listenerX) / (screenWidth / 2.0f);
}
```



```

pan = Math.max(-1.0f, Math.min(1.0f, pan));

float finalLeftVolume;
float finalRightVolume;
if (pan > 0) { // Source is to the right
    finalRightVolume = finalEffectiveVolume;
    finalLeftVolume = finalEffectiveVolume * (1.0f - pan);
} else { // Source is to the left or center
    finalLeftVolume = finalEffectiveVolume;
    finalRightVolume = finalEffectiveVolume * (1.0f + pan);
}

float rate = minRate + rnd.nextFloat() * (maxRate - minRate);
soundPool.play(Objects.requireNonNull(resIdToSoundId.get(resId)), finalLeftVolume,
finalRightVolume, 1, 0, rate);
}

```

● public void setVolume(float volume, boolean persist)

- מעדכנת את עוצמת הקול ומבצעת שמירה חכמה. כדי לא לפגוע בביצועי המשחק, אם persist הוא false (כמו בזמן גרירת סליידר), השמירה מתבצעת ב-Debounce: ה-Handler מבטל שמירות קודמות ומחכה 500 מילישניות של "שקט" לפני הכתיבה לדיסק.

○ קוד:

```

public void setVolume(float volume, boolean persist) {
    this.soundVolume = clamp(volume);
    saveHandler.removeCallbacks(saveRunnable);
    if (persist) {
        saveRunnable.run();
    } else {
        saveHandler.postDelayed(saveRunnable, SAVE_DELAY_MS);
    }
}

```

בסיס נתונים (SharedPreferences)

המחלקה משתמשת ב-SharedPreferences כדי לשמור את הגדרות המשתמש.

- סקירת המידע: המידע נשמר כזוג של מפתח ("sound_volume") וערך (מספר עשרוני המייצג עוצמה).
- פעולות:
 - טעינה: מתבצעת בבנאי בעת הקמת המנהל.
 - שמירה: מתבצעת דרך ה-saveRunnable המופעל על ידי ה-saveHandler.



מדריך למשתמש

פרק זה מציג את הדרישות להתקנת האפליקציה ואת אופן השימוש בה מנקודת מבטו של המשתמש. המדריך מתאר את חוויית השימוש הכללית, מסביר את זרימת הפעולות המרכזיות באפליקציה, מציג את סוגי ההודעות שהמשתמש עשוי לפגוש, ומפרט מגבלות ואילוצים הקשורים לשימוש במערכת.

דרישות התקנה וסביבת עבודה

האפליקציה פותחה ונבדקה במספר סביבות כדי להבטיח תאימות רחבה:

- גרסאות אנדרואיד: אנדרואיד 10 (API 29) ועד אנדרואיד 14 (API 34)
- סוגי מכשירים:
- אמולטורים: Pixel 6, Pixel 7 Pro (בסביבת Android Studio)
- מכשירים פיזיים: Redmi Note 13 Pro.

דרישות כלליות:

- מערכת הפעלה אנדרואיד 8 (API 26) ומעלה
- חיבור אינטרנט פעיל (נדרש לכניסה ראשונית ולסנכרון נתונים)
- נפח אחסון פנוי של כ-50MB

אופן פעולת האפליקציה

עם פתיחת האפליקציה, המשתמש עובר דרך מסך פתיחה (Splash) המבצע טעינה של נתוני המשחק מהענן. במידה והמשתמש אינו מחובר, הוא יועבר למסך התחברות/הרשמה.

זרימת המשחק:

1. התחברות והרשמה: המשתמש נרשם עם אימייל וכינוי. המערכת שולחת אימייל אימות להגברת האבטחה.
2. תפריט ראשי: בחירה בין "סלוטים" (Slot) של משחק - המאפשרים לנהל מספר שמירות שונות על אותו חשבון.
3. עולם המשחק: השחקן שולט בדמות באמצעות ג'ויסטיק וירטואלי. ניתן לתקשר עם דמויות (NPC), לאסוף חפצים ולבצע משימות.
4. מערכת שדרוגים: בכל עליית רמה, השחקן מקבל נקודות שדרוג אותן הוא יכול להשקיע במסך ייעודי כדי לשפר את מדדי החיים, הכוח והסטמינה של הדמות.
5. שמירה אוטומטית: כל התקדמות (סיום משימה, שדרוג דמות) נשמרת אוטומטית בענן, כך שניתן להמשיך מאותה נקודה מכל מכשיר.

אופן המשחק

לאחר ההתחברות, השחקן מועבר ישירות למסך המשגר (Launcher) של חריץ השמירה הראשון. במסך זה, השחקן יכול לצפות בפרטי הפרופיל שלו, לערוך אותם, ולסקור את המדדים הסטטיסטיים הרלוונטיים לשמירה הנוכחית. כניסה לתפריט הפרופיל מתבצעת על ידי לחיצה על סמל הדמות בפינה השמאלית העליונה, שם ניתנת גם האפשרות לרכוש ולהחליף את עיצוב המסגרת של הדמות.

תחילת המשחק מתבצעת בלחיצה על כפתור ה-"PLAY" בפינה הימנית התחתונה. במשחק עצמו, השחקן שולט בדמות באמצעות ג'ויסטיק וירטואלי (Joystick) מובנה, הכולל פקדי פעולה לקפיצה, תזוזה, יצירת אינטראקציה (דיבור) ותקיפה. ההתקדמות בעלילה נעשית על ידי מעקב אחר מערכת המשימות (Quests) המוצגת מתחת למדדי הבריאות של השחקן, ודורשת פתרון בעיות וחשיבה עצמאית.³

³ ראו [נספח ד'](#), איור 1.1: ממשק השליטה (UI) בזמן אמת, כולל הג'ויסטיק ופקדי הפעולה.



מערכת השדרוגים נגישה דרך סמליל הדמות הממוקם משמאל למדדי החיים. לחיצה עליו פותחת את "תפריט השדרוגים", המאפשר גלילה בין מגוון יכולות בחירה אסטרטגית של שדרוג הדמות.⁴ בנוסף, ניתן לעצור את המשחק בכל עת ולגשת לתפריט ההשחיה (Main Menu) וההגדרות דרך סמל ההפסקה בפינה הימנית העליונה.

הודעות למשתמש ומשוב (Alerts & Feedback)

האפליקציה מספקת למשתמש משוב באמצעות הודעות והתראות במצבים שונים:

- **הודעות שגיאה:** מוצגות כאשר יש בעיה בהתחברות, בטעינת הנתונים, או בחיבור לרשת, וכוללות הסבר והנחיה לפעולה.⁵
- **אישורי פעולה:** לפני ביצוע פעולות חשובות (כגון מחיקת סלוט או יציאה מהחשבון), מוצגת הודעת אישור כדי למנוע טעויות.⁶
- **משוב חזותי וקולי:** פעולות כמו השלמת משימה, קבלת פרס או ביצוע פעולה משמעותית מלוות באפקטים גרפיים וצלילים, המסייעים להבין שהפעולה בוצעה בהצלחה.⁷

מגבלות ואילוצים

כדי להבטיח חוויית שימוש תקינה, קיימים מספר מגבלות:

- **מגבלות קלט:**
 - כינוי משתמש: בין 3 ל-16 תווים.
 - סיסמה: מינימום 6 תווים.
- **אילוץ תצוגה:**
 - המשחק מותאם ליחס מסך של 9:16. במכשירים עם יחס שונה, המערכת מבצעת התאמה כדי לשמור על מיקום תקין של האלמנטים במסך.
- **מגבלת רשת:**
 - חיבור אינטרנט נדרש לצורך התחברות ראשונית וסנכרון נתונים. במקרים מסוימים ניתן להמשיך לשחק גם ללא חיבור זמני, כאשר הנתונים נשמרים מקומית ומתעדכנים בענן כאשר החיבור מתחדש.

⁴ ראו [נספח ד'](#), איור 2. תפריט השדרוגים של הדמות.

⁵ ראו [נספח ג'](#), איור 1: דוגמה להודעת שגיאה קופצת בעת כישלון האפליקציה לטעון נתונים מהשרת.

⁶ ראו [נספח ג'](#), איור 2: חלונית אישור בטרם התנתקות מן החשבון, למניעת מחיקה בטעות.

⁷ ראו [נספח ג'](#), איור 3: דוגמה למשוב חזותי המשתלב על גבי מסך המשחק (Canvas) בעת עליית רמה.



רפלקציה

את הפרויקט התחלתי בגישה שאננה ויהירה - הצלחתי בקלות את כל המשימות שהיו היכרות עם מערכת ההפעלה של אנדרואיד סטודיו, מה שגרם לתחילת העבודה על הפרויקט כבר בראשית יא' לפני שניתנה ההוראה. החלטתי שאני מעוניין ליצור משחק, שכן הנושא תמיד עניין והעסיק אותי ורציתי לבדוק את יכולתי. חשבתי להתחיל בצורה בסיסית - לגרום לדמות לעבור מצד אחד של המסך לצד השני, תוך הפעלת אנימציה בסיסית (ישבתי שעות רק לצייר משהו שנראה סביר לפני שעוד התחלתי לעבוד על הקוד שגורם לאותה אנימציה לפעול - טעות), זה היה קשה... אפילו לא אפשרי, חיפשתי פתרונות בגוגל ופגשתי בטווריאל בן כ-30 פרקים המכיל את כל מה שצריך כדי לבנות משחק בסיסי באנדרואיד סטודיו. כך, יומם וליל ישבתי וראיתי את הסרטונים, למדתי דברים חדשים וצורות חשיבה שלא עלו בדעתי והצלחתי לבנות (בהעזקה מוחלטת) משחק בסיסי, ומהר. כבר במהלך הבנייה הראשונית היו לי רעיונות רבים למה לשנות ואיך אפשר לייעל, אבל חיכיתי לסופו - כאשר הוא באמת נגמר והיה זה תורי ליצור את השאר... לא ידעתי מה לעשות, פתאום ההתקדמות המהירה של הוספת פעולה מרכזית למשחק כל יום הפכה להתקדמות איטית וארוכה שרק פעם ב- באמת נחשבה למרכזית. עקב כך, למדתי לייעל את עצמי באמצעות שימוש בבינה מלאכותית. בהתחלה מאוד התנגדתי לכך מפני שרציתי להרגיש ולדעת שאני רשמתי את הקוד ושאיני יודע מה כל דבר עובד ואיך. לבסוף ויתרתי לעצמי והחלטתי שאני כן אתחיל להתייעץ עם אחת כדי לשפר את ההתקדמות ולהוסיף דברים שלא הצלחתי למצוא פתרון אליהם ברשת - אמרתי לעצמי שאני כן אקרא כל פיסת קוד שהיא מביאה לה ובאמת אטמיע אותו בפרויקט באופן המתאים לי, אז אמרתי. ככל שהפרויקט גדל ככה הוא הפך לקשה יותר לשליטה ולא זכרתי כבר מה כל דבר עושה ואיך, אז שמתי לב שאני יכול להשתמש בבינה מלאכותית ישירות בתוך סביבת העבודה שלי, וכך המשכתי עד הסוף.

למדתי המון דברים: החשיבות של ארכיטקטורה נכונה ו"קוד נקי", ומתי השימוש "בקוד נקי מדי" בעצם מפריעה להבנת הפרויקט והאיזון בין השניים; מה זה ומה גורם ל-Race Condition ואיך לטפל בזה, לשמור ולדאוג לכך שהאפליקציה לא תקרוס בגללו; ההבדלים בין פורמטי ה-Assets שאנחנו שומרים בתוך הקובץ שלנו וההשפעה שיש להם על מהירות ההרצה, כמו תמונות מסוג Png, Jpeg, WebP ועוד וההבדלים בין סוגי הקבצים שיש בתוך סוגי הפורמטים האלה, כמו כן גם ההבדלים בין פורמטים של קבצי אודיו ומתי נכון להשתמש בכל אחד, תלוי איזה סוג אודיו נדרש - האם זה אפקט קולי קצר, מוזיקת רקע, גינגל ועוד; למדתי שאני לא יודע ומתמצא בשפת התכנות שהשתמשנו בה (Java) ברמה שאפילו מגרדת את כל מה שאפשר וצריך לדעת; למדתי לחפש ולמצוא באגים בעזרת ה-Logcat ואיך להשתמש גם בבינה מלאכותית כדי לאתר את אותם באגים ולפרש את הודעות השגיאה שה-Logcat מציג.

בסופו של יום לא באמת סיימתי את הפרויקט, לפחות לא מבחינתי. לא סיימתי את המשחק מבחינה עלילתית, יש כמה באגים שאני מודע אליהם שהשארתי לסוף ולא טיפלתי בהם, רציתי להוסיף עוד אפקטים ו-Feedback שהאפליקציה נותנת חזרה בשביל לשפר את חווית המשתמש ועוד. אם היו לי עוד משאבים הייתי מסיים ציורים שהתחלתי ובכך משפר את האנימציות של המשחק, מוסיף עוד אפקטים ופידבק, משפר את הנראות הכוללת בכך שאצור לוגו חדש, מסך "משגר" מעוצב ואולי אשנה קצת את הממשק כפי שהוא עכשיו, הייתי מוסיף שלבים ומשימות צד, עוד Npc's כדי להוסיף למשחק חיים והרגשה שאתה באמת בתוך כפר, איורים לטאמבנייל שמופיע על כל סלוט התקדמות כדי לתת למשתמש דרך לדעת באיזו התקדמות הוא מעוניין ועוד. הדבר העיקרי שלדעתי לא מבוצע טוב באפליקציה הוא הדרך שבה מערכת המשימות פועלת וטוענת את עצמה, היא מאוד לא מודולרית ונהיה יותר ויותר מסובך להוסיף שלבים ומשימות והיא גם אמורה לטעון את מצב העולם האחרון כפי שהוא נשאר לפי ה-Checkpoint האחרון וזה לא עובד כראוי לפעמים, כך שאם היו לי עוד משאבים וזמן בהחלט הייתי מנסה לתקן ולשפר אותה.

לסיכום, זאת הייתה חוויה שאני לא אשכח ואני לא מעוניין לשכוח, נהניתי מאוד מיצירת הפרויקט ואני שמח שאיתגרתי את עצמי בצורה הזאת. הפרויקט עזר לי להבין במה אני מעוניין לעבוד כשאגדל (לפחות בינתיים) ומה מעניין ומרגש אותי.



ביבליוגרפיה

- קארין משחקים. (ללא תאריך). *[Android Game Tutorial]* [פלייליסט וידאו]. YouTube. https://www.youtube.com/playlist?list=PL4rzdwiZLaxYqSDifOi7mUtyzdM6R_2sT
- itch.io. [https://pixel-](https://pixel-boy.itch.io/ninja-adventure-asset-pack) pixel-boy. (2021). *Ninja Adventure - Asset Pack* [boy.itch.io/ninja-adventure-asset-pack](https://pixel-boy.itch.io/ninja-adventure-asset-pack)
- Pixabay. (ללא תאריך). *Sound effects* [ספריית אפקטים קוליים]. <https://pixabay.com/sound-effects/search/sfx/>
- itch.io. [https://srt-](https://srt-studio.itch.io/player-circle-frame-icon) SRT STUDIO. (2023). *Player circle frames* [studio.itch.io/player-circle-frame-icon](https://srt-studio.itch.io/player-circle-frame-icon)

תודות

ברצוני להביע תודה והערכה מיוחדת למורה שלי לפיתוח אפליקציות, אילנה. בזכות ההדרכה, התמיכה וההכוונה המקצועית שלה לאורך הדרך, נחשפתי לעולם המרתק של פיתוח התוכנה והמשחקים. הפרויקט הזה לא היה יוצא לפועל בלעדיה.



נספחים

פרק זה מרכז חומרים משלימים המרחיבים את התיעוד הטכני והויזואלי של הפרויקט

נספח א': קוד המקור המלא של הפרויקט

בשל היקפו הנרחב של קוד המקור, הכולל עשרות מחלקות ומימושים מורכבים של מנוע המשחק, הוחלט להגישו באמצעות מאגר דיגיטלי ב-Google Drive. המאגר מאורגן בצורה היררכית לפי חבילות (Packages) וכולל את כל קבצי ה-Java, ה-XML ומשאבי המערכת.

לצפייה בקוד המקור המלא, ניתן לגשת לכתובת הבאה: [TutorialGame – Google Drive](#)
או לסרוק את קוד ה-QR הבא:





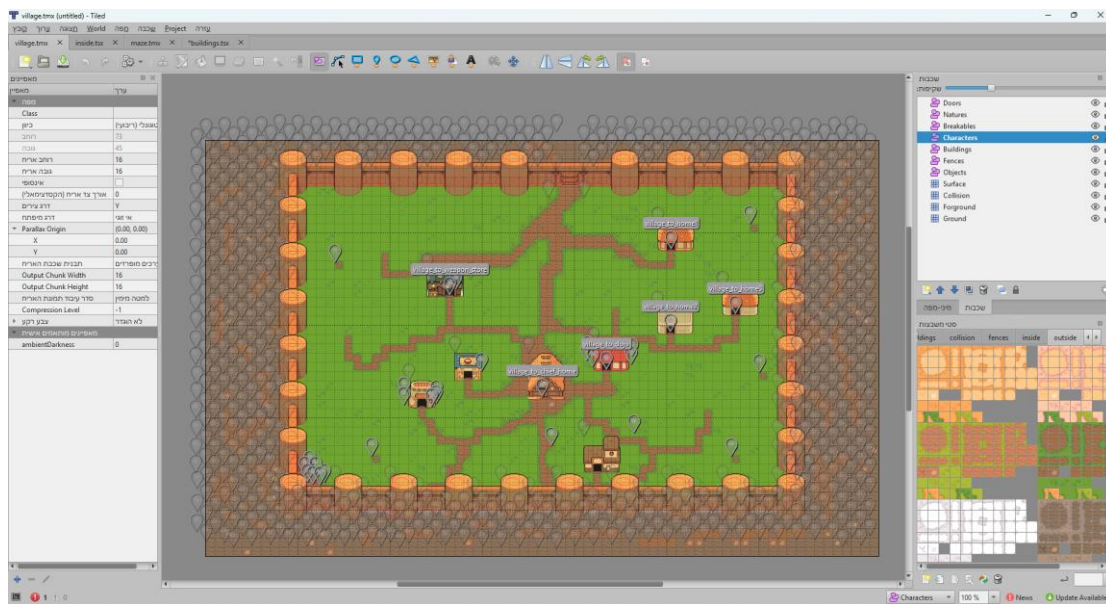
נספח ב': תהליך עיצוב השלבים - המחשה ויזואלית בעזרת Tiled

במסגרת פיתוח המשחק, עלה הצורך במערכת חכמה וגמישה ליצירת שלבים (Level Design) המפרידה בין קוד המשחק (הלוגיקה) לבין העיצוב הוויזואלי. לשם כך, בחרתי להשתמש בתוכנת **Tiled Map Editor** – כלי ליצירת מפות מבוססות-אריחים (Tilemaps) השומר את המידע בפורמט TMX (המבוסס על XML).

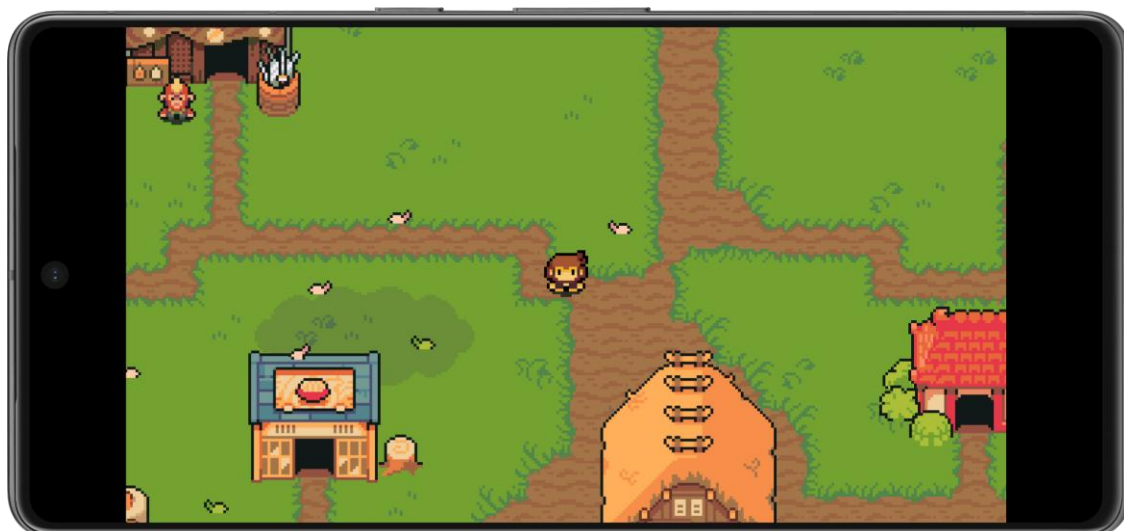
כדי לגשר בין המפות שעוצבו בתוכנה לבין מנוע המשחק באנדרואיד, כתבתי את המחלקה **TmxLoaderUtils**. מחלקה זו משתמשת ב-DOM Parser כדי לקרוא, לנתח ולהמיר את קבצי ה-XML לאובייקטים חיים בזיכרון (**MapLoadData**), אותם מנוע המשחק יודע לרנדור ולנהל.

המערכת שבניתי מנצלת את היכולות של Tiled במספר מישורים מרכזיים:

- 1. הפרדה לשכבות תצוגה ולוגיקה (Tile Layers):** מפות המשחק בנויות ממספר שכבות אריחים כדי לאפשר עומק ויזואלי ולוגי:
 - **שכבות ויזואליות:** שכבות כמו הקרקע (**Ground**) והקירות (**Walls**) המשמשות לרינדור הגרפיקה בלבד.
 - **שכבות לוגיות:** שכבות כמו התנגשויות (**Collision**) וסוגי משטח (**Surface**). שכבת ההתנגשויות, למשל, אינה נראית לשחקן, אך המנוע קורא את מערך הנתונים שלה (CSV) כדי לדעת אילו משבצות חסומות להליכה.
- 2. הצבת ישויות ואובייקטים דינמיים (Object Layers):** בעוד שאריחים הם סטטיים, המשחק מכיל אובייקטים חיים (Entities). עורך ה-Tiled מאפשר להציב אובייקטים וקטוריים במפה. המחלקה שכתבתי סורקת שכבות אובייקטים רבות, כגון: דמויות (**Characters**), מבנים (**Buildings**), טריגרים (**Triggers**), מקורות אור (**Lights**) ודלתות (**Doors**). המערכת מחלצת את מיקום האובייקט (X, Y) ואת ממדיו, ומכפילה אותם בקבוע קנה-המידה של המשחק (**SCALE_MULTIPLIER**) כדי להתאים את המפה לרזולוציית המסכים של מכשירי אנדרואיד.
- 3. שימוש במאפיינים מותאמים אישית (Custom Properties):** זהו אחד היתרונות הבולטים בשימוש ב-Tiled בפרויקט. הזרקתי לוגיקה למפה באמצעות "Custom Properties" (מאפיינים שמוגדרים בעורך ונקראים בקוד), כך שהמפה מחזיקה במידע הנדרש להפעלתה:
 - **הגדרות סביבה:** רמת החושך במפה (**ambientDarkness**), קובץ המוזיקה שיתנגן ברקע, וכמות/סוג המפלצות שיזדמנו (**minMonsters, spawnType**).
 - **מעברי מפות (Doors):** אובייקטים מסוג דלתות מכילים מאפיינים כמו **connectsTo** ו-**targetDoor**, המאפשרים למנוע לדעת לאיזו מפה להעביר את השחקן ובאיזו נקודה להציב אותו.
 - **תאורה וטריגרים:** מקורות אור מכילים תכונות כמו צבע ואופי התאורה (**lightType**), ואובייקטים יכולים לדרוש תנאי התקדמות בעלילה (**requiredCheckpoint**).
- 4. ניהול חבילות גרפיקה (Tilesets):** כדי לחסוך בזיכרון, המערכת קוראת קבצי מקור חיצוניים של אריחים (**tilesets**). כל אריח מקבל מספר מזהה גלובלי (GID). בעת קריאת הנתונים, המערכת מתרגמת את ה-GID לאינדקס המקומי של האריח הרלוונטי, שולפת את המאפיינים הספציפיים שלו, ומכינה אותו לציור על גבי הקנבס (Canvas).



איור ב.1: תכנון מפת המשחק בעורך Tiled, כולל תצוגת חלוקת השכבות (Layers) ומאפייני האובייקטים.



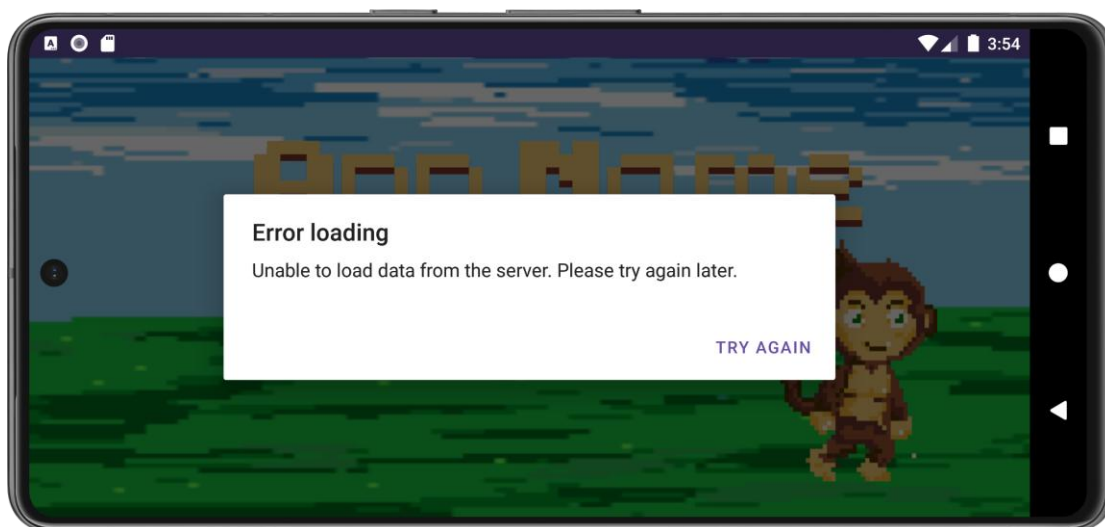
איור ב.2: המפה הסופית כפי שהיא מרונדרת בזמן אמת על ידי מנוע המשחק במכשיר האנדרואיד.



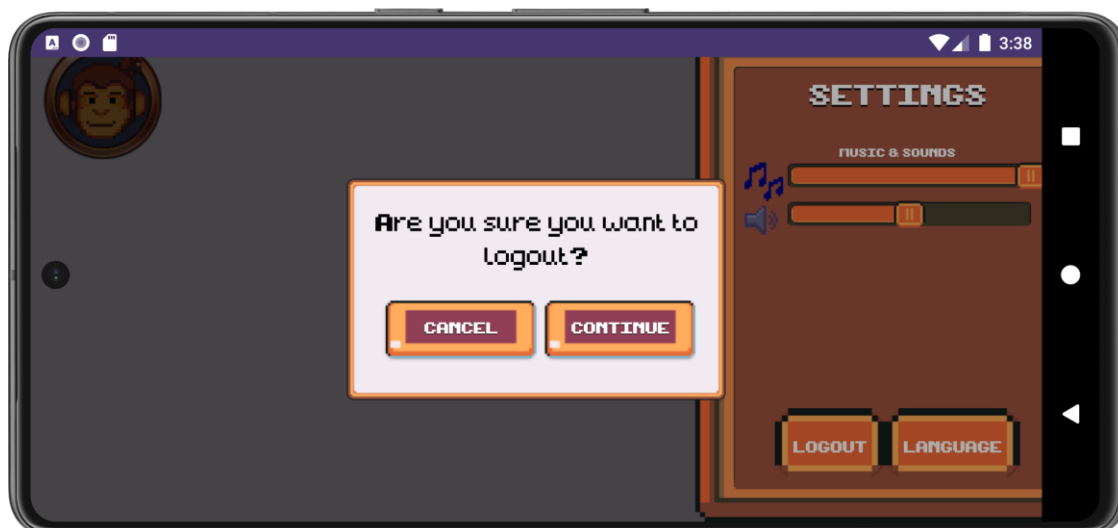
איור ב.3: ניתן לראות גם את שכבת הטבע והגדרות.



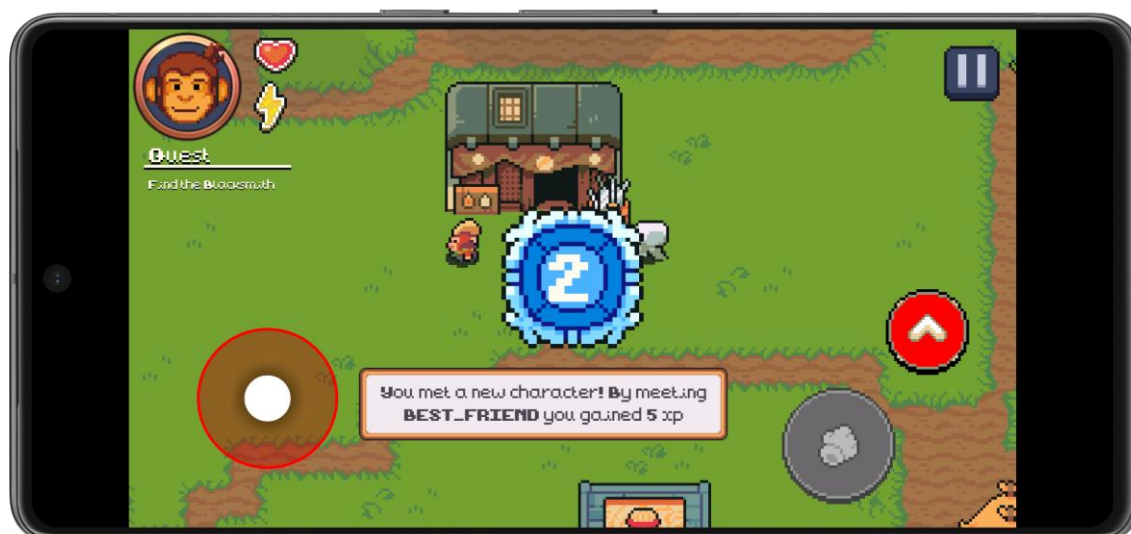
נספח ג': ממשק המשתמש, התראות ומשוב חזותי



איור ג.1: דוגמה להודעת שגיאה קופצת בעת כישלון האפליקציה לטעון נתונים מהשרת.



איור ג.2: חלונית אישור בטרם התנתקות מן החשבון, למניעת מחיקה בטעות.



איור ג.3: דוגמה למשוב חזותי המשתלב על גבי מסך המשחק (Canvas) בעת עליית רמה.



נספח ד': ממשק המשתמש (UI) ותפריטי המשחק



איור ד.1: ממשק השליטה (UI) בזמן אמת, כולל הגיוסטיק ופקדי הפעולה.



איור ד.2: תפריט השדרוגים של הדמות.